

Information Security and Cryptography

Jean-Philippe Aumasson

Willi Meier

Raphael C.-W. Phan

Luca Henzen

The Hash Function BLAKE



Springer

Information Security and Cryptography

Series Editors

David Basin
Kenny Paterson

Advisory Board

Michael Backes
Gilles Barthe
Ronald Cramer
Ivan Damgård
Andrew D. Gordon
Joshua D. Guttman
Christoph Kruegel
Ueli Maurer
Tatsuaki Okamoto
Adrian Perrig
Bart Preneel

More information about this series at <http://www.springer.com/series/4752>

Jean-Philippe Aumasson • Willi Meier
Raphael C.-W. Phan • Luca Henzen

The Hash Function BLAKE

Jean-Philippe Aumasson
Nagravision SA
Kudelski Security
Cheseaux-sur-Lausanne, Switzerland

Willi Meier
Hochschule für Technik
Fachhochschule Nordwestschweiz
Windisch, Switzerland

Raphael C.-W. Phan
Faculty of Engineering
Multimedia University
Cyberjaya, Malaysia

Luca Henzen
Department of IT Security
UBS AG
Zürich, Switzerland

ISSN 1619-7100

ISBN 978-3-662-44756-7

ISBN 978-3-662-44757-4 (eBook)

DOI 10.1007/978-3-662-44757-4

Springer Heidelberg New York Dordrecht London

Library of Congress Control Number: 2014958303

© Springer-Verlag Berlin Heidelberg 2014

This work is subject to copyright. All rights are reserved by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed. Exempted from this legal reservation are brief excerpts in connection with reviews or scholarly analysis or material supplied specifically for the purpose of being entered and executed on a computer system, for exclusive use by the purchaser of the work. Duplication of this publication or parts thereof is permitted only under the provisions of the Copyright Law of the Publisher's location, in its current version, and permission for use must always be obtained from Springer. Permissions for use may be obtained through RightsLink at the Copyright Clearance Center. Violations are liable to prosecution under the respective Copyright Law.

The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.

While the advice and information in this book are believed to be true and accurate at the date of publication, neither the authors nor the editors nor the publisher can accept any legal responsibility for any errors or omissions that may be made. The publisher makes no warranty, express or implied, with respect to the material contained herein.

Printed on acid-free paper

Springer is part of Springer Science+Business Media (www.springer.com)

Foreword

“You’re not allowed to hum for your own algorithm.”

It was a late Friday afternoon in March 2012. John Kelsey, a cryptographer from the United States National Institute of Standards and Technology (NIST), was standing on stage at the Third SHA-3 Candidate Conference.

SHA stands for “Secure Hash Algorithm”. Five years earlier NIST had kicked off an open competition to create a new hash-function standard, SHA-3. NIST’s earlier hash-function standards, SHA-0 and SHA-1 and SHA-2, didn’t come from open competitions; they were designed by the United States National Security Agency. Public research showed in 1998 that SHA-0 was weaker than advertised and showed in 2005 that SHA-1 was weaker than advertised. The design of SHA-2 is not very different from the design of SHA-1, and nobody will be surprised to hear that SHA-2 is also weaker than advertised, although public research so far hasn’t broken SHA-2.

In response to NIST’s call for SHA-3 submissions, 200 cryptographers from around the world formed into teams and designed 64 different hash functions. Years of grueling security analysis and performance analysis then showed that some hash functions were too easy to break and that some were too slow. Others were simply *worrisome*, like SHA-2: it’s not that anyone had figured out how to break them, but a break wouldn’t have been terribly surprising.

NIST ran the First SHA-3 Candidate Conference in Leuven in February 2009 and then a few months later announced 14 second-round candidates. NIST ran the Second SHA-3 Candidate Conference in Santa Barbara in August 2010 and then a few months later announced 5 finalists: BLAKE, Grøstl, JH, Keccak, and Skein. NIST had also declared that in the end there could be only one. Now, at the Third SHA-3 Candidate Conference, NIST was asking for any inputs from the community that could influence its final decision.

“Humming” is a voting mechanism used by the Internet Engineering Task Force, the primary organization developing standards for Internet protocols. Whoever is running the meeting, the “chair”, states an option; everyone in the room who approves of the option then says “mmmmm” in unison. After cycling through all options, the chair summarizes which options received the most hums. One advantage of humming over raising your hand is that it’s more precise: it allows you to hum

more quietly for options that you like but don't have strong feelings about. Humming is also faster than applauding, and hurts your hands less if you're spending all day doing it. One disadvantage of humming is that it's prone to abuse: for example, people who pack the front of the room will be more audible to the chair. Dissenters who say things like "I didn't hear such a loud volume of humming for that option" are considered troublemakers, aren't invited out for beer later, and don't end up having any actual effect on the chair's official notes of the meeting. However, if the chair's goal is simply to see which options have substantial support ("Bush and Gore both seem quite popular"), then humming works reasonably well.

A minute earlier Kelsey had summarized the humming procedure and had said that he would ask the room to hum for each of the SHA-3 finalists. He then named one of the finalists.

I was at the conference. All I could hear at this point was very loud humming from several people sitting in front of me—the submission team for that finalist.

"You're not allowed to hum for your own algorithm," Kelsey said. "Let's try this again." He named the same finalist again.

Deafening silence.

"Wow," Kelsey said, obviously surprised.

The other finalists received more hums. Two of the finalists, BLAKE and Keccak, obviously had much more substantial support than the rest. Both of them also had good reasons for this support. They had very large "security margins": many more rounds of hash computation than were necessary to protect against state-of-the-art attacks. These security margins inspired confidence that improved attacks, even radically improved attacks, would not actually hurt security. BLAKE and Keccak nevertheless offered performance that was never much worse than SHA-2 and often much better.

A closer look shows many ways that NIST could have opted for either BLAKE or Keccak. Software implementations of BLAKE were clearly faster than software implementations of Keccak. Applications that needed higher speeds might opt for hardware accelerators, and accelerators for BLAKE used less hardware area than accelerators for Keccak. On the other hand, as the speed targets increased further, the picture changed: Keccak clearly used less hardware area than BLAKE, and less energy per hashed bit. Keccak also has a "permutation" structure that allows the same hardware to be efficiently reused for applications beyond hashing. As for security, the analysis of Keccak's security seemed reasonably comprehensive, covering all major avenues of attack; but the analysis of BLAKE's security seemed even more comprehensive. As NIST put it later in their final SHA-3 report, the BLAKE security analysis "appears to have a great deal of depth" while the Keccak security analysis "has somewhat less depth."

Some observers tried to guess NIST's final decision by looking at the official evaluation criteria stated in NIST's call for SHA-3 submissions. The detailed list of criteria begins by stating that "The security provided by an algorithm is the most important factor in the evaluation." The discussion of security includes text that I had suggested: "Hash algorithms will be evaluated not only for their resistance against previously known attacks, but also for their resistance against attacks pointed out

during the evaluation process, and for their likelihood of resistance against future attacks.” Obviously the depth of security analysis says something about the likelihood of resistance against future attacks.

NIST had also emphasized security ten years earlier in its call for submissions for the Advanced Encryption Standard (AES), NIST’s previous cryptographic competition. “The security provided by an algorithm is the most important factor in the evaluation,” NIST wrote in the call. “Security was the most important factor in the evaluation,” NIST wrote in its final AES report in 2001.

But let’s go back to the videotape and see how AES was actually chosen. Out of the finalists there were two leading candidates, Rijndael and Serpent. Both candidates had attractive performance features: for example, NIST wrote that “Serpent is well suited to restricted-space environments” and that “pipelined implementations of Serpent [in counter mode] offer the highest throughput of any of the finalists,” while Rijndael was faster than Serpent in software on most CPUs available at the time. As for security, NIST wrote that “Rijndael appears to offer an *adequate* security margin” (emphasis added) while “Serpent appears to offer a *high* security margin.” Ultimately NIST chose Rijndael over Serpent, evidently deciding that the difference in security margin was outweighed by other factors.

NIST announced in October 2012 that it had chosen Keccak as SHA-3. Evidently NIST had decided that the difference in “depth” of security analysis between Keccak and BLAKE was outweighed by other factors. NIST highlighted three factors in its summary of the reasons for choosing Keccak:

- Keccak “offers acceptable performance in software, and excellent performance in hardware.”
- Keccak “has a large security margin, suggesting a good chance of surviving without a practical attack during its working lifetime.”
- Keccak “is also a fundamentally new and different algorithm that is entirely unrelated to the SHA-2 algorithms.” NIST explained that SHA-2 (like BLAKE) was an “ARX design with a key schedule,” whereas Keccak is “a hardware-oriented design that is based entirely on simple bit-oriented operations and moving bits around.”

NIST could just as easily have stated that BLAKE offers excellent performance in software and acceptable performance in hardware; nowhere did NIST suggest that hardware is more important than software. NIST also stated that BLAKE has a large security margin. So in the end it seems that the main reason for selecting Keccak as SHA-3 was primarily because Keccak is “different” from SHA-2.

Perhaps what *you* would like out of a hash function is not something “different” but something *better*: something that is simultaneously stronger and faster. Perhaps what you want is not a complement for SHA-2 but a replacement for SHA-2. I don’t mean to suggest that Keccak is a bad hash function—out of all the hash functions that were submitted to the SHA-3 competition, Keccak is one of my favorites—but if you’re not satisfied with SHA-2 then it’s more likely for your dissatisfaction to be addressed by BLAKE than by SHA-3.

This is the BLAKE book. It tells you *what* BLAKE is and *why* BLAKE is that way. It's written by the top BLAKE experts: the people who designed BLAKE in the first place.

Perhaps BLAKE still isn't fast enough for you. Perhaps performance constraints have forced you to stay with MD5, despite the many known security problems in MD5. You'll then be happy to hear about BLAKE's successor, BLAKE2, which is even faster than MD5 on the CPUs that you most likely care about. BLAKE2 is also described in this book.

Happy hashing!

Oberwolfach, Germany, August 2014

Daniel J. Bernstein

Preface

This book is about the cryptographic hash function BLAKE, one of the five final contenders in the SHA3 competition, out of 64 initial submissions. The SHA3 competition was a public competition held by the US National Institute of Standards and Technology (NIST) aiming to standardize a new Secure Hash Algorithm (SHA), to augment the previous standard, SHA2, following the perceived risk of a cryptanalytic attack.

The SHA3 Hash Competition ended in autumn 2012 with the selection of Keccak as the future US federal standard. Obviously we were disappointed when Keccak was chosen, for BLAKE was considered by many as one of the favorites. Nevertheless, we believe that NIST made the best choice in the circumstances. On the positive side, this gave us the opportunity to create BLAKE2, an improved version of BLAKE that quickly gained traction among developers.

BLAKE was designed between 2007 and 2008, as part of Jean-Philippe’s PhD thesis work at the University of Applied Sciences, Northwestern Switzerland (FHNW), supervised by Willi, and assisted by Raphael and Luca.

We started this book before the selection of Keccak as SHA3 and—let us be honest—we did it because we thought that BLAKE could win and that a book would thus be of interest to many. But after the SHA3 selection, we realized that we needed to do more than what would have been the “SHA3 book,” and this motivated us to put in even more effort. The SHA3 selection announcement also prompted another initiative: the design of BLAKE2.

BLAKE2 was initiated by Jean-Philippe jointly with Samuel Neves (who authored the fastest implementations of BLAKE), Zooko Wilcox-O’Hearn, and Christian Winnerlein. The collaboration stemmed from Twitter discussions and quickly materialized with an improved design inspired by modern applications and platforms. BLAKE2 builds on the cryptanalysis and implementation effort carried out on BLAKE, and was rapidly adopted by developers as a best-of-both hash function: as fast as legacy algorithms MD5 and SHA1, yet with the security of a SHA3 finalist. We thank Samuel, Zooko, and Christian for bringing their unique skills to this project and for the effective teamwork.

We have tried to make this book as accessible as possible, such that most chapters do not require advanced prior knowledge. Our target readers are both:

- developers, engineers, and security professionals who wish to best understand BLAKE and cryptographic hashing in general, so as to best implement and use them;
- applied cryptography researchers and students who need a consolidated reference on BLAKE, and a detailed documentation of the design process.

First of all, we wanted the book to be practice oriented, rather than an elitist academic treatise. This book is therefore much less about proving theorems and describing grand theories than about engineering and craftsmanship. We wanted to provide our readers with:

- An understanding of how BLAKE was designed (what security properties we aimed to achieve, what performance and functional requirements were addressed and how these were established, how components were selected and parametrized, etc.), so that one can critically think about the errors we made and about what was right. In the same spirit, the chapter on BLAKE2 discusses how the modifications from BLAKE were motivated by concrete use cases and applications.
- Guidelines to implement and use BLAKE (as well as BLAKE2), with a focus on software implementation, and an extensive set of test values. Especially with BLAKE2, we provide detailed specifications of modes such as how keyed hashing (that is, message authentication codes and pseudorandom functions) should be implemented, as well as how signaling of parameters should be encoded. This minimizes the responsibility of developers and aims to eventually improve interoperability.

The book includes ten chapters and three appendices, summarized below:

- **Chapter 1: Introduction** sets the stage with a short introduction to cryptographic hashing, the SHA3 competition, and BLAKE. This chapter also introduces notations and endianness conventions.
- **Chapter 2: Preliminaries** reviews applications of cryptographic hashing, and then describes some basic notions: security definitions, constructions, etc. A more technical section describes state-of-the-art collision search methods. SHA1, SHA2, and the SHA3 finalists are briefly presented.
- **Chapter 3: Specification of BLAKE** gives a complete description of the four instances BLAKE-256, BLAKE-512, BLAKE-224, and BLAKE-384.
- **Chapter 4: Using BLAKE** describes several applications of BLAKE instances: simple hashing with or without a salt, Hash-based MAC (HMAC) and Password-Based Key Derivation 2 (PBKDF2) constructions, along with test values.
- **Chapter 5: BLAKE in Software** reviews implementation techniques from portable C and Python to AVR assembly and vectorized code using single instruction, multiple data (SIMD) CPU instructions. We explain how extended instruction sets in Intel, AMD, or ARM chips can be leveraged to implement BLAKE.

- **Chapter 6: BLAKE in Hardware** describes BLAKE’s properties with respect to hardware design for implementation in application-specific integrated circuits (ASIC) or field-programmable gate array devices (FPGA).
- **Chapter 7: Design Rationale** explains in detail why we designed BLAKE the way we did, from NIST’s requirements to the choice of internal parameters.
- **Chapter 8: Security of BLAKE** summarizes the known security properties of BLAKE and describes the best attacks (at the time of writing) on reduced or modified variants.
- **Chapter 9: BLAKE2** presents the successor of BLAKE, starting with motivations and describing in detail the changes made to the original design. This chapter also covers the performance and security aspects of BLAKE2.
- **Chapter 10: Conclusion** concludes the book.
- **Appendix A** provides detailed test vectors.
- **Appendix B** provides a reference portable C implementation of BLAKE.
- **Appendix C** lists third-party software implementations of BLAKE and BLAKE2.

Parts of this book appeared in previous publications, and were revised for the book. This includes material from Jean-Philippe’s PhD thesis, the SHA3 submission, the implementation paper with Samuel, and the BLAKE2 documentation.

Documentation, source code, and the latest cryptanalysis and implementation works on BLAKE and BLAKE2 are available on their respective websites, namely <https://131002.net/blake/> and <https://blake2.net>.

Many people contributed, directly or indirectly, publicly or anonymously, to the development and analysis of BLAKE and BLAKE2, be it through cryptanalysis, security proofs, implementations, review of papers or code, or just encouragement. We would like to thank everyone, and especially:

(Again) the aforementioned co-designers of BLAKE2;

Gaëtan Leurent and Samuel Neves for their work on fast vectorized implementations of BLAKE, which are now the fastest available.

Daniel J. Bernstein for permitting us to reuse ChaCha back in 2007, for running eBASH, and for numerous insights.

Dmitry Chestnykh for his implementations of BLAKE and BLAKE2 in Dart, Go, JavaScript, and Python.

The SHA3 finalists’ teams (Grøstl, JH, Keccak, and Skein) for their fairplay.

The SHA3 team at NIST for their diligent organization of the competition, and especially Bill Burr for his guidance of the process.

Christian Wenzel-Benner, Maïlys Serrano, Pascal Junod, Samuel Neves, Zooko Wilcox-O’Hearn, who proofread all or parts of this book.

Nagravision (Kudelski Group) for supporting BLAKE as well as the preparation of this book.

Vuibroye, Switzerland, June 2014

*Jean-Philippe Aumasson
Willi Meier
Raphael C.-W. Phan
Luca Henzen*

Contents

1	Introduction	1
1.1	Cryptographic Hashing	1
1.2	The SHA3 Competition	2
1.3	BLAKE, in a Nutshell	5
1.4	Conventions	6
2	Preliminaries	9
2.1	Applications	9
2.1.1	Modification Detection	9
2.1.2	Message Authentication	10
2.1.3	Digital Signatures	11
2.1.4	Pseudorandom Functions	12
2.1.5	Entropy Extraction and Key Derivation	13
2.1.6	Password Hashing	13
2.1.7	Data Identification	14
2.1.8	Key Update	14
2.1.9	Proof-of-Work Systems	14
2.1.10	Timestamping	15
2.2	Security Notions	15
2.2.1	Security Models	15
2.2.2	Classical Security Definitions	17
2.2.3	General Security Definition	19
2.3	Black-Box Collision Search	20
2.3.1	Cycles and Tails	20
2.3.2	Cycle Detection	21
2.3.3	Parallel Collision Search	22
2.3.4	Application to Meet-in-the-Middle	22
2.3.5	Quantum Collision Search	23
2.4	Constructing Hash Functions	24
2.4.1	Merkle–Damgård	24
2.4.2	HAIFA	27

2.4.3	Wide-Pipe	27
2.4.4	Sponge Functions	27
2.4.5	Compression Functions	28
2.5	The SHA Family	31
2.5.1	SHA1	31
2.5.2	SHA2	32
2.5.3	SHA3 Finalists	34
3	Specification of BLAKE	37
3.1	BLAKE-256	37
3.1.1	Constant Parameters	37
3.1.2	Compression Function	38
3.1.3	Iteration Mode	40
3.2	BLAKE-512	41
3.2.1	Constant Parameters	41
3.2.2	Compression Function	42
3.2.3	Iteration Mode	42
3.3	BLAKE-224	43
3.4	BLAKE-384	43
3.5	Toy Versions	44
4	Using BLAKE	45
4.1	Simple Hashing	45
4.1.1	Description	45
4.1.2	Hashing a Large File with BLAKE-256	46
4.1.3	Hashing a Bit with BLAKE-512	48
4.1.4	Hashing the Empty String with BLAKE-512	49
4.2	Hashing with a Salt	49
4.2.1	Description	49
4.2.2	Hashing a Bit with BLAKE-512 and a Salt	49
4.3	Message Authentication with HMAC	50
4.3.1	Description	50
4.3.2	Authenticating a File with HMAC-BLAKE-512	50
4.4	Password-Based Key Derivation with PBKDF2	53
4.4.1	Basic Description	53
4.4.2	Generating a Key with PBKDF2-HMAC-BLAKE-224	53
5	BLAKE in Software	55
5.1	Straightforward Implementation	55
5.1.1	Portable C	55
5.1.2	Other Languages	58
5.2	Embedded Systems	60
5.2.1	8-Bit AVR	60
5.2.2	32-Bit ARM	62
5.3	Vectorized Implementation Principle	64

5.4	Vectorized Implementation with SSE Extensions	64
5.4.1	Streaming SIMD Extensions 2 (SSE2)	64
5.4.2	Implementing BLAKE-256 with SSE2	65
5.4.3	Implementing BLAKE-512 with SSE2	66
5.4.4	Implementations with SSSE3 and SSE4.1	70
5.5	Vectorized Implementation with AVX2 Extensions	70
5.5.1	Relevant AVX2 Instructions	71
5.5.2	Implementing BLAKE-512 with AVX2	73
5.5.3	Implementing BLAKE-256 with AVX2	77
5.6	Vectorized Implementation with XOP Extensions	79
5.6.1	Relevant XOP Instructions	80
5.6.2	Implementing BLAKE with XOP	80
5.7	Vectorized Implementation with NEON Extensions	83
5.7.1	Relevant NEON Instructions	83
5.7.2	Implementing BLAKE-256 with NEON	84
5.7.3	Implementing BLAKE-512 with NEON	86
5.8	Performance	88
5.8.1	Speed Summary	89
5.8.2	8-Bit AVR	90
5.8.3	ARM Platforms	91
5.8.4	x86 Platforms (32-bit)	91
5.8.5	amd64 Platforms (64-bit)	92
5.8.6	Other Platforms	93
6	BLAKE in Hardware	97
6.1	RTL Design	97
6.2	ASIC Implementation	98
6.2.1	High-Speed Design	98
6.2.2	Compact Design	100
6.3	FPGA Design	100
6.4	Performance	101
6.4.1	ASIC	102
6.4.2	FPGA	102
6.4.3	Discussion	105
7	Design Rationale	107
7.1	NIST Call for Submissions	107
7.1.1	General Requirements	107
7.1.2	Technical and Security Requirements	109
7.1.3	Could SHA2 Be SHA3?	110
7.2	Needs Analysis	111
7.2.1	Ease of Implementation	112
7.2.2	Performance	113
7.2.3	Security	113
7.2.4	Extra Features	114

7.3	Design Philosophy	114
7.3.1	Minimalism	115
7.3.2	Robustness	119
7.3.3	Versatility	120
7.4	Design Choices	120
7.4.1	General Choices	121
7.4.2	Iteration Mode	122
7.4.3	Core Algorithm	122
7.4.4	Rotation Counts	125
7.4.5	Permutations	126
7.4.6	Number of Rounds	128
7.4.7	Constants	128
8	Security of BLAKE	131
8.1	Differential Cryptanalysis	131
8.1.1	Differences and Differentials	132
8.1.2	Finding Good Differentials	133
8.2	Properties of BLAKE's G Function	133
8.2.1	Basic Properties	134
8.2.2	Differential Properties of G	136
8.3	Properties of the Round Function	141
8.3.1	Bijectivity	141
8.3.2	Diffusion and Low-Weight Differences	142
8.3.3	Invertibility	145
8.3.4	Impossible Differentials	147
8.4	Properties of the Compression Function	151
8.4.1	Finalization	151
8.4.2	Local Collisions	152
8.4.3	Fixed Points	152
8.4.4	Fixed Point Collisions	153
8.4.5	Pseudorandomness	153
8.5	Security Against Generic Attacks	154
8.5.1	Indifferentiability	154
8.5.2	Length Extension	155
8.5.3	Collision Multiplication	155
8.5.4	Multicollisions	156
8.5.5	Second Preimages	157
8.6	Attacks on Reduced BLAKE	158
8.6.1	Preimage Attacks	158
8.6.2	Near-Collision Attack	159
8.6.3	Boomerang Distinguisher	160
8.6.4	Iterative Characteristics	161
8.6.5	Breaking BLOKE	163
8.6.6	Attack on a Variant with Identical Constants	163

9	BLAKE2	165
9.1	Motivations	165
9.2	Differences with BLAKE	166
9.2.1	Fewer Rounds	167
9.2.2	Rotations Optimized for Speed	167
9.2.3	Minimal Padding	168
9.2.4	Finalization Flags	168
9.2.5	Fewer Constants	168
9.2.6	Little-Endianness	169
9.2.7	Counter in Bytes	170
9.2.8	Salt Processing	170
9.2.9	Parameter Block	170
9.3	Keyed Hashing (MAC and PRF)	172
9.4	Tree Hashing	172
9.4.1	Basic Mechanism	173
9.4.2	Message Parsing	174
9.4.3	Special Cases	174
9.4.4	Generic Tree Parameters	175
9.4.5	Updatable Hashing Example	175
9.5	Parallel Hashing: BLAKE2sp and BLAKE2bp	176
9.6	Performance	177
9.6.1	Why BLAKE2 Is Fast in Software	177
9.6.2	64-bit Platforms	178
9.6.3	Low-End Platforms	179
9.6.4	Hardware	180
9.7	Security	180
9.7.1	BLAKE Legacy	180
9.7.2	Implications of BLAKE2 Tweaks	181
9.7.3	Third-Party Cryptanalysis	181
10	Conclusion	185
	References	187
A	Test Vectors	195
A.1	BLAKE-256	195
A.1.1	One-Block Message	195
A.1.2	Two-Block Message	196
A.2	BLAKE-224	198
A.2.1	One-Block Message	198
A.2.2	Two-Block Message	199
A.3	BLAKE-512	201
A.3.1	One-Block Message	201
A.3.2	Two-Block Message	202
A.4	BLAKE-384	205

A.4.1	One-Block Message	205
A.4.2	Two-Block Message	206
B	Reference C Code	209
B.1	blake.h	209
B.2	blake224.c	211
B.3	blake256.c	214
B.4	blake384.c	217
B.5	blake512.c	220
C	Third-Party Software	225
C.1	BLAKE	225
C.2	BLAKE2	226
Index		227

Chapter 1

Introduction

I'm not real happy with saying more than rough principles, and I think that's generally true for more broadly than just this question.

—Bill Burr, First SHA3 Candidate Conference

I am the best cryptographer in the world.

—Stephen Colbert

This introductory chapter presents cryptographic hash functions and their most common applications. It then describes the context of this book, namely NIST's SHA3 competition, and presents a short review of BLAKE's performance and unique properties.

1.1 Cryptographic Hashing

A cryptographic hash function maps a bit string of arbitrary length to a bit string of short, fixed length, typically between 128 and 512 bits. It can thus be viewed as the opposite of a pseudorandom generator, which expands a short, fixed-length string to an arbitrarily long one. Like a cryptographic pseudorandom generator, a cryptographic hash function should achieve various security properties, as discussed in Chapter 2. We shall henceforth simply write *hash function* or just *hash* to refer to a cryptographic hash function,¹ and we shall call its output a *digest* or *hash value*.

Often called a cryptographer's Swiss Army knife, a hash function can underlie many different cryptographic schemes: aside from producing a document's digest to be digitally signed—one of the most common applications—a hash function can serve to construct message authentication codes (MACs), key derivation functions, and even stream ciphers or pseudorandom generators. The nature and volume of the data processed by a hash function vary widely with the application, ranging from four-digit personal identification numbers (PINs) to terabyte disk images. Let us give example applications:

- Code-signing systems such as secure boots (in game consoles, set-top boxes, etc.) or application authentication in smartphones use hash functions to authenticate executed code and prevent execution of third-party malicious code.

¹ A cryptographic hash function is not to be confused with a hash function as used in hash table data structures, as these two types need to satisfy different sets of properties.

- Computer forensics engineers hash and timestamp digital evidence (such as hard drive disks) before further examination as nonmodification proof. They also use hash functions to efficiently and automatically search for illegal content based on its fingerprint.
- Systems generally do not store passwords in the clear but rather their hash value, in order to avoid direct exposure in case of compromise of the database. This practice also ensures that all stored entries have the same length, regardless of the length of the original passwords. Although hash functions should not be used directly, they lie at the basis of password hashing schemes (such as PBKDF2).

Although originally intended to protect only integrity and authenticity, hash functions indirectly contribute to ensuring data confidentiality and availability. Indeed, hash functions are components of encryption schemes [e.g., in RSA with optimal asymmetric encryption padding (OAEP), where SHA1 is commonly used as the hash component and within the mask generation function], and are used for more efficient storage and retrieval of data (e.g., in cloud storage services for proofs of storage, or in key-value stores).

Because of their ubiquity in information systems, it is vital that secure and usable hash functions be made available to industry, government institutions, and individual developers. Until 2004, the field of hash function research was believed stable and the popular MD5 and SHA1 were widely trusted as secure.² This view was revisited with the discovery of collision attacks on MD4, MD5, SHA0, and SHA1 in 2004 and 2005, following breakthrough results by the Chinese researcher Xiaoyun Wang [173,174] and her colleagues. Subsequent years saw the improvement of these attacks, and their extension to reduced versions of functions from the SHA2 family, the latest federal hash standard designed by the National Security Agency (NSA), like its predecessor SHA1.

1.2 The SHA3 Competition

In view of the substantial cryptanalytic progress, the US National Institute of Standards and Technology (NIST)—the same body that standardized the Advanced Encryption Standard (AES) through a public competition in 2000—issued a call in 2007 for a public cryptography competition, codenamed SHA3. NIST planned that the future SHA3 would *augment* (rather than replace) SHA2. NIST also stated that SHA3 should achieve the same security level as SHA2 but with better efficiency, and it was desired that SHA3 have extra features that make it more resilient to future attacks than SHA2.

By the deadline of October 31, 2008, NIST had received 64 submissions. From these, NIST selected 51 in December 2008 to advance to round 1, purely based on minimal acceptance criteria of being complete and proper submissions as per the competition call. Submitters included researchers from both academic and industrial

² “SHA” stands for “secure hash algorithm”; the “MD” prefix means “message digest”.

institutions, such as BT, EADS, École Normale Supérieure, ETH Zürich, Gemalto, Hitachi, IBM, INRIA, Intel, Katholieke Universiteit Leuven, Microsoft, MIT, Orange Labs, Qualcomm, Sagem Sécurité, Sony, STMicroelectronics, the Technion, and the Weizmann Institute.

Candidate submitters were invited to present their algorithms during the First SHA3 Candidate Conference in February 2009 in Leuven, Belgium. In July 2009, NIST announced the 14 candidates to proceed to round 2:

- BLAKE (Switzerland–UK)
- Blue Midnight Wish (Norway)
- CubeHash (USA)
- ECHO (France)
- Fugue (USA)
- Grøstl (Austria–Denmark)
- Hamsi (Belgium)
- JH (Singapore)
- Keccak (Belgium–Italy)
- Luffa (Belgium–Japan)
- Shabal (France)
- SHAvite-3 (Israel)
- SIMD (France)
- Skein (Germany–USA)

The selection of 14 semifinalists was largely according to the evaluation criteria described in the call for proposals, ranked in the order of *security*, *performance*, and *design characteristics*. The security evaluation included analysis of the candidate's resistance against basic attacks such as collision, preimage, second preimage, and length extension, as well as security when the hash function is used as a building block of cryptographic schemes such as MACs or pseudorandom functions. The performance evaluation included metrics in both software (8- to 64-bit systems) and hardware (FPGA and ASIC) platforms. Evaluation of the design characteristics included the flexibility factor, i.e., whether it is parametrizable, versatile across various platforms, and parallelizable, as well as the simplicity factor. That aside, it was also reported that a few round 2 candidates were included due to uniqueness and elegance of design, as NIST wanted to maintain design diversity.

The Second SHA3 Candidate Conference was held in August 2010 in Santa Barbara, CA, USA, where third-party cryptanalysis and implementation results were presented on the 14 round 2 candidates, while the candidate submitters were invited to present brief progress updates on their hash functions.

In December 2010, five finalist candidates were announced to proceed into the final round, namely BLAKE, Grøstl, JH, Keccak, and Skein. NIST justified its choices in a status report published online [135]. NIST emphasized that security was the greatest concern, noting that while none was broken they preferred to be conservative with security while keeping performance in mind. The reasons for some candidates not being selected included:

- the apparent fragility of some algorithms against future attacks;

- large area requirements for hardware implementation;
- substantial tweaks made up to round 2, thus the design deemed not fully mature;
- lack of public cryptanalysis.

The finalists that made it through were generally felt to have an iterated structure readily adjustable to trade security for performance. Similar to the selection for round 2, diversity of designs was also considered during the selection, and the finalists represent different design approaches: AES-based, addition-rotation-XOR (ARX) constructions, the HAsH Iterative FrAmework (HAIFA), and sponge.

The Third SHA3 Candidate Conference was held in March 2012 in Washington, DC with the program focused on third-party cryptanalysis, hardware and software implementations of the finalists, as well as final presentations by the designers.

In October 2012, NIST announced Keccak as the SHA3 winner, citing among other reasons,

the elegant design, large security margin, good general performance, excellent efficiency in hardware, and its flexibility.

In its final report [138], NIST also stated that all five finalists had acceptable performance and security properties, and that any of the five would have made an acceptable choice for SHA3. On the security of the finalists, the report commented as follows:

No finalist has a published attack that, in any real sense, threatens its practical security, (...) Skein has a somewhat larger security margin than Grøstl and JH, and BLAKE and Keccak have large security margins. None of the candidates has an absolutely unacceptable security margin, (...)

The cryptanalysis performed on BLAKE, Grøstl, and Skein appears to have a great deal of depth, while the cryptanalysis on Keccak has somewhat less depth, and that on JH has relatively little depth.

In terms of performance, the report noted that ARX finalists BLAKE and Skein perform well in software, while Keccak is by far the most efficient in hardware, in terms of throughput per area.

Table 1.1 summarizes the timeline of the SHA3 competition. At the time of writing, the Federal Information Processing Standards (FIPS) document officially standardizing SHA3 has yet to be published, though a draft has been released [139].

Table 1.1 Summary timeline of the SHA3 competition.

2007.11.02	Call for submissions, published in the US Federal Register
2008.10.31	Submission deadline (NIST received 64 submissions)
2008.12.10	51 first-round candidates announced
2009.02.25–28	First SHA3 Candidate Conference (Leuven, Belgium)
2009.07.24	14 second-round candidates announced
2010.08.23–24	Second SHA3 Candidate Conference (Santa Barbara, CA, USA)
2010.12.09	5 finalists announced
2012.03.22–23	Third SHA3 Candidate Conference (Washington, DC, USA)
2012.10.02	Keccak announced as the SHA3 winner

1.3 BLAKE, in a Nutshell

BLAKE is a family of four hash functions: BLAKE-224, BLAKE-256, BLAKE-384, and BLAKE-512, whose detailed characteristics appear in Table 1.2. As with SHA2, BLAKE comes with a 32-bit version (BLAKE-256) and a 64-bit version (BLAKE-512), from which the other instances (BLAKE-224 and BLAKE-384) are derived through modified parameters.

Table 1.2 Bit lengths of the parameters of the BLAKE hash functions.

Hash function	Word	Input	Block	Digest	Salt
BLAKE-224	32	$<2^{64}$	512	224	128
BLAKE-256	32	$<2^{64}$	512	256	128
BLAKE-384	64	$<2^{128}$	1,024	512	256
BLAKE-512	64	$<2^{128}$	1,024	384	256

BLAKE is a *user-oriented* design. First of all, we aimed at creating an algorithm that is easy to understand and to implement, even by those nonexperts in cryptography, in order to minimize developers’ coding and debugging effort—the more lines of code, the more bugs, thus the more time and energy needed for a production-ready program. Second, BLAKE is a versatile design that performs well on all platforms, from embedded software systems to resource-constrained hardware. BLAKE is one of the simplest designs submitted to the SHA3 competition, and it can be implemented with low resources (be it hardware gates, lines of code, RAM, ROM, etc.) on most platforms.

At the time of writing, the performance figures for BLAKE include:

- On an Intel Xeon E3-1275 V3 processor (64-bit, 3,500 MHz, Haswell microarchitecture): BLAKE-256 can hash at 6.75 cycles/byte (494 MiBps), and BLAKE-512 can hash at 5.18 cycles/byte (644 MiBps).
- On an Intel Core i7-2600K processor (64-bit, 3,400 MHz, Sandy Bridge microarchitecture): BLAKE-256 can hash at 7.49 cycles/byte (433 MiBps), and BLAKE-512 can hash at 5.77 cycles/byte (562 MiBps).
- On an Atmel ATmega1284p microcontroller (8-bit, 16 MHz, AVR architecture): BLAKE-256 can be implemented with 3,434 bytes of flash memory and 267 bytes of RAM, and can hash at 1,241 cycles/byte (12.59 KiBps).
- On an Intel XScale IXP420 chip (32-bit, 533 MHz, ARMv5TE architecture; a network processor found in routers, NAS servers, etc.): BLAKE-256 can hash at 70 cycles/byte (7.26 MiBps) using 13,160 bytes of ROM and 2,028 bytes of RAM; BLAKE-512 can hash at 167 cycles/byte (3.04 MiBps) with 10,392 bytes of ROM and 1,140 of bytes RAM. Trading off speed and memory, BLAKE-512’s memory consumption can be reduced to 3,716 bytes of ROM or 360 bytes of RAM; BLAKE-512’s memory consumption can be reduced to 7368 bytes of ROM or 947 bytes of RAM.

- On an Intel Atom D510 (32-bit, 1,667 MHz, Bonnell microarchitecture; found in netbooks and mobile devices): BLAKE-256 can hash at 16 cycles/byte (99.36 MiBps), and BLAKE-512 can hash at 18 cycles/byte (88.32 MiBps).
- On 180 nm ASIC: BLAKE-256 with 13.5 kGE (kilogate equivalents) achieves a throughput of 189 Mbps.
- On 90 nm ASIC: BLAKE-256 with 38 kGE achieves a throughput of 11 Gbps, and BLAKE-512 with 79 kGE has throughput of 16 Gbps.

Our utmost goal, however, was security and confidence therein. This is why we built BLAKE on reliable, previously analyzed, components: the HAIFA iteration mode of Biham and Dunkelman [35], the ChaCha function of Bernstein [23], and the *local wide-pipe* of our previous design LAKE [17]. BLAKE thus already looked familiar to cryptanalysts and implementers, which saved precious time and effort during the evaluation process.

As per the HAIFA mode, BLAKE’s compression function is parametrized by a salt and by the number of bits hashed so far, so as to simulate distinct compression functions for each data block processed. Like LAKE, the compression function of BLAKE initializes an internal state with the initial value, the salt, and the counter, and this state is transformed injectively by using an enhanced version of the stream cipher ChaCha’s core function. For better visualization and understanding of the process, the internal state of BLAKE is represented as a 4×4 array of words, similarly to ChaCha or to AES.

1.4 Conventions

Throughout the book, a *word* is either a 32-bit or a 64-bit string, depending on the context: words are 32-bit for BLAKE-224 and BLAKE-256, and are 64-bit for BLAKE-384 and BLAKE-512. Within each word, the most significant bit is stored in the first—i.e., leftmost—position (MSB-zero numbering). A similar big-endian convention applies when converting a data stream into word arrays; for example, a string s of 128 bits is decomposed into four 32-bit words as $s = s_0 || s_1 || s_2 || s_3$ such that the first bit of s is the first bit of s_0 and the last bit of s is the last bit of s_3 . Similarly, an array of bytes is converted to an array of words such that lower-address bytes are more significant within a word.

Most symbols used have their standard meaning: “:=” is assignment of a variable, “||” is concatenation of strings, “+” is integer addition modulo 2^{32} or 2^{64} depending on the word size, “ $\oplus$ ” is XOR, and “ $\gg r$ ” is rotation of r bits towards less significant bits. The symbols $\neg$, $\vee$, and $\wedge$, respectively, denote bitwise negation, logical OR, and logical AND. Hexadecimal numbers are written in `typewriter` style (for example, `0x0` represents the integer 240), and function names are written in sans-serif style.

Data size units follow the International System of Units (SI) standards: one kilobyte (1 kB) is 1,000 bytes, 1,024 bytes is one kibibyte (1 KiB); one kilobit (1 kb) is 1,000 bits, 1,024 bits is one kibibit (1 Kib); likewise for megabytes (MB), mebibytes

(MiB), megabits, etc. Throughput values are denoted accordingly; e.g., one megabit per second is 1 Mbps, one gibibyte per second is 1 GiBps, etc.

Speed of software implementations is reported in terms of efficiency on the platform considered (in cycles per byte) and actual throughput in *bytes* per second at the CPU's nominal frequency, whereas speed of hardware implementations is reported in terms of throughput in *bits* per second at the frequency considered (for example the maximal frequency when speed is the optimization factor).

Chapter 2

Preliminaries

There are $2^{18446744073709551616} - 1$ possible [SHA-256] inputs.

—Thomas Pornin

It seems very hard to make a mathematical definition that captures the idea that human beings can't find collisions in SHA1.

—Mihir Bellare

This chapter introduces the reader to cryptographic hash functions, starting with an informal review of the most common applications, from modification detection and digital signature to key update and timestamping. We then present slightly more formally the security notions associated with hash functions, discussing in particular what being “one-way” means (which is less simple than it sounds). Getting more technical, we review state-of-the-art generic collision search methods, and constructions of hash functions. Finally, we conclude with an overview of the SHA1 and SHA2 standards, as well as of the SHA3 finalists.

2.1 Applications

As previously commented, hash functions are a cryptographer’s Swiss Army knife, for they can serve a large variety of purposes in security systems, acting as one-way functions, collision-resistant functions, or pseudorandom functions, among others.

We review the most common applications of hash functions, starting with the best known, namely modification detection, message authentication, and digital signatures.

2.1.1 Modification Detection

Modification detection is probably the oldest and most straightforward application of cryptographic hash functions, as it only aims to protect integrity of data rather than authenticity—no secret key is involved; one just computes $H(M)$ and compares it with the received value for verification. This is used in a model where a communication channel is secure but unreliable (accidental transmission errors may occur).

In this context, hash values are often called *checksums* when used for modification detection. Checksums are, for example, added at the end of a transmitted packet so that the recipient can check that the received hash value matches the value computed from the received data. Simple, insecure, checksum algorithms such as cyclic redundancy checks (CRCs) are widely used for detection of accidental errors, due to their simplicity and efficiency (as found in trailers of Ethernet frames). However, secure modification codes should protect not only against accidental modification but also against malicious ones. In particular, they should be second-preimage-resistant hash functions, to avoid forgery of data matching the published checksum.

An example of the use of hash functions as checksums is by websites proposing the download of software packages: the website publishes a URL to a software package along with the hash of the target file so that users can verify that they downloaded the legitimate data. This protects against accidental errors as well as straightforward malicious man-in-the-middle modifications of the downloaded content, but not against more clever attackers who would adapt the checksum to the modified file. Also, mere hashing with no secret key does not authenticate the origin of the file (unless, indirectly, if within an HTTPS tunnel).

Checksums are typically used in peer-to-peer file-sharing systems. For example, BitTorrent protects the integrity of data transferred by hashing individually each piece (between 32 kB and 4 MB) of a file with SHA1, and recording it in the torrent file. As previously mentioned, computer forensics uses hash functions to provide proofs of nonmodification of collected evidence. Plenty of other applications use hash functions for ensuring data integrity: intrusion detection systems (e.g., Artillery, Samhain), version control systems (e.g., Git, Perforce), integrity-checking filesystems (e.g., ZFS), cloud storage systems (e.g., OpenStack Swift), distributed filesystems (e.g., Tahoe-LAFS), etc.

2.1.2 Message Authentication

Message authentication codes (MACs) are essentially keyed versions of modification detection codes, and thus provide authenticity in addition to integrity protection—i.e., they ensure that the hash obtained was computed by a party sharing the secret key. The values $H_K(M)$ produced by MACs are sometimes called *authenticators* or just *MACs*. Not all MACs are constructed as keyed hash functions, however: UMAC [44, 110] and Poly1305-AES [25] are examples of MACs that instead rely on a universal hash function combined with a pseudorandom function or an encryption function.

The main security requirement of a MAC is resistance to *forgery*, i.e., the inability for an adversary to determine a legitimate authenticator for some message. As with the definition of preimage resistance, different notions of forgery exist depending on the distribution of the message. From the weakest to the strongest notions:

- **Existential forgery** puts no restriction on the message; it can, for example, be a meaningless, random-looking value;

- **Selective forgery** obliges the attacker to choose the message prior to the attack, for example, as being in a “weak” subset of messages;
- **Universal forgery** is the ability to create a valid signature for any given message.

In all definitions, the attacker is assumed to be able to request a valid MAC of any message of its choice. A forgery is thus successful if the values returned have not been obtained with such a query. It is known that a keyed hash function is a secure MAC if it is pseudorandom.

The most common hash-function-based construction of MACs is the NIST standard HMAC, which for a hash function H , a key K , and a message M returns

$$\text{HMAC-}H(K, M) = H((K \oplus \text{opad}) \| H((K \oplus \text{ipad}) \| M)) ,$$

where K is padded with zero bytes 00 to fill a data block, $\text{opad} = 5c5c \dots 5c$, and $\text{ipad} = 3636 \dots 36$. We refer to [109, 132] for a complete specification detailing particular cases—like how to handle keys longer than a block.

In practice, MACs are often sent jointly with the ciphertext of some message, the authenticator being computed on the plaintext or on the ciphertext; for example, IPsec’s encapsulated security payloads are protected with HMAC-SHA1 (i.e., a MAC is computed on the encrypted data). A disadvantage of HMAC, however, is its suboptimal efficiency on short data: at least two blocks of data have to be processed, and three if the outer hash cannot be precomputed.

2.1.3 Digital Signatures

Digital signature schemes are *asymmetric* (i.e., public-key) cryptographic schemes composed of three algorithms:

- A **key generation** algorithm Gen , which given a security level as parameter creates a key pair (sk, pk) , where sk is the (secret) signing key and pk is the (public) verification key
- A **signing algorithm** Sign that takes a signing key and data M and that returns a signature $s := \text{Sign}_{sk}(M)$
- A **verification algorithm** Verif that takes a verification key, a signature s , and data M and that returns $\text{Verif}_{pk}(s, M)$ either valid or invalid

Common signature schemes rely on hard mathematical problems (RSA, discrete logarithm) and involve big-number arithmetic operations, which makes them orders of magnitude slower than typical block ciphers or hash functions. Moreover, signature schemes generally accept inputs of limited and small length, such as fewer than 2,048 bits. For these reasons, the signature of a message is computed by applying the signing algorithm to the messages digest rather than to the message itself (the so-called hash-then-sign paradigm). Typical signature schemes are RSA as per the Public-Key Cryptographic Standard #1 (PKCS#1) [89] and the Digital Signature Standard (DSS) [133].

The main security requirement for a signature scheme is that *forgery* of valid signature-and-message pairs should be infeasible except for the signer, even when many such pairs are known. It is easy to see that, if the hash function used is not second-preimage resistant, one can forge a valid signature-and-message pair by “re-cycling” a known pair. In fact, collision resistance is necessary if the attacker can choose messages to be signed by the legitimate party. Collision resistance is also necessary to ensure nonrepudiation of signatures by the signer.

Many different types of signatures have been proposed, with various functionalities and security requirements, for example:

- **Undeniable signatures** are verified through interactions between the signer and the prover rather than with a single algorithm on the verifier side, and allow the signer to prove the invalidity of a signature. These two features allow the signer to choose who can verify a given signature.
- **Group signatures** allow a member of a group to anonymously sign data on behalf of the group.
- **Randomized signatures** are like normal signatures but with randomized hashing rather than deterministic hashing. This allows one to drop the requirement of collision resistance for a weaker form called target collision resistance.

One of the most common uses of digital signatures is in HTTPS-secured websites, which can prove their identity by sending a *certificate* signed by a certification authority (CA), verified on the client side using the public key of the CA embedded in one’s browser. Signatures are also used to prevent the execution of arbitrary code on smartphones and game consoles via the implementation of a chain of trust, although these protection mechanisms are regularly broken due to flaws in design and/or implementation.

2.1.4 Pseudorandom Functions

Pseudorandom functions (PRFs) are objects that satisfy *computational indistinguishability* from uniform outputs. In practice hash-based PRFs (as defined in Definition 5) are seldom used isolatedly, but rather as part of a protocol whose security relies on the indistinguishability property of the PRF.

A common way to construct a hash-based PRF is to use the HMAC construction (see Section 2.1.2); for example, the Internet Key Exchange (IKEv2) protocol of the IPsec suite uses HMAC-SHA-256 as a PRF to set up a shared secret key [97, 98].

PRFs are found in many widely used protocols. For example, the transport layer security (TLS) handshake makes use of two dedicated PRF constructions (analyzed in detail in [69]). The Kerberos V authentication protocol also needs PRFs to ensure provable security. Finally, PRFs are at the core of certain password-based key derivation and hashing schemes, as discussed in the next section. Fundamentally, MACs and PRFs are identical objects.

2.1.5 Entropy Extraction and Key Derivation

The ability of hash functions to eliminate structures and symmetries of related inputs to produce random-looking outputs is leveraged for the following applications:

- **Entropy extraction**, that is, exploiting the possibly nonuniform¹ randomness of some entropy pool to produce uniformly distributed strings, thus maximizing the per-bit entropy. For a formal definition of entropy extractors and theoretical results, we refer the reader to the work of Dodis et al. [62]
- **Key derivation**, that is, the generation of cryptographic keys from secret and public parameters; for example, from a serial number, timestamp, and secret global key. A common application is password-based key derivation, for which the notion of key stretching [101] was introduced to mitigate brute-force attacks and simulate extra entropy by enforcing additional computation per password.

The PKCS standard PBKDF2 is a common password-based key derivation function [94, 95]. It uses a pseudorandom function to produce a key from a salt and a password, using a variable number of iterations of the PRF—the more iterations, the slower the brute-force. The standard recommends at least 1,000 iterations. The use of PBKDF2 with only one iteration in a previous version of the Blackberry software was shown to be a major security flaw² For comparison, Apple’s mobile OS iOS3 used 2,000 iterations, and its subsequent versions 10,000 iterations.

2.1.6 Password Hashing

Password databases are regularly “dumped” from servers by malicious intruders. To mitigate the risk to users, a good practice is to store hashes of the passwords rather than the passwords themselves. However, a simple $H(\text{password})$ exposes users’ passwords to efficient time-memory tradeoffs—commonly known as rainbow tables, the main technique employed. Slightly better is to hash with a salt, but this still leaves passwords exposed to brute-force and dictionary attacks with tools such as John the Ripper or Hashcat, especially when ran on graphical processing units (GPUs).

A much better practice is to use a dedicated *password hashing function*, that is sufficiently slow (among other properties) to mitigate brute-force and dictionary attacks. Such functions include the PBKDF2 construction (originally intended for key derivation) and the dedicated designs bcrypt [150] and scrypt [143].

These last years, several major organizations fell victim of “passwords thefts”, actually the extraction of databases of weak hashes (sometimes as simple as salt-less MD5) following a compromise of one of their servers. Hundreds of thousands

¹ A source of bits from a physical phenomenon may appear random, yet with respect to a distribution that is not the ideal, uniform, one. There may, for example, be more ones than zeroes, or the pattern “1010” may occur more often than the pattern “0101,” etc.

² Cf. vulnerability CVE-2010-3741.

of users suffered the publication of their password, which often is reused through email accounts, social network services, etc.

To address the lack of research and solutions for secure password hashing—and password protection more generally—the Password Hashing Competition³ was initiated in 2013, on the same principle as the SHA3 competition: the community is invited to submit password hashing schemes, which will be evaluated by a panel of experts. The submission deadline was set to March 31, 2014, and the selection of one or more designs is expected in Q2 2015.

2.1.7 Data Identification

The (practical) uniqueness of a file’s fingerprint through a hash function is exploited in forensics investigations to facilitate identification of files within a large file system. The short length of fingerprints allows compact storage of large databases of target files. Target files are, for example, illegal content, specific system files, or files known to be dangerous. Refined techniques, such as piecewise hashing, are used to deal with partially modified files.

Hash functions have for example been used by antimalware tools to assist in identification of malicious payloads.

2.1.8 Key Update

Two or more parties that share a secret key K can agree to update as $K := H(K)$ at predefined times, so that the compromise of a K does not compromise earlier K ’s, thanks to the preimage resistance of H . This property is called *forward security*, and is also known as *backtracking resistance* in the context of PRNGs.

If the update of the key depends on the data exchanged—for instance, if K is updated as $K := H(K, M)$, where M is an aggregate of all the data exchanged since the last update—then the property of *backward security* (also known as *prediction resistance*) can be achieved; that is, the compromise of a K does not compromise future K ’s if the attack does not observe all the traffic.

2.1.9 Proof-of-Work Systems

Proof-of-work systems aim to force a party to perform some resource-consuming operation prior to performing some task, such as sending email, accessing to a ser-

³ <https://password-hashing.net>

vice, etc. These aim to deter massive execution of a task and thus prevent abuse (e.g., spam email) or denial of service.

The one-wayness and unpredictability of hash functions is exploited by proof-of-work systems such as the famous Hashcash,⁴ originally designed as an antispam measure. Given a header containing metadata (such as email address, timestamp, etc.), Hashcash clients seek a nonce that, when combined with the header, hashes to a value with a given number of leading 0 bits. Initially, Hashcash used SHA-1 and searches for a hash value with 20 leading zeroes. The famous cryptocurrency Bitcoin relies on Hashcash with a “double” SHA-256 instead of SHA1, and an adapted number of leading zeroes, varying over time: initially set to 32 in 2009, it is 63 at the time of writing, representing about $2^{63}/10$ double hashes per minute. Litecoin uses scrypt [143] rather than SHA-256, a password hashing scheme designed to use significant memory and thus mitigate the efficiency of GPUs, FPGAs, and ASICs.

2.1.10 Timestamping

One can use hash functions to commit to data while keeping it hidden, with the ability to later reveal the data as evidence of earlier knowledge. The said data can, for example, be a scientific result, a document establishing intellectual property, financial forecasts, informants’ names, etc. A commercial trusted timestamping service can be used to guarantee the exact time of publication, although one may choose to just publish it on (say) Twitter.

The properties exploited are (a strong form of) collision resistance and preimage resistance of the hash function. It was shown that MD5 cannot guarantee secure commitments, when researchers “predicted” the outcome of the 2008 US presidential election [166] by revealing the MD5 digest of the president’s name a year before the vote.

2.2 Security Notions

2.2.1 Security Models

Informally, a hash function is *preimage resistant* if it is hard to find data mapping to a given hash value. Although this definition sounds self-explanatory, one notices that “hard” needs to be precisely defined. There are essentially three definitions of “hard,” whose acceptance depends on the community considered:

- *Theoretical cryptographers* and complexity theorists often define a problem as hard if it can be proven that solving it requires a number of operations growing

⁴ <http://hashcash.org>.

faster than any polynomial function of the “size” of the problem. Although this definition captures well the intuitive notion of hardness for scalable classes of problems, it is not relevant for hash functions with fixed parameters, as used in practice.

- *Applied cryptographers* seldom use the term “hard” in security definitions. They rather consider a hash function preimage resistant if there is no method substantially faster than brute force to compute a preimage of some hash value. The exact definition of “substantially” is disputed, as well as that of “some value”—this point is discussed further in this section.
- *Security practitioners* tend to have a more pragmatic view, and understand “hard” as “infeasible in practice.” That is, they tend to be satisfied with a theoretically suboptimal security level, as long as actual applications are not threatened and that the security level remains of an adequate order of magnitude; for example, a collision attack with time complexity 2^{120} and memory 2^{64} instead of only time 2^{128} is not really a concern for security. More pragmatically, and from a risk analysis standpoint, cryptography remains strong enough as long as the cost of breaking it overwhelms that of breaking other components of the system, which are generally much more fragile: software correctness, user behavior.

It follows that different communities have different definitions of a “break” and of an “attack.” This can cause misunderstandings, for example, when information is relayed in general media: In 2009 cryptographers showed how to recover a key [41] of the AES-192 block cipher within approximately 2^{176} evaluations of the cipher, instead of 2^{192} ideally. This attack does break AES-192 according to the definition of applied cryptographers, although it is clearly infeasible.⁵ Nonetheless, several news sites published headlines such as “New AES Attack,” which caused some users to believe that their AES-192 keys were at risk. Such results, reducing the theoretical complexity but leaving it impractically high, are sometimes called *certificational attacks*.

This book uses the definition of applied cryptographers, which is also that of NIST in its call for SHA3 submissions, namely, that SHA3 should achieve n -bit security against preimage attacks to be considered unbroken. In other words, any method substantially faster than the generic 2^n -time search is viewed as an attack. More generally, a function is considered to be broken when a method does “something” (such as finding multicollisions, input/output linear relations, etc.) more efficiently than the best generic attack, regardless of whether that “something” can be exploited to compromise a system’s assets. The reasoning is that, if it can resist a nuclear bomb today, it can certainly resist a Molotov cocktail, or any explosive devices that may be crafted by an imaginative attacker in the next 20 years.

Recall the above informal definition of preimage resistance: “if it is hard (...) a given hash value.” Observe that this raises (at least) two other issues:

- How can one be sure that the problem is indeed hard (whatever “hard” means)? and

⁵ To give an order of magnitude, there are approximately $\approx 2^{166}$ atoms on Earth, and fewer than 2^{59} seconds have passed since the Big Bang.

- How is the hash value given to the attacker?

While the first question relates to deep questions in complexity theory, the second one will find an answer in the next section.

2.2.2 Classical Security Definitions

We now formally define the properties that a cryptographic hash should achieve to be called secure. We distinguish between unkeyed and keyed hash functions. The latter, denoted H_K , are parametrized by a secret key K held only by legitimate users; attackers who know H but not K cannot compute H_K . This is a simple way to simulate a *secret algorithm*, since hiding a 128-bit key is easier than hiding a program or algorithm, and generating a large number of keys is easier than generating a large number of algorithms with similar security properties.

Ideally, knowledge of H should not help an attacker who does not know K , compared with one who also does not know H . In both the unkeyed and keyed settings, a hash function is assumed to accept data of arbitrary length (up to some bound) as input, and to produce n -bit hash values; for instance, SHA1 formally accepts data of length up to $2^{64} - 1$ bits (that is, almost 16,384 pebibytes) and produces 160-bit hash values.

2.2.2.1 Unkeyed Hash Functions

A rigorous definition of preimage resistance can be of mainly two kinds: range based, and codomain based, as described below.⁶

Definition 1 (Preimage resistance, range based). A hash function H is *preimage resistant* if, given a random n -bit string h , finding M such that $H(M) = h$ requires approximately 2^n evaluations of H .

Definition 2 (Preimage resistance, codomain based). A hash function H is *preimage resistant* if, given $H(M)$ for some unknown random input M , finding M' such that $H(M') = H(M)$ requires approximately 2^n evaluations of H .

The 2^n bound follows from the fact that finding preimages for an ideal hash function cannot be done with fewer than about 2^n evaluations of the function; that is, bruteforce⁷ is optimal.

⁶ A random function $f : \{0, 1\}^n \rightarrow \{0, 1\}^n$ has $\{0, 1\}^n$ as codomain—the set of what may possibly come out of f —but a range that is a strict subset of $\{0, 1\}^n$, since a significant number of n -bit strings would not admit preimages; in other words, f would not be *surjective*.

⁷ Note that we talk of “bruteforce” rather than “exhaustive search,” for the latter applies to (say) key recovery for a block cipher, but not to preimage search.

Definitions 1 and 2 only differ in the way the challenge value is chosen. It is easy to see that, if the hash function behaves like a random function, then the distribution of the challenge is the same in the two definitions, making them equivalent. However, both definitions are imperfect, because they can define functions that are obviously weak as preimage resistant; for example, consider the following hash functions H :

- For all inputs, H evaluates to the all-zero string. This function is preimage resistant according to Definition 1, but not according to Definition 2.
- For all n -bit inputs, H evaluates to the all-zero string; for other inputs, H behaves like a random function. This function is preimage resistant according to both definitions, but is clearly insecure.

Fortunately, these examples are pathological cases that are unlikely to be met by actual human-designed hash functions. The above definitions are thus sufficient for practical purposes, and the identification of insecure functions that may happen to satisfy those notions is left to common sense.

We now define second-preimage resistance and collision resistance.

Definition 3 (Second-preimage resistance). A hash function H is *second-preimage resistant* if, given $H(M)$ for some known random data M , finding $M' \neq M$ such that $H(M') = H(M)$ requires approximately 2^n evaluations of H .

Note that, unlike in the codomain-based version of preimage resistance, M' must be distinct from M . Again, finding second preimages has complexity approximately 2^n for an ideal function.

Definition 4 (Collision resistance). A hash function H is *collision resistant* if finding distinct M and M' such that $H(M) = H(M')$ requires approximately $2^{n/2}$ evaluations of H .

The $2^{n/2}$ bound follows from the well-known birthday attack, the key idea being that with $2^{n/2}$ values of $H(M)$, one can construct approximately 2^n candidate pairs for $H(M) = H(M')$. Note that collision search algorithms with complexity about $2^{n/2}$ can be implemented with only negligible memory, without storing all $2^{n/2}$ values (see Section 2.3).

2.2.2.2 Keyed Hash Functions

Keyed hash functions incorporate a secret key K of k bits, and thus can only be computed by parties knowing K . Keyed hash functions are at the basis of message authentication codes and of pseudorandom functions. Definitions of security of keyed hash functions substantially differ from and extend the classical definitions. For the sake of simplicity, we only give informal definitions, which assume that the attacker is able to query H_K as a black box to obtain $H_K(M)$ for the messages of its choice:

Definition 5 (Pseudorandomness). A hash function H is *pseudorandom* if, for a random K , distinguishing H_K from a random function needs approximately 2^k queries to H_K .

Definition 6 (Unpredictability). A hash function H is *unpredictable* if for a random K , finding $H_K(M)$ for some unqueried M needs approximately 2^k queries to H_K .

Those two notions are similar but not identical: pseudorandomness implies unpredictability, but not the other way around.

2.2.3 General Security Definition

A general definition of a secure hash function is *a function that behaves like an ideal hash function*, which is, admittedly, a bit tautological. A less imprecise definition is given by Ferguson, Schneier, and Kohno [67]:

An attack on a hash function is a non-generic method of distinguishing the hash function from an ideal hash function.

In other words, if one can “do something” for a hash function that one cannot do with the same (or lesser) effort for an ideal hash function (or for any other hash function), then this “distinguishes” it from an ideal one. The method employed is called a *distinguisher*; for example, a method to find preimages in 2^{n-4} is an attack, for ideal hash functions only admit preimage attacks in 2^n . More generally, a method more efficient than the best *generic attack*—i.e., one that works for any hash function—is a distinguisher.

There are some caveats, though:

- First, any hash function specified as an algorithm (rather than as an abstract “oracle”) admits a trivial distinguisher because there exists a compact expression—namely, the algorithm—of the output as a function of the input. For an ideal hash function, such an expression is unlikely to exist. Actually, the most compact representation of a random hash function has exponential length; in other words, the program would not even fit in a computer’s memory.
- Second, many distinguishers do not impact the actual security of the hash function, suggesting that the ideal hash function considered is too high an ideal for any practical purpose.
- Third, distinguishers are difficult to rigorously define formally, and there is no standard definition accepted by the community. Nevertheless, a distinguisher is generally an “elephant” test: you recognize it when you see it.

Whatever the goal of an attack, it should be compared with the generic method not only in terms of computational complexity, but also of memory requirements, probability of success, parallelism, and more generally in terms of cost-effectiveness when actually implemented (in the physical world).

The general definition of security has the advantage of capturing all security-critical properties such as preimage resistance, but a hash function that fails to satisfy it is not necessarily insecure. In cryptography theory, however, so-called security proofs of schemes that use hash functions as an underlying primitive generally assume that hash functions do satisfy that general definition. Some thus argue that it is risky to use a nonideal hash function in such a provably secure scheme. For more about this, we refer the reader to the notion of *indifferentiability* and its related literature (e.g. [55, 126, 154]).

2.3 Black-Box Collision Search

Generic collision search methods are one of the most interesting problems related to hash functions, due in part to the elegance of the techniques. Such methods do not depend on the internals of the hash functions, and rather view them as black boxes assumed to behave as random functions. Below we describe state-of-the-art methods applicable to cryptographic hash functions as well as to any function that behaves sufficiently randomly. Such functions include the core functions of public-key schemes based on factoring or discrete logarithms.

The general collision search problem is, given a function F with a finite range, to find distinct inputs x and x' such that $F(x)$ equals $F(x')$. Collision search is an important tool in cryptanalysis, most notably to evaluate the security of discrete logarithm-based schemes, to perform meet-in-the-middle attacks, or to find collisions in components of hash functions.

We refer to Joux's book [91, Chaps. 6–8] for a detailed overview of a comprehensive review of collision search methods.

2.3.1 Cycles and Tails

Let $\mathcal{F}_n$ denote the set of all functions from a domain D of size n to a codomain of size n , with n finite; for example, if D consists of all b -bit strings, then $n = 2^b$. Let F be a random element of $\mathcal{F}_n$ —that is, a random mapping from and to n -element sets. A folklore result is that the range of F is expected to contain $n(1 - 1/e) \approx 0.63n$ distinct elements. Therefore, F is expected to have collisions $F(x) = F(x')$, $x \neq x'$. Efficient methods for finding such collisions exploit the structure of F as a collection of *cycles*.

Consider the infinite sequence $\{x_i = F(x_{i-1})\}_{0 < i}$, for some arbitrary starting value x_0 . Because D is finite, this sequence will eventually begin to cycle, that is, to repeat an identical sequence indefinitely. Hence, there exist two smallest integers $\mu \geq 0$ (the *tail length*) and $\lambda \geq 1$ (the *cycle length*) such that $x_i = x_{i+\lambda}$ for every $i \geq \mu$. Such a structure then yields a collision at the point where the cycle begins: $F(x_{\mu-1}) = F(x_{\mu+\lambda-1}) = x_\mu$.

The *birthday paradox* illustrates well the above structure: in a sequence of random numbers in $\{1, \dots, n\}$, the expected number of draws before a number occurs twice is asymptotically $\sqrt{\pi n/2} \approx 1.25\sqrt{n}$. This is because the expected values of the tail length μ and of the cycle length λ sum to $\sqrt{\pi n/8} + \sqrt{\pi n/8} = \sqrt{\pi n/2}$. This value is sometimes called the *rho length*, because of the rho shape of the sequence, as noticed by Pollard [146].

A trivial collision search algorithm repeats the following: pick random points x and x' , return them as a collision if $F(x)$ equals $F(x')$, otherwise pick another pair of random points. About n trials are required, since x and x' collide with probability $1/n$. A less trivial algorithm exploits the existence of cycles by storing a sequence $\{x_i = F(x_{i-1})\}_{0 \leq i < \sqrt{\pi n/2}}$, sorting it, and looking for a collision. State-of-the-art methods eliminate the large memory requirements and the cost of sorting a large list. In the following we review these methods, starting with explicit cycle-detection methods, then presenting modern techniques optimized for efficiency on parallel computing infrastructure. Finally, we explain how to apply these methods to concrete cryptanalytic problems.

2.3.2 Cycle Detection

The low-memory cycle-detection method of Floyd is at the base of Pollard's rho method for factoring and computing discrete logarithms.⁸ It is based on the following observation [108, 3.1, Ex. 6]:

Theorem 1. *For a periodic sequence $x_0, x_1, x_2, \dots$, there exists a unique $i > 0$ such that $x_i = x_{2i}$ and the smallest such i lies in the range $\lambda \leq i \leq \lambda + \mu$.*

The values λ and μ are the cycle length and tail length, as defined in Section 2.3.1.

Based on Theorem 1, Floyd's method picks a starting value $x_0 = x'_0$ and compares the values $x_i = F(x_{i-1})$ and $F(F(x'_{i-1}))$, $i \geq 1$. The expected number of iterations before reaching a match is $\sqrt{\pi^5 n/288} \approx 1.03\sqrt{n}$ [19].

Floyd's algorithm detects that the sequence has reached a cycle but does not return the values of λ and μ , nor a collision for F . This can be done as follows, once $x_i = x_{2i}$ is found: generate x_j and x_{j+i} , $j \geq 0$, until finding $x_j = x_{j+i}$; at the first equality we have $j = \mu$. If none of the values x_{j+i} equals x_j then $\lambda = \mu$, otherwise λ is the smallest such j . This operation costs on average $2\sqrt{\pi n/2}$ evaluations of F . Finally, detecting the cycle and locating the collision with the above method costs

$$3\sqrt{\pi^5 n/288} + 2\sqrt{\pi n/2} \approx (3.09 + 2.51)\sqrt{n} = 5.60\sqrt{n}$$

⁸ "Floyd's algorithm" was actually first described in [108], and credited to Floyd without citation. Floyd's 1967 paper [70] describes an algorithm for listing cycles in a directed graph, but that differs from the cycle-detection algorithm considered here.

evaluations of F , and requires negligible memory (storage of a few x_i 's). Slightly more efficient variants of Floyd's algorithm were proposed by Brent [19] and Teske [169]. Sedgewick et al. showed how to eliminate the redundant computations by using a small amount of memory [161], but their algorithm is not as general as Floyd's (in particular, it cannot be combined with Pollard's rho factoring method).

2.3.3 Parallel Collision Search

A disadvantage of Floyd's algorithm (and thus of Pollard's rho method) is that it cannot be parallelized efficiently: m processors (e.g., CPU cores) do not provide a $1/m$ reduction of complexity. In other words, speed does not grow linearly with the number of processors. This is because one has to wait for a given invocation of F to end before the next can begin. Efficient parallelization of collision search takes a different approach, by using *distinguished points*. The idea of using distinguished points (i.e., points that have some predefined easily checkable property, such as having ten leading zero bits) was proposed by Quisquater and Delescaille [151] for searching collisions for the Data Encryption Standard (DES), and earlier noted by Rivest [60, p.100] in the context of Hellman's time-memory tradeoff. Below we describe a simple method for efficient parallelization of collision search using distinguished points, due to van Oorshot and Wiener [140].

Let m be the number of processors available, and consider some easily checked property $P \subsetneq D$ that a random point satisfies with probability $\theta < 1$. To perform the search, each processor runs the following procedure:

1. Select a starting value x_0 ;
2. Iterate $x_i = F(x_{i-1})$, $i > 0$, until a distinguished point $x_d \in P$ is reached;
3. Add x_d (along with x_0 and d) to a common list for all processors;
4. Repeat the process (that is, go to 1.).

The algorithm halts when a same distinguished point appears twice in the common list, which means that two distinct sequences $(x_0, \dots, x_i)$ and $(x'_0, \dots, x'_j)$ lead to a same value $x_i = x'_j$ (one should ensure that a same starting value is not used twice). With high probability, one will easily deduce a collision from these two sequences (if the first sequence leads to the starting point of the second, then no collision will be found). Details can be found in [140].

The above algorithm runs in time about $\sqrt{\pi n/2}/m + 2.5/\theta$ to locate a collision, hence parallelism provides a linear speedup of the search.

2.3.4 Application to Meet-in-the-Middle

Parallel collision search using distinguished points can be directly applied to find collisions for hash functions. It can also be adapted to compute discrete logarithms

in cyclic groups. Here we show how it can be used to perform meet-in-the-middle (MitM) attacks.

The problem considered is, given two functions F_1 and F_2 in $\mathcal{F}_n$, to find x and x' (not necessarily distinct) such that $F_1(x)$ equals $F_2(x')$. A solution can be found by defining an easily checked property P , and by considering the function

$$F(x) = \begin{cases} F_1(x) & \text{if } x \in P \\ F_2(x) & \text{otherwise} \end{cases}.$$

Under reasonable assumptions on F_1 and F_2 , and assuming that a random x satisfies P with probability $1/2$, a collision $F(x) = F(x')$ will be useful as soon as x satisfies P but x' does not—meaning that the collision is between F_1 and F_2 . When the cost of computing F_1 and F_2 significantly differs (for example, if one of them represents a shortcut preimage attack on some component), the property P can be adapted to optimize the complexity of the attack, so that F_1 is called more often than F_2 , for example (e.g., if F_2 is slower to evaluate).

Note that the MitM problem considered here, and often encountered in cryptanalysis, differs from what is called MitM in [140]. Indeed, the latter attack looks for a single “golden value,” and its complexity heavily depends on the domain size, whereas in the former the complexity only depends on the range size.

2.3.5 Quantum Collision Search

Although they do not exist (yet), and are sometimes believed to be physically impossible to construct (see for example [117]), quantum computers do represent a potential threat for cryptography. Indeed, efficient quantum algorithms exist for factoring integers and solving discrete logarithms, two problems whose alleged hardness guarantees the security of RSA, DSA, Diffie–Hellman, elliptic-curve cryptography, etc. Solutions in a world with efficient quantum computers are proposed in [27].

Symmetric cryptography is also concerned, to a lesser extent, with quantum attacks: using quantum Fourier transform and Grover’s algorithm [73], a quantum search algorithm can recover an n -bit key in time about $2^{n/2}$, with negligible memory. This would require to double the length of hash values for the same preimage resistance.

Finding a (black-box) collision with a quantum algorithm takes $\Omega(2^{n/3})$ queries [1, 111]. The quantum search algorithm was adapted by Brassard, Høyer, and Tapp [48] to find collisions in time $O(2^{n/3})$, but it requires space $O(2^{n/3})$ of read-only quantum memory (for a detailed cost analysis, see [26]). This makes quantum collision search significantly *less efficient* than classical parallel search, which needs space of only $O(2^{n/6})$ to find collisions in time $O(2^{n/3})$. Therefore, quantum computers are unlikely to be a major threat as far as collisions are concerned.

2.4 Constructing Hash Functions

All general-purpose hash functions split the data to be hashed into blocks of fixed length and process them iteratively using a *compression function*. The compression function takes fixed-length input and produces fixed-length output. The combination of calls to a compression function to process arbitrary-length input is called an *iteration mode*.

We present the classical iteration mode used by MD5, SHA1, and SHA2 (the so-called Merkle–Damgård construction) as well as the state-of-the-art modes used by most recent designs, such as SHA3 candidates. Finally, we review constructions of compression functions based on block ciphers.

2.4.1 Merkle–Damgård

The Merkle–Damgård (shorthand MD) iteration mode proceeds in two steps: first, a preprocessing of the data to hash (the *padding* step), then the actual processing of the data. Below we describe those two steps and present some properties of the MD construction.

Note that the details of the MD construction may slightly vary in the literature. Here we describe how it is used in SHA1 and SHA2, as per the NIST standard [137].

2.4.1.1 Data Padding

The data to hash can be of arbitrary bit length. However, the iteration mode processes blocks of m bits. It is thus necessary to transform the data received into a sequence of m -bit blocks in an invertible way, so as to avoid trivial collision; in other words, the original data should be uniquely defined given the data after padding. We shall henceforth refer to the data before padding as the “original data,” and to the data after padding as the “padded data.”

In SHA1, SHA-224, and SHA-256, the block length m is 512 bits. Padding of ℓ -bit data proceeds as follows:

1. Append a “1” bit to the end of the data;
2. Append $k \geq 0$ “0” bits, where k is the smallest solution to the equation $\ell + 1 + k \equiv 448 \pmod{512}$;
3. Append a 64-bit unsigned big-endian representation of ℓ .

This procedure guarantees that the bit length of the padded data is a multiple of 512.

In SHA-384 and SHA-512 m is 1,024 bits. Padding is similar, except that k should satisfy $\ell + 1 + k \equiv 896 \pmod{1,024}$ and that ℓ is represented on 128 bits.

2.4.1.2 Data Processing

After data padding, a MD hash function processes a sequence of blocks $M_0, \dots, M_{N-1}$ using a compression function compress by doing

$$h := \text{compress}(M_i, h) \quad \text{for } i = 0, \dots, N-1,$$

where the *chaining value* h is initialized to some predefined *initial value* (IV). The hash value returned is the final value of h obtained after processing M_{N-1} .

In SHA1, h is 160-bit and the IV is the four 32-bit words

67452301 efc dab89 98badcfe 10325476 c3d2e1f0 .

2.4.1.3 Security Properties

We summarize the main security properties of the MD mode.

Security Reductions

It can be shown that a collision $H(M) = H(M')$ on a MD hash function always implies a collision $\text{compress}(h, M_i) = \text{compress}(h', M'_i)$ for the underlying compression function. One can thus reduce⁹ the collision resistance of the hash function to that of its compression function. We call *internal collision* any collision for the compression function that occurs before processing the last data block.

A similar reduction exists with respect to preimage resistance; clearly, if one can find preimages of the hash function, then one can also find preimages of the compression function.

If padding of the data length is omitted, the collision resistance reduction no longer holds. To show this, suppose one knows a data block M_0 such that

$$\text{compress}(h, M_0) = h$$

when h is set to the IV. It follows that, for any data M , the strings $M_0 \| M$ and $M_0 \| M_0 \| M$ hash to the same value.

Multicollision Attack

Observe that, in the MD mode, the chaining values are of the same length as the hash value. One can thus search for a collision of chaining values at the same cost as for the hash value, namely $2^{n/2}$ calls to the compression function, approximately.

⁹ In the sense of a complexity reduction.

As discovered by Joux [90], one can find a *multicollision*¹⁰ for a MD hash function with much less effort than ideally, by proceeding as follows:

1. Find blocks M_0^0 and M_0^1 such that $\text{compress}(h, M_0^0) = \text{compress}(h, M_0^1)$, for h set to the IV.
2. Find blocks M_1^0 and M_1^1 such that $\text{compress}(h, M_1^0) = \text{compress}(h, M_1^1)$, for h set to $\text{compress}(h, M_0^0)$.
3. Repeat the procedure for N blocks, to obtain in total N pairs (M_i^0, M_i^1) .

Thus all strings of the form $M_0^\star \| M_1^\star \| \dots \| M_{N-1}^\star$, where $\star$ is a wildcard symbol for either 0 or 1, will hash to the same value. As there are 2^N such strings, we call them a 2^N -multicollision. The computational effort is about $N \cdot 2^{n/2}$ calls to the compression function.

This multicollision attack is only feasible in practice if it is feasible to find a collision in the first place. A collision-resistant function is thus also multicollision resistant.

Fixed Points and Second Preimages

In the “Security Reductions” paragraph above we explained how the data length padding thwarts an attack exploiting one fixed point $h = \text{compress}(h, M)$. What about two fixed points? One may exploit two fixed points in such a way that the two forged inputs have the same length, that is, such that (any) fixed points are iterated the same number of times for both. This is the idea behind Dean’s second preimage attack [59] on MD functions.

Dean’s idea to bypass the length padding was improved by Kelsey and Schneier, who exploited Joux’s multicollision trick to produce “expandable messages” (we refer to [100] for details of the attack). Using this attack, second preimages of 2^k -block messages can be found in time approximately 2^{n-k} , instead of 2^n ideally.

Length Extension

The length extension property allows one to determine the hash value of $M = M_p \| P \| M_s$ given only the hash value $H(M_p)$ and the suffix M_s . Here P stands for the padding data appended to M_p when computing $H(M_p)$. To find $H(M)$, it thus suffices to compute $\tilde{H}(M_s)$, where $\tilde{H}$ is identical to H except that it uses the value $H(M_p)$ as an IV.

One can use length extension in forgery attacks against MACs computed as $H(K \| M)$, where K is the key and M is the data to authenticate. This construction is known as *prefix MAC*, and is thus insecure when combined with a MD hash function.

¹⁰ A k -multicollision is a set of k distinct inputs that hash to the same value.

2.4.2 HAIFA

The Hash Iterative FrAmework (HAIFA) [35] is an enhanced version of the MD iteration mode that aims at improving security and flexibility. The main differences from MD are that:

- The compression function in HAIFA takes as additional inputs a salt and a counter of the number of bits already processed;
- The IV and the padding depend on the length of the hash value, which is of variable size (but not longer than the chaining value).

The counter foils attacks based on fixed points, including the second-preimage attack of Kelsey and Schneier. The built-in salt encourages and simplifies the use of a salt in password storage and digital signatures.

BLAKE uses a simplified version of HAIFA that retains all of HAIFA's desirable properties.

2.4.3 Wide-Pipe

The so-called wide-pipe [122] mode is similar to the MD mode except that the chaining value is larger than the returned hash value. A second compression function must thus be used to produce the final digest from the last chaining value—this function can be as simple as a simple truncation of a subset of the bits.

The wide-pipe mode mitigates attacks based on internal collisions, such as Joux's multicollisions or Kelsey and Schneier second preimages. This comes at a price, however: because the internal state is larger, more memory is necessary, and potentially more computation in order to achieve the same security as with a smaller chaining value.

The compression function BLAKE uses a construction that was called *local wide-pipe*, for it creates a larger internal state within the compression function, whereas chaining values have the same length as the final digest.

2.4.4 Sponge Functions

Sponge functions [29, 30] use a construction that deviates from the compression-based MD mode: given a chaining value h , the new chaining value is computed as $P(h \oplus M_i)$, where the data block M_i is significantly smaller than h and where P is a permutation, which may be efficiently invertible.

As depicted in Figure 2.1, a sponge function consists of a state of width $b = r + c$ bits, where:

- r is called the *rate*, and corresponds to the data block size;

- c is called the *capacity*, and defines the security level, being approximately $c/2$ bits.

The sponge function then modifies the internal state by a sequence of data block injections followed by an application of a permutation function (which may also be a noninvertible transform).

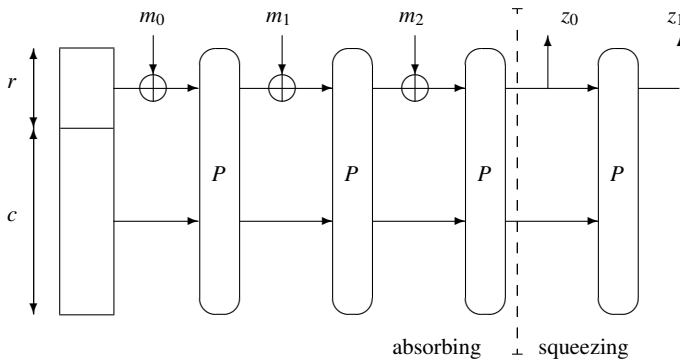


Fig. 2.1 The sponge construction, for the example of a 4-block ($4r$ -bit) padded message.

Like the MD mode, sponge functions benefit from security reductions; for example, it has been proven that the function behaves ideally if the underlying permutation behaves ideally (cf. 2.2.3). An advantage of sponge functions is their flexibility: it is straightforward to vary parameters to achieve various efficiency/security tradeoffs. Examples of sponge functions are the SHA3 candidate Keccak and the lightweight hash function QUARK [14].

2.4.5 Compression Functions

Compression functions are the main building block of Merkle–Damgård and wide-pipe hash functions. As their name suggests, they return fewer bits than input bits received. However, unlike file compression methods, they do not aim to retain the information from the input (which would imply at least partial invertibility). Contrary to that, they aim to behave as random functions, thus eliminating any structure in their input values.

One strategy to construct compression functions is to *reuse block ciphers*—that is, keyed permutation, invertible transforms—to create a noninvertible transform. The main motivations for this approach are:

- **Confidence:** If the security of a hash function is reducible to that of its underlying block cipher, then using a well-analyzed block cipher gives more confidence than a new algorithm.

- **Compact implementations:** The code used for encryption with the block cipher may be reused by the hash function, thus reducing the space occupied by the cryptographic components in a program.

Another significant advantage specific to the reuse of AES is *speed*: native AES instructions in recent AMD, ARM, or Intel processors significantly speed up AES, and hash functions may take advantage of this. Besides faster execution, native AES instructions also avoid the risk of cache-timing attacks, as demonstrated on table-based implementations of AES.

Counterarguments to block cipher-based hashing are:

- **Structural problems:** Generally the block and key lengths of block ciphers do not match the values required for hash functions; e.g., AES uses 128-bit blocks, whereas a general-purpose hash function should return digests of at least 224 bits. One thus has to use constructions with several instances of the block cipher, which is less efficient.
- **Slow key schedule:** The initialization of block ciphers is typically slow, which motivates the use of fixed-key permutations rather than families of permutations. However, results indicate that this approach cannot yield compression functions that are both efficient and secure [43, 155, 156]. A proposal for fixing this problem was to use a *tweakable* block cipher [121], where an additional input, the tweak, is processed much faster than a key.

We now briefly summarize the historical development of block cipher-based hashing.

The idea of making hash functions out of block ciphers dates back at least to Rabin [153], who proposed in 1978 to hash $(m_1, \dots, m_\ell)$ as

$$\text{DES}_{m_\ell}(\dots(\text{DES}_{m_1}(\text{IV})\dots).$$

Subsequent works devised less straightforward schemes, with one or two calls to the block cipher within a compression function [112, 125, 129, 148, 152].

In 1993, Preneel, Govaerts, and Vandewalle (PGV) [149] considered 64 block cipher-based compression functions and identified 12 secure ones,¹¹ including

$$\begin{aligned} &E_h(M) \oplus M \\ &E_h(h \oplus M) \oplus h \oplus M \\ &E_h(M) \oplus h \oplus M \\ &E_h(M) \oplus M \\ &E_M(h) \oplus h \\ &E_M(h \oplus M) \oplus h \oplus M \end{aligned}$$

where $E_h(M)$ denotes encryption of the data block M with h as a key (see Figure 2.2). The fifth construction in this list is known as the Davies–Meyer construction and is used by MD5, SHA1, and SHA2. Note that some of the constructions implicitly assume that M and h have the same length. Some constructions have the

¹¹ A formal analysis of the security of these constructions can be found in [45].

undesirable property of easily found fixed points; for example, a fixed point for the construction $E_M(h) \oplus h$ can be found by choosing an arbitrary M and computing $h' := E_M^{-1}(0)$, leading to $E_M(h') \oplus h' = h'$ (see Section 2.4.1.3 for attacks exploiting fixed points).

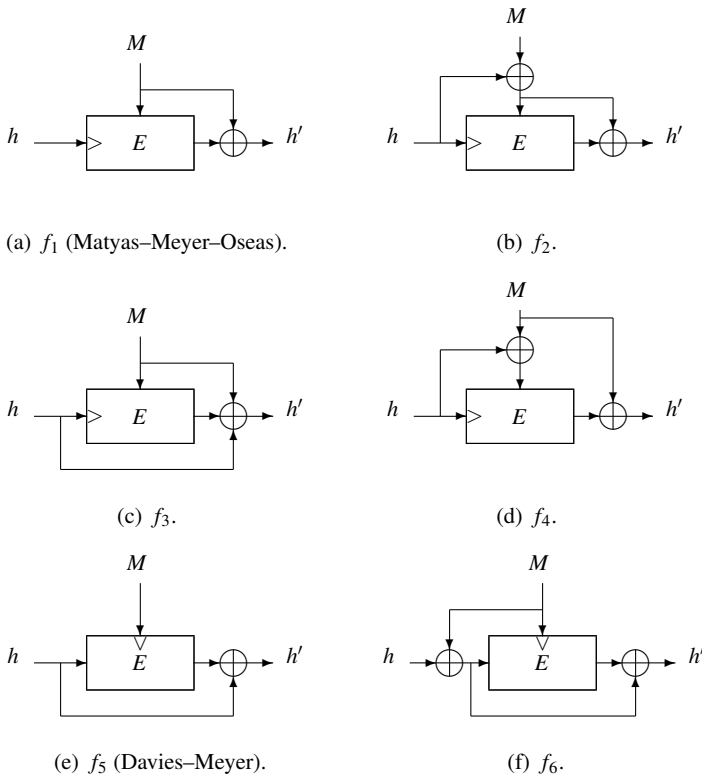


Fig. 2.2 Block cipher-based compression functions f_1 to f_6 by Preneel et al. [149], where a mark denotes the key input (we assume keys and message blocks of the same size).

Note that the so-called PGV schemes cannot be proved collision resistant under the pseudorandom permutation (PRP) assumption only; to see this, take a block cipher E and construct the block cipher $\tilde{E}$ as

$$\tilde{E}_k(m) = \begin{cases} k & \text{if } m = k \\ E_k(k) & \text{if } m = E_k^{-1}(k) \\ E_k(m) & \text{otherwise} \end{cases}.$$

If the Matyas–Meyer–Oseas (MMO) construction [125] $E_{h_{i-1}}(m_i) \oplus m_i$ is instantiated with $\tilde{E}$, then collisions are easy to find, yet $\tilde{E}$ inherits the PRP property from E .

BLAKE uses a construction similar to $E_M(h) \oplus h$, but where the block cipher is replaced by a noninvertible function—itsself built on a block cipher. This makes finding points difficult to find, among other properties.

Examples of pre-SHA3 designs based on block ciphers are Whirlpool [20], Maelstrom [68], and Grindahl [107] (subsequently broken [144]), which all build on AES. Some submissions to the SHA3 competition were based on AES: ECHO, Fugue, LANE, SHAMATA, SHAvite-3, and Vortex, to name a few. They all use an ad hoc construction to make a compression function out of AES. However, the security of AES as a block cipher is not always sufficient for the security of the compression function; for example, SHAMATA and Vortex were broken [12, 85] (ironically, one attack on Vortex works because AES is a good block cipher).

2.5 The SHA Family

This section gives a brief overview of the NIST-approved hash functions SHA1 and SHA2, and of the SHA3 candidates. Below we copy NIST's 2006 statement regarding the use of SHA1 and SHA2 [131]:

The SHA-2 family of hash functions (i.e., SHA-224, SHA-256, SHA-384 and SHA-512) may be used by Federal agencies for all applications using secure hash algorithms. Federal agencies should stop using SHA-1 for digital signatures, digital time stamping and other applications that require collision resistance as soon as practical, and must use the SHA-2 family of hash functions for these applications after 2010. After 2010, Federal agencies may use SHA-1 only for the following applications: hash-based message authentication codes (HMACs); key derivation functions (KDFs); and random number generators (RNGs). Regardless of use, NIST encourages application and protocol designers to use the SHA-2 family of hash functions for all new applications and protocols.

In spite of that recommendation, SHA1 remains widely used, either for legacy reasons, efficiency reasons, or acceptance of the risk—which may or may not be justified.

2.5.1 SHA1

The NIST standard SHA1 [137] was designed by the US National Security Agency (NSA) and published in 1995. SHA1 superseded SHA0, a function almost identical to SHA1 published in 1993 and later withdrawn. Later, in 1998, a collision for SHA0 was published [49].

SHA1 produces 160-bit digests, and is thus expected to provide 80-bit security against collision attacks. As of 2013, SHA1 is with MD5 the most widely used hash function.

2.5.1.1 Internals

SHA1 follows the MD construction. The compression function of SHA1 processes 512-bit data blocks and 160-bit chaining values. It initializes an internal state of five 32-bit words a, b, c, d, e with the current chaining value, and transforms it by repeating a step function 80 times and XORing the final state with the initial state. The step function for the first 16 steps does

$$\begin{aligned} \text{ch} &= (b \wedge c) \vee ((\neg b) \wedge d) \\ \text{temp} &= \text{ch} + e + m_i + 5A827999 \\ e &= d \\ d &= c \\ c &= (b \lll 30) \\ b &= a \\ a &= (a \lll 5) + \text{temp} \end{aligned}$$

where m_i is the i th word of the data block $i = 0, \dots, 15$. The subsequent steps of SHA1 use different Boolean functions to compute ch , different constants to compute temp , and words w_i of the *expanded* data block computed as

$$w_i := (w_{i-3} \oplus w_{i-8} \oplus w_{i-14} \oplus w_{i-16}) \lll 1, \quad i = 16, \dots, 79$$

with $w_i := m_i$ for $i = 0, \dots, 15$.

2.5.1.2 Security

The first reported attack on the full SHA1 was a collision attack with complexity 2^{69} by Chinese researcher Xiaoyun Wang and her team [174], in 2005. As of 2011, attacks with complexity as low as 2^{57} [123] and 2^{52} [127] have been claimed. However, the refined complexity analysis by Manuel [124] argued that these estimates were too optimistic, and that the most efficient attack known then [93] may have complexity between 2^{65} and 2^{69} . In 2013, Stevens' refined analysis [165] led to a collision attack with estimated complexity of 2^{61} .

2.5.2 SHA2

The NIST standard SHA2 [137] was designed by the US National Security Agency (NSA) and published in 2001. SHA2 is a family of four hash functions: SHA-224, SHA-256, SHA-384, and SHA-512, which produce digests of bit size equal to their respective suffixes. SHA2 is supported by an increasing number of products, with SHA-256 being the most common instance.

2.5.2.1 Internals

Like SHA1, all functions of the SHA2 family follow the Merkle–Damgård construction. SHA-256 and SHA-512 are the two main instances of the family. They respectively work on 32- and 64-bit words, and thus use distinct algorithms. SHA-224 and SHA-384 are derived from SHA-256 and SHA-512, by using distinct initial values and truncating the final output.

The compression function of SHA-256 processes 512-bit data blocks and 256-bit chaining values. It initializes an internal state of eight 32-bit words a, b, c, d, e, f, g, h with the current chaining value, and transforms it by repeating a step function 64 times and XORing the final state with the initial state. The step function of SHA-256 does the following:

$$\begin{aligned}
 \sigma_0 &= (a \ggg 2) \oplus (a \ggg 13) \oplus (a \ggg 22) \\
 \sigma_1 &= (e \ggg 6) \oplus (e \ggg 11) \oplus (e \ggg 25) \\
 \text{maj} &= (a \wedge b) \oplus (a \wedge c) \oplus (b \wedge c) \\
 \text{ch} &= (e \wedge f) \oplus ((\neg e) \wedge g) \\
 \text{temp0} &= \sigma_0 + \text{maj} \\
 \text{temp1} &= \sigma_1 + \text{ch} + \text{const}_i + h + w_i \\
 h &= g \\
 g &= f \\
 f &= e \\
 e &= d + \text{temp1} \\
 d &= c \\
 c &= b \\
 b &= a \\
 a &= \text{temp0} + \text{temp1}
 \end{aligned}$$

The values const_i are predefined step-dependent constants, and w_i 's are words of expanded data block. For $i = 0, \dots, 15$, w_i is equal to the data block word m_i , and for $i = 16, \dots, 63$ w_i is recursively defined as

$$\begin{aligned}
 s_0 &:= (w_{i-15} \ggg 7) \oplus (w_{i-15} \ggg 18) \oplus (w_{i-15} \gg 3) \\
 s_1 &:= (w_{i-2} \ggg 17) \oplus (w_{i-2} \ggg 19) \oplus (w_{i-2} \gg 10) \\
 w_i &:= w_{i-16} + s_0 + w_{i-7} + s_1
 \end{aligned}$$

The compression function of SHA-512 uses 64-bit words, and thus processes 1,024-bit blocks and 512-bit chaining values.

2.5.2.2 Security

No attack is known on any of the four SHA2 instances. The best known results are attacks on versions with a reduced number of steps: in 2009, Aoki, Guo, Matusiewicz, Sasaki, and Wang [5] described preimage attacks on 43-step SHA-256 and 46-step SHA-512 that are twice as fast as bruteforce. The best known colli-

sion attacks [84] find collisions on 24-step SHA-256 and SHA-512 with respective complexities of $2^{28.5}$ and 2^{53} .

2.5.3 SHA3 Finalists

We give a brief overview of the other SHA3 finalists, highlighting their unique qualities and strengths compared with other algorithms.

2.5.3.1 Grøstl

Designed by a team of seven active cryptanalysts from the Technical University of Graz, Austria and from the Technical University of Denmark, Grøstl is the only SHA3 finalist that reuses components of the AES. Grøstl follows a wide-pipe MD construction, with a compression function returning

$$P(h \oplus M) \oplus Q(M) \oplus h ,$$

where P and Q are two permutations inspired by the AES, and h and M have the same length (512 or 1,024 bits). The security of Grøstl was extensively analyzed by its designers as well as third parties, leading to a well-understood design. These were the main reasons for its selection as a finalist.

2.5.3.2 JH

JH was designed by Hongjun Wu, from Nanyang Technological University, Singapore. JH is the only SHA3 finalist to use 4-bit Sboxes combined with a linear diffusion layer, in the spirit of the AES finalist Serpent. The chopped wide-pipe MD construction of JH relies on a novel compression function that computes

$$E(h \oplus (M \parallel 0 \dots 0)) \oplus (0 \dots 0 \parallel M) ,$$

where E is a permutation, h is 1,024-bit, and M as well as the strings of zeroes are 512-bit. Due to its bit-oriented structure, JH is a good performer in hardware implementations. JH made the finals due to its security margin, all-round performance, and innovative design.

2.5.3.3 Keccak

Keccak is a creation of Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche, from the semiconductor companies STMicroelectronics and NXP. Keccak is a sponge function based on a bit-oriented permutation. It is thus very fast in

hardware implementations. Keccak was selected due to its security margin, high throughput, and simplicity of design.

2.5.3.4 Skein

Skein is the brainchild of a team of eight researchers and engineers from industry (BT, Hifn, Intel, Microsoft, PGP) and academia (Bauhaus-Universität Weimar, UCSD, University of Washington). Skein uses a construction similar to HAIFA, and builds on a compression function based on a tweakable block cipher in modified MMO mode

$$E_h(M) \oplus M ,$$

where E is a tweakable block cipher (called Threefish) using only modular addition, rotation, and XOR. Due to its construction optimized for 64-bit software architectures, Skein is the best performer on high-end desktop and server platforms. Skein was selected as a finalist due to its security margin and speed in software.

Chapter 3

Specification of BLAKE

Engineering is damn hard work.
—Harold Goldberg

This chapter gives a complete specification of the hash function family BLAKE. It first describes the two main instances, BLAKE-256 and BLAKE-512, and then their variants BLAKE-224 and BLAKE-384. Finally, it describes the toy versions BLOKE, FLAKE, BLAZE, and BRAKE.

3.1 BLAKE-256

The hash function BLAKE-256 operates on 32-bit words and returns a 256-bit hash value. This section defines BLAKE-256, going from its constant parameters to its compression function, then to its iteration mode.

3.1.1 Constant Parameters

BLAKE-256 uses the same 256-bit initial value (IV) as SHA-256, namely¹

$$\begin{aligned} \text{IV}_0 &= 6a09e667 & \text{IV}_1 &= bb67ae85 & \text{IV}_2 &= 3c6ef372 & \text{IV}_3 &= a54ff53a \\ \text{IV}_4 &= 510e527f & \text{IV}_5 &= 9b05688c & \text{IV}_6 &= 1f83d9ab & \text{IV}_7 &= 5be0cd19 \end{aligned}$$

BLAKE-256 uses 16 word constants, chosen as the first digits of π :

$$\begin{aligned} u_0 &= 243f6a88 & u_1 &= 85a308d3 & u_2 &= 13198a2e & u_3 &= 03707344 \\ u_4 &= a4093822 & u_5 &= 299f31d0 & u_6 &= 082efa98 & u_7 &= ec4e6c89 \\ u_8 &= 452821e6 & u_9 &= 38d01377 & u_{10} &= be5466cf & u_{11} &= 34e90c6c \\ u_{12} &= c0ac29b7 & u_{13} &= c97c50dd & u_{14} &= 3f84d5b5 & u_{15} &= b5470917 \end{aligned}$$

¹ These constants correspond to the first 32 bits of the fractional parts of the square roots of the first eight prime numbers.

Ten permutations of the set $\{0, \dots, 15\}$ are used by all BLAKE functions, defined in Table 3.1.

Table 3.1 Permutations of $\{0, \dots, 15\}$ used by BLAKE.

σ_0	:	[	0,	1,	2,	3,	4,	5,	6,	7,	8,	9,	10,	11,	12,	13,	14,	15	]
σ_1	:	[	14,	10,	4,	8,	9,	15,	13,	6,	1,	12,	0,	2,	11,	7,	5,	3	]
σ_2	:	[	11,	8,	12,	0,	5,	2,	15,	13,	10,	14,	3,	6,	7,	1,	9,	4	]
σ_3	:	[	7,	9,	3,	1,	13,	12,	11,	14,	2,	6,	5,	10,	4,	0,	15,	8	]
σ_4	:	[	9,	0,	5,	7,	2,	4,	10,	15,	14,	1,	11,	12,	6,	8,	3,	13	]
σ_5	:	[	2,	12,	6,	10,	0,	11,	8,	3,	4,	13,	7,	5,	15,	14,	1,	9	]
σ_6	:	[	12,	5,	1,	15,	14,	13,	4,	10,	0,	7,	6,	3,	9,	2,	8,	11	]
σ_7	:	[	13,	11,	7,	14,	12,	1,	3,	9,	5,	0,	15,	4,	8,	6,	2,	10	]
σ_8	:	[	6,	15,	14,	9,	11,	3,	0,	8,	12,	2,	13,	7,	1,	4,	10,	5	]
σ_9	:	[	10,	2,	8,	4,	7,	6,	1,	5,	15,	11,	9,	14,	3,	12,	13,	0	]

3.1.2 Compression Function

The compression function of BLAKE-256 takes four values as input:

- A 256-bit chaining value $h = h_0 \parallel \dots \parallel h_7$.
- A 512-bit data block $m = m_0 \parallel \dots \parallel m_{15}$
- A 128-bit salt $s = s_0 \parallel s_1 \parallel s_2 \parallel s_3$
- A 64-bit bit counter value $t = t_0 \parallel t_1$

These inputs represent 960 bits (30 words) in total. The value returned as output is a new 256-bit chaining value, denoted h' .

The compression function works in three steps:

- Initialization of an internal state
- Iteration of 14 rounds to transform this state
- Finalization to return h' from the final state value

These three steps are detailed below.

3.1.2.1 Initialization

The 512-bit (16-word) internal state of the compression function is represented as a 4×4 array. It is initialized with h , s , t , and word constants as

$$\begin{pmatrix} v_0 & v_1 & v_2 & v_3 \\ v_4 & v_5 & v_6 & v_7 \\ v_8 & v_9 & v_{10} & v_{11} \\ v_{12} & v_{13} & v_{14} & v_{15} \end{pmatrix} := \begin{pmatrix} h_0 & h_1 & h_2 & h_3 \\ h_4 & h_5 & h_6 & h_7 \\ s_0 \oplus u_0 & s_1 \oplus u_1 & s_2 \oplus u_2 & s_3 \oplus u_3 \\ t_0 \oplus u_4 & t_0 \oplus u_5 & t_1 \oplus u_6 & t_1 \oplus u_7 \end{pmatrix}.$$

3.1.2.2 Round Function Iteration

Once the state v is initialized, the compression function iterates a series of 14 rounds. A round is a transformation of the state v that computes

$$\begin{aligned} G_0(v_0, v_4, v_8, v_{12}) \quad G_1(v_1, v_5, v_9, v_{13}) \quad G_2(v_2, v_6, v_{10}, v_{14}) \quad G_3(v_3, v_7, v_{11}, v_{15}) \\ G_4(v_0, v_5, v_{10}, v_{15}) \quad G_5(v_1, v_6, v_{11}, v_{12}) \quad G_6(v_2, v_7, v_8, v_{13}) \quad G_7(v_3, v_4, v_9, v_{14}) \end{aligned}$$

where, at round number $0 \leq r \leq 13$, the transform $G_i(a, b, c, d)$ is defined as

$$\begin{aligned} a &:= a + b + (m_{\sigma_r(2i)} \oplus u_{\sigma_r(2i+1)}) \\ d &:= (d \oplus a) \ggg 16 \\ c &:= c + d \\ b &:= (b \oplus c) \ggg 12 \\ a &:= a + b + (m_{\sigma_r(2i+1)} \oplus u_{\sigma_r(2i)}) \\ d &:= (d \oplus a) \ggg 8 \\ c &:= c + d \\ b &:= (b \oplus c) \ggg 7 \end{aligned}$$

The first four calls $G_0, \dots, G_3$ can be computed in parallel, because each of them transforms a distinct column of the array. We thus call the procedure of computing $G_0, \dots, G_3$ a *column step*. Similarly, the last four calls $G_4, \dots, G_7$ process distinct diagonals and thus can be parallelized as well, being called a *diagonal step*.

Note that a round can also be viewed as a sequence of:

1. A column step
2. A rotation of the i -th row, $i = 0, \dots, 3$, of i positions towards the left (similar to the ShiftRows operation in AES)
3. A column step
4. A rotation of the i -th row, $i = 0, \dots, 3$, of i positions towards the right

This observation will prove useful to implement BLAKE in a single-instruction multiple-data (SIMD) manner.

3.1.2.3 Finalization

After the round iteration, the new chaining value $h' = h'_0 \parallel \dots \parallel h'_7$ returned by the compression function is defined as a combination of the final state value with the h and s input:

$$\begin{aligned}
h'_0 &:= h_0 \oplus s_0 \oplus v_0 \oplus v_8 \\
h'_1 &:= h_1 \oplus s_1 \oplus v_1 \oplus v_9 \\
h'_2 &:= h_2 \oplus s_2 \oplus v_2 \oplus v_{10} \\
h'_3 &:= h_3 \oplus s_3 \oplus v_3 \oplus v_{11} \\
h'_4 &:= h_4 \oplus s_0 \oplus v_4 \oplus v_{12} \\
h'_5 &:= h_5 \oplus s_1 \oplus v_5 \oplus v_{13} \\
h'_6 &:= h_6 \oplus s_2 \oplus v_6 \oplus v_{14} \\
h'_7 &:= h_7 \oplus s_3 \oplus v_7 \oplus v_{15}
\end{aligned}$$

3.1.3 Iteration Mode

The previous section described how data blocks are processed with the compression function. We now explain how the iteration mode of BLAKE-256 works, that is, how the compression function is used to hash a message of arbitrary length. The iteration mode is a simplified version of HAIFA (see Section 2.4.2).

3.1.3.1 Data Padding

The data to be hashed (length of at least 1 bit, and of at most $2^{64} - 1$ bits²) is first padded such that its length reaches a multiple of 512. It is then split into 512-bit blocks, in order to be processed iteratively with the compression function.

Data padding works in two steps:

1. Append to the data a bit “1” followed by the minimal (possibly zero) number of bits “0” so that the total length is congruent to 447 modulo 512. Thus, at least one bit and at most 512 are appended.
2. Append to the data a bit “1” followed by a 64-bit unsigned little-endian representation of the original data bit length.

For data M of $1 \leq \ell \leq 2^{64} - 1$ bits, padding can thus be represented as

$$M := M \parallel 8000 \dots 0001 \parallel \ell ,$$

where ℓ is represented as a 64-bit little-endian integer.

For example, data padding transforms the 8-bit original data `ab` to the 512-bit padded data `ab800000...000000010000000000000008`.

3.1.3.2 Iterated Hash

Once padding is done, the padded data M is viewed as a sequence of 512-bit blocks $M_0, M_1, \dots, M_{N-1}$. BLAKE-256 then computes the digest of M by doing for $i = 0$

² About 2,300 petabytes.

to $N - 1$

$$h := \text{compress}(h, M_i, s, \ell_i) ,$$

where h is initialized to the IV specified in Section 3.1.1. The hash value returned is the final value of h . The salt s is optional, and set to zero by default. For $0 \leq i \leq N - 1$, the bit counter value ℓ_i is defined as the number of original data bits in $M_0 \parallel \dots \parallel M_i$, that is, excluding the bits added by the data padding. If the last block contains no bit of the original data then ℓ_{N-1} is zero; for example, a 1,020-bit input leads to padded data of 1,536 bits (three blocks) with $\ell_0 = 512$, $\ell_1 = 1,020$, $\ell_2 = 0$.

The 128-bit ℓ_i 's are represented as unsigned little-endian integers, and the bit counter $t = t_0 \parallel t_1$ is set such that t_0 contains the less significant half of ℓ_i 's encoding.

3.2 BLAKE-512

The hash function BLAKE-512 is similar to BLAKE-256, but operates on 64-bit instead of 32-bit words. Compared with BLAKE-256, all lengths of variables are thus doubled: a hash value or a chaining value is 512-bit, a data block is 1,024-bit, a salt is 256-bit, and a bit counter is 128-bit. In the following we only describe the differences between BLAKE-512 and BLAKE-256.

3.2.1 Constant Parameters

BLAKE-512 uses the same 512-bit initial value as SHA-512, namely³

$$\begin{aligned} \text{IV}_0 &= 6a09e667f3bcc908 & \text{IV}_1 &= bb67ae8584caa73b \\ \text{IV}_2 &= 3c6ef372fe94f82b & \text{IV}_3 &= a54ffa53a5f1d36f1 \\ \text{IV}_4 &= 510e527fade682d1 & \text{IV}_5 &= 9b05688c2b3e6c1f \\ \text{IV}_6 &= 1f83d9abfb41bd6b & \text{IV}_7 &= 5be0cd19137e2179 \end{aligned}$$

Like BLAKE-256, BLAKE-512 uses 16 word constants chosen as the first digits of π :

$$\begin{aligned} u_0 &= 243f6a8885a308d3 & u_1 &= 13198a2e03707344 \\ u_2 &= a4093822299f31d0 & u_3 &= 082efa98ec4e6c89 \\ u_4 &= 452821e638d01377 & u_5 &= be5466cf34e90c6c \\ u_6 &= c0ac29b7c97c50dd & u_7 &= 3f84d5b5b5470917 \\ u_8 &= 9216d5d98979fb1b & u_9 &= d1310ba698dfb5ac \\ u_{10} &= 2ffd72dbd01adfb7 & u_{11} &= b8e1afed6a267e96 \\ u_{12} &= ba7c9045f12c7f99 & u_{13} &= 24a19947b3916cf7 \\ u_{14} &= 0801f2e2858efc16 & u_{15} &= 636920d871574e69 \end{aligned}$$

³ These constants correspond to the first 64 bits of the fractional parts of the square roots of the first eight prime numbers.

Since the constants of BLAKE-256 are a subset of the constants of BLAKE-512, only the latter have to be stored to implement both functions.

BLAKE-512 uses the same permutations as BLAKE-256 (see Table 3.1).

3.2.2 Compression Function

The compression function of BLAKE-512 takes four values as input:

- A 512-bit chaining value $h = h_0 \parallel \dots \parallel h_7$
- A 1,024-bit data block $m = m_0 \parallel \dots \parallel m_{15}$
- A 256-bit salt $s = s_0 \parallel s_1 \parallel s_2 \parallel s_3$
- A 128-bit bit counter value $t = t_0 \parallel t_1$

These inputs represent 1,920 bits (30 words) in total.

The compression function of BLAKE-512 is similar to that of BLAKE-256 with words of 64 bits instead of 32 bits, and with the difference that it makes 16 rounds instead of 14, and that $G_i(a, b, c, d)$ uses different rotation distances, as shown below:

$$\begin{aligned}
 a &:= a + b + (m_{\sigma_r(2i)} \oplus u_{\sigma_r(2i+1)}) \\
 d &:= (d \oplus a) \ggg 32 \\
 c &:= c + d \\
 b &:= (b \oplus c) \ggg 25 \\
 a &:= a + b + (m_{\sigma_r(2i+1)} \oplus u_{\sigma_r(2i)}) \\
 d &:= (d \oplus a) \ggg 16 \\
 c &:= c + d \\
 b &:= (b \oplus c) \ggg 11
 \end{aligned}$$

At round $r \geq 10$, the permutation used is $\sigma_{r \bmod 10}$; for example, at the last round ($r = 15$), the permutation $\sigma_{15 \bmod 10} = \sigma_5$ is used.

3.2.3 Iteration Mode

BLAKE-512 follows an iteration mode identical to that of BLAKE-256, but adapted to 64-bit words. Data padding thus works as follows for BLAKE-512:

1. Append to the data a bit “1” followed by the minimal (possibly zero) number of bits “0” so that the total length is congruent to 895 modulo 1,024. Thus, at least one bit and at most 1,024 are appended.
2. Append to the data a bit “1” followed by a 128-bit unsigned little-endian representation of the original data bit length.

For data M of $1 \leq \ell \leq 2^{128} - 1$ bits, padding can thus be represented as

$$M := M \parallel 8000 \dots 0001 \parallel \ell,$$

where ℓ is represented as a 128-bit little-endian integer. Note that BLAKE-512 accepts data of up to $2^{128} - 1$ bits.

The iterated hash algorithm of BLAKE-512 is the same as that of BLAKE-256, with adapted lengths of variables.

3.3 BLAKE-224

The hash function BLAKE-224 returns a 224-bit (28-byte) hash value. BLAKE-224 is identical to BLAKE-256 except that:

- It uses the 256-bit initial value of SHA-224:

$$\begin{aligned} \text{IV}_0 &= \text{c1059ed8} & \text{IV}_1 &= \text{367cd507} & \text{IV}_2 &= \text{3070dd17} & \text{IV}_3 &= \text{f70e5939} \\ \text{IV}_4 &= \text{ffc00b31} & \text{IV}_5 &= \text{68581511} & \text{IV}_6 &= \text{64f98fa7} & \text{IV}_7 &= \text{befa4fa4} \end{aligned}$$

- In the data padding, the bit “1” preceding the data length is replaced by a bit “0”; the padded data is thus formed as

$$M := M \parallel 8000 \dots 0000 \parallel \ell.$$

- The hash value returned is truncated to its first 224 bits; that is, the iterated hash returns $h_0 \parallel \dots \parallel h_6$ instead of $h_0 \parallel \dots \parallel h_7$.

3.4 BLAKE-384

The hash function BLAKE-384 returns a 384-bit (48-byte) hash value. BLAKE-384 is identical to BLAKE-512 except that:

- It uses the 512-bit initial value of SHA-384:

$$\begin{aligned} \text{IV}_0 &= \text{cbbb9d5dc1059ed8} & \text{IV}_1 &= \text{629a292a367cd507} \\ \text{IV}_2 &= \text{9159015a3070dd17} & \text{IV}_3 &= \text{152fec8d8f70e5939} \\ \text{IV}_4 &= \text{67332667ffc00b31} & \text{IV}_5 &= \text{8eb44a8768581511} \\ \text{IV}_6 &= \text{db0c2e0d64f98fa7} & \text{IV}_7 &= \text{47b5481dbefa4fa4} \end{aligned}$$

- In the data padding, the bit “1” preceding the data length is replaced by a bit “0”; the padded data is thus formed as

$$M := M \parallel 8000 \dots 0000 \parallel \ell.$$

- The hash value returned is truncated to its first 384 bits; that is, the iterated hash returns $h_0 \parallel \dots \parallel h_5$ instead of $h_0 \parallel \dots \parallel h_7$.

3.5 Toy Versions

To encourage cryptanalysts to analyze BLAKE, we published soon after the SHA3 submission deadline four “toy versions” of BLAKE. These families of hash functions—BLOKE, FLAKE, BLAZE, and BRAKE—are similar to BLAKE, except that they exclude one (or more) of its component features:

- In BLOKE, the permutations $\sigma_0, \dots, \sigma_9$ are all set to only the identity permutation $[0, 1, \dots, 15]$.
- FLAKE makes no feedforward of h and s , that is, the finalization of the compression function sets

$$\begin{aligned} h'_0 &:= v_0 \oplus v_8 \\ h'_1 &:= v_1 \oplus v_9 \\ h'_2 &:= v_2 \oplus v_{10} \\ h'_3 &:= v_3 \oplus v_{11} \\ h'_4 &:= v_4 \oplus v_{12} \\ h'_5 &:= v_5 \oplus v_{13} \\ h'_6 &:= v_6 \oplus v_{14} \\ h'_7 &:= v_7 \oplus v_{15} \end{aligned}$$

- BLAZE replaces all the 16 word constants by zero in the G function (but not in the state initialization).
- BRAKE incorporates all the modifications of BLOKE, FLAKE, and BLAZE.

Chapter 8 presents cryptanalysis results suggesting that BRAKE may not be the weakest of the four versions.

Chapter 4

Using BLAKE

Let H be a secure hash function—we'll get back to what that means in a second.
—Matthew Green

This chapter shows how BLAKE can be used in common hash-based cryptographic schemes. For each scheme, we provide a basic description and a concrete example showing how the data to be hashed is formed, as well as some intermediate values and the final result. Examples can be seen as detailed test vectors, and aim to be reproducible so that developers can check their implementations against various use cases. This chapter may be used as a set of test vectors, but does not aim to be an authoritative specification, let alone a recommendation, of the standard schemes considered.

4.1 Simple Hashing

4.1.1 Description

We consider a simple application of a BLAKE instance as specified in Chapter 3, without using any other scheme or primitive. Such straightforward hashing is, for example, used in digital signature schemes, where the hash function execution is generally independent of the signature algorithm used. Indeed, the hash function serves to create a condensed representation of a message that will fit the short input length mandated by signature schemes (for example, ECDSA signatures on an n -bit curve cannot process inputs of more than n bits; thus 160-bit SHA1 hashes are typically used in combination with 160-bit curves).

4.1.2 Hashing a Large File with BLAKE-256

In this example, we use BLAKE-256 to hash an ISO image of the latest Ubuntu Linux distribution.¹ The corresponding file, `ubuntu-12.04-beta1-desktop-amd64.iso`, hashes with SHA-256 to the following value:

```
6e5c0dcda1dbd6673940137e66bfe6828b5e4288f9a28194cb089384439e2377 .
```

The file is 734,310,400 bytes long (about 700 MiB). This is exactly 11,473,600 blocks of 512 bits. The last block processed by BLAKE-256 will thus be

```
8000...00010000000015e258000 ,
```

where `15e258000` is the hexadecimal representation of the number of data *bits*, equal to $734,310,400 \times 8 = 5,874,483,200$.

The first data block will consist of the first 64 bytes of the file, that is, as shown with the `xxd` hexdump utility:

```
$ xxd -l 64 -g 4 ubuntu-12.04-beta1-desktop-amd64.iso
00000000: 33ed9090 90909090 90909090 90909090  3.....
00000010: 90909090 90909090 90909090 90909090  .....
00000020: 33edfa8e d5bc007c fbfc6631 db6631c9  3.....|.f1.f1.
00000030: 66536651 06578edd 8ec552be 007cbf00  fSfQ.W....R...|..
```

The second data block consists of the 64 subsequent bytes, that is:

```
$ xxd -l 64 -s 64 -g 4 ubuntu-12.04-beta1-desktop-amd64.iso
00000040: 06b90001 f3a5ea4b 06000052 b441bbaa  .....K...R.A..
00000050: 5531c930 f6f9cd13 721681fb 55aa7510  U1.0....r...U.u.
00000060: 83e10174 0b66c706 f106b442 eb15eb00  ...t.f.....B....
00000070: 5a51b408 cd1383e1 3f5b510f b6c64050  ZQ.....?[Q...@P
```

We wrote a command-line program that hashes a file with BLAKE-256, and that for each compression function call prints out the counter value *t*, the data block *m*, the initial and final *v* state, and the new chaining value obtained. The output of this program for the first two blocks is then

```
t:      00000000 00000200 (512)
m:      33ed9090 90909090 90909090 90909090
        90909090 90909090 90909090 90909090
        33edfa8e d5bc007c fbfc6631 db6631c9
        66536651 06578edd 8ec552be 007cbf00
v init: 6a09e667 bb67ae85 3c6ef372 a54ff53a
        510e527f 9b05688c 1f83d9ab 5be0cd19
        243f6a88 85a308d3 13198a2e 03707344
        a4093a22 299f33d0 082efa98 ec4e6c89
v end:  f56757f0 0ab1d588 db3c4ceb 0acbc871
        f3e05df8 beb2f92d 25d89f01 0664f176
```

¹ Downloaded on March 8, 2012 from <http://mirror.switch.ch/ftp/mirror/ubuntu-cdimage/precise/ubuntu-12.04-beta1-desktop-amd64.iso>

```

          b9c711ae 3a58dd28 23008c19 47bd4f67
          8a698640 476803f5 35023e58 69a1cb38
h:        26a9a039 8b8ea625 c4523380 e839722c
          288789c7 62df9254 0f5978f2 3425f757

t:        00000000 00000400 (1024)
m:        06b90001 f3a5ea4b 06000052 b441bbaa
          5531c930 f6f9cd13 721681fb 55aa7510
          83e10174 0b66c706 f106b442 eb15eb00
          5a51b408 cd1383e1 3f5b510f b6c64050
v init:   26a9a039 8b8ea625 c4523380 e839722c
          288789c7 62df9254 0f5978f2 3425f757
          243f6a88 85a308d3 13198a2e 03707344
          a4093c22 299f35d0 082efa98 ec4e6c89
v: end    143d44b3 ebba2d7f dee6a1da 64829dc7
          cddc54db 2bbdc870 c497f835 e071c83e
          2897657e 79d788a1 acd6a152 09f06f3b
          bc347c0e 55b4f590 7c4e066f 360e155d
h:        1a0381f4 19e303fb b6623308 854b80d0
          596fa112 1cd6afb4 b78086a8 e25a2a34

```

Above, and as per the specifications in Chapter 3, *t* denotes the counter (with its decimal value in parentheses), *m* the message block, *v: init* the initial value of the internal state, *v: end* its final value after the round iteration, and *h* the intermediate hash value.

We skip the details of the processing of the 11,473,597 subsequent blocks. Eventually, the last 512 bits of the file hashed happen to be all zero:

```

$ xxd -s -64 -g 4 ubuntu-12.04-beta1-desktop-amd64.iso
2bc4afc0:00000000 00000000 00000000 00000000 .....
2bc4afd0:00000000 00000000 00000000 00000000 .....
2bc4afe0:00000000 00000000 00000000 00000000 .....
2bc4aff0:00000000 00000000 00000000 00000000 .....

```

When processing this block, the penultimate call to the compression function reports the following values:

```

t:        00000001 5e258000 (1579515904)
m:        00000000 00000000 00000000 00000000
          00000000 00000000 00000000 00000000
          00000000 00000000 00000000 00000000
          00000000 00000000 00000000 00000000
v init:   55fb7f55 c2e76afa 3005b380 4e7e6bc4
          71824950 d2a93bc6 f2c7fb8b adc03ac1
          243f6a88 85a308d3 13198a2e 03707344
          fa2cb822 77bab1d0 082efa99 ec4e6c88
v end:    7c567963 0f4f21d0 c61a708d 73565ce1
          71877dfc 448b584e f12901f0 a59a7722
          aflfef05 66a3fb44 1b1f3624 7e0a30c9
          a85f5e1d 2a2386bd e0f2c36e f9f4f87f
h:        86b2e933 ab0bb06e ed00f529 432207ec
          a85a6ab1 bc01e535 e31c3915 f1aeb59c

```

Finally the last block compressed by BLAKE-256 is as determined above, and includes only padding bits (and no bits from the file processed):

```

t:      00000001 5e258000 (1579515904)
m:      80000000 00000000 00000000 00000000
        00000000 00000000 00000000 00000000
        00000000 00000000 00000000 00000000
        00000000 00000001 00000001 5e258000
v init: 86b2e933 ab0bb06e ed00f529 432207ec
        a85a6ab1 bc01e535 e31c3915 f1aeb59c
        243f6a88 85a308d3 13198a2e 03707344
        a4093822 299f31d0 082efa98 ec4e6c89
v end:  a31d955c e092597e 346fc3b2 19e531dc
        77241594 8650703e 24f5493d 70aa2682
        652377b6 6abab9ba ed7a5440 26d887d8
        1f7b4024 ca91e42b d77c001c 7d452b5d
h:      408c0bd9 212350aa 341562db 7c1fb1e8
        c0053f01 f0c07120 10957034 fc41b843

```

BLAKE-256 thus hashes the file `ubuntu-12.04-beta1-desktop-amd64.iso` to

```
408c0bd9212350aa341562db7c1fb1e8c0053f01f0c0712010957034fc41b843 .
```

4.1.3 Hashing a Bit with BLAKE-512

As required by NIST, BLAKE instances hash data whose length can be any number of bits, that is, not necessarily an integral number of bytes (although use cases justifying that requirement are unclear, the byte being the standard atomic data unit, and bytes being octets on all reasonable platforms); for example, the BLAKE-512 digest of the bit “1” is obtained by hashing the 1,024-bit block consisting of:

- A bit “1” (data hashed)
- Another bit “1” (signaling the beginning of padding bits)
- 445 contiguous copies of the “0” bit
- A bit “1” (differentiator between BLAKE-512 and BLAKE-384)
- The 128-bit encoding of the integer 1 (the data length).

That is, the counter and the block processed are:

```

t:      0000000000000000 0000000000000001 (1)
m:      c000000000000000 0000000000000000
        0000000000000000 0000000000000000
        0000000000000000 0000000000000000
        0000000000000000 0000000000000000
        0000000000000000 0000000000000000
        0000000000000000 0000000000000000
        0000000000000000 0000000000000000
        0000000000000000 0000000000000001
        0000000000000000 0000000000000001

```

The hash value returned is

```
69269d2195e03088f928a24a4539849727e47dc46d2596f12b2c88491776f20c
31b1526912aec62f29e6641221ca2a67e149857be5e6e08fc3f49ec5d7b7138c .
```

4.1.4 Hashing the Empty String with BLAKE-512

BLAKE can hash the empty string, that is, data of length zero. The padded data is constructed according to the specification, such that the first bit of the padded data is also the first bit of padding. Hashing the empty string may find applications when the hash of an optional parameter is part of a construction or encoding; for example, the OAEP 2.1 encryption scheme standard requires the hash of a “label,” and by default, when no label is defined, the hash of the empty string is used.

The counter and the block processed by the first and only compression are thus

```
t:      0000000000000000 0000000000000000 (0)
m:      8000000000000000 0000000000000000
        0000000000000000 0000000000000000
        0000000000000000 0000000000000000
        0000000000000000 0000000000000000
        0000000000000000 0000000000000000
        0000000000000000 0000000000000000
        0000000000000000 0000000000000000
        0000000000000000 0000000000000001
        0000000000000000 0000000000000000
```

Note that the message length encoded in the last 128 bits is zero. The hash value returned is

```
a8cfbbd73726062df0c6864dda65defe58ef0cc52a5625090fa17601e1eecd1b
628e94f396ae402a00acc9eab77b4d4c2e852aaaa25a636d80af3fc7913ef5b8 .
```

4.2 Hashing with a Salt

4.2.1 Description

BLAKE offers built-in support for hashing with a salt of up to 128 bits for BLAKE-224 and BLAKE-256, and of up to 256 bits for BLAKE-384 and BLAKE-512. As specified in Chapter 3, the 4-word salt is XORed to four constants and defines the initial value of the internal state words v_8, v_9, v_{10}, v_{11} .

4.2.2 Hashing a Bit with BLAKE-512 and a Salt

When hashing the bit “1” with a salt, the counter and the block processed are the same as in Section 4.1.3. The only difference is in the initial state of the compression function; for example, with a salt set to the 256-bit string 00010203...1e1f, the initial state is set to

```
6a09e667f3bcc908 bb67ae8584caa73b
3c6ef372fe94f82b a54ff53a5f1d36f1
```

```

510e527fade682d1 9b05688c2b3e6c1f
1f83d9abfb41bd6b 5be0cd19137e2179
243e688b81a60ed4 1b1080250f7d7d4b
b4182a313d8a27c7 1037e083f0537296
452821e638d01376 be5466cf34e90c6d
c0ac29b7c97c50dd 3f84d5b5b5470917

```

instead of (note the differences on the fifth and sixth lines)

```

6a09e667f3bcc908 bb67ae8584caa73b
3c6ef372fe94f82b a54ff53a5f1d36f1
510e527fade682d1 9b05688c2b3e6c1f
1f83d9abfb41bd6b 5be0cd19137e2179
243f6a8885a308d3 13198a2e03707344
a4093822299f31d0 082efa98ec4e6c89
452821e638d01376 be5466cf34e90c6d
c0ac29b7c97c50dd 3f84d5b5b5470917

```

The hash value returned is

```

14a9838f1bd061a49d4573c5aca39e84e0d29d088896b3ec69977dd4d7427388
fb8c1170074ddbfe2e6027d269cad5bc9bfff76dbcfef6b573923461773d63dd8 .

```

4.3 Message Authentication with HMAC

4.3.1 Description

HMAC is a common hash-based MAC construction defined in 1996 by Bellare, Canetti, and Krawczyk [21, 109], now standardized in FIPS 198-1 [132]. Although HMAC is commonly used in combination with SHA1 (as in IPsec, TLS, SSH), it is a generic construction that accepts any hash function as its building block: Given a hash function H and a key K , remember from Section 2.1.2 that $\text{HMAC-}H$ produces an authentication tag by computing

$$\text{HMAC-}H(K, M) := H((K \oplus \text{opad}) \| H((K \oplus \text{ipad}) \| M)) ,$$

where $\text{opad} = 5c5c \dots 5c$, and $\text{ipad} = 3636 \dots 36$.

4.3.2 Authenticating a File with HMAC-BLAKE-512

This example applies HMAC-BLAKE-512 to compute a MAC of the documentation² of the hash function Skein [66]. The PDF file `skein1.3.pdf` hashes with SHA-256 to

² Downloaded on March 8, 2012 from <http://www.skein-hash.info/sites/default/files/skein1.3.pdf>.

```
79de9ee16dbf79d635e0270574efbb0c74a73f16b02badc0253d08065d9df24a .
```

This file is 479,368 bytes long, that is, 3,745 blocks of 1,024 bits plus 8 extra bytes, which are

```
$ xxd -s -8 -g 8 skein1.3.pdf
0075080: 360a2525454f460a                6.%%EOF.
```

The inner call to BLAKE-512 processes a first data block of 1,024 bits including the key, followed by the 479,368 bytes of skein1.3.pdf. In total BLAKE-512 thus processes 479,496 bytes, so the last block compressed is

[illegible]

where 3_{a8840} is the number of data bits hashed, that is, $479,496 \times 8 = 3,835,968$.

Let us consider the 256-bit key `00010203...1e1f`. The first block compressed is thus formed by XORing each of those 32 bytes with `36`, that is,

36373435323330313e3f3c3d3a3b383926272425222320212e2f2c2d2a2b2829,

followed by 36 bytes, that is, zero bytes XORed with ipad.

The intermediate values for the first block of the inner hash are then

```
t: 0000000000000000 00000000000000400 (1024)
m: 3637343532333031 3e3f3c3d3a3b3839
    2627242522232021 2e2f2c2d2a2b2829
    3636363636363636 3636363636363636
    3636363636363636 3636363636363636
    3636363636363636 3636363636363636
    3636363636363636 3636363636363636
    3636363636363636 3636363636363636
    3636363636363636 3636363636363636
v init: 6a09e667f3bcc908 bb67ae8584caa73b
        3c6ef372fe94f82b a54ff53a5f1d36f1
        510e527fade682d1 9b05688c2b3e6c1f
        1f83d9abfb41bd6b 5be0cd19137e2179
        243f6a8885a308d3 13198a2e03707344
        a4093822299f31d0 082efa98ec4e6c89
        452821e638d01777 be5466cf34e9086c
        c0ac29b7c97c50dd 3f84d5b5b5470917
v end:  6192846e97f903ef 2585ced5c6228a3a
        849aeb269cbfe542 126ef941345ed90d
        12a8e81e1367fd7e d44c5fb6f71cd790
        007a309e04bc1c74 6c582c2e0358dfba
        7eab33a2a2123716 33a7c385153d50bf
        3a78d1ab70fabafc af417e4bffc5b1c1
        4e5910d170985e3f 44c6098d4b27338b
        282ec1f62eb81a5f 7d3cd4b173d1951e
h: 753051abc657fdf1f1 ad45a3d557647dbe
    828cc9ff12d1a795 1860723094865e3d
```

```
0df faab0ce192190 0b8f3eb797058804
37d728c3d145bb43 4a84358663f76bdd
```

The next data block processed consists of the first 128 bytes of skein1.3.pdf, that is:

```
$ xxd -l 128 -g 8 skein1.3.pdf
00000000: 255044462d312e34 0a25d0d4c5d80a33 %PDF-1.4.%....3
00000010: 2030206f626a203c 3c0a2f4c656e6774 0 obj <<./Lengt
00000020: 6820363532202020 202020200a2f4669 h 652 ./Fi
00000030: 6c746572202f466c 6174654465636f64 lter /FlateDecod
00000040: 650a3e3e0a737472 65616d0a78daa594 e,>>.stream.x...
00000050: cb6edb301045f7f9 0a2d252062489194 .n.0.E...-% bH..
00000060: c89de1a679b56903 586d164517844c5b ....y.i.Xm.E..L[
00000070: ac65d2d0a3a98102 fdf592a6e4da8112 .e.....
```

with a counter t set to 2,048, the compression function returns the following chaining value:

```
h:      1fbc3339ae861404 ab35a79d5e57b33e
        127d280eadf86bf7 ae357cb6e1ce4aca
        c44db962bc44e223 cd197d6111fe5a34
        53b100fd0f1a3fba a9f8b2b5b72d8760
```

Eventually the digest computed by the inner hash is

```
489c75e9113cd6488615eb5f959e6210ee383f0688bd154ec77f615940f08deb
559e4c5997d00bb965d14163a1fb18e319f7040485eca5286ebfdb2c47105d6.
```

The outer hash first compresses the key XORed with 5c bytes, that is,

```
m:      5c5d5e5f58595a5b 5455565750515253
        4c4d4e4f48494a4b 4445464740414243
        5c5c5c5c5c5c5c5c 5c5c5c5c5c5c5c5c
        5c5c5c5c5c5c5c5c 5c5c5c5c5c5c5c5c
        5c5c5c5c5c5c5c5c 5c5c5c5c5c5c5c5c
        5c5c5c5c5c5c5c5c 5c5c5c5c5c5c5c5c
        5c5c5c5c5c5c5c5c 5c5c5c5c5c5c5c5c
        5c5c5c5c5c5c5c5c 5c5c5c5c5c5c5c5c
        5c5c5c5c5c5c5c5c 5c5c5c5c5c5c5c5c
```

The second block processed consists of the 512-bit digest of the inner hash followed by 512 bits of padding. In total $1,024 + 512 = 1,536$ bits are hashed, thus the counter and the padding include the hexadecimal value 600:

```
t:      0000000000000000 0000000000000600 (1536)
m:      489c75e9113cd648 8615eb5f959e6210
        ee383f0688bd154e c77f615940f08deb
        559e4c5997d00bb9 65d14163a1fb18e3
        19f7040485eca528 6ebfdbfb2c47105d6
        8000000000000000 0000000000000000
        0000000000000000 0000000000000000
        0000000000000000 0000000000000001
        0000000000000000 0000000000000600
```

The digest returned by the outer hash, and thus by the HMAC instance, is finally

```
3352282b5d69728fb01e041553b85c0b28cea046ba418d6d6372dfe44eccc762
3a1f70adfbe0f6b0b94054b90d829037272e8d9477a6994084f0b3b4798f140c.
```

4.4 Password-Based Key Derivation with PBKDF2

4.4.1 Basic Description

PBKDF2 is a password-based key derivation scheme standardized as PKCS #5 v2.0 [94, 95] and is a NIST recommendation [136]. PBKDF2 is, for example, used to derive an encryption key for disk encryption software given a user's password. It is also often used to store password hashes on web services' servers.

Like HMAC, PBKDF2 is a generic construction that takes as a parameter a pseudorandom function (PRF). PBKDF2 is most often instantiated with HMAC-SHA1, thus building the key-derivation scheme PBKDF2-HMAC-SHA1, but different PRFs are sometimes used (HMAC-SHA-256 in Truecrypt, an AES-based PRF in iOS, etc.). What matters the most for security, however, is less the PRF than the number of iterations and the per-user unicity and unpredictability of the salt.

Given a PRF F , a password P , a salt s , an iteration count c , and an output length $d \leq n$, PBKDF2- F returns the first d bytes of

$$U_1 \oplus U_2 \oplus \cdots \oplus U_c$$

with U_i recursively defined as

$$\begin{aligned} U_1 &= F(P, s || 00000001) \\ U_i &= F(P, U_{i-1}), \quad 2 \leq i \leq c \end{aligned}$$

where P is used as the key of the PRF F . We refer to [94] for a complete specification of PBKDF2.

4.4.2 Generating a Key with PBKDF2-HMAC-BLAKE-224

We generate a 128-bit AES key with PBKDF2-HMAC-BLAKE-224, using “password” (that is, the byte string 70617373776f7264) as a password, the 16-byte all-ff string as a salt, and $c = 4,000$ iterations (the same amount as, for example, PBKDF2-HMAC-SHA1 in Apple iWork, at the time of writing).

The first call to HMAC-BLAKE-224, which plays the role of the PRF in our instance of PBKDF2, takes our 8-byte password as a key and the 20-byte string

```
ffffffffffffffffffffffffffffffffffff00000001.
```

As in the previous HMAC example in Section 4.3.2, the first block compressed includes the key XORed to and followed by the ipad padding, while the second block includes the above 20 bytes followed by padding as per the BLAKE-224 specification. In total, 672 bits are processed by the inner hash:

```
t:      00000000 000002a0 (672)
```

```

m:      ffffffff ffffffff ffffffff ffffffff
        00000001 80000000 00000000 00000000
        00000000 00000000 00000000 00000000
        00000000 00000000 00000000 000002a0
v init: e2fbd5f0 c417206f 065e4b52 9c6cf60e
        f8c6adad b5b8ffb8 a5085f38 c086428b
        243f6a88 85a308d3 13198a2e 03707344
        a4093a82 299f3370 082efa98 ec4e6c89
v end:  c7a52759 88cb795e ea22e4e9 34b77be2
        8188ac76 6d1e8ca9 5024284f 5295f99d
        d39cb55e 7f7858e6 f889f85a 17543931
        be0cd419 e4b9f866 1561994b df5b60b8
h:      f6c247f7 33a401d7 14f557e1 bf8fb4dd
        c742d5c2 3c1f8b77 e04dee3c 4d48dbae

```

The outer hash processes a first block including the key, then a second block including the 224-bit inner digest, such that a total of 736 bits are hashed by the outer hash:

```

t:      00000000 000002e0 (736)
m:      f6c247f7 33a401d7 14f557e1 bf8fb4dd
        c742d5c2 3c1f8b77 e04dee3c 80000000
        00000000 00000000 00000000 00000000
        00000000 00000000 00000000 000002e0
v init: 223a6966 2210384a 5a7b5eee dfe3f84d
        f4a42150 b366a78e d5eed611 25b0467c
        243f6a88 85a308d3 13198a2e 03707344
        a4093ac2 299f3330 082efa98 ec4e6c89
v end:  1d7d9614 1e6f2ddf a121c651 77e766cd
        7b6f90ec 0f997f01 778f6e1a bc9cf9dd
        b51a667e d0522ac9 46d716c5 db1a480b
        1e02c1d6 1397d6ef 336d69a6 86015258
h:      8a5d990c ec2d3f5c bd8d8e7a 731ed68b
        91c9706a af680e60 910cd1ad 1f2dedf9

```

This ends the first iteration of HMAC-BLAKE-224. After the next 3,999 iterations, PBKDF2-HMAC-BLAKE-224 XORs the 4,000 outputs produced and returns their 16 first bytes:

```
aca483bab3f495bd9ce7cfe01cebbcb81 .
```

Note that this example is not a recommendation of PBKDF2-HMAC-BLAKE-224, although this is a highly secure password-based key derivation scheme: State-of-the-art so-called password hashing schemes provide higher security against bruteforce attacks, as observed within the Password Hashing Competition.³

³ <https://password-hashing.net>.

Chapter 5

BLAKE in Software

Be quick, be quiet, and be on time.
—Kelly Johnson

This chapter explains how to implement BLAKE on software platforms, from simplistic and portable C implementations to assembly for 8-bit AVR microcontrollers. We focus on the implementation of the compression function, as opposed to the operation mode, the latter being straightforward to implement and not performance-critical. Optimized C and assembly implementations of BLAKE for various platforms are included in the SUPERCOP software, available for download from the eBACS project [28]. Complete reference C implementations of BLAKE-256 and BLAKE-512 are given in Appendix B.

5.1 Straightforward Implementation

A straightforward implementation of BLAKE aims to implement the specification of the algorithm without any performance optimization target. As we will see below, BLAKE allows clear and quickly written implementations in various languages.

5.1.1 Portable C

We present a detailed tutorial to implement the compression function of BLAKE-256, in a top-down manner.

5.1.1.1 Interface, Declarations, and Definitions

First, we define an interface that reproduces the definition in Section 3.1.2, where unmodified values are defined as `const` arguments. We also declare an array of 16 32-bit words to hold the internal state of the compression function, and an integer counter variable:

```

void blake256_compress(
    uint32_t *h,
    const uint32_t *m,
    const uint32_t *s,
    const uint32_t *t )
{
    uint32_t v[16];
    int i;

```

Note that we use the types defined in `stdint.h`; e.g., `uint32_t` is a 32-bit unsigned integer type.

We then define variables for the σ_i permutations and for the 32-bit constants:

```

const int sigma[][16] = {
    { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9,10,11,12,13,14,15 },
    { 14,10, 4, 8, 9,15,13, 6, 1,12, 0, 2,11, 7, 5, 3 },
    { 11, 8,12, 0, 5, 2,15,13,10,14, 3, 6, 7, 1, 9, 4 },
    { 7, 9, 3, 1,13,12,11,14, 2, 6, 5,10, 4, 0,15, 8 },
    { 9, 0, 5, 7, 2, 4,10,15,14, 1,11,12, 6, 8, 3,13 },
    { 2,12, 6,10, 0,11, 8, 3, 4,13, 7, 5,15,14, 1, 9 },
    { 12, 5, 1,15,14,13, 4,10, 0, 7, 6, 3, 9, 2, 8,11 },
    { 13,11, 7,14,12, 1, 3, 9, 5, 0,15, 4, 8, 6, 2,10 },
    { 6,15,14, 9,11, 3, 0, 8,12, 2,13, 7, 1, 4,10, 5 },
    { 10, 2, 8, 4, 7, 6, 1, 5,15,11, 9,14, 3,12,13, 0 },
    { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9,10,11,12,13,14,15 },
    { 14,10, 4, 8, 9,15,13, 6, 1,12, 0, 2,11, 7, 5, 3 },
    { 11, 8,12, 0, 5, 2,15,13,10,14, 3, 6, 7, 1, 9, 4 },
    { 7, 9, 3, 1,13,12,11,14, 2, 6, 5,10, 4, 0,15, 8 } };

const uint32_t u[16] = {
    0x243f6a88,0x85a308d3,0x13198a2e,0x03707344,
    0xa4093822,0x299f31d0,0x082efa98,0xec4e6c89,
    0x452821e6,0x38d01377,0xbe5466cf,0x34e90c6c,
    0xc0ac29b7,0xc97c50dd,0x3f84d5b5,0xb5470917};

```

5.1.1.2 Initialization

The 16-word internal state is initialized as per Section 3.1.2.1, by setting its first eight words to the current chaining value, and its last eight words to a combination of the salt, counter, and constants:

```

for(i=0; i< 8;++i) v[i] = h[i];
v[ 8] = s[0] ^ 0x243f6a88;
v[ 9] = s[1] ^ 0x85a308d3;
v[10] = s[2] ^ 0x13198a2e;
v[11] = s[3] ^ 0x03707344;
v[12] = t[0] ^ 0xa4093822;
v[13] = t[0] ^ 0x299f31d0;
v[14] = t[1] ^ 0x082efa98;
v[15] = t[1] ^ 0xec4e6c89;

```

5.1.1.3 Round Function Iteration

To define the round function iteration, we first define the G function with the following macros:

```
#define ROT(x,n) (((x)<<(32-n)) | ((x)>>(n)))

#define G(a,b,c,d,e) \
    v[a] += (m[sigma[i][e]] ^ u[sigma[i][e+1]]) + v[b]; \
    v[d] = ROT( v[d] ^ v[a],16 ); \
    v[c] += v[d]; \
    v[b] = ROT( v[b] ^ v[c],12 ); \
    v[a] += (m[sigma[i][e+1]] ^ u[sigma[i][e]]) + v[b]; \
    v[d] = ROT( v[d] ^ v[a], 8 ); \
    v[c] += v[d]; \
    v[b] = ROT( v[b] ^ v[c], 7 );
```

The macro ROT defines 32-bit right rotation. G takes as parameter four internal state words a, b, c, and d, as well as an integer e (the value 2i in Section 3.1.2.2). G also uses the round counter variable i, ranging from 0 to 13. Here, the use of a macro simplifies the code, compared with an explicit repetition or with the definition of another function.

The iteration of 14 rounds is then simply coded as

```
for(i=0; i<14; ++i)
{
    /* column step */
    G( 0, 4, 8,12, 0 );
    G( 1, 5, 9,13, 2 );
    G( 2, 6,10,14, 4 );
    G( 3, 7,11,15, 6 );
    /* diagonal step */
    G( 0, 5,10,15, 8 );
    G( 1, 6,11,12,10 );
    G( 2, 7, 8,13,12 );
    G( 3, 4, 9,14,14 );
}
```

5.1.1.4 Finalization

As specified in Section 3.1.2.3, finalization forms the updated chaining value as a combination of the initial chaining value, the salt, and the final state after 14 rounds:

```
for(i=0; i<8; ++i) h[i] ^= v[i] ^ v[i+8] ^ s[i%4];
```

This closes the definition of the BLAKE-256 compression function.

5.1.1.5 Remark About Endianness

Data to be hashed is often passed to the programming interface as an array of bytes, rather than as an array of words. It is thus important to use the correct endianness when converting an array of bytes to an array of words. As defined in Section 1.4, BLAKE follows a big-endian convention, like SHA1 and the SHA2 family (but unlike the little-endian MD5 and BLAKE2, see Section 9.2.6). That is, our straightforward implementations could be modified as follows to receive a message block as an array of bytes:

```
void blake256_compress(
    uint32_t *h,
    const uint8_t *block,
    const uint32_t *s,
    const uint32_t *t )
{
    uint32_t v[16];
    uint32_t m[16];
    int i;

#define U8TO32_BIG(p) \
    (((uint32_t)((p)[0]) << 24) | ((uint32_t)((p)[1]) << 16) | \
     ((uint32_t)((p)[2]) << 8) | ((uint32_t)((p)[3])      ))

    for(i=0; i<16;++i) m[i] = U8TO32_BIG(block + i*4);
```

Here the macro `U8TO32_BIG` ensures the big-endian conversion of an array of bytes to an unsigned 32-bit word, regardless of the machine's endianness. On big-endian machines, one may simply write the following (violating strict aliasing, though):

```
for(i=0; i<16;++i) m[i] = ((uint32_t*)block)[i];
```

5.1.2 Other Languages

The definition of BLAKE is consistently simple across programming languages, since the compression function mainly consists in initializing variables and iterating G. As an illustration, we show code snippets from BLAKE-256 implementations in Go, Haskell, and Python:

The Go implementation of BLAKE-256 by Dmitry Chestnykh¹ defines G as

```
v0 += (m[si[0]] ^ cst[si[0+1]]) + v4
v12 = (v12^v0)<<(32-16) | (v12^v0)>>16
v8 += v12
v4 = (v4^v8)<<(32-12) | (v4^v8)>>12
v0 += (m[si[0+1]] ^ cst[si[0]]) + v4
v12 = (v12^v0)<<(32-8) | (v12^v0)>>8
```

¹ Available from <https://github.com/dchest/blake256/>.

```
v8 += v12
v4 = (v4^v8)<<(32-7) | (v4^v8)>>7
```

The Haskell implementation of BLAKE-256 by Kevin Cantu² defines G as

```
a' = a + b + (messageword (i2) `xor` constant (i2 + 1))
d' = (d `xor` a') `rotateR` rot0
c' = c + d'
b' = (b `xor` c') `rotateR` rot1
a'' = a' + b' + (messageword (i2 + 1) `xor` constant (i2))
d'' = (d' `xor` a'') `rotateR` rot2
c'' = c' + d''
b'' = (b' `xor` c'') `rotateR` rot3
```

The Python implementation of BLAKE-256 by Larry Bugbee³ defines G as

```
def G(a, b, c, d, i):
    va = v[a]
    vb = v[b]
    vc = v[c]
    vd = v[d]

    sri = SIGMA[round][i]
    sril = SIGMA[round][i+1]

    va = ((va + vb) + (m[sri] ^ cxx[sril])) & MASK
    x = vd ^ va
    vd = (x >> rot1) | ((x << (WORDBITS-rot1)) & MASK)
    vc = (vc + vd) & MASK
    x = vb ^ vc
    vb = (x >> rot2) | ((x << (WORDBITS-rot2)) & MASK)

    va = ((va + vb) + (m[sril] ^ cxx[sri])) & MASK
    x = vd ^ va
    vd = (x >> rot3) | ((x << (WORDBITS-rot3)) & MASK)
    vc = (vc + vd) & MASK
    x = vb ^ vc
    vb = (x >> rot4) | ((x << (WORDBITS-rot4)) & MASK)

    v[a] = va
    v[b] = vb
    v[c] = vc
    v[d] = vd
```

One can observe that, in those three languages, an implementation of G is only a translation of the specification's pseudocode to the language's syntax, and only consists in simple arithmetic operations; that is, no language-specific function or data structure is required for a fast implementation of BLAKE's core.

² Available from <https://github.com/killerswan/Haskell-BLAKE>.

³ Available from <http://www.seanet.com/~bugbee/crypto/blake/>.

5.2 Embedded Systems

Embedded systems may successfully compile straightforward portable C implementations of BLAKE, however these often offer suboptimal performance compared with a dedicated implementation. This section discusses how efficient implementations of BLAKE can be written for two common embedded architectures: AVR (8-bit) and ARMv6 (32-bit). It relies on the work of von Maurich [82] and Osvik [142] (for AVR) and of Schwabe, Yang, and Yang [160] (for ARMv6).

5.2.1 8-Bit AVR

Atmel AVR is a modified Harvard 8-bit reduced instruction set computing (RISC) architecture found in microcontroller processors, as used in a number of industrial applications, such as automotive systems. AVR processors are also found in consumer electronics, for example, in hand controllers of the Microsoft Xbox game console.

AVR processors have 32 registers of 8 bits, and 16-bit instructions operating on 8-bit operands. The instructions `rol` and `ror` perform left and right 1-bit rotation-through-carry, rather than shift of an arbitrary distance, as found in high-end processors. Since 8-bit processors seldom have high-bandwidth connections, memory footprint is generally a more critical factor than speed.

As a modified Harvard architecture, AVR uses flash memory for instructions and SRAM for data. Fetching constant data from flash therefore requires a special instruction named “load program memory” (`lpm`), which is slower than its counterpart fetching data from SRAM.

The assembly code snippets below are from the implementation by Ingo von Maurich.⁴ We refer to [7] for a complete documentation of the AVR instruction set.

5.2.1.1 Implementing the G Function

At round r , the G function consists in loading indices $\sigma_r(2i)$ and $\sigma_r(2i+1)$, loading the message and constant words at these positions, and performing a chain of additions, XORs, and rotations.

Loading indices, message, and constant words from the processor’s memories, as well as preparing the input, is a significant source of latency. The `lpm` instruction that loads an index from σ (residing in flash) has a latency of three cycles, and the `ld` instruction that loads bytes from the message and constant words (residing in RAM) has a latency of two cycles (as used with post-increment). This implies at least 38 cycles per G function to load the four message and constant words used. As

⁴ As published on <https://bitbucket.org/vmingo/blake256-avr-asm/>.

discussed in [82], this figure may be slightly reduced by preloading the σ table to SRAM.

Addition of 32-bit words is done using the `add` instruction followed by `adc` with carry (instruction `adc`), which both take one cycle per instruction, that is, four cycles; for example, adding d to c is done with the following code:

```
add c_lo,d_lo
adc c_ml,d_ml
adc c_mh,d_mh
adc c_hi,d_hi
```

In this code snippet (and in subsequent ones) a 32-bit variable is represented as four bytes; for example, the c variable of G is represented as the bytes `c_lo`, `c_ml`, `c_mh`, and `c_hi`, from the least to most significant byte.

The XOR between two words is simply the XOR between each pair of bytes at the same position. This is done with the `eor` instruction as follows:

```
eor b_lo,c_lo
eor b_ml,c_ml
eor b_mh,c_mh
eor b_hi,c_hi
```

Rotation by 16 bits is the most efficient of all rotations in G on AVR, as it takes only three cycles by swapping the two 16-bit halves of a word, using the `movw` instruction:

```
movw temp,d_hi
movw d_hi,d_ml
movw d_ml,temp
```

Rotation by 8 bits needs to swap bytes rather than 16-bit words, which takes five cycles:

```
mov temp2,d_mh
mov d_mh,d_hi
mov d_hi,d_lo
mov d_lo,d_ml
mov d_ml,temp2
```

Rotation by 7 bits is relatively simple, as it is equivalent to a swap of bytes followed by a 1-bit rotation. Thus only one rotate instruction is needed for each byte. As `rol` has a latency of one cycle, this gives a total of 10 cycles:

```
mov temp2,b_mh
mov b_mh,b_hi
mov b_hi,b_lo
mov b_lo,b_ml
mov b_ml,temp2
lsl b_lo
rol b_ml
rol b_mh
rol b_hi
adc b_lo,temp
```

Rotation by 12 bits is the most expensive, since 12 is four units away from a multiple of four. This implies four rotations per byte, plus an add-with-carry to move the most significant bit to the first position. It is more efficient to implement the 12-bit rotation as a 16-bit rotation followed by a 4-bit inverse rotation, since rotating by 16 bits is more efficient than by 8 bits. This gives a total of 23 cycles:

```

movw temp,b_hi
movw b_hi,b_ml
movw b_ml,temp
clr temp2
lsl b_lo
rol b_ml
rol b_mh
rol b_hi
adc b_lo,temp2
lsl b_lo
rol b_ml
rol b_mh
rol b_hi
adc b_lo,temp2
lsl b_lo
rol b_ml
rol b_mh
rol b_hi
adc b_lo,temp2
lsl b_lo
rol b_ml
rol b_mh
rol b_hi
adc b_lo,temp2

```

Similar techniques can be used to implement the rotations of BLAKE-512: rotations by 32 and 16 bits are swaps of 16-bit words; rotation by 25 bits is a rotation by 24 bits (swap of bytes) followed by a 1-bit rotation; rotation by 11 bits is a swap of bytes followed by a 3-bit rotation. Since words in BLAKE-512 are 64-bit, instead of 32-bit for BLAKE-256, many more AVR assembly instructions are necessary.

5.2.2 32-Bit ARM

BLAKE is easily implemented on ARM architectures as a chain of XORs, additions, and rotations, using ARM or Thumb-2 instructions. Optimizations for the ARMv6 architecture (as found in the ARM11 series of processors) were presented by Schwabe, Yang, and Yang at the Third SHA3 Candidate Conference [160], and are summarized below.

The first optimization technique is the use of a “rotated” representation of part of the state to exploit ARM’s rotate-and-add capability: the ARMv6 architecture allows to make the operation $x := y \odot (z \ggg n)$ for essentially the same cost as $x := y \odot z$, where $\odot$ is an arithmetic operation. However, one cannot directly use this

feature to implement BLAKE, since G instead computes (say) $b := (b \oplus c) \ggg 12$. The trick of Schwabe, Yang, and Yang is to set $b := b \oplus c$ rather than $b := (b \oplus c) \ggg 12$, and then to set $a := a + (b \ggg 12)$ rather than $a := a + b$. The value b is thus stored in a rotated state, and the subsequent values are adapted to use operands in the correct rotated state. A similar technique is used for all rotations.

For example, whereas a direct implementation of G_0 in the first round would implement the following pseudocode:

```

v0 := v0 + v4
v0 := v0 + (m0 ⊕ u1)
v12 := (v12 ⊕ v0) ≫≫ 16
v8 := v8 + v12
v4 := (v4 ⊕ v8) ≫≫ 12
v0 := v0 + v4
v0 := v0 + (m1 ⊕ u0)
v12 := (v12 ⊕ v0) ≫≫ 8
v8 := v8 + v12
v4 := (v4 ⊕ v8) ≫≫ 7

```

the code by Schwabe, Yang, and Yang rather implements this modified version:

```

v0 := v0 + (v4 ≫≫ 0)
v0 := v0 + ((m0 ⊕ u1) ≫≫ 0)
v12 := v12 ⊕ (v0 ≫≫ 0)
v8 := v8 + (v12 ≫≫ 16)
v4 := v4 ⊕ (v8 ≫≫ 0)
v0 := v0 + (v4 ≫≫ 12)
v0 := v0 + ((m1 ⊕ u0) ≫≫ 0)
v12 := v12 ⊕ (v0 ≫≫ 16)
v8 := v8 + (v12 ≫≫ 24)
v4 := v4 ⊕ (v8 ≫≫ 20)

```

The second optimization technique aims to minimize the load and store operations (ARMv6 only has 14 usable 32-bit registers, whereas BLAKE-256 uses $16 + 16 + 16$ words for the message, constants, and internal state). This is achieved by first pushing the 16 constants on the stack, then at each compression loop pushing the 16 message words (saving a register containing the pointer to the message), and by optimizing the register allocation and the instruction scheduling.

We refer to the article of Schwabe, Yang, and Yang for details of their implementation [160], and to their “arm11” implementation included in SUPERCOP.

5.3 Vectorized Implementation Principle

A straightforward C implementation of BLAKE, such as the example in Section 5.1.1, uses one XOR instruction to perform the first $d \oplus a$ operation in G_0 then three other XOR instructions to perform the first $d \oplus a$ operations in G_1 , G_2 , and G_3 . Vectorized instructions allow the processor to perform those four operations with a single instruction: for BLAKE-256, a first 128-bit register is initialized with the concatenation of the four d operands, a second 128-bit register is initialized with the concatenation of the four a operands, then a vectorized XOR instruction is executed with those two registers as operands. Similarly, all operations in G can be vectorized. A vectorized implementation of BLAKE-256 thus computes the column step of the round function by working with four 128-bit registers, each register holding one variable of each column. The diagonal step is computed by rotating the state's rows—that is, the 128-bit registers—in order to apply the same operations as in the computation of the column step (see the observation at the end of Section 3.1.2.2).

The subsequent sections discuss the concrete usage of vectorized instructions available in recent microprocessors.

5.4 Vectorized Implementation with SSE Extensions

This section discusses the use of vectorized instructions available in the SSE family of single-instruction multiple-data (SIMD) instructions.

5.4.1 Streaming SIMD Extensions 2 (SSE2)

Intel's first set of instructions supporting all 4-way 32-bit SIMD operations necessary to implement BLAKE-256 is the Streaming SIMD Extensions 2 (SSE2) set. SSE2 includes vector instructions on 4×32 -bit words for integer addition, XOR, word-wise left and right shift, as well as word shuffle. This is all one needs to implement BLAKE-256's round function, as rotations can be simulated by two shifts and an XOR. BLAKE-512 can also use SSE2 (though with less benefit than BLAKE-256), thanks to the support of 2-way 64-bit SIMD operations.

SSE2 instructions operate on 128-bit XMM registers, rather than 32- or 64-bit general-purpose registers. In 64-bit mode (x86-64 a/k/a amd64 architecture) 16 XMM registers are available, whereas only eight are available in 32-bit mode (x86). The SSE2 instructions are supported by all recent Intel and AMD desktop and laptop processors (Intel's Xeon, Celeron, Core i7, etc.; AMD's Athlon 64, Opteron, etc.) as well as by common low-voltage processors, as found in netbooks (Intel's Atom; VIA's C7 and Nano).

In addition to inline assembly, C(++) programmers can use SSE2 instructions via *intrinsic functions* (or simply *intrinsics*), which are extensions built into most

compilers. Intrinsics allow to enforce the use of SSE2 instructions by the processor, enable the use of C syntax and variables instead of assembly language and hardware registers, and let the compiler optimize instruction scheduling for better performance. Tables 5.1 and 5.2 show intrinsics corresponding to some assembly mnemonics used to implement BLAKE-256 and BLAKE-512, respectively. A complete reference to SSE2 intrinsics for the Intel compiler can be found in [86] (these are also supported in `gcc`).

Table 5.1 Main SSE2 instructions used to implement BLAKE-256.

Assembly	Intrinsic	Description
<code>padd</code>	<code>_mm_add_epi32</code>	4-way 32-bit integer addition
<code>pxor</code>	<code>_mm_xor_si128</code>	XOR of two 128-bit registers
<code>psll</code>	<code>_mm_srli_epi32</code>	4-way 32-bit left-shift
<code>psrl</code>	<code>_mm_srli_epi32</code>	4-way 32-bit right-shift
<code>pshufd</code>	<code>_mm_shuffle_epi32</code>	4-way 32-bit word shuffle

Table 5.2 Main SSE2 instructions used to implement BLAKE-512.

Assembly	Intrinsic	Description
<code>paddq</code>	<code>_mm_add_epi64</code>	2-way 64-bit integer addition
<code>pxor</code>	<code>_mm_xor_si128</code>	XOR of two 128-bit registers
<code>psllq</code>	<code>_mm_srli_epi64</code>	2-way 64-bit left-shift
<code>psrlq</code>	<code>_mm_srli_epi64</code>	2-way 64-bit right-shift
<code>punpcklqdq</code>	<code>_mm_unpacklo_epi64</code>	interleave high 64-bit words
<code>punpckhqdq</code>	<code>_mm_unpackhi_epi64</code>	interleave low 64-bit words

5.4.2 Implementing BLAKE-256 with SSE2

To help understand the principle of SIMD implementations of BLAKE, we first present a simple SSE2 implementation of BLAKE-256's column step, similar to the `sse2` implementation in SUPERCOP [28]. The v internal state is stored in four XMM registers defined as `_mm128i` type and aliased `row1`, `row2`, `row3`, and `row4`. These respectively correspond to the first four rows of the 4×4 array representation described in Section 3.1.2.1.

First, one initializes a 128-bit XMM register aliased `buf1` with the four message words $m_{\sigma_r[2i]}$. Another XMM register aliased `buf2` is initialized with the four constants $u_{\sigma_r[2i+1]}$. Then `buf1` and `buf2` are XORed together into `buf1`, and the result is added to `row1`:

```
buf1 = _mm_set_epi32(m[sig[r][6]], m[sig[r][4]],
```

```

        m[sig[r][2]], m[sig[r][0]]);
buf2 = _mm_set_epi32(u[sig[r][7]], u[sig[r][5]],
        u[sig[r][3]], u[sig[r][1]]);
buf1 = _mm_xor_si128( buf1, buf2 );
row1 = _mm_add_epi32( row1, buf1 );

```

One can already prepare the XMM register containing the XOR of the permuted message and constants for the next message input:

```

buf1 = _mm_set_epi32(m[sig[r][7]], m[sig[r][5]],
        m[sig[r][3]], m[sig[r][1]]);
buf2 = _mm_set_epi32(u[sig[r][6]], u[sig[r][4]],
        u[sig[r][2]], u[sig[r][0]]);
buf1 = _mm_xor_si128( buf1, buf2 );

```

The subsequent operations are only vectorized XOR, integer addition, and shifts:

```

row1 = _mm_add_epi32( row1, row2 );
row4 = _mm_xor_si128( row4, row1 );
row4 = _mm_xor_si128( _mm_srli_epi32( row4, 16 ),
        _mm_slli_epi32( row4, 16 ));
row3 = _mm_add_epi32( row3, row4 );
row2 = _mm_xor_si128( row2, row3 );
row2 = _mm_xor_si128( _mm_srli_epi32( row2, 12 ),
        _mm_slli_epi32( row2, 20 ));
row1 = _mm_add_epi32( row1, buf1 );
row1 = _mm_add_epi32( row1, row2 );
row4 = _mm_xor_si128( row4, row1 );
row4 = _mm_xor_si128( _mm_srli_epi32( row4, 8 ),
        _mm_slli_epi32( row4, 24 ));
row3 = _mm_add_epi32( row3, row4 );
row2 = _mm_xor_si128( row2, row3 );
row2 = _mm_xor_si128( _mm_srli_epi32( row2, 7 ),
        _mm_slli_epi32( row2, 25 ));

```

At the end of a column step, each register is word-rotated to perform the diagonal step as a column step on the rotated state, as observed in Section 3.1.2.2.

```

row2 = _mm_shuffle_epi32( row2, _MM_SHUFFLE(0,3,2,1) );
row3 = _mm_shuffle_epi32( row3, _MM_SHUFFLE(1,0,3,2) );
row4 = _mm_shuffle_epi32( row4, _MM_SHUFFLE(2,1,0,3) );

```

The `_mm_shuffle_epi32` intrinsic takes as second argument an immediate value (a constant integer literal) expressed as a predefined macro. We refer to [86, p.65] for details of the `_MM_SHUFFLE` macro.

5.4.3 Implementing BLAKE-512 with SSE2

Since XMM registers are only 128-bit, BLAKE-512 can only use 2-way SIMD operations over vectors of two 64-bit words, whereas BLAKE-256 can use 4-way SIMD operations over vector of four 32-bit words. In the code below (similar to the `sse2` implementation in SUPERCOP), the `v` internal state is stored in eight XMM

registers defined as `__m128i` type and aliased `row1a`, `row1b`, `row2a`, `row2b`, `row3a`, `row3b`, `row4a`, and `row4b`. These correspond to each of the two halves of each row of the state.

The implementation of round `r` starts with the loading of the permuted message and constant words for the first two instances of `G` of the column step (G_0 and G_1):

```
buf2a = _mm_set_epi64( ( __m64 )m[sig[r][ 2]],
                       ( __m64 )m[sig[r][ 0]] );
buf1a = _mm_set_epi64( ( __m64 )u[sig[r][ 3]],
                       ( __m64 )u[sig[r][ 1]] );
```

The pairs of message words and constants are then XORed together, and the result is added to the vector of the two words in the first two columns of the first row of the state:

```
buf1a = _mm_xor_si128( buf1a, buf2a );
row1a = _mm_add_epi64( _mm_add_epi64( row1a, buf1a ), row2a );
```

The subsequent operations are vectorized XOR, integer addition, and shifts over the words of the first two columns:

```
row1a = _mm_add_epi64( _mm_add_epi64( row1a, buf1a ), row2a );
row4a = _mm_xor_si128( row4a, row1a );
row4a = _mm_xor_si128( _mm_srli_epi64( row4a, 32 ),
                      _mm_slli_epi64( row4a, 32 ) );
row3a = _mm_add_epi64( row3a, row4a );
row2a = _mm_xor_si128( row2a, row3a );
row2a = _mm_xor_si128( _mm_srli_epi64( row2a, 25 ),
                      _mm_slli_epi64( row2a, 39 ) );
```

Similar code is used for the second part of `G`, still over the words of the first two columns:

```
buf2a = _mm_set_epi64( ( __m64 )m[sig[r][ 3]],
                       ( __m64 )m[sig[r][ 1]] );
buf1a = _mm_set_epi64( ( __m64 )u[sig[r][ 2]],
                       ( __m64 )u[sig[r][ 0]] );
buf1a = _mm_xor_si128( buf1a, buf2a );
row1a = _mm_add_epi64( _mm_add_epi64( row1a, buf1a ), row2a );
row4a = _mm_xor_si128( row4a, row1a );
row4a = _mm_xor_si128( _mm_srli_epi64( row4a, 16 ),
                      _mm_slli_epi64( row4a, 48 ) );
row3a = _mm_add_epi64( row3a, row4a );
row2a = _mm_xor_si128( row2a, row3a );
row2a = _mm_xor_si128( _mm_srli_epi64( row2a, 11 ),
                      _mm_slli_epi64( row2a, 53 ) );
```

To compute G_2 and G_3 , only the indices change:

```
buf2a = _mm_set_epi64( ( __m64 )m[sig[r][ 6]],
                       ( __m64 )m[sig[r][ 4]] );
buf1a = _mm_set_epi64( ( __m64 )u[sig[r][ 7]],
                       ( __m64 )u[sig[r][ 5]] );
buf1a = _mm_xor_si128( buf1a, buf2a );
row1b = _mm_add_epi64( _mm_add_epi64( row1b, buf1a ), row2b );
```

```

row4b = _mm_xor_si128( row4b, row1b );
row4b = _mm_xor_si128( _mm_srli_epi64( row4b, 32 ),
    _mm_slli_epi64( row4b, 32 ) );
row3b = _mm_add_epi64( row3b, row4b );
row2b = _mm_xor_si128( row2b, row3b );
row2b = _mm_xor_si128( _mm_srli_epi64( row2b, 25 ),
    _mm_slli_epi64( row2b, 39 ) );
buf2a = _mm_set_epi64( ( __m64 )m[sig[r][ 7]],
    ( __m64 )m[sig[r][ 5]] );
buf1a = _mm_set_epi64( ( __m64 )u[sig[r][ 6]],
    ( __m64 )u[sig[r][ 4]] );
buf1a = _mm_xor_si128( buf1a, buf2a );
row1b = _mm_add_epi64( _mm_add_epi64( row1b, buf1a ), row2b );
row4b = _mm_xor_si128( row4b, row1b );
row4b = _mm_xor_si128( _mm_srli_epi64( row4b, 16 ),
    _mm_slli_epi64( row4b, 48 ) );
row3b = _mm_add_epi64( row3b, row4b );
row2b = _mm_xor_si128( row2b, row3b );
row2b = _mm_xor_si128( _mm_srli_epi64( row2b, 11 ),
    _mm_slli_epi64( row2b, 53 ) );

```

Diagonalization can be performed with only assignment and unpacking instructions:

```

buf1a = row4a;
buf2a = row2a;
row4a = row3a;
row3a = row3b;
row3b = row4a;
row4a = _mm_unpackhi_epi64( row4b,
    _mm_unpacklo_epi64( buf1a, buf1a ) );
row4b = _mm_unpackhi_epi64( buf1a,
    _mm_unpacklo_epi64( row4b, row4b ) );
row2a = _mm_unpackhi_epi64( row2a,
    _mm_unpacklo_epi64( row2b, row2b ) );
row2b = _mm_unpackhi_epi64( row2b,
    _mm_unpacklo_epi64( buf2a, buf2a ) );

```

Then the diagonal step is similar to the column step, with adapted indices:

```

/* diagonal step for G4 and G5 */
buf2a = _mm_set_epi64( ( __m64 )m[sig[r][10]],
    ( __m64 )m[sig[r][ 8]] );
buf1a = _mm_set_epi64( ( __m64 )u[sig[r][11]],
    ( __m64 )u[sig[r][ 9]] );
buf1a = _mm_xor_si128( buf1a, buf2a );
row1a = _mm_add_epi64( _mm_add_epi64( row1a, buf1a ), row2a );
row4a = _mm_xor_si128( row4a, row1a );
row4a = _mm_xor_si128( _mm_srli_epi64( row4a, 32 ),
    _mm_slli_epi64( row4a, 32 ) );
row3a = _mm_add_epi64( row3a, row4a );
row2a = _mm_xor_si128( row2a, row3a );
row2a = _mm_xor_si128( _mm_srli_epi64( row2a, 25 ),
    _mm_slli_epi64( row2a, 39 ) );
buf2a = _mm_set_epi64( ( __m64 )m[sig[r][11]],
    ( __m64 )m[sig[r][ 9]] );

```

```

buf1a = _mm_set_epi64( ( __m64 )u[sig[r][10]],
                       ( __m64 )u[sig[r][ 8]] );
buf1a = _mm_xor_si128( buf1a, buf2a );
row1a = _mm_add_epi64( _mm_add_epi64( row1a, buf1a ), row2a );
row4a = _mm_xor_si128( row4a, row1a );
row4a = _mm_xor_si128( _mm_srli_epi64( row4a, 16 ),
                       _mm_slli_epi64( row4a, 48 ) );
row3a = _mm_add_epi64( row3a, row4a );
row2a = _mm_xor_si128( row2a, row3a );
row2a = _mm_xor_si128( _mm_srli_epi64( row2a, 11 ),
                       _mm_slli_epi64( row2a, 53 ) );

/* diagonal step for G6 and G7 */
buf2a = _mm_set_epi64( ( __m64 )m[sig[r][14]],
                       ( __m64 )m[sig[r][12]] );
buf1a = _mm_set_epi64( ( __m64 )u[sig[r][15]],
                       ( __m64 )u[sig[r][13]] );
buf1a = _mm_xor_si128( buf1a, buf2a );
row1b = _mm_add_epi64( _mm_add_epi64( row1b, buf1a ), row2b );
row4b = _mm_xor_si128( row4b, row1b );
buf2a = _mm_set_epi64( ( __m64 )m[sig[r][15]],
                       ( __m64 )m[sig[r][13]] );
row4b = _mm_xor_si128( _mm_srli_epi64( row4b, 32 ),
                       _mm_slli_epi64( row4b, 32 ) );
row3b = _mm_add_epi64( row3b, row4b );
row2b = _mm_xor_si128( row2b, row3b );
buf1a = _mm_set_epi64( ( __m64 )u[sig[r][14]],
                       ( __m64 )u[sig[r][12]] );
row2b = _mm_xor_si128( _mm_srli_epi64( row2b, 25 ),
                       _mm_slli_epi64( row2b, 39 ) );
buf1a = _mm_xor_si128( buf1a, buf2a );
row1b = _mm_add_epi64( _mm_add_epi64( row1b, buf1a ), row2b );
row4b = _mm_xor_si128( row4b, row1b );
row4b = _mm_xor_si128( _mm_srli_epi64( row4b, 16 ),
                       _mm_slli_epi64( row4b, 48 ) );
row3b = _mm_add_epi64( row3b, row4b );
row2b = _mm_xor_si128( row2b, row3b );
row2b = _mm_xor_si128( _mm_srli_epi64( row2b, 11 ),
                       _mm_slli_epi64( row2b, 53 ) );

```

Finally the state is shuffled back to its initial configuration:

```

buf1a = row3a;
row3a = row3b;
row3b = buf1a;
buf1a = row2a;
buf2a = row4a;
row2a = _mm_unpackhi_epi64( row2b,
                             _mm_unpacklo_epi64( row2a, row2a ) );
row2b = _mm_unpackhi_epi64( buf1a,
                             _mm_unpacklo_epi64( row2b, row2b ) );
row4a = _mm_unpackhi_epi64( row4a,
                             _mm_unpacklo_epi64( row4b, row4b ) );
row4b = _mm_unpackhi_epi64( row4b,
                             _mm_unpacklo_epi64( buf2a, buf2a ) );

```

5.4.4 Implementations with SSSE3 and SSE4.1

The SSE2 instruction set was followed by the SSE3, SSSE3, SSE4.1, and SSE4.2 extensions [86], which brought additional instructions to operate on XMM registers. It was found that some of those instructions could be of benefit to BLAKE, and implementations exploiting SSSE3 and SSE4.1 instructions have been submitted to SUPERCOP by Samuel Neves:

- The `ssse3` implementation of BLAKE-256 uses the `pshufb` instruction (intrinsic `_mm_shuffle_epi8`) to perform rotations of 16 and 8 bits, as well as the initial conversion of the message from little-endian to big-endian byte order, since both can be expressed as byte shuffles (in the `sse2` implementations rotations were implemented as two shifts and an XOR). This brings a significant speedup on Core 2 based on the Penryn microarchitecture, which introduced a dedicated shuffle unit to complete `pshufb` within one micro-operation, against four on the first Core 2 chips [53].
- The `sse41` implementation of BLAKE-256 uses the `pblendw` instruction (`_mm_blend_epi16`) in combination with SSE2's `pshufd`, `pslldq`, and others to load m and u words according to the σ permutations without using table lookups.

In general, the `ssse3` implementation is faster than `sse2`, and `sse41` is faster than both.⁵ For example, the 20110708 measurements of SUPERCOP on `sandy0` (a machine equipped with a Sandy Bridge Core i7, without AVX activated) reported `sse41` as the fastest implementation of BLAKE-256, with the `ssse3` and `sse2` implementations being, respectively, 4% and 24% slower.

The SUPERCOP software included the `vect128` and `vect128-mmhack` implementations of BLAKE-256 by Leurent, which are almost as fast as the `sse41` implementation. The main singularity of Leurent's code is its implementation of the σ permutations: `vect128` "byte-slices" each message word across four XMM registers and uses the `pshufb` instruction to reorder them according to σ ; `vect128-mmhack` instead uses MMX and general-purpose registers to store and unpack the message words in the correct order into XMM registers.

5.5 Vectorized Implementation with AVX2 Extensions

In 2008 Intel announced the Advanced Vector Extensions (AVX), introducing 256-bit-wide vector instructions. These improve on the previous SSE extensions, which work on 128-bit XMM registers. In addition to SIMD operations extending SSE's capabilities from 128- to 256-bit width, AVX brings to implementers non-destructive operations with a 3- and 4-operand syntax (including for legacy 128-bit

⁵ See the benchmarks results on <http://bench.cr.yp.to/results-sha3.html>.

SIMD extensions), as well as relaxed memory alignment constraints, compared with SSE.

AVX operates on 256-bit SIMD registers called YMM, divided into two 128-bit lanes, such that the low lanes (lower 128 bits) are aliased to the respective 128-bit XMM registers. Most instructions work “in-lane”; that is, each source element is applied only to other elements of the same lane. Some more expensive “cross-lane” instructions do exist, most notably shuffles.

AVX2 is an extension of AVX announced in 2011 that promotes most of the 128-bit SIMD integer instructions to 256-bit capabilities. AVX2 supports 4-way 64-bit XOR, integer addition, and shifts, thus enabling SIMD implementations of BLAKE-512. AVX2 also includes instructions to perform any-to-any permutation of words over a 256-bit register and vectorized table lookup to load elements in memory to YMM registers (see the instructions `vperm*` and `vpgatherd*` in Section 5.5.1).

AVX is supported by Intel processors based on the Sandy Bridge microarchitecture (and future ones). The first processors commercialized were Core i7 and Core i5 in January 2011. AVX2 was introduced in Intel’s Haswell 22 nm architecture, which was released in June 2013.

All the code presented in this section was written by Samuel Neves in the context of our joint project on vectorized implementations of BLAKE [130], as presented at the Third SHA3 Candidate Conference. The correctness of the implementations was tested with Intel’s Software Development Emulator, and the Yasm assembler compiled them.

5.5.1 Relevant AVX2 Instructions

We focus on a small subset of the AVX2 instructions, presenting for each a brief explanation of what it does. For a better understanding, the most sophisticated instructions are also described with an equivalent description in C syntax using only general-purpose registers. Table 5.3 summarizes the main instructions along with their C intrinsic functions.

5.5.1.1 ARX SIMD

To implement add-rotate-XOR (ARX) algorithms with AVX2, the following instructions are available: `vpaddq` for 8-way 32-bit integer addition, `vpaddq` for 4-way 64-bit integer addition, `vpxor` for 256-bit-wide XOR, and `vpsllvd`, `vpsrlvd`, `vpsllvq`, and `vpsrlvq` for variable left and right shift of 32- and 64-bit words (that is, each word within a YMM register may be shifted by a different value).

Table 5.3 Intrinsics of main AVX2 instructions useful to implement BLAKE.

Assembly	Intrinsic	Description
<code>vpaddq</code>	<code>_mm256_add_epi64</code>	4-way 64-bit integer addition
<code>vpaddq</code>	<code>_mm256_add_epi64</code>	4-way 64-bit integer addition
<code>vpxor</code>	<code>_mm256_xor_si256</code>	XOR of the two 256-bit values
<code>vpsllvd</code>	<code>_mm256_sllv_epi32</code>	8-way 32-bit left-shift
<code>vpsllvq</code>	<code>_mm256_sllv_epi64</code>	4-way 64-bit left-shift
<code>vpsrlvd</code>	<code>_mm256_srlv_epi32</code>	8-way 32-bit right-shift
<code>vpsrlvq</code>	<code>_mm256_srlv_epi64</code>	4-way 64-bit right-shift
<code>vpermd</code>	<code>_mm256_permute8x32_epi32</code>	shuffle of the eight 32-bit words
<code>vpermq</code>	<code>_mm256_permute4x64_epi64</code>	shuffle of the four 64-bit words
<code>vpgatherdd</code>	<code>_mm256_i32gather_epi32</code>	8-way 32-bit table lookup
<code>vpgatherdq</code>	<code>_mm256_i32gather_epi64</code>	4-way 64-bit table lookup

5.5.1.2 Cross-Lane Permuters

AVX2 provides instructions to realize any permutation of 32- and 64-bit words within a YMM register, through the following instructions: `vpermd` shuffles 32-bit words of a full YMM register across lanes using two YMM registers as inputs: one as source, the other as the permutation's indices:

```
uint32_t a[8], b[8], c[8];
for (i=0; i < 8; ++i) c[i] = a[b[i]];
```

`vpermq` is similar to `vpermd` but shuffles 64-bit words and takes an immediate operand instead as the permutation:

```
uint64_t a[4], c[4]; int b;
for (i=0; i < 4; ++i) c[i] = a[(b >> (2*i)) % 4];
```

5.5.1.3 Vectorized Table Lookups

The “gather” instructions are among the most remarkable of the AVX2 extensions: `vpgatherdd` performs eight table lookups in parallel, as in the code below:

```
uint8_t *b; uint32_t scale, idx[8], c[8];
for (i=0; i < 8; ++i) c[i] = *(uint32_t) (b + idx[i]*scale);
```

`vpgatherdq` is quite similar to `vpgatherdd`, but works on four 64-bit words:

```
uint8_t *b; uint32_t scale, idx[4]; uint64_t c[4];
for (i=0; i < 4; ++i) c[i] = *(uint32_t) (b + idx[i]*scale);
```

5.5.1.4 Insertion and Extraction

AVX2 offers a number of instructions to manipulate words and YMM registers, of which the most relevant for us are the following:

The `vpinsrd` instruction (already in AVX), also accessible by its intrinsic `_mm_insert_epi32`, inserts one 32-bit word into a specified position in an XMM register, as follows:

```
uint32_t c[8], a; int imm;
c[imm] = a;
```

`vpblendd` (`_mm256_blend_epi32`), similar to the SSE4.1 `pblendw` instruction, permits the selection of words from two different sources according to an immediate index, placing them in a third destination register:

```
uint32_t a[8], b[8], c[8]; int sel;
for(i=0; i < 8; ++i)
    if((sel>>i)&1) c[i] = b[i];
    else c[i] = a[i];
```

`vextracti128` (`_mm256_extracti128_si256`) as well as `vinseri128` (`_mm256_inseri128_si256`) extract and insert an XMM register into the lower or upper halves of a YMM register. `vextracti128` is equivalent to

```
uint32_t a[8], c[4]; int imm;
for(i=0; i < 4; ++i) c[i] = a[i + 4*imm];
```

while `vinseri128` is equivalent to

```
uint32_t a[8], b[4], c[8]; int imm;
for(i=0; i < 8; ++i) c[i] = a[i];
for(i=0; i < 4; ++i) c[i+4*imm] = b[i];
```

5.5.2 Implementing BLAKE-512 with AVX2

This section first presents a basic SIMD implementation of BLAKE-512, using AVX2's 4-way 64-bit SIMD instructions in the same way that BLAKE-256 uses SSE2's 4-way 32-bit instructions. We then discuss optimizations exploiting instructions specific to AVX2. For ease of understanding, we present C code using intrinsics for AVX2 instructions, followed by excerpts of an assembly implementation.

5.5.2.1 Basic SIMD C Implementation

AVX2 provides instructions to write a straightforward SIMD implementation of BLAKE-512 similar to the `sse2` implementation of BLAKE-256 in Section 5.4.2, except that 256-bit YMM registers are used to hold four 64-bit words instead of 128-bit XMM registers being used to hold four 32-bit words.

The code below implements the column step of BLAKE-512's round function, that is, it computes the first four instances of `G` in parallel. The 4×4 state of 64-bit words is stored in four YMM registers defined as `__m256i` type and aliased `row1`, `row2`, `row3`, and `row4`.

```

buf1 = _mm256_set_epi64x(m[sig[r][6]], m[sig[r][4]],
                        m[sig[r][2]], m[sig[r][0]]);
buf2 = _mm256_set_epi64x(u[sig[r][7]], u[sig[r][5]],
                        u[sig[r][3]], u[sig[r][1]]);
buf1 = _mm256_xor_si256(buf1, buf2);
row1 = _mm256_add_epi64(_mm256_add_epi64(row1, buf1), row2);
row4 = _mm256_xor_si256(row4, row1);
row4 = _mm256_xor_si256(_mm256_srli_epi64(row4, 32),
                        _mm256_slli_epi64(row4, 32));
row3 = _mm256_add_epi64(row3, row4);
row2 = _mm256_xor_si256(row2, row3);
buf1 = _mm256_set_epi64x(u[sig[r][6]], u[sig[r][4]],
                        u[sig[r][2]], u[sig[r][0]]);
buf2 = _mm256_set_epi64x(m[sig[r][7]], m[sig[r][5]],
                        m[sig[r][3]], m[sig[r][1]]);
buf1 = _mm256_xor_si256(buf1, buf2);
row2 = _mm256_xor_si256(_mm256_srli_epi64(row2, 25),
                        _mm256_slli_epi64(row2, 39));
row1 = _mm256_add_epi64(_mm256_add_epi64(row1, buf1), row2);
row4 = _mm256_xor_si256(row4, row1);
row4 = _mm256_xor_si256(_mm256_srli_epi64(row4, 16),
                        _mm256_slli_epi64(row4, 48));
row3 = _mm256_add_epi64(row3, row4);
row2 = _mm256_xor_si256(row2, row3);
row2 = _mm256_xor_si256(_mm256_srli_epi64(row2, 11),
                        _mm256_slli_epi64(row2, 53));
row2 = _mm256_permute4x64_epi64(row2, _MM_SHUFFLE(0, 3, 2, 1));
row3 = _mm256_permute4x64_epi64(row3, _MM_SHUFFLE(1, 0, 3, 2));
row4 = _mm256_permute4x64_epi64(row4, _MM_SHUFFLE(2, 1, 0, 3));

```

A simple optimization consists in implementing the rotation by 32 bits using the `vpshufd` instruction, which implements “in-lane” shuffle of 32-bit words. That is, the line

```

row4 = _mm256_xor_si256(_mm256_srli_epi64(row4, 32),
                        _mm256_slli_epi64(row4, 32));

```

can be replaced by

```

row4 = _mm256_shuffle_epi32(row4, _MM_SHUFFLE(2, 3, 0, 1));

```

Similarly, the rotations by 16 bits can be implemented using `vpshufb` in a similar fashion as in the `ssse3` implementation (see Section 5.4.4):

```

row4 = _mm256_shuffle_epi8(row4, r16);

```

where `r16` is the alias of a YMM register containing the index values for the byte of `row4` at its respective lane and position.

5.5.2.2 Parallelized Message Loading

As observed in Section 5.5.1, the `vpgatherdq` instruction can be used to load words from arbitrary memory addresses. To load message words according to the σ_r permutation, one would thus write the following C code:

```

__m256i m0 = _mm_i32gather_epi64(m, sigma[r][0], 8);
__m256i m1 = _mm_i32gather_epi64(m, sigma[r][1], 8);
__m256i m2 = _mm_i32gather_epi64(m, sigma[r][2], 8);
__m256i m3 = _mm_i32gather_epi64(m, sigma[r][3], 8);

```

where the `sigma[r][i]`'s are of `__m128i` type, and where each 32-bit word holds an index of the permutation. As each `sigma[r][i]` holds four indices, `sigma[r][0]` to `sigma[r][3]` hold the 16 indices of the σ_r permutation.

Such a sequential implementation of four `vpgatherdq`'s is expected to only add an extra latency equivalent to that of a single `vpgatherdq`, since the subsequent instructions only depend on the first call, and therefore may not stall while the three other loads are executed. This assumes that `vpgatherdq` is pipelined, and that the subsequent loads can start one cycle after the first one.

5.5.2.3 Message Caching

As discussed in the previous section, loading the message words according to the σ permutations takes a considerable number of cycles, compared with an arithmetic operation. A potential optimization consists in eliminating redundancies due to the reuse of six of the ten σ permutations, in the first and last six rounds; that is, a same permuted message is used twice for the permutations $\sigma_0, \sigma_1, \dots, \sigma_5$. An implementation strategy could thus be:

1. in rounds 0 to 5: compute the permuted messages, and store the result in memory (preferably in unused YMM registers);
2. in rounds 6 to 9: compute the permuted messages without storing the result;
3. in rounds 10 to 15: do not compute the permuted messages, but rather use the registers set in step 1.

To save six vectorized XORs, one should store the permuted message already XORed with the constants, as the latter are reused as well.

The above strategy would require 24 YMM registers only to store the permuted message—as a BLAKE-512 message block is 1,024-bit, occupying four YMM registers—whereas only 16 are available and at least six are necessary to implement the round function. The 24 YMM registers represent 768 bytes of memory, which fits comfortably in most processors' L1 cache, but induces a potential performance penalty due to the latency of L1 accesses.

Eventually, it turned out that message caching does not speed-up implementations. However we deemed interesting to report this optimization attempt, as it may be useful for other algorithms, or for BLAKE on other platforms.

5.5.2.4 Assembly Code Excerpts

The G function of BLAKE-512 can be coded as follows, with permuted message as YMM registers in arguments `%1` and `%2` (comments are prefixed with a semicolon):

```

%macro VPROTRQ 2
    vpsllq ymm8, %1, 64-%2 ; x << 32-c
    vpsrlq %1, %1, %2 ; x >> c
    vpxor %1, %1, ymm8
%endmacro

; ymm0-3: State
; ymm4-7: m_{\sigma} xor u_{\sigma}
; ymm8-9: Free temp registers
; ymm10-13: m
%macro G 2
    vpaddq ymm0, ymm0, %1 ; row1 + buf1
    vpaddq ymm0, ymm0, ymm1 ; row1 + row2
    vpxor ymm3, ymm3, ymm0 ; row4 ^ row1
    vpshufd ymm3, ymm3, 10110001b ; row4 >>> 32

    vpaddq ymm2, ymm2, ymm3 ; row3 + row4
    vpxor ymm1, ymm1, ymm2 ; row2 ^ row3
    VPROTRQ ymm1, 25 ; row2 >>> 25

    vpaddq ymm0, ymm0, %2 ; row1 + buf1
    vpaddq ymm0, ymm0, ymm1 ; row1 + row2
    vpxor ymm3, ymm3, ymm0 ; row4 ^ row1
    vpshufb ymm3, ymm3, ymm15 ; row4 >>> 16

    vpaddq ymm2, ymm2, ymm3 ; row3 + row4
    vpxor ymm1, ymm1, ymm2 ; row2 + row3
    VPROTRQ ymm1, 11 ; row2 >>> 11
%endmacro

```

Message loading with `vpgatherdq` can be coded as follows, with support of message caching:

```

%macro MSGLOAD 1

    vpcmpeqq ymm14, ymm14, ymm14 ; FF..FF
    vmovdqa xmm8, [perm + %1*64 + 00]
    vpgatherdq ymm4, [rsp + 8*xmm8], ymm14

    vpcmpeqq ymm14, ymm14, ymm14 ; FF..FF
    vmovdqa xmm9, [perm + %1*64 + 16]
    vpgatherdq ymm5, [rsp + 8*xmm9], ymm14

    vpcmpeqq ymm14, ymm14, ymm14 ; FF..FF
    vmovdqa xmm8, [perm + %1*64 + 32]
    vpgatherdq ymm6, [rsp + 8*xmm8], ymm14

    vpcmpeqq ymm14, ymm14, ymm14 ; FF..FF
    vmovdqa xmm9, [perm + %1*64 + 48]
    vpgatherdq ymm7, [rsp + 8*xmm9], ymm14

    vpxor ymm4, ymm4, [const_z + 128*%1 + 00]
    vpxor ymm5, ymm5, [const_z + 128*%1 + 32]
    vpxor ymm6, ymm6, [const_z + 128*%1 + 64]
    vpxor ymm7, ymm7, [const_z + 128*%1 + 96]

```

```

%ifdef CACHING
%if %1 < 6
    vmovdqa    [rsp + 128 + %1*128 + 00], ymm4
    vmovdqa    [rsp + 128 + %1*128 + 32], ymm5
    vmovdqa    [rsp + 128 + %1*128 + 64], ymm6
    vmovdqa    [rsp + 128 + %1*128 + 96], ymm7
%endif
%endif

%endmacro

```

Diagonalization, undiagonalization, and a round look as follows:

```

%macro DIAG 0
    vpermq    ymm1, ymm1, 0x39
    vpermq    ymm2, ymm2, 0x4e
    vpermq    ymm3, ymm3, 0x93
%endmacro

%macro UNDIAG 0
    vpermq    ymm1, ymm1, 0x93
    vpermq    ymm2, ymm2, 0x4e
    vpermq    ymm3, ymm3, 0x39
%endmacro

%macro ROUND 1
    MSGLOAD %1
    G ymm4, ymm5
    DIAG
    G ymm6, ymm7
    UNDIAG
%endmacro

```

5.5.3 Implementing BLAKE-256 with AVX2

This section shows how BLAKE-256 can benefit from AVX2. Unlike BLAKE-512, BLAKE-256 is not naturally adaptable to 256-bit vectors, as there is a maximum of four G_i independently running functions per round. Nevertheless, it is possible to take advantage of AVX2 to speed-up BLAKE-256.

5.5.3.1 Optimized Message Loading

The first way to improve message loads is by using the `vpgatherdd` instruction from the AVX2 instruction set. To perform the full 16-word message permutation required in each round, only four operations are required:

```

_m128i m0 = _mm_i32gather_epi32(m, sigma[r][0], 4);

```

```

_mm128i m1 = _mm_i32gather_epi32(m, sigma[r][1], 4);
_mm128i m2 = _mm_i32gather_epi32(m, sigma[r][2], 4);
_mm128i m3 = _mm_i32gather_epi32(m, sigma[r][3], 4);

```

This can be further improved by using only two YMM registers to store the permuted message:

```

_mm256i m01 = _mm256_i32gather_epi32(m, sigma[r][0], 4);
_mm256i m23 = _mm256_i32gather_epi32(m, sigma[r][1], 4);

```

The individual 128-bit blocks of message are accessible through the `vextracti128` instruction.

One must also consider the possibility that `vpgatherdd` will not have acceptable performance, perhaps due to specific processor design idiosyncrasies; AVX2 can still help us, via the `vpermd` and `vpblendd` instructions:

```

tmp0 = _mm256_permutevar8x32_epi32(m01, sigma00);
tmp1 = _mm256_permutevar8x32_epi32(m23, sigma01);
tmp2 = _mm256_permutevar8x32_epi32(m01, sigma10);
tmp3 = _mm256_permutevar8x32_epi32(m23, sigma11);
m01 = _mm256_blend_epi32(tmp0, tmp1, mask0);
m23 = _mm256_blend_epi32(tmp2, tmp3, mask1);

```

In the above code, we permute the elements from the first YMM register into their proper order in the permutation, after which we permute the elements from the second. A simple blend instruction suffices to obtain the correct permutation. We repeat the process for the second part of the permutation. Once again, individual 128-bit blocks are available via `vextracti128`.

5.5.3.2 Message Caching

Message caching, as introduced in Section 5.5.2.3 for BLAKE-512, can be applied to BLAKE-256. Due to the smaller number of redundant permuted messages (4) and the lower message size, the full state ($4 \times 4 \times 128$ bits) can be stored in eight YMM registers. This leaves the possibility of either storing all entries, or of keeping some in registers. Permuted messages are easily stored using the `vinseri128` instruction:

```

// First 4 permuted elements
cache_reg = _mm256_inseri128_si256(cache_reg, buf1, 0);
...
// Second 4 permuted elements
cache_reg = _mm256_inseri128_si256(cache_reg, buf1, 1);
_mm256_store_si256(&cache[r], cache_reg);

```

In rounds 10 and above, we can retrieve the cached permutations with a simple load and extract:

```

cache_reg = _mm256_load_si256(&cache[r]);
buf1 = _mm_extracti128(cache_reg, 0);
...
buf1 = _mm_extracti128(cache_reg, 1);

```

Like for BLAKE-512, one should store the message words already XORed with the constants.

5.5.3.3 Tree Hashing

Observe that AVX2 allows to use the 256-bit width of YMM registers to compute two keyed permutations in parallel, that is, where each 128-bit lane of YMM registers processes an independent block: the instruction `vpaddq` can perform the two 4-way additions in parallel, a single `vpermd` can rotate two rows in the (un)diagonalization step, etc. Overall, it is easy to see that compressing two blocks with this technique will be close to twice as fast as two single-stream compressions.

This technique may be exploited to implement a tree hashing mode, wherein two independent nodes or leaves are processed in parallel. In particular, a binary tree hashing mode processing a 2^{n-1} -block message could be implemented with 2^{n-1} double compressions rather than $2^n - 1$ compressions (if leaves are as large as a message block). With a binary tree of fixed depth two and variable leaf size, such that the message is split in two halves of equal block length, a parallel implementation with AVX2 is likely to be twice as fast as than standard serial hashing.

If the classical (nontree) mode is used, BLAKE can also benefit from this technique to hash two messages simultaneously. Note that the indices of the message blocks need not be synchronized, as different counter values may be used for each of the two blocks processed in parallel.

When combined with multi-core and multithreading technologies (as implemented in new processors), we expect this technique to allow extremely high speed for both tree hashing and multistream processing.

5.6 Vectorized Implementation with XOP Extensions

In 2007, AMD announced its SSE5 set of new instructions. These featured 3-operand instructions, more powerful permutations, native integer rotations, and fused-multiply-add capabilities. After the announcement of AVX, however, SSE5 was shelved in favor of AVX plus XOP, FMA4, and CVT16. The XOP instruction set [2] extends AVX with new integer multiply-and-accumulate (`vpmac*`), rotation (`vprot*`), shift (`vpsha*`, `vpshl*`), permutation (`vpper*`), and conditional move (`vpcmov`) instructions working on XMM registers. These instructions have latency of at least two cycles. XOP instructions are integrated in AMD's Bulldozer microarchitecture, which first appeared in the FX-series 32 nm processors released in October 2011.

All the code presented in this section was written by Samuel Neves in the context of our joint project on vectorized implementations of BLAKE [130], as presented at the Third SHA3 Candidate Conference.

5.6.1 Relevant XOP Instructions

We present the most useful XOP instructions for implementing BLAKE.

5.6.1.1 Rotation

Whereas SSE and AVX require rotations to be implemented with a combination of two shifts and an XOR, XOP introduces rotate instructions with either fixed or variable counts: the 3-operand `vprotd` (intrinsics `_mm_roti_epi32` and `_mm_rot_epi32`) sets its destination XMM register to the four 32-bit words from a source register rotated by possibly different counts (positive for left rotation, negative for right); `vprotdq` (intrinsics `_mm_roti_epi64` and `_mm_rot_epi64`) is the equivalent instruction for 2-way 64-bit vectorized rotation.

5.6.1.2 Conditional Move

The `vpcmov` instruction (intrinsic `_mm_cmov_si128`) takes four operands among which a destination register has each of its bits set to the corresponding bit of either the first or the second source operand, depending on a selector third operand; this is similar to the “?” ternary operator in C. `vpcmov` accepts XMM or YMM registers as operands; for the latter, the instruction is equivalent to

```
uint64_t a[4], b[4], c[4], d[4];
for (i=0; i < 4; ++i) d[i] = (a[i] & c[i]) | (b[i] & ~c[i]);
```

5.6.1.3 Byte Permutation

With the `vpperp` instruction, XOP offers more than a simple byte permutation: given two source XMM registers (that is, 256 bits) and a 16-byte selector, `vpperp` fills the destination XMM register with bytes that are either a byte chosen from the two source registers, or a constant either `00` or `ff`. Furthermore, bitwise logical operations can be applied to source bytes (invert, reverse, etc.).

5.6.2 Implementing BLAKE with XOP

This section shows the main XOP-specific optimizations for BLAKE-256 and BLAKE-512, with a focus on the former. Although only a limited number of XOP instructions can be exploited, they provide a significant speedup compared with implementations using AVX but not XOP. The latest version of our `xop` implementations can be found in SUPERCOP.

5.6.2.1 Faster Rotations

The first optimization is straightforward, as it just consists in doing rotations with the dedicated `vprotd` instruction. In BLAKE-256, rotations by 16 and 8, previously implemented with SSSE3's `pshufb`, can also be replaced with `vprotd`. The first half of G can thus be coded as

```
row1 = _mm_add_epi32( _mm_add_epi32( row1, buf), row2 );
row4 = _mm_xor_si128( row4, row1 );
row4 = _mm_roti_epi32(row4, -16);
row3 = _mm_add_epi32( row3, row4 );
row2 = _mm_xor_si128( row2, row3 );
row2 = _mm_roti_epi32(row2, -12);
```

Similarly, `vprotq` can be used in BLAKE-512.

For BLAKE-256, we save two instructions per 12 or 7 rotation, thus eight instructions per round, and 112 per compression. In the Bulldozer microarchitecture, shifts (`vpslld` and `vpslrd`) have a latency of three cycles, and `vpxor` of two: a rotation thus takes six cycles, as the shifts can be pipelined within the execution unit (assuming a new instruction can start at every cycle). Since `vprotd` has latency of two, we can expect to save four cycles per rotation, thus 224 cycles per compression, that is, 3.5 cycles per byte. This figure may be slightly lower in practice, due to the pipelining of other instructions during the execution of the shift-shift-XOR.

For BLAKE-512, we save four instructions per 25 or 11 rotation, thus 16 instructions per round, and 256 per compression. On Bulldozer, we can expect the rotations (without `vprotq`) to complete in eight cycles, due to the pipelining of the four 3-cycle latency shifts. Assuming the two `vprotq`'s are pipelined as well to complete in three cycles, we save five cycles per rotation, thus 320 per compression, that is, 2.5 cycles per byte. Again, the context may slightly lower this estimate in practice.

5.6.2.2 Optimized Message Loading

XOP can be used to implement BLAKE's message permutation without memory lookups, that is, by reorganizing the words $m_0, \dots, m_{15}$ within registers, similarly to the approach in Section 5.5.3.1. The key operation is `vpperm`'s conditional moves, which allow us to copy up to four arbitrary message words out of eight into an XMM register; for example in the first column step of the first round, an XMM register needs be loaded with m_0, m_2, m_4, m_6 ; with XMM registers `m0` and `m1`, respectively, holding m_0 to m_3 and m_4 to m_7 , this can be done as

```
selector = _mm_set_epi32( 0x1b1a1918, 0x13121110,
                        0x0b0a0908, 0x03020100 );
s0 = _mm_perm_epi8(m0, m1, selector);
```

A complete definition of the `vpperm` selector can be found in [2, p235]. Note that, unlike message words, constant words can be loaded directly, to be XORed with the message:

```
s1 = _mm_set_epi32(0xec4e6c89, 0x299f31d0, 0x3707344, 0x85a308d3);
buf = _mm_xor_si128(s0, s1);
```

The same procedure can be followed when the four message words to be loaded span three or four message registers—that is, where the i -th register, $i = 0, 1, 2, 3$, holds m_{4i} to m_{4i+1} . An example of the latter case occurs in the first message load of the fourth round, where we need the following code:

```
s0 = _mm_perm_epi8(m0, m1,
                  _mm_set_epi32(SEL(0), SEL(0), SEL(3), SEL(7)));
s0 = _mm_perm_epi8(s0, m2,
                  _mm_set_epi32(SEL(7), SEL(2), SEL(1), SEL(0)));
s0 = _mm_perm_epi8(s0, m3,
                  _mm_set_epi32(SEL(3), SEL(5), SEL(1), SEL(0)));
s1 = _mm_set_epi32(0x3f84d5b5, 0xc0ac29b7, 0x85a308d3, 0x38d01377);
buf = _mm_xor_si128(s0, s1);
```

where `SEL` is a macro that forms the appropriate selector.

Each round requires four message loads (two in each step). Of the ten permutations:

1. 6 use two message registers (thus one `vpperm`)
2. 30 use three message registers (thus two `vpperm`'s)
3. 4 use four message registers (thus three `vpperm`'s)

In total, 78 calls to `vpperm` are necessary to implement the first ten permutations (e.g., when message caching is used), and 94 if the first rounds' loads are recomputed (see Table 5.4 for the detailed distribution). These numbers may be reduced with new implementation techniques eliminating redundancies, for example, by reusing previously loaded messages to avoid 3-`vpperm` loads.

Note that one could use `vpinsrd` instead of `vpperm` for single-word insertions. This does not improve speed, however, as `vpinsrd` has a latency of 12 cycles on Bulldozer, as opposed to simply two for `vpperm`, due to the decoupling of integer and floating-point units.

Table 5.4 Number of message loads requiring either one, two, or three calls to `vpperm`, as a function of the permutation.

Registers	<code>vpperm</code>	Permutation (round) index									
		0	1	2	3	4	5	6	7	8	9
2	1	4	-	-	-	-	-	-	-	1	1
3	2	-	4	4	3	3	4	4	3	2	3
4	3	-	-	-	1	1	-	-	1	1	-

5.7 Vectorized Implementation with NEON Extensions

NEON extensions are available in ARM processors of the Cortex family, such as the Cortex-A9 in the Apple iPad 2 and iPhone 4S. They offer to implementers 16 128-bit registers (also seen as 32 64-bit registers) and a rich set of SIMD instructions operating on vectors of 8-, 16-, 32-, or 64-bit words, whereas the basic ARM architecture has only 32-bit registers.

This section gives a brief overview of how to use NEON to implement BLAKE. We refer to ARM’s manuals⁶ for a complete reference on NEON, and to Leurent’s code for complete NEON implementations of BLAKE-256 and BLAKE-512 (implementations `vect128` and `vect128-neon` in SUPERCOP [28]).

5.7.1 Relevant NEON Instructions

NEON instructions relevant to implement BLAKE essentially do the same vectorized operations as their SSE2 counterparts, but work with different data types and have different names. Tables 5.5 and 5.6 show the main NEON instructions for SIMD implementations of BLAKE-256 and BLAKE-512, respectively.

Note that NEON has stricter typing than SSE2; whereas SSE2 defines a single data type for all unsigned 128-bit data (`__m128i`, internally defined as a structure of 8- to 64-bit vectors), NEON has different data types for different vectorizations of the data; for example, a 128-bit register viewed as a vector of four 32-bit unsigned integers should be declared as `uint32x4_t`, whereas a vector of 16 unsigned bytes should be declared as `uint8x16_t`. Therefore, different intrinsics have to be used for identical operations dependent on the vector types of the data; that is why NEON has (say) `veorq_u32` and `veorq_u8` intrinsics for 128-bit XOR. Although some compilers allow casting to another vector type (e.g., gcc), the right way to convert vectors is to use the `vreinterpret` intrinsics; for simplicity, we allow casts in the code snippets below.

Table 5.5 Main NEON instructions used to implement BLAKE-256.

Assembly	Intrinsic	Description
<code>vadd.i32</code>	<code>vaddq_u32</code>	4-way 32-bit integer addition
<code>veor</code>	<code>veorq_u32</code>	4-way 32-bit XOR
<code>vsli.32</code>	<code>vsliq_n_u32</code>	4-way 32-bit left-shift and insert
<code>vsri.32</code>	<code>vsriq_n_u32</code>	4-way 32-bit right-shift and insert
<code>vext.32</code>	<code>vextq_u32</code>	4-way 32-bit word shuffle
<code>vtbl.8</code>	<code>vtbl2_u8</code>	vectorized byte look up

⁶ <http://infocenter.arm.com>.

Table 5.6 Main NEON instructions used to implement BLAKE-512.

Assembly	Intrinsic	Description
vadd.i64	vaddq_u64	2-way 64-bit integer addition
veor	veorq_u64	2-way 64-bit XOR
vsli.i64	vsliq_n_u64	2-way 64-bit left-shift and insert
vsri.i64	vsriq_n_u64	2-way 64-bit right-shift and insert
vext.64	vextq_u64	4-way 32-bit word shuffle
vrev64.32	vrev64q_u32	reverse of 32-bit words
vtbl.8	vtbl2_u8	vectorized byte look up

5.7.2 Implementing BLAKE-256 with NEON

We present a basic SIMD implementation of the round function of BLAKE-256 using NEON. In the code below, the variables `row1`, `row2`, `row3`, and `row4` are defined as `uint32x4_t`, and correspond to 128-bit NEON registers. The variables `m1`, `m2`, `m3`, and `m4` contain the permuted message words (we refer to Leurent’s `vect128` implementation for an efficient implementation of message loading). The value `i` is the index of the round, in $0, 1, \dots, 13$.

```

/* column step */
m0 = veorq_u32( m0, u[4 * ( i % 10 )] );
A = vaddq_u32( vaddq_u32( A, m0 ), B );
D = veorq_u32( A, D );
D = ( uint32x4_t )PERMUTE( ( uint8x16_t )D, rot16 );
C = vaddq_u32( C, D );
B = veorq_u32( B, C );
B = ROT( B, 12 );
m1 = veorq_u32( m1, u[4 * ( i % 10 ) + 1] );
A = vaddq_u32( vaddq_u32( A, m1 ), B );
D = veorq_u32( D, A );
D = ( uint32x4_t )PERMUTE( ( uint8x16_t )D, rot8 );
C = vaddq_u32( C, D );
B = veorq_u32( B, C );
B = ROT( B, 7 );

/* diagonalize */
B = vextq_u32( B, B, 1 );
C = vextq_u32( C, C, 2 );
D = vextq_u32( D, D, 3 );

/* diagonal step */
m2 = veorq_u32( m2, u[4 * ( i % 10 ) + 2] );
A = vaddq_u32( vaddq_u32( A, m2 ), B );
D = veorq_u32( A, D );
D = ( uint32x4_t )PERMUTE( ( uint8x16_t )D, rot16 );
C = vaddq_u32( C, D );
C = vaddq_u32( C, D );
B = veorq_u32( B, C );
B = v32_rotate( B, 12 );
m3 = veorq_u32( m3, u[4 * ( i % 10 ) + 3] );

```

```

A = vaddq_u32( vaddq_u32( A, m3 ), B );
D = veorq_u32( D, A );
D = ( uint32x4_t )PERMUTE( ( uint8x16_t )D, rot8 );
C = vaddq_u32( C, D );
B = veorq_u32( B, C );
B = ROT( B, 7 );

/* undiagonalize */
B = vextq_u32( B, 3 );
C = vextq_u32( C, 2 );
D = vextq_u32( D, 1 );

```

The code above uses the following macros and constants:

```

#define ROT(x,n) ({
    uint32x4_t t__ __attribute__((unused));
    t__ = vsliq_n_u32(t__, x, 32-(n));
    t__ = vsriq_n_u32(t__, x, n);
    t__;
})

#define PERMUTE(x,s) ({
    uint8x8x2_t x__;
    x__.val[0] = vget_low_u8(x);
    x__.val[1] = vget_high_u8(x);
    vcombine_s8(vtbl2_u8(x__,vget_low_u8(s)),
    vtbl2_u8(x__,vget_high_u8(s)));
})

static const uint32x4_t u[] =
{
    {{0x85a308d3, 0x03707344, 0x299f31d0, 0xec4e6c89}},
    {{0x243f6a88, 0x13198a2e, 0xa4093822, 0x082efa98}},
    {{0x38d01377, 0x34e90c6c, 0xc97c50dd, 0xb5470917}},
    {{0x452821e6, 0xbe5466cf, 0xc0ac29b7, 0x3f84d5b5}},
    {{0xbe5466cf, 0x452821e6, 0xb5470917, 0x082efa98}},
    {{0x3f84d5b5, 0xa4093822, 0x38d01377, 0xc97c50dd}},
    {{0xc0ac29b7, 0x13198a2e, 0xec4e6c89, 0x03707344}},
    {{0x85a308d3, 0x243f6a88, 0x34e90c6c, 0x299f31d0}},
    {{0x452821e6, 0x243f6a88, 0x13198a2e, 0xc97c50dd}},
    {{0x34e90c6c, 0xc0ac29b7, 0x299f31d0, 0xb5470917}},
    {{0x3f84d5b5, 0x082efa98, 0x85a308d3, 0xa4093822}},
    {{0xbe5466cf, 0x03707344, 0xec4e6c89, 0x38d01377}},
    {{0x38d01377, 0x85a308d3, 0xc0ac29b7, 0x3f84d5b5}},
    {{0xec4e6c89, 0x03707344, 0xc97c50dd, 0x34e90c6c}},
    {{0x082efa98, 0xbe5466cf, 0x243f6a88, 0x452821e6}},
    {{0x13198a2e, 0x299f31d0, 0xa4093822, 0xb5470917}},
    {{0x243f6a88, 0xec4e6c89, 0xa4093822, 0xb5470917}},
    {{0x38d01377, 0x299f31d0, 0x13198a2e, 0xbe5466cf}},
    {{0x85a308d3, 0xc0ac29b7, 0x452821e6, 0xc97c50dd}},
    {{0x3f84d5b5, 0x34e90c6c, 0x082efa98, 0x03707344}},
    {{0xc0ac29b7, 0xbe5466cf, 0x34e90c6c, 0x03707344}},
    {{0x13198a2e, 0x082efa98, 0x243f6a88, 0x452821e6}},
    {{0xc97c50dd, 0x299f31d0, 0x3f84d5b5, 0x38d01377}},
    {{0xa4093822, 0xec4e6c89, 0xb5470917, 0x85a308d3}},

```

```

    {{0x299f31d0, 0xb5470917, 0xc97c50dd, 0xbe5466cf}},
    {{0xc0ac29b7, 0x85a308d3, 0x3f84d5b5, 0xa4093822}},
    {{0xec4e6c89, 0x03707344, 0x13198a2e, 0x34e90c6c}},
    {{0x243f6a88, 0x082efa98, 0x38d01377, 0x452821e6}},
    {{0x34e90c6c, 0x3f84d5b5, 0x85a308d3, 0x38d01377}},
    {{0xc97c50dd, 0xec4e6c89, 0xc0ac29b7, 0x03707344}},
    {{0x243f6a88, 0xa4093822, 0x082efa98, 0xbe5466cf}},
    {{0x299f31d0, 0xb5470917, 0x452821e6, 0x13198a2e}},
    {{0xb5470917, 0x38d01377, 0x03707344, 0x452821e6}},
    {{0x082efa98, 0x3f84d5b5, 0x34e90c6c, 0x243f6a88}},
    {{0x13198a2e, 0xec4e6c89, 0xa4093822, 0x299f31d0}},
    {{0xc0ac29b7, 0xc97c50dd, 0x85a308d3, 0xbe5466cf}},
    {{0x13198a2e, 0xa4093822, 0x082efa98, 0x299f31d0}},
    {{0xbe5466cf, 0x452821e6, 0xec4e6c89, 0x85a308d3}},
    {{0x34e90c6c, 0x3f84d5b5, 0xc0ac29b7, 0x243f6a88}},
    {{0xb5470917, 0x38d01377, 0x03707344, 0xc97c50dd}},
};

static const uint8x16_t rot8 = {{
    1, 2, 3, 0, 5, 6, 7, 4, 9, 10, 11, 8, 13, 14, 15, 12
}};

static const uint8x16_t rot16 = {{
    2, 3, 0, 1, 6, 7, 4, 5, 10, 11, 8, 9, 14, 15, 12, 13
}};

```

The macro ROT uses the shift and insert instructions to perform a rotation with two instructions, avoiding an explicit merge of the two shifted vectors (as in Section 5.1.1.3). The macro PERMUTE is used to perform rotations by 8 and 16 bits through a permutation of bytes; the instruction `vtbl.8` is used to perform vectorized table lookup at the given indices (`rot8` and `rot16`).

5.7.3 Implementing BLAKE-512 with NEON

A basic implementation of BLAKE-512 with NEON uses two 128-bit registers typed `uint64x2_t` for each row of the internal state, and uses two 2-way 64-bit operations to perform each of the 4-way parallel operations. In the code below, the variables `row1`, `row2`, `row3`, and `row4` are thus typed `uint32x4_t`, as well as `m1`, `m2`, `m3`, and `m4`, which contain the permuted message words. A round can thus be implemented as follows:

```

/* column step */
t0 = veorq_u64( m0, u[8 * ( i % 10 ) + 0] );
t1 = veorq_u64( m1, u[8 * ( i % 10 ) + 1] );
A0 = vaddq_u64( vaddq_u64( A0, t0 ), B0 );
A1 = vaddq_u64( vaddq_u64( A1, t1 ), B1 );
D0 = veorq_u64( A0, D0 );
D1 = veorq_u64( A1, D1 );

```

```

D0 = ( uint64x2_t ) ( vrev64q_u32( ( uint32x4_t ) D0, 1 ) );
D1 = ( uint64x2_t ) ( vrev64q_u32( ( uint32x4_t ) D1, 1 ) );
C0 = vaddq_u64( C0, D0 );
C1 = vaddq_u64( C1, D1 );
B0 = veorq_u64( B0, C0 );
B1 = veorq_u64( B1, C1 );
B0 = ROT( B0, 25 );
B1 = ROT( B1, 25 );
t0 = veorq_u64( m2, u[8 * ( i % 10 ) + 2] );
t1 = veorq_u64( m3, u[8 * ( i % 10 ) + 3] );
A0 = vaddq_u64( vaddq_u64( A0, t0 ), B0 );
A1 = vaddq_u64( vaddq_u64( A1, t1 ), B1 );
D0 = veorq_u64( D0, A0 );
D1 = veorq_u64( D1, A1 );
D0 = ( uint64x2_t ) ( PERMUTE( ( uint8x16_t ) D0, rot16 ) );
D1 = ( uint64x2_t ) ( PERMUTE( ( uint8x16_t ) D1, rot16 ) );
C0 = vaddq_u64( C0, D0 );
C1 = vaddq_u64( C1, D1 );
B0 = veorq_u64( B0, C0 );
B1 = veorq_u64( B1, C1 );
B0 = ROT( B0, 11 );
B1 = ROT( B1, 11 );

/* diagonalize */
SHUFFLE1( B0, B1 );
SHUFFLE2( C0, C1 );
SHUFFLE3( D0, D1 );

/* diagonal step */
t0 = veorq_u64( m4, u[8 * ( i % 10 ) + 4] );
t1 = veorq_u64( m5, u[8 * ( i % 10 ) + 5] );
A0 = vaddq_u64( vaddq_u64( A0, t0 ), B0 );
A1 = vaddq_u64( vaddq_u64( A1, t1 ), B1 );
D0 = veorq_u64( A0, D0 );
D1 = veorq_u64( A1, D1 );
D0 = ( uint64x2_t ) ( vrev64q_u32( ( uint32x4_t ) D0, 1 ) );
D1 = ( uint64x2_t ) ( vrev64q_u32( ( uint32x4_t ) D1, 1 ) );
C0 = vaddq_u64( C0, D0 );
C1 = vaddq_u64( C1, D1 );
B0 = veorq_u64( B0, C0 );
B1 = veorq_u64( B1, C1 );
B0 = ROT( B0, 25 );
B1 = ROT( B1, 25 );
t0 = veorq_u64( m6, u[8 * ( i % 10 ) + 6] );
t1 = veorq_u64( m7, u[8 * ( i % 10 ) + 7] );
A0 = vaddq_u64( vaddq_u64( A0, t0 ), B0 );
A1 = vaddq_u64( vaddq_u64( A1, t1 ), B1 );
D0 = veorq_u64( D0, A0 );
D1 = veorq_u64( D1, A1 );
D0 = ( uint64x2_t ) ( PERMUTE( ( uint8x16_t ) D0, rot16 ) );
D1 = ( uint64x2_t ) ( PERMUTE( ( uint8x16_t ) D1, rot16 ) );
C0 = vaddq_u64( C0, D0 );
C1 = vaddq_u64( C1, D1 );
B0 = veorq_u64( B0, C0 );

```

```

B1 = veorq_u64( B1, C1 );
B0 = ROT( B0, 11 );
B1 = ROT( B1, 11 );

/* undiagonalize */
SHUFFLE3( B0, B1 );
SHUFFLE2( C0, C1 );
SHUFFLE1( D0, D1 );

```

This code makes use of the `PERMUTE` macro and of the `rot16` constants defined in Section 5.7.2, as well as of the following:

```

#define ROT(x,n) ({
    uint64x2_t t__ __attribute__((unused));
    t__ = vsliq_n_u64(t__, x, 64-(n));
    t__ = vsriq_n_u64(t__, x, n);
    t__;
})

#define SHUFFLE1(x, y) ({
    uint64x2_t t__, u__;
    t__ = vextq_u64(x, y, 1);
    u__ = vextq_u64(y, x, 1);
    x = t__;
    y = u__;
})

#define SHUFFLE2(X, Y) do {
    uint64x2_t t__ = X;
    X = Y;
    Y = t__;
} while(0)

#define SHUFFLE3(x, y) ({
    uint64x2_t t__, u__;
    t__ = vextq_u64(x, y, 1);
    u__ = vextq_u64(y, x, 1);
    y = t__;
    x = u__;
})

```

5.8 Performance

We present speed measurements for BLAKE-256 and BLAKE-512 resulting from automated benchmarks on various platforms. Most figures presented for 32- and 64-bit processors are courtesy of the eBACS project [28]. Benchmarks for most ARMs and microcontrollers are courtesy of the XBX project [177].

Note that the performance figures reported below are only the best at the time of writing, and remain subject to improvement.

5.8.1 Speed Summary

The speed figures reported in the subsequent sections are in cycles per byte, which is the most relevant metric to accurately and fairly compare the speed of cryptographic algorithms. Using cycles per byte (or per any other data unit) has the advantage of making the speed measurements independent of the processor's frequency. Indeed, frequency has a high variance among processors, and can even vary during the operation of a single processor (for example, when the processor incorporates a “dynamic overclocking” technology).

Nevertheless, users are obviously interested in the actual speed of a hash function, that is, in the amount of data processed per unit time. The cycle-per-byte unit is then irrelevant, except as a preliminary step to determine the said actual speed. We thus report data-per-second figures in Table 5.7, deduced from the cycles-per-byte figures and the nominal operating frequency of each processor.

The processors selected include:

- NVIDIA Tegra 2, a system-on-chip (SoC) based on two ARM Cortex A9 cores (32-bit) that do not include the NEON extensions. Tegra 2 has been integrated in a number of tablets, such as the ASUS Eee Pad, Samsung Galaxy Tab, Sony Tablet S, etc. The Tegra 2 used has frequency 1 GHz, but there exist models operating at 1.2 GHz (Tegra 250 3D).
- Qualcomm Snapdragon S3 APQ8060, a SoC based on Qualcomm's Scorpion core (32-bit), an implementation of the ARMv7 architecture similar to the Cortex A8. It was, for example, used in the Samsung Galaxy S II smartphone. Snapdragon includes NEON extensions. The Snapdragon used has frequency 1.7 GHz, but operates in the Galaxy S II at 1.2 GHz.
- AMD FX-8120, a 64-bit server and desktop processor based on the Bulldozer microarchitecture. The FX-8120 has four cores and supports eight threads, and it includes the SSE family of extensions as well as AVX and XOP.
- AMD E-450, a 64-bit processor for netbooks and other portable devices, based on the Bobcat microarchitecture. The E-450 has a single core, and includes the SSE family of extensions up to SSSE3, plus AMD's SSE4a.
- Intel Core i7-2600K, a 64-bit desktop processor based on the Sandy Bridge microarchitecture. This processor has four cores, and includes the SSE family of extensions as well as AVX.
- Intel Core i3-2310M, a 64-bit laptop processor based on the Sandy Bridge microarchitecture. This processor has two cores, and includes the SSE family of extensions as well as AVX.
- Intel Xeon E3-1275 V3, a 64-bit server processor based on the Haswell microarchitecture. This processor has four cores, and includes the SSE family of extensions as well as AVX and AVX2.
- IBM POWER7, a 64-bit server processor based on the Power ISA v.2.06 microarchitecture. This processor can have four, six, or eight cores, and we do not know how many cores has the version used here (this is unlikely to affect results, as benchmarks run on a single core).

Note that the difference of frequencies between Tegra 2 and Snapdragon significantly influences their relative speeds in Table 5.7, but that other models of the same SoC family may have different frequencies, and even the same model may run at different frequencies depending on the application. This highlights the importance of a cycles per byte of a frequency-agnostic metric.

Since large amounts of data can consist of a few huge messages (for example, when checking the integrity of file systems) or of many small messages (for example, when data comes from network traffic), we report speeds on both long messages and 64-byte messages.

Table 5.7 Speed of BLAKE-256 and BLAKE-512 in mebibytes (2^{20} bytes) per second.

Processor	Frequency (MHz)	BLAKE-256		BLAKE-512	
		Long	64	Long	64
NVIDIA Tegra 2	1,000	31	12	16	6
Qualcomm Snapdragon S3	1,782	73	13	76	12
AMD FX-8120	3,100	249	104	430	147
AMD E-450	1,650	87	8	153	7
Intel Core i7-2600K	3,400	433	198	562	222
Intel Core i3-2310M	2,100	267	122	353	136
Intel Xeon E3-1275 V3	3,500	494	230	644	271
IBM POWER7	3,550	68	29	134	45

We observe in Table 5.7 that the highest speed is achieved on the high-frequency Haswell processor, with respectively 494 and 644 mebibytes per second for BLAKE-256 and BLAKE-512. Mobile processors (Tegra 2, Snapdragon, E-450, Core i3) show lower speeds, but sufficient ones for any typical application. The POWER7, besides a higher frequency than other processors, shows a relatively poor performance. This may be due to the present lack of dedicated implementation of BLAKE for this architecture.

5.8.2 8-Bit AVR

The publications reporting AVR implementations of BLAKE by von Maurich [82] and Osvik [142] considered the initial version of BLAKE, and thus report speed figures for 10 rounds rather than 14. The memory figures, however, are not expected to vary much.

The implementation of BLAKE-256 by von Maurich [82], as adapted to the final version of BLAKE, occupies 251 bytes of RAM and 1,780 bytes of ROM (code size), and runs on long messages at 456 cycles per byte.

The implementation by Osvik [142] of BLAKE-32 (the initial submission with 10 rounds) occupies 206 bytes of RAM and 2,076 bytes of ROM, and runs on long

messages at 263 cycles per byte. Adding a 40% overhead to estimate the speed of BLAKE-256 (the final submission with 14 rounds), we obtain 368 cycles per byte.

5.8.3 *ARM Platforms*

Table 5.8 reports performance figures for BLAKE-256 on a multitude of ARM-based platforms, including:

- speed when hashing a message of 1,536 bytes (in cycles per byte)
- RAM consumption, to store temporary variables (in bytes)
- RAM consumption, to store code and constants (in bytes)

For each platform, Table 5.8 reports figures for three implementations respectively optimizing each of these metrics. Only one line is given for benchmarks from SUPERCOP, which does not record memory consumptions. Table 5.9 reports similar measurements for BLAKE-512.

The processors with NEON instructions use SIMD instructions as described in Section 5.7, thanks to the implementations published by Leurent. Note that the NEON-enabled processors allow the use of instructions operating on 64-bit words, whereas others only have 32-bit instructions. This explains why BLAKE-512 is considerably faster on NEON-enabled platforms.

5.8.4 *x86 Platforms (32-bit)*

32-bit processors using the x86 architecture include recent low-power notebook processors and older desktop and server processors. We also consider 64-bit processors operating in 32-bit mode, to address the cases when a 32-bit OS is running on a 64-bit machine. Tables 5.10 and 5.11 report speed measurements (in cycles per byte) for long messages as well as messages of 576 and 64 bytes. In those tables, the last column indicates which SIMD extensions (if any) were necessary to achieve the reported speed. Note that processors' support of SIMD extensions varies: for example, the Athlon K7 does not even include SSE2 (but only MMX), and AMD processors did not include SSSE3 until the Bobcat and Bulldozer microarchitectures.

Lower message length leads to a higher cycles per byte count, due to the overheads mainly caused by the hash finalization; for example, when a 64-byte message is processed by BLAKE-512, 128 bytes are actually hashed because the padding imposes an additional 64-byte block.

More recent processors tend to perform better due to their more advanced microarchitectures, which allow the execution of more instructions per cycle in parallel— thanks to several arithmetic logic units (ALUs)—and include the most recent instruction set extensions.

Table 5.8 Performance of BLAKE-256 on selected ARM platforms, with speed in cycles per byte for 1,536-byte messages, and memory in bytes.

Core	Architecture	Hardware	NEON	Speed	RAM	ROM
ARM920T	ARMv4T	Atmel AT91RM9200		78	716	25,488
ARM920T	ARMv4T	Atmel AT91RM9200		603	272	3,952
ARM920T	ARMv4T	Atmel AT91RM9200		150	284	2,052
XScale	ARMv5TE	Intel IXP420		91	2,028	13,160
XScale	ARMv5TE	Intel IXP420		276	360	6,456
XScale	ARMv5TE	Intel IXP420		149	408	3,716
Cortex-M0	ARMv6-M	NXP LPC1114		115	772	9,124
Cortex-M0	ARMv6-M	NXP LPC1114		372	280	1,152
Cortex-M0	ARMv6-M	NXP LPC1114		372	280	1,152
Cortex-M3	ARMv7-M	TI LM3S811		49	508	12,496
Cortex-M3	ARMv7-M	TI LM3S811		210	280	1,320
Cortex-M3	ARMv7-M	TI LM3S811		210	280	1,320
Cortex-A8	ARMv7-A	TI DM3730	✓	24	404	4304
Cortex-A8	ARMv7-A	TI DM3730	✓	104	280	1,472
Cortex-A8	ARMv7-A	TI DM3730	✓	112	304	1296
Cortex-A8	ARMv7-A	Freescall i.MX515	✓	20	-	-
Cortex-A9	ARMv7-A	TI OMAP 4460	✓	23	-	-
Cortex-A9	ARMv7-A	NVIDIA Tegra 2		32	-	-
Scorpion	ARMv7-A	Qualcomm Snapdragon S3	✓	27	-	-

5.8.5 amd64 Platforms (64-bit)

64-bit processors using the amd64 architecture are found in servers, desktops, laptops, and now in most notebooks as well as some tablets. Tables 5.12 and 5.13 report benchmarks for recent (at the time of writing) and less recent processors, including lower-power mobile processors such as AMD’s E-450 or Intel’s Atom N435. As in Tables 5.10 and 5.11, speed is given in cycles per byte for long, 576-byte, and 64-byte messages.

Since amd64 is an extension of the x86 architectures, BLAKE (or any other algorithm) is at least as fast in 64-bit mode as in 32-bit mode. BLAKE-512 is often considerably faster on 64-bit platforms thanks to the availability of 64-bit arithmetic operations. However, it is still fast on 32-bit platforms that include SIMD instruction set extensions.

We do not include the Core i7 used in Section 5.8.1, since it has the same core as the Core i3 considered and thus very similar benchmark results. However, we include the latest benchmarks from eBASH on a Intel Xeon with the Haswell microarchitecture, as available at the time of completing the book.

Table 5.9 Performance of BLAKE-512 on selected ARM platforms, with speed in cycles per byte for 1,536-byte messages, and memory in bytes.

Core	Architecture	Hardware	NEON	Speed	RAM	ROM
ARM920T	ARMv4T	Atmel AT91RM9200		157	1,076	15,188
ARM920T	ARMv4T	Atmel AT91RM9200		423	488	5,052
ARM920T	ARMv4T	Atmel AT91RM9200		423	488	5,052
XScale	ARMv5TE	Intel IXP420		197	1,140	28,764
XScale	ARMv5TE	Intel IXP420		392	948	15,684
XScale	ARMv5TE	Intel IXP420		225	1,056	7,368
Cortex-M0	ARMv6-M	NXP LPC1114		265	824	5,876
Cortex-M0	ARMv6-M	NXP LPC1114		409	560	1,476
Cortex-M0	ARMv6-M	NXP LPC1114		406	560	1,476
Cortex-M3	ARMv7-M	TI LM3S811		177	916	8,768
Cortex-M3	ARMv7-M	TI LM3S811		228	516	1,776
Cortex-M3	ARMv7-M	TI LM3S811		228	516	1,776
Cortex-A8	ARMv7-A	TI DM3730	✓	32	2,104	12020
Cortex-A8	ARMv7-A	TI DM3730	✓	387	529	4101
Cortex-A8	ARMv7-A	TI DM3730	✓	135	540	1700
Cortex-A8	ARMv7-A	Freescall i.MX515	✓	21	-	-
Cortex-A9	ARMv7-A	TI OMAP 4460	✓	25	-	-
Cortex-A9	ARMv7-A	NVIDIA Tegra 2		64	-	-
Scorpion	ARMv7-A	Qualcomm Snapdragon S3	✓	28	-	-

Table 5.10 Performance of BLAKE-256 on 32-bit (x86) processors, and 64-bit processors restricted to x86 mode (second part of the table).

Processor	Microarchitecture (core)	Long	576	64	SIMD
AMD Athlon	K7 (Pluto)	22.60	25.78	51.12	
AMD Athlon 64 3800+	K8 (ClawHammer)	27.66	31.45	61.66	
Intel Pentium 3	P6 (Coppermine)	24.20	27.82	56.53	
Intel Pentium 4	Netburst (Willamette)	25.88	34.42	72.44	SSE2
Intel Atom Z520	Bonnell (Silverthorne)	18.70	21.67	44.69	SSSE3
VIA Eden ULV	Esther	42.36	48.60	98.03	SSE2
AMD FX-8120	Bulldozer (Zambezi)	12.49	14.42	30.09	XOP
Intel Core i3-2310M	Sandy Bridge (206a7)	7.72	8.98	19.00	AVX

5.8.6 Other Platforms

Tables 5.14 and 5.15 present performance measurements for platforms excluded from the previous sections. These processors include:

- ICT Loongson 3A, a 64-bit processor developed by the Institute of Computing Technology of the Chinese Academy of Sciences, and based on the MIPS64 ar-

Table 5.11 Performance of BLAKE-512 on 32-bit (x86) processors, and 64-bit processors restricted to x86 mode (second part of the table).

Processor	Microarchitecture (core)	Long	576	64	SIMD
AMD Athlon	K7 (Pluto)	57.08	64.31	121.78	
AMD Athlon 64 3800+	K8 (ClawHammer)	68.31	76.97	144.73	
Intel Pentium 3	P6 (Coppermine)	72.50	82.34	156.77	
Intel Pentium 4	Netburst (Willamette)	40.90	47.61	102.25	SSE2
Intel Atom Z520	Bonnell (Silverthorne)	29.62	34.84	76.25	SSSE3
VIA Eden ULV	Esther	49.78	57.14	115.73	SSE2
AMD FX-8120	Bulldozer (Zambezi)	8.12	10.17	24.61	XOP
Intel Core i3-2310M	Sandy Bridge (206a7)	7.20	8.54	19.62	AVX

Table 5.12 Performance of BLAKE-256 on 64-bit (amd64) processors.

Processor	Microarchitecture (core)	Long	576	64	SIMD
AMD FX-8120	Bulldozer (Zambezi)	11.83	13.64	28.09	XOP
AMD E-450	Bobcat (Ontario)	18.00	20.60	41.11	
AMD A8-3850	K10 (Llano)	12.60	14.45	104.53	
AMD Athlon 64 X2	K8 (Windsor)	13.61	15.57	30.94	
Intel Xeon E3-1275 V3	Haswell (306c3)	6.75	7.56	14.52	AVX
Intel Core i3-2310M	Sandy Bridge (206a7)	7.49	8.48	16.38	AVX
Intel Atom N435	Bonnell (Pineview)	16.11	18.93	41.50	SSE2
Intel Xeon E5620	Nehalem (Westmere-EP)	8.52	9.76	19.69	SSE4.1
Intel Core 2 Duo E8400	Core (Wolfdale)	8.65	9.97	20.67	SSE4.1
VIA Nano U3500	Isaiah	13.33	15.65	33.98	SSE4.1

Table 5.13 Performance of BLAKE-512 on 64-bit (amd64) processors.

Processor	Microarchitecture (core)	Long	576	64	SIMD
AMD FX-8120	Bulldozer (Zambezi)	6.88	8.44	19.97	XOP
AMD E-450	Bobcat (Ontario)	10.22	11.91	25.92	
AMD A8-3850	K10 (Llano)	7.06	8.35	64.30	
AMD Athlon 64 X2	K8 (Windsor)	7.60	8.91	18.58	
Intel Xeon E3-1275 V3	Haswell (306c3)	5.18	5.92	12.28	AVX
Intel Core i3-2310M	Sandy Bridge (206a7)	5.66	6.87	14.69	AVX
Intel Atom N435	Bonnell (Pineview)	12.78	15.10	34.12	
Intel Xeon E5620	Nehalem (Westmere-EP)	7.14	8.37	18.19	SSE4.1
Intel Core 2 Duo E8400	Core (Wolfdale)	7.02	8.19	16.88	
VIA Nano U3500	Isaiah	10.94	12.68	26.64	

- chitecture. It is used in laptops, and in KD-60-I, a Chinese supercomputer that includes 80 quadcore Loongson 3A processors.
- Cell, a processor developed by Sony, Toshiba, and IBM, and famous for being used in Sony’s PlayStation 3 gaming console. The Cell’s cores are eight synergistic processings units (SPU), which mostly work on 128-bit operands and include SIMD instructions for 4- and 8-way parallelism.

- IBM POWER7, a 64-bit server processor based on the Power ISA v.2.06 microarchitecture. This processor can have four, six, or eight cores, and we do not know how many cores the version used here has (this is unlikely to affect the results, as benchmarks run on a single core).
- Sun UltraSPARC III, a 64-bit processor mostly used in servers and dating back to 2011. It is based on the SPARC v9 architecture.
- HP Itanium II, a 64-bit processor mostly used in enterprise information systems, and based on Intel’s Itanium architecture.

Table 5.14 Performance of BLAKE-256 on processors other than AVR, ARM, x86, and amd64.

Processor	Architecture	Long	576	64
ICT Loongson 3A	MIPS64	33.60	43.43	117.88
Cell	Cell	33.30	40.76	100.62
IBM POWER7	Power	48.98	57.56	112.98
Sun UltraSPARC III	SPARC v9	45.36	51.31	98.92
HP Itanium II	Itanium	18.68	22.75	55.58

Table 5.15 Performance of BLAKE-512 on processors other than AVR, ARM, x86, and amd64.

Processor	Architecture	Long	576	64
ICT Loongson 3A	MIPS64	20.59	28.75	90.19
Cell	Cell	32.15	40.49	106.88
IBM POWER7	Power	25.13	31.41	73.28
Sun UltraSPARC III	SPARC v9	26.02	30.14	63.30
HP Itanium II	Itanium	5.28	8.49	38.62

Chapter 6

BLAKE in Hardware

Hardware is to software as a valley is to the glacier on it.
—Vaughan Pratt

This chapter analyzes the suitability of BLAKE for hardware implementation and surveys state-of-the-art architectures that cover a large portion of potential applications for ASIC and FPGA. Before entering into the specification of the various implementations, we introduce some basic notions of digital design and related characterization figures. The central part describes generic and application-specific architectures of BLAKE, while we conclude the chapter with a performance review of the most relevant implementation documented so far.

6.1 RTL Design

In the last decade, digital communication has drastically increased in speed. Dedicated processors in the form of digital signal processing (DSP) systems, field-programmable devices, or instruction set extensions have been widely employed in the implementation of security protocols. Security is indeed forced to cope with modern transmission rates. Software implementations of cryptographic primitives have the great advantage of being portable and with a short time-to-market, however even the most advanced processors are inefficient in terms of area with respect to dedicated hardware. RTL¹ design of symmetric ciphers as well as hash functions therefore becomes crucial. Complementary metal–oxide–semiconductor (CMOS) technologies and modern FPGA devices provide the benchmark to evaluate their suitability for hardware.

Instead of using the number of instructions or the code complexity, digital designs are mainly evaluated and compared through the maximal achievable frequency² (often given in MHz), total circuit size (gate equivalents for ASIC and slices for FPGA), and power dissipation. Further metrics can be derived by com-

¹ Register-transfer level design is the most common design methodology to characterize synchronous digital circuits. Common hardware description languages are VHDL and Verilog.

² This for synchronous designs.

binning these three values with other parameters of the architecture, for example, the data path width. Normally, when a designer implements hardware code targeting a specific technology or device, he tries to optimize at least one of these parameters, depending on the final application (e.g., size and power for RFID or frequency and therefore throughput for high-speed encryptors). This aspect implies that one single algorithm may not be the most efficient for all application fields.

6.2 ASIC Implementation

We present generic RTL hardware architectures of BLAKE that are optimized for frequency and speed, throughput per unit area, and finally low size and low-power.

6.2.1 High-Speed Design

The iterative nature of most of the compression functions in modern cryptographic hash algorithms forces the designer to implement in the hardware description language a single round coupled with a memory block, typically in the form of registers. Each clock cycle of a round is computed by the logic part, and the internal variables after being updated are stored within the registers. In MD-like modes of operation, an additional memory block is needed for the temporary chain variable. In the case of BLAKE, registers are used for the internal state v , the chain variable h , the message block m , and the salt s . As it is used only at the beginning of the compression process, the counter value t does not require dedicated memory. This translates to a sequential area of 1,408 bits for BLAKE-256 and 2,816 bits for BLAKE-512 plus some additional registers for the control unit. In Figure 6.1, a typical block diagram of a BLAKE architecture is illustrated. The main components are the five register blocks and the combinational logic of the parallel G transformations.

6.2.1.1 Round Function Scalability

The repetition in BLAKE of the transform G_i throughout the rounds ensures a high degree of scalability. The number of dedicated logic blocks that compute G_i varies according to the target application. Obviously, with more blocks the final throughput increases as well as the size of the circuit. A natural choice falls on the numbers 8, 4, and 1. Architectures with four parallel G maximize the speed-to-area ratio—in other words, hardware efficiency—and one message block is indeed computed within 28 clock cycles for BLAKE-256, and 32 for BLAKE-512.

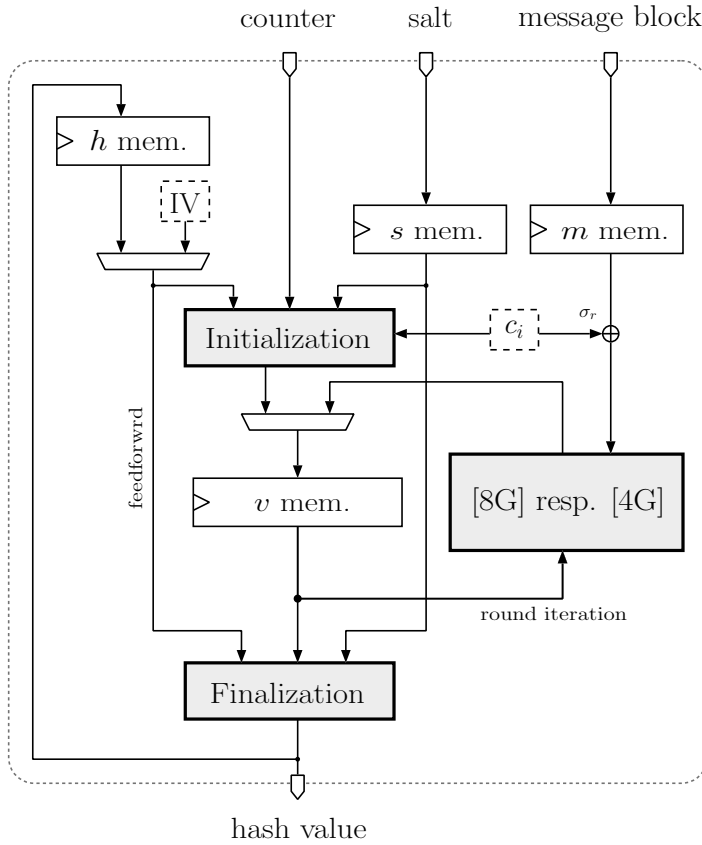


Fig. 6.1 Main architecture of a typical BLAKE hardware implementation.

6.2.1.2 Message Block Handling

As pointed out in [81], the longest logical path, i.e., the path that determines the maximum operating frequency, propagates from the selection of the message word m_i , through the G operations and ends the internal state registers. Two solutions have been proposed to shorten the period to the original ChaCha internal round, i.e., four xor gates and four modular additions. Tillich et al. [170] insert in their high-speed four-G design an additional pipeline register at the output of the permutation table. Allegedly, the output of the xor operation between the permuted message words and the constants is stored and in the following round provided to the G functions. Henzen et al. introduce in [81] a round rescheduling. They exploit the flow dependency of G computations to anticipate by one cycle the additions $a + m_{\sigma_r(2i)} \oplus u_{\sigma_r(2i+1)}$ and $a + m_{\sigma_r(2i+1)} \oplus u_{\sigma_r(2i)}$ (see the flow diagram in Figure 6.2). This solution is more cost-efficient in terms of area, since the message words and

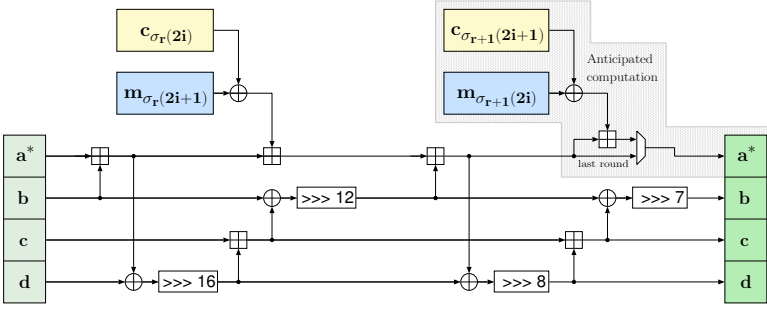


Fig. 6.2 Rescheduling of the G computations. Anticipating the addition of the message and the constant permits achievement of the optimal timing for RTL designs.

the constants are stored with the a variables of G without the use of extra pipeline registers.

6.2.2 Compact Design

Compact architectures of BLAKE are limited by the expensive storage requirement of the internal variables. This sort of area bottleneck forces optimizations towards the computational part of BLAKE. Since the compression function works on modular addition, the straightforward way to reduce the circuit size is to limit the number of addition blocks. In [81] a low-cost BLAKE-256 implementation for $0.18\ \mu\text{m}$ CMOS is described. The authors suggest an architecture with a single modular adder combined with clock-gating latch-based memories. The extensive use of time-sharing within the logical components combined with the alternative design methodology of the sequential blocks leads to a compact core that occupies 13,575 GEs. Smaller designs targeting less than 10 kGE in ASIC are therefore mainly limited by the memory, being in [81] about 70% of the total area.

6.3 FPGA Design

Modern FPGA chips integrate several macroblock components as dedicated memories, high-speed transceivers, or embedded processors. Often the use of these building blocks offers great advantages in terms of speed and size. Specifically for hash functions such as BLAKE, storing constants and variables in block RAMs (BRAMs) or employing adders allocated in digital signal processing (DSP) units can lead to faster and more compact implementations. In this section, we describe the most significant contributions of BLAKE architectures that exploit embedded FPGA re-

sources. We want to demonstrate how BLAKE can be efficiently implemented in field-programmable processors that make dedicated components available.

The first design was proposed in 2010 by Beuchat et al. [32]. These authors introduced a compact processor that fits in a minimum amount of logic in Spartan and Virtex devices from the FPGA manufacturer Xilinx. They implement a single arithmetic unit that is able to compute modular addition or xor operation, by exploiting the carry control bit within a standard Xilinx slice. In order to improve timing, rotation and multiplexing are further isolated using pipeline registers. The resulting architecture is a four times pipelined logic unit that can interleave the G computations of the horizontal and diagonal steps. The constants and the variables are stored in dual-port BRAM, whereas the instruction memory of the control unit is stored in a ROM unit. In total, two BRAMs are used for BLAKE-256, and three for BLAKE-512. The smallest architecture is 194 Mbps BLAKE-256, which fits in only 52 slices of a Virtex-6 chip (see results in Table 6.1).

A second lightweight architecture of BLAKE-256 is described in [96] by Kaps et al. Constants, salt, counter, initial and chaining hash values, along with the message block are stored in a single BRAM. The internal state is stored in dynamic RAM units—i.e. specific slices configured as memory. They feed the logic part, which is implemented as a half quasi-pipelined G function. The final architecture is a 349 Mbps BLAKE-256 using 163 slices of a Virtex-6 chip. For comparison we provide in Table 6.1 the logic-only architecture (cf. [96]).

According to Umar et al. [162], BLAKE is one among the SHA3 finalists that can profit from the use of DSP units to allocate the integer addition. Modern FPGA chips indeed provide multiple DSPs with large multipliers and adders. Computing the four modular additions of the G transformation through DSP adders reduces the amount of logic required. However, no significant speed advantages can be asserted. A complete design that stores the message block and the constants in dedicated BRAMs and exploit DSPs leads to a total area reduction of 60%, compared with a pure logic implementation. As can be seen from the values in Table 6.1, the implementation of Umar et al. reach a throughput of 1,534 Mbps using 8 DSPs, 12 BRAMs, and 662 slices.

6.4 Performance

We provide hardware performance figures for BLAKE-256 and BLAKE-512 with the latest references at the time of writing. In order to review in depth the suitability of BLAKE for ASIC and FPGA implementations and to pinpoint the differences with SHA2, we include the performance of the SHA-256 and SHA-512 functions. The results derive from projects that implement both algorithms with identical design methodologies, conditions, and technologies and mainly focus on maximizing the throughput-to-area ratio.

6.4.1 ASIC

For the CMOS design, Gürkaynak et al. [77] fabricated a 65 nm ASIC hosting two different architectures of BLAKE-256: one optimized for a target throughput of 2.488 Gbps (the ETHZ project [77]), and one optimized for throughput-to-area ratio (the GMU project [71]). Compared with the SHA2 architecture implemented on the same chip, BLAKE achieves similar speed values but requiring twice the area of SHA2. A similar study, led by Guo et al. [76] and culminating in a complete 130 nm chip, demonstrated faster but larger architectures of BLAKE-256, but with similar throughput-to-area ratios to the SHA2 architecture. Table 6.3 lists the main performance figures of these three ASIC projects. Also, comparing the power dissipation, BLAKE generally results in a higher energy-per-bit ratio.

Table 6.1 Overview of various BLAKE implementations in state-of-the-art FPGAs.

	Hash size	Frequency	Speed	Resources			Device
		[MHz]	[Mbps]	Slices	BRAMs	DSPs	
Beuchat et al. [6, 32]	256	456.0	194	52	2	0	xc6vlx75t-2
Kaps et al. [96]	256	233.0	412	248	1	0	xc5vlx20-2
Kaps et al. [96]	256	253.8	448	271	0	0	xc5vlx20-2
Kaps et al. [96]	256	197.6	349	163	1	0	xc6vlx75t-1
Kaps et al. [96]	256	268.8	475	166	0	0	xc6vlx75t1
Umar et al. [162]	256	86.9	1,534	662	12	8	xc5
Umar et al. [162]	256	105.4	1,861	726	13	0	xc5
Beuchat et al. [6, 32]	512	374.0	280	81	3	0	xc6vlx75t-2
Kerckhof et al. [102]	512	240	183	192	0	0	xc6vlx75t-1
Kerckhof et al. [102]	512	304	232	215	0	0	xc6vlx75t-1

6.4.2 FPGA

Gaj et al. [71] published in 2012 one of the most comprehensive analyses of FPGA performance of SHA3 finalists and the SHA2 functions. The evaluation includes several architectures with different design styles and computes the principal performance figures for four FPGA devices. The architectures use a generic core and do not employ embedded resources of the FPGAs.

Table 6.2 provides a snapshot of the most significant results from that work. Comparing the throughput values, BLAKE-256 and BLAKE-512 are about five times as fast as the SHA2 algorithm in four FPGA processors. The higher speed, mainly due to the parallel processing in the BLAKE compression function, causes an increase in the circuit size. The final area/speed ratio of BLAKE is on average half that of SHA2.

Table 6.2 FPGA performance figures of the BLAKE function, with area measured in adaptive look-up tables (ALUTs).

Algorithm	Device	Throughput [Mbps]	Area [ALUTs]	Throughput-to-area [Mbps/ALUTs]
BLAKE-256	Xilinx Virtex 5	7,547	3,495	2.16
SHA-256	Xilinx Virtex 5	1,401	396	3.54
BLAKE-512	Xilinx Virtex 5	560	386	1.45
SHA-512	Xilinx Virtex 5	2,013	798	2.52
BLAKE-256	Xilinx Virtex 6	8,056	2,530	3.18
SHA-256	Xilinx Virtex 6	1,634	239	6.84
BLAKE-512	Xilinx Virtex 6	10,706	5,267	2.03
SHA-512	Xilinx Virtex 6	2,381	513	4.64
BLAKE-256	Altera Stratix III	7,583	6,267	1.21
SHA-256	Altera Stratix III	1,656	959	1.73
BLAKE-512	Altera Stratix III	9,980	12,074	0.83
SHA-512	Altera Stratix III	2,128	1,995	1.07
BLAKE-256	Altera Stratix IV	8,063	6,271	1.29
SHA-256	Altera Stratix IV	1,798	959	1.87
BLAKE-512	Altera Stratix IV	11,075	12,082	0.92
SHA-512	Altera Stratix IV	2,378	1,996	1.19

Table 6.3 CMOS performance figures for 256-bit hash sizes.

Algorithm	Technology [nm]	Latency [cycles]	Frequency [MHz]	Throughput [Gbps]	Area [kGEs]	Throughput-to-area [kpbs/GE]	Power [mW]	Energy-per-bit [mJ/Gb]
BLAKE-256 [77]	65	57	378.07	3.396	39.96	84.979	53.78 ^a	21.616 ^a
SHA-256 [77]	65	67	563.38	4.305	24.30	177.156	13.13 ^a	5.277 ^a
BLAKE-256 [71]	65	29	409.84	7.236	43.02	168.179	26.25 ^a	10.551 ^a
SHA-256 [71]	65	65	687.29	5.414	25.14	215.342	10.23 ^a	4.112 ^a
BLAKE-256 [76]	130	30	125	2.13	34.15	62.47	21.33 ^b	25.00 ^b
SHA-256 [76]	130	68	200	1.51	21.67	69.54	5.18 ^b	13.76 ^b

^a Measured with a target speed of 2.488 Gbps.
^b Measured with a fixed clock frequency of 50 MHz.

6.4.3 Discussion

Similarly to software engineers, the task of hardware engineers implementing BLAKE can simply consist in translating the specification to a hardware description language (HDL) syntax, and to implementing signaling appropriately. Implementation techniques to trade circuit area for latency are straightforward, and consist of standard horizontal or vertical folding. Unlike a hardware-oriented design such as Keccak, efficiency is impacted by the use of integer addition, which on the other hand improves software efficiency for the same security level.

Finally, note what NIST's final report comments regarding hardware performance of BLAKE:

In hardware, BLAKE is one of the most flexible SHA-3 finalists, since it can be folded vertically and horizontally by two or four, and pipelines readily within a single round. BLAKE gives the best performance of any algorithm for very compact FPGA implementations, and the same would probably be true for ASIC implementations. (...) High-performance implementations of BLAKE in FPGAs or ASICs typically require about twice the size of SHA-2, with about the same throughput, so BLAKE's throughput/area ratio is roughly half that of SHA-2.

Indeed, BLAKE's greatest advantage for hardware designers is its high flexibility, although its maximal speed and efficiency are lower than those of Keccak, partly due to BLAKE's use of integer addition for high speed in software.

Chapter 7

Design Rationale

We designed the best hash function we could.
—Bruce Schneier

This chapter explains why we designed BLAKE in the way we did, answering questions such as

- Why is there a counter input to the compression function?
- Why only use integer addition, XOR, and rotation?
- Why 14 and 16 rounds?
- Why an optional salt?

We attempted to make design choices according to requirements derived from the identified needs of future SHA3 users, as in a typical engineering project. This chapter is structured as follows: Section 7.1 first summarizes the requirements defined by NIST in its call for proposals, from minimal acceptance criteria to strict security requirements. Section 7.2 then reports an informal needs analysis, as the basis of our general design philosophy, which is exposed in Section 7.3. Section 7.4 presents concrete design choices for each component of BLAKE, in top-down order.

7.1 NIST Call for Submissions

NIST published the call for SHA3 submissions in November 2007 in the Federal Register.¹ We summarize the main requirements imposed on the SHA3 submissions, as well as the key evaluation criteria considered by NIST.

7.1.1 General Requirements

Informal requirements of SHA3 are first stated in the “Background” section of the FR notice:

¹ The official journal of the US government, shorthand FR.

Since SHA3 is expected to provide a simple substitute for the SHA2 family of hash functions, certain properties of the SHA2 hash functions must be preserved, including the input parameters; the output sizes; the collision resistance, preimage resistance, and second-preimage resistance properties; and the ‘one-pass’ streaming mode of execution.

Here “input parameters” should not be understood as “length of data blocks,” but rather as type, minimal and maximal sizes of the inputs. In the same paragraph, NIST lists examples of desirable features:

the selected SHA3 algorithm may offer efficient integral options, such as randomized hashing, that fundamentally improve security, or it may be parallelizable, more efficient to implement on some platforms, more suitable for certain applications, or may avoid some of the incidental ‘generic’ properties (such as length extension) of the Merkle–Damgård construct that often result in insecure applications.

As observed later, resistance to length extension was actually a strict requirement for SHA3, rather than only a desirable property.

NIST further states that it

expects SHA3 to have a security strength that is at least as good as the hash algorithms currently specified in FIPS 180-2, and that this security strength will be achieved with significantly improved efficiency.

This statement has been a source of discussion, because:

1. There is no known method to find numbers x and y such that x rounds of (say) SHA-256 have the same “cryptographic strength” as y rounds of SHA3 with a 256-bit digest.
2. The relative efficiency of SHA2 and SHA3 varies significantly across platforms. One specific platform may be chosen, but this would obviously bias the comparison.

It is therefore impossible to rigorously determine whether a given SHA3 finalist is more efficient than SHA2 at identical security strength. That said, a number of the 64 submissions turned out to be both slower and weaker than SHA2.

The second section of the FR notice is entitled “Requirements for Candidate Algorithm Submission Packages” and contains the “nomination requirements” for the SHA3 standard, as well as acceptable optional features.

After formal requirements on the documentation (need for a complete and intelligible specification, explanations for important design decisions, preliminary analysis, etc.), NIST suggests that

the submitted algorithm may include a tunable security parameter, such as the number of rounds, which would allow the selection of a range of possible security/performance trade-offs.

For most submissions, including BLAKE, this parameter was indeed the number of rounds. NIST explicitly states that

[it] is open to, and encourages, submissions of hash functions that differ from the traditional Merkle–Damgård model, using other structures, chaining modes, and possibly additional inputs.

This statement arguably encouraged the submission of original and innovative modes, as found in a number of submissions.

NIST then asks that submitters include

a statement of the algorithm's estimated computational efficiency and memory requirements in hardware and software across a variety of platforms.

Submissions were also required to include a series of test vectors—or “known answer tests”—as well as “Monte Carlo tests.” NIST provided C prototypes for reference implementations of submitted algorithms, as well as a C program to compute the said test results.

Submissions were further required to be

available worldwide on a royalty free basis during the period of the hash function competition.

Algorithms covered by a US or foreign patent (or patent application) were not formally excluded, but submitters are required to disclose this fact. As far as we can tell, none of the round 2 submissions was covered by a patent or patent application filed by its designers. Furthermore, most of the source code was published under permissive licenses, when a license was specified.

7.1.2 Technical and Security Requirements

The technical requirements appear in the sections “Minimum Acceptability Requirements” and “Evaluation Criteria” of the FR notice. The minimal requirements are straightforward:

The algorithm shall be implementable in a wide range of hardware and software platforms.

Strictly speaking, this requirement is satisfied by any algorithm, as any hardware or software platform is a universal computer, able to implement any algorithm.

The candidate algorithm shall be capable of supporting message digest sizes of 224, 256, 384, and 512 bits, and shall support a maximum message length of at least $2^{64} - 1$ bits.

That is, SHA3 was planned to support the same digest sizes as the SHA2 family. Note that NIST does not impose that this functionality should be achieved with one, two (like SHA2), or more distinct basic algorithms. It was expected, however, that four significantly distinct algorithms for the four required digest sizes would be perceived negatively.

The security requirements are probably the most interesting part of the call for submissions. Interestingly, the first to appear in the FR concerned keyed schemes, with the following statements:

When the candidate algorithm is used with HMAC to construct a PRF as specified in the submitted package, that PRF must resist any distinguishing attack that requires much fewer than $2^{n/2}$ queries and significantly less computation than a preimage attack.

(...)

Any additional PRF constructions specified for use with the candidate algorithm must provide the security that is claimed in the submission document.

Note that the latter statement concerns optional PRF constructions, and does not specify the security level required. Another optional feature is explicit support for randomized hashing. For this application, NIST provides a concrete attack scenario that proposed algorithms should resist:

The attacker chooses a message M_1 of length at most 2^k bits. The specified construct is then used on M_1 with a randomization value r_1 that has been randomly chosen without the attacker's control after the attacker has supplied M_1 . Given r_1 , the attacker then attempts to find a second message M_2 and randomization value r_2 that yield the same randomized hash value.

In other words, the attacker has to find a second preimage for the hash function such that a random value is part of the input.

In the section “Additional Security Requirements of the Hash Functions”, NIST defines concrete security bounds for hash functions producing an n -bit digest:

- collision resistance of approximately $n/2$ bits,
- preimage resistance of approximately n bits,
- second-preimage resistance of approximately $n - k$ bits for any message shorter than 2^k bits,
- resistance to length-extension attacks, and
- any m -bit hash function specified by taking a fixed subset of the candidate function's output bits is expected to meet the above requirements with m replacing n .

A security of “approximately n bits” means that the amount of computation needed to break the security notion with high probability is of the order of 2^n elementary operations. Here an elementary operation is an evaluation of the hash function, but the exact definition is not extremely important.

NIST finally comments that

[these] requirements are believed to be satisfiable by fairly standard hash algorithm constructions; any result that shows that the candidate algorithm does not meet these requirements will be considered to be a serious attack.

Among the 64 submissions received, more than 20 were shown not to meet these requirements, including more than 10 for which practical attacks were found. However, all five finalists appear to easily satisfy the security requirements imposed.

7.1.3 Could SHA2 Be SHA3?

Does SHA2 satisfy the requirements defined by NIST for SHA3? The answer is clearly negative since all SHA2 instances are vulnerable to the length-extension attack, whereas NIST's call imposes resistance to that attack for SHA3. Nevertheless,

the length extension property does not affect the security of SHA2 when properly used (for example, when using any of the constructions recommended by NIST, such as HMAC or randomized hashing). To the best of our understanding, however, SHA2 complies with all the other requirements for SHA3.

In terms of performance, SHA2 is noticeably slower than SHA1, but turned out to be more efficient than most SHA3 submissions, although it is outperformed on recent platforms by two of the five finalists. Moreover, implementations of SHA2 require relatively low memory and hardware area compared with most of the SHA3 submissions.

Note that, according to NIST in the SHA3 call for proposals,

SHA3 is intended to augment the existing NIST-approved hash algorithm toolkit, which includes the SHA2 family of hash functions.

SHA3 is thus not a replacement for the SHA2 standard family of hash functions. As it turned out, NIST picked a winner that complements well the SHA2 standards.

7.2 Needs Analysis

As stated above, SHA3 will be included in the list of cryptographic hash functions approved by NIST, and will be a Federal Information Processing Standard, namely FIPS-202. Like its predecessors SHA1 and SHA2, the scope of the SHA3 standard is not restricted to a subset of applications or platforms, but aims to be appropriate wherever a cryptographic hash function is required (password-based key derivation a.k.a. password hashing being excluded, for it requires specific, slow, hash functions); that is, the current federal standard FIPS 180-4 (Secure Hash Standard), as prepared by NIST, defines the applicability of SHA1 and SHA2 as follows [137, p.V]:

This Standard is applicable to all Federal departments and agencies for the protection of sensitive unclassified information that is not subject to Title 10 United States Code Section 2315 (10 USC 2315) and that is not within a national security system (...). This standard shall be implemented whenever a secure hash algorithm is required for Federal applications, including use by other cryptographic algorithms and protocols. (...) The secure hash algorithms specified herein may be implemented in software, firmware, hardware or any combination thereof.

Furthermore, although the mission of NIST is to provide standards to US government agencies and businesses, SHA3 is expected to become, sooner or later, a de facto worldwide standard, like AES. Potential users of SHA3 will thus be numerous and diverse, with heterogeneous and sometimes contradictory needs. Implementations will be realized in languages ranging from JavaScript or C to hardware HDLs and 8-bit assemblers; implementers will range from experienced cryptography engineers to junior developers.

Identifying the needs of SHA3 users is thus a challenging task. This section attempts to identify these main needs, and in particular to determine what would encourage each class of users to choose SHA3 rather than SHA2.

7.2.1 *Ease of Implementation*

Academic evaluations of cryptographic algorithms generally focus on the efficiency of the fastest implementation for a specific platform, rather than that of portable code. The efficiency of an algorithm is then often associated to that of its *fastest* implementation on mainstream platforms, regardless of the complexity, security, and portability of that implementation.

In particular, the cost of writing efficient implementations is generally overlooked. However, developers seldom have the time or the skills to write the best possible implementation for the platform considered. Instead, they aim to minimize the implementation time necessary to write a fast enough and working code, before moving on to another part of the project, hoping to deliver the software on time. An option is sometimes to reuse existing open-source software, but that is not always appropriate, for reasons that can be either

- Technical: inadequate API, language standard, efficiency, etc.
- Legal: inadequate licenses (typically GPL or other “viral” licenses), patents, etc.
- Other: internal policy forbidding the use of open-source software, etc.

An algorithm that is difficult to implement and to test can not only delay the project and/or increase its cost, but can also dramatically alter the quality of the end product if bugs have not been identified in time. Moreover, the common lack of rigorous unit testing of cryptographic algorithms tends to leave many corner case bugs: implementations are often tested with only a handful of test vectors that do not cover special cases (such as unusual input lengths, malformed input, etc.).

Clearly, algorithms requiring sophisticated implementation and testing techniques are thus undesirable—especially given the relative simplicity of SHA2 and of its implementations. We therefore considered ease of implementation as a major need. Relevant criteria include:

- **Inherent simplicity** of the algorithm: modularity, design symmetries, number of distinct components (the fewer the better), complexity of the primitive operations, etc.
- **Clarity** of specification: how concise and intelligible is a comprehensive description of the algorithm? Can any part of the description be misinterpreted? Etc.
- **Prerequisites** required to understand and implement the algorithm: mathematical notions (e.g., polynomials over finite fields), programming techniques (e.g., bitslicing), etc.
- Is the algorithm easily **translated** to an imperative programming language’s syntax from the pseudocode of the specification? Counterexamples are AES (a round never reproduces the textbook specification, but instead uses large precomputed tables) and the AES finalist Serpent (the S-box is described as a look-up table but implemented as a logical circuit).
- **Failure-friendliness**: what is the risk of misunderstanding and of coding errors? Is there a risk of confusion due to unintuitive endianness, word size, etc.?
- **Ease of testing and debugging**: are there many components to be tested? How long and diverse is the code (the more lines of code, the more bugs)? Etc.

Our design philosophy based on those criteria is exposed in Section 7.3.1.

7.2.2 Performance

We assumed that SHA3 would be considered a failure by the public if it were perceived as noticeably slower than SHA2, regardless of its perceived security margin.

Although many applications could use a function two or three times slower than SHA2 without any perceptible performance degradation, there are applications where faster hashing noticeably affects costs and/or user experience: revision control systems, file systems supporting integrity checking, or cloud storage systems integrating deduplication features (e.g., ZFS). Moreover, the most popular benchmark platforms are laptop, desktop, and server microprocessors from the two mainstream CPU vendors. We thus required that BLAKE be consistently faster than (or about as fast as) SHA2 across high-end software platforms.

In embedded software applications, memory footprint—RAM and ROM—is often more critical than speed, for example on smaller microcontrollers embedded in consumer products. Depending on the application, speed should also be competitive with that of SHA2, on platforms from 8-bit to 32-bit architectures; for example, the use of only 64-bit arithmetic can benefit high-end processors, but penalizes low-end platforms.

Hardware designers are generally mainly concerned with the area occupied by a “reasonable implementation,” that is, one that is optimized neither for the highest speed nor for the lowest area. Speed in hardware is seldom critical, however too large an area is the most common obstacle to the deployment of a cryptographic algorithm. Like for software platforms, users expect SHA3 to improve over SHA2 in at least one aspect, be it speed, size, or diversity of architectures.

7.2.3 Security

Unlike performance, there is no known metric to reliably estimate the security of a cryptographic algorithm—that is, the overall amount of effort required to mount a useful attack on the algorithm, given the current technology. One therefore often talks of *confidence* in security, and of “security margin.” To estimate the security margin of an algorithm, a common heuristic is to compute the ratio between the total number of rounds and the number of rounds broken (for some definition thereof).

Although comparing security margins does not always make sense—like saying that SHA1 is more secure with 3,000 than with 300 rounds—it may be the best tool available to compare algorithms in terms of security. To be widely deployed, SHA3 should thus have a security margin at least as high as that of SHA2 at the time of selection, and this for any reasonable definition of “broken.”

At the time of writing, a method is known [103] to compute preimages slightly faster than expected for a version of SHA-256 with 45 rounds out of 64, giving a security margin of $64/45 \approx 1.42$. Another attack works on only 28 steps and finds collisions in practical time [128], however the security margin metric does not consider attacks that are practical but target fewer rounds. On SHA-512, a preimage attack allegedly slightly more efficient than a generic attack was found [103] on 57 rounds out of 80, giving a security margin of $80/57 \approx 1.40$, a value similar to the security margin of SHA-256 at the time.

7.2.4 Extra Features

Besides serving as a drop-in replacement for SHA2 with better performance and/or security margin, SHA3 may offer functionalities not found in the previous hash standards. An example of such functionality is keyed hashing, with MACs as the main application (and PRF, an equivalent object in terms of security): today MACs and PRFs are often instantiated with HMAC-SHA1 or HMAC-SHA-256, and then used to implement encrypt-then-MAC, PBKDF2, etc. However, HMAC is overly complicated for what it does—keyed hashing—and is suboptimal for short messages, due to its two calls to the hash function. The possibility to build a simpler and more efficient MAC, either implicitly or explicitly, may thus be appreciated by users and standardization organizations.²

Another example of a relevant extra feature would be support for a *salt*, that is, an additional short input that aims to diversify the hash function. A salt can be used to implement randomized hashing, to replace constructions such as RMX [78, 134]. A salt may also be used to personalize an implementation of SHA3, for example, to ensure that each product or customer is using a distinct algorithm, yet all the algorithms fully comply with the definition of the SHA3 standard.

One may imagine a number of other extra features: integration of parameters for tree hashing and/or parallel hashing, personalization, (password-based) key-derivation, etc. However the more features are supported, the more complex the specification and implementation of the algorithm. Users may be satisfied by transparent support of more functionalities than in SHA3, however the algorithm should remain as simple as a basic hash function to simplify its analysis and implementation during the SHA3 competition.

7.3 Design Philosophy

Our general philosophy was to design a cryptographic algorithm that would satisfy all users regardless of their background and their expertise in cryptography. In

² BLAKE supports simple prefix-MAC implicitly, and BLAKE2 explicit the support with well-defined signaling.

other words, BLAKE does not aim to be optimized for a single application or with respect to a single metric, but rather with respect to a homogeneous aggregation of several notions. We acknowledge that the SHA3 competition is an *engineering* project, and thus not the ideal venue for highly experimental or sophisticated algorithms. We derived our requirements from NIST’s evaluation criteria and from the users’ needs (as analyzed in Section 7.2), and refrained from using too complex or innovative techniques. Indeed, the following analogy can be made. SHA3 is more like an automotive system than a mobile application: it cannot be updated once put in production, and even minor bugs can have dramatic consequences (in theory, the FIPS standard could be updated, but that would be embarrassing for NIST and troublesome for users). It thus makes sense to engineer SHA3 as a component of an aerospace system, by favoring robustness and simplicity over sophistication and novelty, in order to maximize confidence and to minimize analysis efforts.

In terms of efficiency, we wished to design an algorithm that performed at least as well as SHA2 on any platform with respect to at least one metric—be it speed, code or circuit size, efficiency, memory consumption, etc.

The rest of this section describes the three pillars of our design philosophy: simplicity and minimalism, prior art reuse, and versatility.

7.3.1 Minimalism

Designing a complicated and secure algorithm is fairly easy; examples are plentiful in the literature, industry, and cryptographic competitions. However, such algorithms, although never broken, are never used.

As in many undertakings, the difficulty lies in doing things in the simplest possible way, and creating a system that consumes no more resources than necessary (be it computing power or human brainpower). Because the ultimate goal of a cryptographic algorithm is to be *used* rather than to eternally remain the sole object of academic research, we support a notion of elegance that is more concerned with minimalism and simplicity than with mathematical beauty—an elitist and subjective notion. The rest of this section discusses the notion of simplicity applied to cryptographic algorithms, explains its advantages, and how to realize it.

7.3.1.1 General Definitions

The designers of Rijndael (AES) distinguish “*simplicity of specification*” from “*simplicity of analysis*” [58, 5.2], and summarize the former as

[making] use of a limited number of operations [that] can be easily explained.

Simplicity of analysis is then defined as

the ability to demonstrate and understand in what way the cipher offers protection against known types of cryptanalysis.

Daemen and Rijmen note that simplicity of specification does not necessarily imply simplicity of analysis, and that the converse holds as well.

One may distinguish another dimension of simplicity: *simplicity of implementation*, and more precisely simplicity to write a reasonably efficient implementation, regardless of the platform. Indeed, as Rijndael/AES illustrates, simplicity of specification does not necessarily imply simplicity of implementation, because notions that seem simple on paper may not be simple to translate to a programming language.

Obviously, notions of simplicity are relative to the context: an experienced cryptographer with a mathematical background and a programmer may disagree on the simplicity of a cryptographic algorithm (for example, “simple finite-field arithmetic” sounds like an oxymoron to many), just like equally experienced programmers on different platforms may have different notions of simplicity of implementation (for example, 64-bit arithmetic is simple when writing C for a 64-bit platform, but less so when programming in 8-bit assembly).

Simplicity of analysis is an even fuzzier notion, as it strongly depends on the current body of knowledge regarding attacks and proof techniques. History has shown that algorithms placing too much confidence on “proving security” against a subset of attacks had lesser resilience to other (and new) attacks; for example, VSH [54] claimed provable security against collision attacks but is not preimage resistant [157]; the SHA-3 candidate FSB [8] needs postprocessing by a “real” hash function to eliminate structural biases.

7.3.1.2 Benefits of Simplicity

In the security engineering literature it is common to read things like “complexity is the main enemy of security” [67] and thus that more simplicity tends to mean more security.³ Dan Geer, for example, argues that [72]

complexity provides both opportunity and hiding places for attackers.

Similar arguments apply to cryptography, to some extent: a clear and succinct specification, few lines of code, few components, and simple operations will encourage cryptanalysts to analyze the algorithm and to report any finding. Conversely, many algorithms remain unbroken even with very few rounds because nobody made the effort to understand their cryptic specification.⁴ Daemen and Rijmen sum it up by saying that [58, 5.2]

the simplicity of a cipher contributes to the appeal it has for cryptanalysts, and in the absence of successful cryptanalysis, to its cryptographic credibility.

Perhaps the most obvious advantage of simplicity is that it dramatically reduces the work time, and thus the global cost of an implementation. As one can observe

³ This statement admits exceptions when complexity is a desirable feature (for example, to make reverse engineering more difficult), or when specific complexity metrics are considered [163].

⁴ Finding references is left as an exercise to the reader.

in industry, the workflow of understanding the specification of a cryptographic algorithm and implementing, debugging, and testing it can range from a few hours to several weeks, depending on the complexity of the design (and on the implementation language and platform). From a more general economic standpoint, it is undesirable to spend more resources than necessary to achieve a given task; in other words, why make things orders of magnitude more complicated than they could be?

7.3.1.3 Implementing Simplicity

We discuss several properties that define or relate to the simplicity of a cryptographic algorithm.

Conciseness

Simplicity is often synonymous with *conciseness*: of the pseudocode, and of the actual source code. Conciseness of the pseudocode (and of the specification in general) means less effort and energy required to understand the algorithm. It generally implies shorter source code, and thus shorter development time. Conciseness can be achieved by replicating similar and/or identical operations, which requires the existence of symmetries within the algorithm's structure.

Symmetries

A general strategy to minimize the size of the description of a program is to introduce *symmetries*, that is, structural similarities; for example, the iteration of a round function is a design symmetry that allows one to specify the algorithm as

1. a parametrized round function
2. a number of iterations
3. the parameters to use for each of the rounds

Introducing design symmetries simplifies the understanding of the algorithm by reducing the amount of information to digest. It also simplifies implementation by allowing reuse of the same code in different parts of the program. Within a round, symmetry may be introduced through *modularity*, that is, defining a component as the sequential or parallel application of similar subcomponents.

Clearly, the amount of symmetry in a design has to be limited, as the goal of cryptanalysis is precisely to identify and exploit structures in the design. Design symmetries should thus be combined in such a way that the function defined is structureless. To break linearity over some algebraic structure—a particular kind of symmetry—one can introduce linear operations over another algebraic structure; for example, an algorithm combining XOR, integer addition, and wordwise rotations can break linearity with respect to each of those three operators.

Diversity

The notion of *diversity* applied to cryptographic algorithms relates to the variety of components and operators employed.

In security engineering, more diverse operations may be seen as either enlarging the attack surface (potentially increasing the general risk of attacks) or applying defense in depth (mitigating the impact of attacks). Although the latter principle has sometimes been applied in cryptography—for example, with the AES candidate MARS—it is generally deemed irrelevant, contrary to the former: a number of algorithms have been attacked by exploiting a single weakness in one of their components. The designers of the AES candidate Twofish share this view, writing [159, 6.3]

Cryptographic design does not lend itself to the adage of not putting all your eggs in one basket. Since any particular ‘basket’ has the potential of breaking the entire cipher, it makes more sense to use as few baskets as possible—and to scrutinize those baskets intensely.

Instead of using many distinct operators and components, it is sufficient to use two types of algebraically incompatible functions, as found in many substitution–permutation algorithms: these iterate a round function composed of:

- A linear transform, such as a bit permutation (as in PRESENT [47]) or a matrix multiplication (as in AES),
- A nonlinear transform, such as the substitution of each byte by its image through an “S-box”.

This approach minimizes the number of distinct operators and components to what is necessary to achieve cryptographic strength. An alternative to S-boxes is the explicit combination of incompatible operators, which has the advantage of avoiding pitfalls of S-box implementations (see Section 7.4.3.2). This approach was pioneered with the block cipher IDEA, designed by Massey and Lai in 1991 [113]: IDEA works on 16-bit words and combines XOR, (16-bit) integer addition, and multiplication modulo $2^{16} + 1$ (not 2^{16}), a strategy that proved effective, as IDEA remains practically unbroken after more than 20 years of cryptanalysis attempts.

Prior Knowledge

A criterion often overlooked is the *prior knowledge* required to understand the algorithm; for example, understanding AES internals requires knowledge of mathematical notions related to finite-field algebra, such as modular inverse, polynomials over finite fields, etc. Although this is basic algebra that is well understood by most cryptographers, it is often not familiar to software engineers, who thus have to make extra effort to fully understand AES’s operations and optimize it if necessary. As shown by designs such as RC4 or Salsa20, a minimal set of simplistic operations is sufficient for fast and secure algorithms.

Isomorphism

The life of implementers is made much easier when the paper specification is “isomorphic” to a typical implementation; that is, implementing the algorithm is essentially just translating the specification document to a given programming language. Again, AES is a counterexample: whereas textbooks describe an AES round as the sequence SubBytes, ShiftRows, MixColumns, and AddRoundKey, any reasonable implementation for high-end processors uses large precomputed tables, as described in [58, 4.2]. The AES finalist Serpent is not much different: whereas it is described as using an S-box as a 4-bit lookup table, fast software implementations actually implement the S-box as a sequence of logical operations.

Extra Features

Finally, the addition of invasive extra features often complicates the specification of an algorithm and increases the risk of implementation errors (for example, by confusing the signaling for two different modes of operation). It is thus preferred that any additional feature be supported transparently, with only minimal changes to the basic design.

7.3.2 Robustness

As stated in the introduction of this section, it will not be possible to fix SHA3 should a problem occur after it is selected and deployed (although SHA0 was fixed to SHA1 shortly after being defined). As designers of a SHA3 candidate, we thus followed the same approach as NASA engineers did when sending a rover to Mars: build on solid components using recent technology but not too recent to reduce the risk of undetected bugs. An advantage of this approach is that the resulting design will already look familiar to cryptanalysts and implementers, thus saving precious time during the evaluation process. We deemed it essential to build on previous knowledge and work from the community—be it about security or performance—in order to cope with the low resources available to analyze SHA3 candidates. Indeed, the literature is rich enough in secure and well-analyzed schemes to save us the task of designing yet other new schemes with little added value but their novelty.

A potential disadvantage of a conservative approach is that the resulting design may not look extremely innovative. But as explained above, our point of view is that a competition like AES or SHA3 is more about consolidating knowledge acquired during the past years of research than about proposing brand-new approaches.

7.3.3 *Versatility*

Versatility is defined in [58, 5.1.4] as the property of being “efficient on the widest range of processors possible.” More generally, a more versatile algorithm performs well on all platforms, software or hardware, which assumes in the first place that it can be implemented and executed on all reasonable platforms.

An algorithm optimized for a specific platform is unlikely to be the most versatile, since optimization consists in adapting the algorithm to best exploit the resources of the target platform: register size, instruction set, type of memory, etc.; for example, one may wish to optimize an algorithm for the most recent desktop processors, by exploiting 64-bit arithmetic, SIMD instruction extension sets, etc. However, focusing on such a sophisticated platform will strongly penalize low-end devices, which are equipped with only basic instructions on words of at most 32 bits. Conversely, optimizing for 8-bit microcontrollers may yield a high level of efficiency (e.g., in terms of security/speed), but it may under-exploit features of more powerful processors.

Another disadvantage of optimization is that it tends to complicate the specification, for example, by introducing sequences of operations minimizing a processor’s stalls.

We thus imposed the following guidelines:

- Choose an algorithm that can exploit features of common processors but not to the point of significantly penalizing other platforms, and making sure that it remains fast when restricted to the most basic instructions.
- When having to choose between optimizing the algorithm internals for security or for efficiency, opt for the latter and add rounds if necessary (see the choice of rotation constants in Section 7.4.4).
- Offer several degrees of parallelism, following a general trend in recent and future processors (mainly with SIMD instructions and instruction-level parallelism), and enabling a larger design space of hardware architectures.
- Ensure that a basic portable reference C implementation does compile for and is reasonably efficient on all platforms. Make the writing of the reference implementation as language-agnostic as possible, by using only the most basic instructions.
- Generally, refrain from any optimization, be it for software or hardware platforms, that would significantly penalize another platform.

7.4 Design Choices

This section explains how we designed BLAKE, based on NIST’s requirements and on the above design philosophy, as derived from our analysis of users’ needs. Going top-down, we present and justify all the major choices, from the high-level interface to the rotation constants in the core algorithm.

7.4.1 General Choices

As required by NIST, we designed BLAKE to exhibit the following properties:

- produce digests of 224, 256, 384, and 512 bits
- support maximum message length of at least $2^{64} - 1$ bits
- process data in a one-pass streaming mode (that is, only read each message block once)

In addition, we imposed BLAKE to:

- Have exactly the same interface as SHA2 (message block length, etc.), so that BLAKE fits as a drop-in replacement for SHA2 in most applications. In particular, like SHA2 and SHA1 (but unlike MD5), BLAKE parses input byte arrays to 32- or 64-bit words in a big-endian way.
- Perform well on all software or hardware platforms, and allow several degrees of space/time performance tradeoff through vertical and horizontal folding.
- Support an optional salt input to implement randomized hashing and diversification, but minimize the impact on the design. As observed in the specifications, the salt takes the place of constants and thus does not impose additional algorithmic operations.

Like SHA2, we chose to design two similar algorithms working, respectively, with 32- and 64-bit words. Compared with a monolithic design, this allows the 64-bit BLAKE-512 to take full advantage of 64-bit arithmetic implemented in desktop and server processors, whereas the 32-bit BLAKE-256 is more compact and better suited for 8- to 32-bit processors.

We excluded the following features:

- Reduction to a supposedly hard problem: The relative failure of provably secure hash functions shows the limitations of this approach; although of theoretical interest, such designs tend to be inefficient, and their highly structured constructions expose them to attacks with respect to security notions other than the one being proven.
- Homomorphic or incremental properties: The few advantages of homomorphic and incremental hash functions are not worth their cost; more importantly, these properties are undesirable in many applications.
- Specification for variable-length hashing: in practice, users can just truncate the hash values, and there is only marginal demand for hash values of more than 512 bits (specific constructions can be used to produce more than 512 bits).
- Explicit definition of MAC and RNG modes: adding explicit support for MAC and RNG functionalities would have required additional definitions and specifications. We opted for a simpler design, since BLAKE's operation mode is known (for example) to allow secure prefix-MAC constructions, if needed in a future version of the SHA3 standard.

7.4.2 Iteration Mode

The iteration mode of BLAKE is a stripped version of the Hash Iterative Framework (HAIFA, see Section 2.4.2), as proposed by Biham and Dunkelman “to solve many of the pitfalls of the Merkle–Damgård construction” [35, 3]. We chose HAIFA because it is the simplest, minimal iteration mode that fixes Merkle–Damgård, and that supports salt. In addition, in its original version HAIFA supports variable-length hashing, by using an IV and a padding that depend on the digest size. Since BLAKE does not aim to produce digests of arbitrary length, we simplify HAIFA by defining specific IVs and by minimizing the padding difference (i.e., one bit is sufficient to differentiate BLAKE-256 from BLAKE-224). Furthermore, HAIFA provides the highest security level, namely indistinguishability from a random oracle. Security properties are studied in detail in Section 8.5.1.

The iteration mode of BLAKE is the so-called *narrow-pipe*, that is, where the chaining values are of the same length as the digest, as opposed to wide-pipe modes, which use larger chaining values. A counterargument is that narrow-pipe designs provide lower theoretical security than wide-pipe designs [104], but such objections are irrelevant and lie far beyond the scope of the SHA3 security requirements.

Internally to the compression function BLAKE, uses a *local wide-pipe*, as introduced in the LAKE hash function [17]: an internal state twice as large as the chaining value is initialized with the salt and the counter, and transformed with a key permutation parametrized by the data block. The larger state of the local wide-pipe allows to simply process the additional inputs, ensures that no internal collision exists for a fixed data block, and makes fixed points difficult to find (and thus to exploit). The finalization step shrinking the state size thus provides an additional security layer, by hiding the final internal state when the IV is known. Compared with a wide-pipe construction with chaining values as large as the local wide-pipe, the BLAKE-256 mode of operation saves 256 bits of memory by storing a 256-bit rather than a 512-bit chaining value to perform feedforward.

An objection to this construction is that using the chaining value as a key of the permutation would exclude internal collisions for distinct messages. However, this type of construction, as adopted by Skein [66], is less resilient to powerful side-channel attacks, since the data block can be recovered from any internal state (see [51]).

7.4.3 Core Algorithm

The core algorithm of BLAKE is based on ChaCha [23], a stream cipher designed by Daniel J. Bernstein as a variant of Salsa20 [24]. We explain why we chose ChaCha, and how we transformed it to a (64-bit) block cipher.

7.4.3.1 ChaCha Core

ChaCha is a variant of Salsa20, a stream cipher submitted in 2005 to the eSTREAM competition, a project of the ECRYPT European network of excellence, “to promote the design of efficient and compact stream ciphers suitable for widespread adoption.”⁵ Thanks to its simplicity and high speed, Salsa20 has been included in several products and projects (the libraries NaCl and Crypto++, the Tahoe-LAFS cloud storage system, the KeePass password manager, the scrypt password hashing scheme, etc.). Salsa20 defines a quarterround function invertibly mapping four 32-bit words (y_0, y_1, y_2, y_3) to (z_0, z_1, z_2, z_3) as follows:

$$\begin{aligned} z_1 &:= y_1 \oplus ((y_0 + y_3) \lll 7) \\ z_2 &:= y_2 \oplus ((z_1 + y_0) \lll 9) \\ z_3 &:= y_3 \oplus ((z_2 + z_1) \lll 13) \\ z_0 &:= y_0 \oplus ((z_3 + z_2) \lll 18) \end{aligned}$$

The quarterround is then applied to each column, and to each round, of a 4×4 state of 32-bit words. ChaCha instead transforms four words a, b, c, d as follows:

$$\begin{aligned} a &:= a + b \\ d &:= (d \oplus a) \ggg 16 \\ c &:= c + d \\ b &:= (b \oplus c) \ggg 12 \\ a &:= a + b \\ d &:= (d \oplus a) \ggg 8 \\ c &:= c + d \\ b &:= (b \oplus c) \ggg 7 \end{aligned}$$

and then transforms the state by applying the above function to columns and diagonals, instead of columns and rows. Quoting its designer [23],

ChaCha, like Salsa20, uses 4 additions and 4 XORs and 4 rotations to invertibly update 4 32-bit state words. However, ChaCha applies the operations in a different order, and in particular updates each word twice rather than once. (...) Obviously the ChaCha quarter-round, unlike the Salsa20 quarter-round, gives each input word a chance to affect each output word.

Clearly, ChaCha satisfies our desideratum of simplicity, given its minimalism and design symmetry: it consists of a minimal set of basic operations, and repeats the same pattern of addition, rotation, and XOR for each of the four words transformed, and this for each column and diagonal, for each of the rounds.

The ChaCha core, as used in BLAKE, can be seen as repeated computations of the G: eight per round, thus 112 in BLAKE-256 and 128 in BLAKE-512. Using many simple iterations of a simple function rather than few of a complicated function has the following advantages, as explained by the designers of the SHA3 finalist Skein [66, 8.1]:

⁵ <http://www.ecrypt.eu.org/stream/>.

There are advantages to using many simple rounds. The resultant algorithm is easier to understand and analyze. Implementations can be chosen to be small and slow by iterating every round, large and fast by unrolling all rounds, or somewhere in between.

Note the similarity between Salsa20/ChaCha and AES: both view the state as a 4×4 array and transform each column independently. SIMD implementations of ChaCha and BLAKE perform the diagonal step as a shift of the rows followed by a transform of the columns, on the model of AES.

7.4.3.2 No S-Boxes

In cryptographic primitives, “S-boxes” are generally lookup tables with an index in $[0; 2^n - 1]$ (i.e., with n -bit input) and a value in $[0; 2^m - 1]$ (i.e., with m -bit output). A common choice of parameters is $n = m = 8$ in such a way that each S-box value is unique, and thus that the function defined is a permutation (as in AES). Indeed, S-boxes are just a method to perform certain types of function, rather than a specific type of function; in particular, it is obvious that all functions implemented through S-boxes can be implemented as a sequence of logical operations. Along those lines, Bernstein writes [22, 1]:

S-boxes are a feature of software, not of the mathematical functions computed by that software. For example, one can write AES software that does not use S-boxes, or (much faster) AES software that uses S-boxes.

Note however that the block cipher Serpent [3] relies on 4-to-4-bit S-boxes, as defined in its specification, but these are generally implemented as a sequence of logical operations [141].

Even before choosing ChaCha as core algorithm, we decided not to rely on S-boxes, for essentially the same reasons as those why Salsa20 does not use S-boxes:

The basic counterargument is that a simple integer operation takes one or two 32-bit inputs rather than one 8-bit input, so it effectively mangles several 8-bit inputs at once. It is not obvious that a series of S-box lookups—even with rather large S-boxes, as in AES, increasing L1 cache pressure on large CPUs and forcing different implementation techniques for small CPUs—is faster than a comparably complex series of integer operations.

A further argument against S-box lookups is that, on most platforms, they are vulnerable to timing attacks [22, 2].

One may argue that, like in Serpent, S-boxes could be made small and implemented as a small set of logical operations—as they are generally in hardware implementations. However, as previously noted in Section 7.2.1, this complicates the implementation by requiring specific techniques that differ from the specification, and introduces the risk of less secure implementations based on table lookups.

7.4.3.3 Pure ARX

BLAKE only uses integer addition, XOR, and rotation—it is a so-called ARX algorithm.⁶ These three operations are sufficient to design a secure algorithm, as they form a *universal* set of operations; that is, any computable function can be expressed as a combination of addition, XOR, and rotation. In particular, chaining XOR and integer addition ensures that the algorithm is not linear with respect to either of those operations, and rotations ensure that any input bit can influence any output bit.

We avoid the use of logical OR or AND operators, because they generally do more harm than good to cryptographic algorithms: OR and AND have the ability to destroy information, namely differences in their operands. This can be exploited in differential attacks to form a collision from two distinct values, as illustrated by attacks on MD5 and SHA1 [173, 174].

Rotations, unlike additions and XORs, generally have no dedicated instruction and have to be simulated with two shift operations (or more on platforms that only have 1-bit shifts). However, some CPUs can perform the two shifts in parallel, some have native rotation instructions (like the instruction `vprotq` in AMD’s Bulldozer), and some rotations can be performed by just reordering the bytes (see, e.g., Section 5.2.1.1).

The rotation counts are fixed rather than dependent on the data, to prevent attackers from controlling the operations in order to use “weak rotations,” for example, by forcing all the counts to be zero; history has shown that data-dependent rotations are generally a bad idea [11, 106].

7.4.4 Rotation Counts

BLAKE-256 uses the same rotation counts as ChaCha, namely 16, 12, 8, and 7. As shown in Sections 5.2.1.1 and 5.4.4, counts that are multiples of 8 can be implemented by just reordering bytes, which is often faster than shifting the words, as the byte alignment allows to implement the rotation by swapping bytes rather than by using shift instructions. Indeed, many 8-bit microcontrollers have only 1-bit shifts of bytes, so rotation by (e.g.) 3 bits is particularly expensive; implementing a rotation by a mere permutation of bytes greatly speeds up ARX algorithms. Rotation of 7 thus has the advantage that it is just one bit away from 8, which is an advantage on platforms with only 1-bit shift instructions (such as 8-bit AVR).

Since ChaCha was only specified for 32-bit words, we had to select rotations for the 64-bit version used in BLAKE-512. We chose 32, 25, 16, and 11 so that, like in BLAKE-256, two rotation counts are multiples of 8 and one is one-bit away from a multiple of 8. We checked several sets of rotation counts and picked the one that seemed to provide the best diffusion, among those satisfying the above criteria.

⁶ An abbreviation introduced in 2009 by Weinmann, originally “AXR” [176].

It is conjectured that the exact values of the rotation counts have relatively low influence on the security of the algorithm, as long as the values are not obviously bad for diffusion (e.g., all zero, or all one). It is observed in [10] that

[finding] really bad rotation counts for ARX algorithms turns out to be difficult. For example, randomly setting all rotations in BLAKE-512 or Skein to a value in $\{8, 16, 24, \dots, 56\}$ may allow known attacks to reach slightly more rounds, but no dramatic improvement is expected.

This conjecture is based on discussions with some authors of differential cryptanalysis attacks on BLAKE as well as on our own experiments.

7.4.4.1 Message Injection

BLAKE injects the message into the internal state by using it as a *key* of a keyed permutation, similarly to the so-called Davies–Meyer construction used by MD5, SHA1, and SHA2. This type of injection is thus very common among hash functions, and relatively well understood.

Each message word is injected exactly once within each round through:

1. an XOR to a constant, different at each round, for a simple diversification of the message word
2. an integer addition to the internal state

Therefore, any two different message words will give different values in the internal state, as opposed to an injection that would use logical OR or AND.

A message injection can be seen as a tradeoff between the injection rate (that is, the amount of bits injected per unit time) and the amount of diffusion between two consecutive injections (to avoid “perturb-and-correct” attacks). In BLAKE, we attempt to address this tradeoff by ensuring that a single message word affects up to four internal state words before the next message word injection. This is achieved by injecting two message words per G function; after several prototype designs, we deemed that injecting four words is too much, and one not enough. To break any symmetry, notably to mitigate perturb-and-correct attacks, each message is injected at a different position in each round, and a number of criteria are imposed on these positions, as described in the next section.

7.4.5 Permutations

The permutations $\sigma_0, \dots, \sigma_9$ were chosen to meet several security criteria. First we ensure that the same input difference does not appear twice at the same place (to complicate “correction” of differences in the state). Second, for a random message all values $(m_{\sigma_r(2i)} \oplus c_{\sigma_r(2i+1)})$ and $(m_{\sigma_r(2i+1)} \oplus c_{\sigma_r(2i)})$ should be distinct with high probability. For chosen messages, this guarantees that each message word will be XORed with different constants, and thus apply distinct transformations to the state

through rounds. It also implies that no pair (m_i, m_j) is input twice in the same G_i . Finally, the position of the inputs should be balanced: in a round, a given message word is input either in a column step or in a diagonal step, and appears either first or second in the computation of G_i . We ensure that each message word appears as many times in a column step as in a diagonal step, and as many times first as second within a step. To summarize:

1. no message word should be input twice at the same point;
2. no message word should be XORed twice with the same constant;
3. each message word should appear exactly five times in a column step and five times in a diagonal step;
4. each message word should appear exactly five times in first position in G and five times in second position.

This is equivalent to saying that, in the representation of permutations in Section 3.1.1 (also see Table 7.1):

1. for all $i = 0, \dots, 15$, there should exist no distinct permutations σ, σ' such that $\sigma(i) = \sigma'(i)$;
2. no pair (i, j) should appear twice at an offset of the form $(2k, 2k + 1)$, for all $k = 0, \dots, 7$;
3. for all $i = 0, \dots, 15$, there should be five distinct permutations σ such that $\sigma(i) < 8$, and five such that $\sigma(i) \geq 8$;
4. for all $i = 0, \dots, 15$, there should be five distinct permutations σ such that $\sigma(i)$ is even, and five such that $\sigma(i)$ is odd.

We implemented an automated search for sets of permutations matching the above criteria, and selected an arbitrary output of our program after checking manually that it did verify the said criteria.

Round	G_0		G_1		G_2		G_3		G_4		G_5		G_6		G_7	
0	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	14	10	4	8	9	15	13	6	1	12	0	2	11	7	5	3
2	11	8	12	0	5	2	15	13	10	14	3	6	7	1	9	4
3	7	9	3	1	13	12	11	14	2	6	5	10	4	0	15	8
4	9	0	5	7	2	4	10	15	14	1	11	12	6	8	3	13
5	2	12	6	10	0	11	8	3	4	13	7	5	15	14	1	9
6	12	5	1	15	14	13	4	10	0	7	6	3	9	2	8	11
7	13	11	7	14	12	1	3	9	5	0	15	4	8	6	2	10
8	6	15	14	9	11	3	0	8	12	2	13	7	1	4	10	5
9	10	2	8	4	7	6	1	5	15	11	9	14	3	12	13	0

Table 7.1 Permutations of message and constant words.

Table 7.1 shows the selected set of permutations. In most implementations of BLAKE, the permutations are encoded as a lookup table, so that at each round the program first fetches the index specific to this round, and then the message word

with that index. However, on some recent CPUs it is faster to use vectorized instructions to reorder message words rather than to load indices from memory, as reported in Section 5.4.4.

7.4.6 Number of Rounds

Selecting the number of rounds for a new cryptographic primitive is perhaps the most delicate choice as it depends on several unknown factors, including the future findings of cryptanalysts and the choices of other entrants in the competition. As noted in the Rijndael book [58, 5.1.5],

The criteria of security and efficiency are applied by all cipher designers. There are cases in which efficiency is sacrificed to obtain a higher security margin. The challenge is to come up with a cipher design that offers a reasonable security margin while optimizing efficiency.

The initial submission of BLAKE to the SHA3 competition had 10 and 14 rounds, respectively, for the 32-bit and the 64-bit versions. This choice was motivated by the previous cryptanalysis results on Salsa20 and ChaCha, as well as by our own cryptanalysis of BLAKE. The final version of BLAKE increased these values to 14 and 16. This tweak was not motivated by a perceived threat to the versions with fewer rounds, but rather by the excellent performance of BLAKE; we believed that increasing the “security margin”—which was already comfortable—would be an advantage, compared with the other algorithms in the competition.

Like 10 and 14, 14 and 16 are even numbers; this is not a coincidence: an even number of rounds simplifies hardware architectures that make two rounds within one clock cycle. Using 16, a multiple of 8, simplifies hardware architectures unrolling four rounds, and thus computing the compression function in four cycles.

7.4.7 Constants

BLAKE-256 uses the same 256-bit initial value (IV) as SHA-256, and BLAKE-512 the same 512-bit IV as SHA-512, respectively:

$$\begin{aligned} \text{IV}_0 &= 6a09e667 & \text{IV}_1 &= bb67ae85 & \text{IV}_2 &= 3c6ef372 & \text{IV}_3 &= a54ff53a \\ \text{IV}_4 &= 510e527f & \text{IV}_5 &= 9b05688c & \text{IV}_6 &= 1f83d9ab & \text{IV}_7 &= 5be0cd19 \end{aligned}$$

and

$$\begin{aligned} \text{IV}_0 &= 6a09e667f3bcc908 & \text{IV}_1 &= bb67ae8584caa73b \\ \text{IV}_2 &= 3c6ef372fe94f82b & \text{IV}_3 &= a54ff53a5f1d36f1 \\ \text{IV}_4 &= 510e527fade682d1 & \text{IV}_5 &= 9b05688c2b3e6c1f \\ \text{IV}_6 &= 1f83d9abfb41bd6b & \text{IV}_7 &= 5be0cd19137e2179 \end{aligned}$$

Using these IVs has two benefits: First, if both BLAKE-256 and BLAKE-512 are implemented, only 512 bits of IV have to be stored since the IV of BLAKE-256

is a subset of that of BLAKE-512 (namely, the higher-order 32 bits of each word). Second, if both BLAKE and SHA2 are implemented, the same IVs can be used for both, saving at least 64 bytes of memory.

The 16 u_i constants were chosen as the digits of π : our requirement was just that the words “look random,” and in particular be all distinct and have about as many ones as zero bits. Note that the block cipher Blowfish [158] also uses π digits as constants, and thus shares some constants with BLAKE.

Chapter 8

Security of BLAKE

Security is the absence of unmitigated surprise.

—Dan Geer

We invite all young and experienced cryptanalysts to ignore our security arguments and boldly attack Keccak as if your life depended on it.

—Keccak team

This chapter summarizes the security properties of BLAKE, as well as the attacks found on reduced or modified versions. First, we present a bottom-up analysis of the properties of BLAKE's building blocks, necessary for the understanding of more advanced results. Then actual attacks on reduced versions of the hash function or of its components (compression function, permutation) are described. The focus is on differential cryptanalysis, the tool of choice for analyzing—and ultimately breaking—hash functions.

8.1 Differential Cryptanalysis

We start with a succinct reminder of the principle of differential cryptanalysis: this family of methods was introduced by Biham and Shamir in the late 1980s, and first applied to DES [39] (see [33] for historical anecdotes). It has since become the favorite tool of cryptanalysts, because of its generality (differences can be considered with respect to XOR, addition, etc.), and because it is often the technique that works best. Differential cryptanalysis is also more intuitive than linear cryptanalysis, which is another family of cryptanalysis methods.

Differential cryptanalysis exploits correlations between the difference in the input and the corresponding difference in the output of a cryptographic algorithm. It covers a broad class of attacks, from simple distinguishers observing an output bit's bias towards one or zero, to advanced techniques such as boomerang attacks, related-key attacks, and combinations thereof. This section introduces some basic definitions and applications of differential cryptanalysis, and finally overviews some more advanced techniques.

8.1.1 Differences and Differentials

The following general description considers a block cipher, that is, a keyed permutation, as found at the core of SHA1, SHA2, BLAKE, or BLAKE2. Nevertheless, most of the techniques generalize to arbitrary functions, which may or may not be invertible, and are not necessarily keyed.

Let E be a block cipher with κ -bit key and n -bit blocks. In the context of differential attacks, a *differential* for E is a pair $(\Delta_{\text{in}}, \Delta_{\text{out}}) \in \{0, 1\}^n \times \{0, 1\}^n$, where Δ_{in} is called the *input difference*, and Δ_{out} the *output difference*. One associates to a differential the probability that a random input *conforms* to it, that is, the value

$$p_{\Delta} = \Pr_{k,m} (E_k(m \oplus \Delta_{\text{in}}) = E_k(m) \oplus \Delta_{\text{out}}) .$$

Here the probability is taken over the space of all keys and all messages, but depending on the applications it may be more relevant to consider probability for a fixed message or for a fixed key.

Ideally, p_{Δ} should be approximately equal to 2^{-n} for all Δ 's (it cannot be equal to that value for all differences, but the distribution should be statistically close to the one expected for an ideal function). Therefore, if a differential with probability $p_{\Delta} \gg 2^{-n}$ exists, E no longer qualifies as a pseudorandom permutation. Note that we consider differences with respect to XOR, which is the most common type of difference, but not the only one used; for example, collision attacks on MD5 [173] used differences with respect to integer addition.

Suppose that E_k can be decomposed as

$$E_k = E_k^N \circ E_k^{N-1} \circ \cdots \circ E_k^2 \circ E_k^1 ,$$

where $E^1, \dots, E^N$ are block ciphers with κ -bit key and n -bit blocks, and $\circ$ denotes the composition of functions (that is, $f \circ g(x) = f(g(x))$). A *differential characteristic*¹ for E is a sequence of differentials $\Delta^1, \dots, \Delta^N$, where $\Delta_{\text{in}}^i = \Delta_{\text{out}}^{i-1}$, $1 < i \leq N$. An input to E_k conforms to the differential characteristic if the consecutive differences when evaluating m and $m \oplus \Delta_{\text{in}}^1$ are, respectively, $\Delta_{\text{out}}^1, \dots, \Delta_{\text{out}}^N$. The probability associated with a differential characteristic Δ , under some independence assumption,² is the product of the probabilities associated with each differential in the characteristic, that is,

$$p_{\Delta} \approx p_{\Delta}^1 \times p_{\Delta}^2 \times \cdots \times p_{\Delta}^N .$$

For actual ciphers, the independence assumption does not necessarily hold. In the worst case, contradictions in the conditions imposed by two consecutive differentials imply that the characteristic cannot be satisfied, thus that it has probability zero.

Differential characteristics are typically used on sequences of *rounds*, that is, when E_k^i represents the i -th round of the function (be it a block cipher, a stream cipher, or a hash function). When all rounds are identical, one may search for *itera-*

¹ Also known as “differential path” or “differential trail.”

² Namely, a hypothesis of stochastic equivalence, see [113].

tive differentials (i.e., such that $\Delta_{\text{in}} = \Delta_{\text{out}}$) on the round function to form an *iterative characteristic* of the form $\Delta^1, \dots, \Delta^N = \Delta_{\text{in}}, \dots, \Delta_{\text{in}}$.

8.1.2 Finding Good Differentials

Finding good differentials generally means finding differentials Δ that hold with a high probability p_Δ . These are often found by making *linear approximations* of the function attacked; for example, suppose that some function only includes the operations $+$, $\oplus$, and $\gg$. If one replaces all additions by XORs, then the function behaves linearly, with respect to XOR, therefore an input difference always leads to the same output difference. Now note that $x + y$ equals $x \oplus y$ if and only if $x \wedge y = 0$, that is, when no carry appears in the addition. Heuristically, when the input difference has a low weight, and when there is a small number of additions, the propagation of the difference will follow that of the linearized model with non-negligible probability.

To estimate the probability of a differential found by linear approximation, one has to estimate the probability that all active³ integer additions behave like XORs, with respect to the input difference considered. Under reasonable independence assumptions, the problem can be reduced to estimating the probability that each individual addition behaves linearly given a random input, which is

$$p_{\Delta, \Delta'} = \Pr_{x,y}((x \oplus \Delta) + (y \oplus \Delta') = (x + y) \oplus (\Delta \oplus \Delta')) .$$

We have $p_{\Delta, \Delta'} = 2^{-w}$, where w is the Hamming weight of $\Delta \vee \Delta'$, excluding the weight of the most significant bit. Note that we do not require the addition to behave fully linearly, but just that no carry perturbs the diffusion of the differences. The more general problem of the differential behavior of addition has been studied in [119, 120].

Differentials may also be *nonlinear*: [119] provides an algorithm that, given two differences in two summands, returns the output difference that has the highest probability, which is not necessarily linear.

8.2 Properties of BLAKE's G Function

The G function is the core of BLAKE and the source of its security against differential attacks, which are a broad class of attacks that, for example, include the methods used to find collisions on MD5. Actually, most of the collision attacks on cryptographic hash functions can be described as differential attacks, irrespective of the transformation for which differences are considered (be it XOR, integer addition, or

³ An operation is called active when it includes a difference from the characteristic considered.

word rotation). This section thus focuses on the differential properties of G , with an emphasis on XOR differentials, which are by far the most commonly exploited in cryptanalytic attacks.

8.2.1 Basic Properties

We shall focus on the G function of BLAKE-256, but most observations apply (or can be adapted) to that of BLAKE-512 as well.

8.2.1.1 Operations

Recall from Chapter 3 that the G function at the core of BLAKE is defined for BLAKE-256 as

$$\begin{aligned} a &:= a + b + (m_{\sigma_r(2i)} \oplus u_{\sigma_r(2i+1)}) \\ d &:= (d \oplus a) \ggg 16 \\ c &:= c + d \\ b &:= (b \oplus c) \ggg 12 \\ a &:= a + b + (m_{\sigma_r(2i+1)} \oplus u_{\sigma_r(2i)}) \\ d &:= (d \oplus a) \ggg 8 \\ c &:= c + d \\ b &:= (b \oplus c) \ggg 7 \end{aligned}$$

and for BLAKE-512 as

$$\begin{aligned} a &:= a + b + (m_{\sigma_r(2i)} \oplus u_{\sigma_r(2i+1)}) \\ d &:= (d \oplus a) \ggg 32 \\ c &:= c + d \\ b &:= (b \oplus c) \ggg 25 \\ a &:= a + b + (m_{\sigma_r(2i+1)} \oplus u_{\sigma_r(2i)}) \\ d &:= (d \oplus a) \ggg 16 \\ c &:= c + d \\ b &:= (b \oplus c) \ggg 11 \end{aligned}$$

The differences between the two functions are that BLAKE-256 works with 32-bit words whereas BLAKE-512 works with 64-bit words, and the adapted rotation indices. Both G functions take a similar set of arguments and perform the same sequence of operations, consisting of six integer additions, six XORs, and four word rotations.

The three operators used ($+$, $\oplus$, and $\ggg$, so-called ARX) are computationally universal; that is, they are sufficient to implement any computable function. To see this, observe that:

1. any computable function can be expressed with only XOR and AND gates (algebraic normal form);

2. an XOR between two bits can be performed with the wordwise XOR operator;
3. an AND can be performed with integer addition by setting the two operand bits as least significant bits (LSBs) of two word registers, and taking the second LSB (the carry) as a result;
4. finally (and depending on the computation model) the rotation operator can be used to move the result of the AND back to the LSB of a register.

In particular, the ARX operators are sufficient to implement a secure cryptographic function; for example, AES can be described as a sequence of additions, XORs, and rotations, although this would lead to slow implementations. More generally, any S-box can be expressed as a sequence of ARX operations.

8.2.1.2 Invertibility

Given a message m and a round index r , the inverse function of G is defined as follows for BLAKE-256, by reversing each operation of the original function:

$$\begin{aligned}
 b &:= c \oplus (b \lll 7) \\
 c &:= c - d \\
 d &:= a \oplus (d \lll 8) \\
 a &:= a - b - (m_{\sigma_r(2i+1)} \oplus u_{\sigma_r(2i)}) \\
 b &:= c \oplus (b \lll 12) \\
 c &:= c - d \\
 d &:= a \oplus (d \lll 16) \\
 a &:= a - b - (m_{\sigma_r(2i)} \oplus u_{\sigma_r(2i+1)})
 \end{aligned}$$

Hence, for any (a', b', c', d') one can efficiently compute the unique (a, b, c, d) such that $G(a, b, c, d) = (a', b', c', d')$, given i and m . In other words, G is a permutation of the set $\{0, 1\}^{128}$.

8.2.1.3 Diffusion

Diffusion is informally defined as the ability of the function to quickly spread a small change in the input through the whole internal state. Diffusion has also been called the *avalanche effect*; for example, G injects message words such that any change, i.e., difference in a message word, affects the four words output. Tables 8.1 and 8.2 show the average number of output bits modified by G , given a random one-bit difference in the input, for each input word. The words that diffuse the most are the first introduced in the chain of operations of G , namely a and b , and the word that is the most affected is the last modified, b .

Table 8.1 Average number of changes in each output word given a random bit flip in each input word.

in\out	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>
<i>a</i>	4.6	11.7	10.0	6.5
<i>b</i>	6.6	14.0	11.5	8.4
<i>c</i>	2.4	6.6	4.8	2.4
<i>d</i>	2.4	8.4	6.7	3.4

Table 8.2 Average number of changes in each output word given a random bit flip in each input word, in the XOR-linearized model.

in\out	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>
<i>a</i>	4.4	9.9	8.2	6.3
<i>b</i>	6.3	12.4	9.8	8.1
<i>c</i>	1.9	3.9	2.9	1.9
<i>d</i>	1.9	4.9	3.9	2.9

8.2.2 Differential Properties of G

We present some differential properties of the G function, that is, properties related to the propagation of input differences within G. We focus on differences with respect to the XOR operation, or *bit differences*, as opposed to differences with respect to integer addition and subtraction.

We first consider the case of differences in the message words only, and then the general case with input differences in the state. Finally, we discuss properties of the inverse G function, G^{-1} .

We introduce specific notations for intermediate and final values of (a, b, c, d) , as shown below:

$$\begin{aligned}
\hat{a} &:= a + b + (m_{\sigma_r(2i)} \oplus u_{\sigma_r(2i+1)}) \\
\hat{d} &:= (d \oplus \hat{a}) \ggg 16 \\
\hat{c} &:= c + \hat{d} \\
\hat{b} &:= (b \oplus \hat{c}) \ggg 12 \\
a' &:= \hat{a} + \hat{b} + (m_{\sigma_r(2i+1)} \oplus u_{\sigma_r(2i)}) \\
d' &:= (\hat{d} \oplus a') \ggg 8 \\
c' &:= \hat{c} + d' \\
b' &:= (\hat{b} \oplus c') \ggg 7
\end{aligned}$$

We thus use the following notations to denote differences:

- Δa : initial difference in a
- $\Delta \hat{a}$: difference in the intermediate value of a
- $\Delta a'$: final difference in a
- Δ_j : difference in m_j

Similar notations are used for differences in b, c, d , and m_j . We generally denote i as the index of G (when necessary), and the indices of m and u words as $j = \sigma_r(2i)$

and $k = \sigma_r(2i + 1)$. We also use the operators $\wedge$ (AND) and $\vee$ (OR), both to connect logical statements and as bitwise operators.

For instance, if $\Delta a = \Delta_j = 0$ and $\Delta b = 80 \dots 00$, then $\Delta \hat{a} = 80 \dots 00$, because $\hat{a}$ is defined in G as $a + b + (m_j \oplus u_k)$, which propagates a difference in the most significant bit (MSB) of a to the result with probability one, due to the absence of carry induced by this difference.

8.2.2.1 Fixed Points

A *fixed point* for G is a value of (a, b, c, d) such that $G(a, b, c, d) = (a, b, c, d)$, in other words, a value for which G behaves as the identity function. Too many fixed points are undesirable, since they may be exploited to attack the hash function.

For G where the m and u words are fixed, the only fixed point is $(0, 0, 0, 0)$. To see this, observe that to have $a' = a$, we need $\hat{b} = -b$; to have $c' = c$, we need $\hat{d} = d'$. Analyzing the necessary conditions for those to hold shows a contradiction with $b' = b$ and $d' = d$, leaving only the all-zero value as solution.

In general, the existence and value of a fixed point depend on the value of the m and u words, therefore the use of distinct u words at each call of G ensures that a fixed point for an instance of G is unlikely to also be a fixed point for another instance of G within the compression function.

8.2.2.2 Differences in the Message Words Only

All statements below assume zero difference in the state words, that is, $\Delta a = \Delta b = \Delta c = \Delta d = 0$.

Proposition 1. *If $\Delta_j = 0$ and $\Delta_k \neq 0$, then $\Delta a' \neq 0$, $\Delta b' \neq 0$, $\Delta c' \neq 0$, and $\Delta d' \neq 0$.*

Proof. If there is no difference in m_j then there is no difference in a, b, c , and d after the first four lines of G. Thus a difference Δ in m_k always gives a nonzero difference Δ' in a . Then, each of the final values is computed by combining a word having no difference with a word that has a difference; since all the operations are invertible, all final values have a nonzero difference. $\square$

Proposition 2. *If $\Delta_j \neq 0$, then*

$$\begin{array}{l|l} (\Delta a' = 0) \Rightarrow (\Delta d' \neq 0) & (\Delta c' = 0) \Rightarrow (\Delta b' \neq 0) \wedge (\Delta d' \neq 0) \\ (\Delta b' = 0) \Rightarrow (\Delta c' \neq 0) & (\Delta d' = 0) \Rightarrow (\Delta a' \neq 0) \wedge (\Delta c' \neq 0) \end{array}$$

Proof. We show that, in the output, a and d cannot be both free of difference, as well as d and c , and b and c . By a similar argument as in the proof of Proposition 1, after the first four lines of G the four state words have nonzero differences. In particular, the state has differences $(\Delta', \Delta'' \ggg 12, \Delta'', \Delta' \ggg 16)$, for some nonzero Δ' and Δ'' . Suppose that we obtain $\Delta a' = 0$. Then we must have $\Delta d' = (\Delta' \ggg 24)$. Hence a

and d cannot be both free of difference. Similarly, canceling the difference Δ'' in c requires a difference in d , thus c and d cannot be both free of difference. Finally, to cancel the difference in b , c must have a difference, thus b and c cannot be both free of difference. $\square$

Two corollaries immediately follow from Proposition 1 and Proposition 2:

Corollary 1. *If $(\Delta_j \vee \Delta_k) \neq 0$, then there are differences in at least two output words.*

Corollary 2. *All differentials with an output difference of one of the following forms are impossible:*

$$\begin{array}{cccc} (\Delta, 0, 0, 0) & (0, \Delta, 0, 0) & (\Delta, 0, 0, \Delta') & (\Delta, 0, \Delta', 0) \\ (0, 0, \Delta, 0) & (0, 0, 0, \Delta) & (\Delta, \Delta', 0, 0) & (0, \Delta, \Delta', 0) \end{array}$$

for some nonzero Δ and Δ' , and for any Δ_j and Δ_k .

Note that output differences of the form $(0, \Delta, 0, \Delta')$ are possible; for instance, if $\Delta_k = (\Delta_j \ggg 4)$, then the output difference obtained by linearization is $(0, \Delta_j \ggg 3, 0, \Delta_j)$. For such a Δ_j , the highest probability 2^{-28} is achieved for $\Delta = 88888888$.

A consequence of Corollary 2 is that a difference in at least one word of $m_7, \dots, m_{15}$ gives differences in at least two output words after the first round. This yields the following upper bounds on the probabilities of differential characteristics.

Proposition 3. *A differential characteristic with input difference Δ_j , Δ_k has probability:*

- at most 2^{-1} if $\Delta_j = 0$ and $\Delta_k \neq 0$
- at most 2^{-6} if $\Delta_j \neq 0$ and $\Delta_k = 0$
- at most 2^{-5} if $\Delta_j \neq 0$ and $\Delta_k \neq 0$

Proof. We prove each of the three statements separately:

- A possible differential characteristic when linearizing additions with $|\Delta_j| = 0$ and $w = |\Delta_k| \neq 0$ (that is, when the Hamming weight of Δ_k is the positive integer w) has output differences

$$(\Delta_k, \Delta_k \ggg 15, \Delta_k \ggg 8, \Delta_k \ggg 8)$$

for BLAKE-256. If $(\Delta_k \wedge 80 \dots 0080)$ is zero, then the differential characteristic is followed with probability 2^{-2w} ; if it equals $800 \dots 00$ or $00 \dots 0080$, with probability 2^{-2w+1} ; if it equals $80 \dots 0080$, with probability 2^{-2w+2} . Clearly, the probability is maximized for $w = 1$ and Δ_k either $80 \dots 00$ or $00 \dots 0080$, giving probability $1/2$. Since at least one non-MSB difference must be active for any difference, probability is at most $1/2$, a bound which we could match.

- Suppose all additions behave as XORs (that is, no carries are propagating). Sums of the four additions then have the following differences:

$$\begin{aligned}
& 0 + \Delta_j \\
& 0 + (\Delta_j \ggg 16) \\
& \Delta + (\Delta_j \ggg 28) \\
& (\Delta_j \ggg 16) + ((\Delta_j \ggg 4) \oplus (\Delta_j \ggg 8) \oplus (\Delta_j \ggg 24))
\end{aligned}$$

for BLAKE-256. When $w = 1$: the logical OR of the summands is, respectively, 1, 1, 2, and 4, so 8 in total. Rotation by zero and by 16 appears twice each, thus if Δ_j equals 80000000 or 00008000, then two of the eight bits are MSBs. This differential characteristic is thus followed with probability 2^{-6} when Δ_j equals 80000000 or 00008000.

It is easy to see that a higher probability cannot be obtained when $w > 1$: indeed, the probability cannot be less than 2^{-4w+4} ; when $w = 2$, weights excluding MSB are at least 1, 1, 3, and 3, which gives a probability of 2^{-8} . Hence, 2^{-6} is the highest probability.

- First observe that, if $w = |\Delta_j| > 1$, then after the first four lines, a , b , and c have at least $w - 1$ differences, excluding the MSB. Hence, the differential characteristic for the second part of G is followed with probability at least $2^{2 \times (w-1) + w-1} = 2^{3w-3}$, because a , b , and c appear in the two additions. This bound is maximized to 2^{-3} for $w = 2$. A refined analysis shows that when $w = 2$ a differential characteristic cannot have probability greater than 2^{-6} , even considering nonlinear differentials.

Suppose that $w = |\Delta_j| = 1$ and that the first part of G is traversed with probability $1/2$; that is, m_j has difference $\Delta \in \{80000000, 00008000\}$, and intermediate values of (a, b, c, d) have differences

$$(\Delta, \Delta \ggg 16, \Delta \ggg 16, \Delta \ggg 28),$$

which is one of the following differences:

$$\begin{aligned}
& (80000000, 00000008, 00008000, 00008000) \\
& (00008000, 00080000, 80000000, 80000000).
\end{aligned}$$

When $\Delta = 80000000$, there are two optimal choices of a difference in m_k (80008008 and 80000008), which both give total probability 2^{-5} . When $\Delta = 00008000$, the optimal choice of a difference in m_k is 80088000, which also gives total probability 2^{-5} .

□

8.2.2.3 Differences in Any Input Word

The results below no longer assume zero input difference in the state words. The first proposition states necessary conditions to produce collisions with G (an obvious necessary condition being the introduction of differences in at least one of the message words):

Proposition 4. *If $\Delta a' = \Delta b' = \Delta c' = \Delta d' = 0$, then $\Delta b = \Delta c = 0$.*

Proof. By Proposition 6, in G^{-1} a difference in m_j and/or m_k cannot affect b and c , hence a collision for G needs no difference in b and c . $\square$

In other words, a collision for G requires zero difference in the initial b and c ; for instance, collisions can be obtained for certain differences Δa , Δ_j , and zero differences in the other input words. Indeed at line 1 of the description of G , Δa propagates to $(a + b)$ with probability $2^{-\|\Delta a\|}$, Δ_j propagates to $(m_j \oplus u_j)$ with probability one, and finally Δa eventually cancels Δ_j .

The following result directly follows from Proposition 4:

Corollary 3. *The following classes of differentials for G are impossible:*

$$\begin{aligned} (\Delta, \Delta', \Delta'', \Delta''') &\mapsto (0, 0, 0, 0) \\ (\Delta, 0, \Delta'', \Delta''') &\mapsto (0, 0, 0, 0) \\ (\Delta, \Delta', 0, \Delta''') &\mapsto (0, 0, 0, 0) \end{aligned}$$

for nonzero Δ' and Δ'' , and any Δ , Δ''' , Δ_j , and Δ_k .

Many other classes of impossible differentials for G exists; for example, if $\Delta a' \neq 0$ and $\Delta b' = \Delta c' = \Delta d' = 0$, then $\Delta b = 0$.

Proposition 5. *The only differential characteristics with probability one give $\Delta a' = \Delta b' = \Delta c' = \Delta d' = 0$ and have either*

- $\Delta_j = \Delta a = 800 \dots 00$ and $\Delta b = \Delta c = \Delta d = \Delta_k = 0$;
- $\Delta_k = \Delta a = \Delta d = 800 \dots 00$ and $\Delta b = \Delta c = \Delta_j = 0$;
- $\Delta_j = \Delta_k = \Delta d = 800 \dots 00$ and $\Delta a = \Delta b = \Delta c = 0$.

Proof. The difference $(800 \dots 00)$ is the only difference whose differential probability is one. Hence probability-1 differential characteristics must only have differences active in additions. By enumerating all combinations of MSB differences in the input, one observes that the only valid ones have either MSB difference in Δ_j and Δa , in Δ_k and Δa and Δd , or in Δ_j and Δ_k and Δd . $\square$

For constants u_i equal to zero, more probability-1 differentials can be obtained using differences with respect to integer addition. However, in this case simple attacks exist (see Section 8.6.6).

8.2.2.4 Properties of G^{-1}

We start with basic differential properties of the inverse of G , as these will be useful in the subsequent analysis of G . Recall that, at round r , the inverse of G of BLAKE-256 computes

$$\begin{aligned}
b &:= c \oplus (b \lll 7) \\
c &:= c - d \\
d &:= a \oplus (d \lll 8) \\
a &:= a - b - (m_k \oplus u_j) \\
b &:= c \oplus (b \lll 12) \\
c &:= c - d \\
d &:= a \oplus (d \lll 16) \\
a &:= a - b - (m_j \oplus u_k)
\end{aligned}$$

where $j = \sigma_r(2i)$ and $k = \sigma_r(2i + 1)$. Unlike G , its inverse G^{-1} has low flow dependency: two consecutive lines can be computed simultaneously and independently, with concurrent access to one variable.

Many properties of G^{-1} can be deduced from the properties of G ; for example, probability-1 differential characteristics for G^{-1} can be directly obtained from Proposition 5. We report two particular properties of G^{-1} . The first one follows directly from the description of G^{-1} .

Proposition 6. *In G^{-1} , the final values of b and c do not depend on the message words m_j and m_k . In particular, b depends only on the initial b , c , and d .*

That is, when inverting G , the initial b and c depend only on the choice of the image (a, b, c, d) , not on the message.

The following property follows from the observation in Proposition 3:

Proposition 7. *There exists no differential characteristic that gives collisions with probability one.*

Properties of G^{-1} are exploited in Section 8.3.4 to find impossible differentials.

8.3 Properties of the Round Function

Recall that the round function of BLAKE is the following sequence of evaluations of G , where those on the same line can be carried out independently (e.g., in parallel):

$$\begin{aligned}
&G_0(v_0, v_4, v_8, v_{12}) \quad G_1(v_1, v_5, v_9, v_{13}) \quad G_2(v_2, v_6, v_{10}, v_{14}) \quad G_3(v_3, v_7, v_{11}, v_{15}) \\
&G_4(v_0, v_5, v_{10}, v_{15}) \quad G_5(v_1, v_6, v_{11}, v_{12}) \quad G_6(v_2, v_7, v_8, v_{13}) \quad G_7(v_3, v_4, v_9, v_{14})
\end{aligned}$$

8.3.1 Bijectivity

Because G is a permutation, a round is a permutation of the inner state v for any fixed message, and the inverse round does

$$\begin{aligned}
&G_4^{-1}(v_0, v_5, v_{10}, v_{15}) \quad G_5^{-1}(v_1, v_6, v_{11}, v_{12}) \quad G_6^{-1}(v_2, v_7, v_8, v_{13}) \quad G_7^{-1}(v_3, v_4, v_9, v_{14}) \\
&G_0^{-1}(v_0, v_4, v_8, v_{12}) \quad G_1^{-1}(v_1, v_5, v_9, v_{13}) \quad G_2^{-1}(v_2, v_6, v_{10}, v_{14}) \quad G_3^{-1}(v_3, v_7, v_{11}, v_{15})
\end{aligned}$$

where G^{-1} is the inverse G described in Section 8.2.1.2.

In other words, given a message and the value of v after r rounds, one can determine the value of v at rounds $r - 1$, $r - 2$, etc., and thus the initial value of v . Therefore, for a same message a sequence of rounds is a permutation of the internal states. It follows that internal collisions for two distinct chaining values and an identical message block do not exist.

8.3.2 Diffusion and Low-Weight Differences

After one round, all 16 words of the internal state are affected by a modification of one bit in the input (be it the message, the salt, or the chain value). Here we illustrate diffusion through rounds with a concrete example, for the *null message* and the *null initial state*. The arrays below represent the *differences* in the state after each step of the first two rounds (column step, diagonal step, column step, diagonal step), for a difference in the least significant bit of v_0 :

$$\begin{array}{ll}
 \text{column step} & \begin{pmatrix} 00000037 & 00000000 & 00000000 & 00000000 \\ e06e0216 & 00000000 & 00000000 & 00000000 \\ 37010b00 & 00000000 & 00000000 & 00000000 \\ 37000700 & 00000000 & 00000000 & 00000000 \end{pmatrix} \quad (\text{weight } 34) \\
 \text{diagonal step} & \begin{pmatrix} 0000027f & 10039015 & 5002b070 & c418a7d4 \\ 66918cc7 & 1cbeee25 & f1a8535f & c111ad29 \\ f8d104f0 & 6f08c6f9 & 5f77131e & e4291fe7 \\ 151703a7 & 705002b0 & f2c22207 & 7f001702 \end{pmatrix} \quad (\text{weight } 219) \\
 \text{column step} & \begin{pmatrix} 944f85fd & a044ccb3 & 9476a6bc & 24b6adac \\ a729bbe9 & 6549bc3d & 3a330361 & 7318b20d \\ 7bf5f768 & 7831614b & cf44c968 & 53d886e2 \\ 5a1642b3 & 41b00ea0 & a7115a95 & 7ac791d1 \end{pmatrix} \quad (\text{weight } 249) \\
 \text{diagonal step} & \begin{pmatrix} dfc2d878 & f9faae7a & 2d804d9a & 3ef58b7f \\ fc91af81 & d78e2315 & 55048021 & 0811cc46 \\ fb98af71 & dc27330e & 47a19b59 & edde442e \\ f042bb72 & 1c7a59ab & ac2effa4 & 2e76390b \end{pmatrix} \quad (\text{weight } 264)
 \end{array}$$

For comparison, in the linearized model (i.e., where all additions are replaced by XORs), we have

$$\begin{aligned}
&\text{column step} \begin{pmatrix} 00000011 & 00000000 & 00000000 & 00000000 \\ 20220202 & 00000000 & 00000000 & 00000000 \\ 11010100 & 00000000 & 00000000 & 00000000 \\ 11000100 & 00000000 & 00000000 & 00000000 \end{pmatrix} \quad (\text{weight } 14) \\
&\text{diagonal step} \begin{pmatrix} 00000101 & 10001001 & 10011010 & 02202000 \\ 40040040 & 22022220 & 00202202 & 00222020 \\ 01110010 & 20020222 & 01111101 & 00111101 \\ 01110001 & 10100110 & 22002200 & 01001101 \end{pmatrix} \quad (\text{weight } 65) \\
&\text{column step} \begin{pmatrix} 54500415 & 13012131 & 02002022 & 20331103 \\ 2828a0a8 & 46222006 & 04006046 & 64646022 \\ 00045140 & 30131033 & 12113132 & 10010011 \\ 00551045 & 23203003 & 03121212 & 01311212 \end{pmatrix} \quad (\text{weight } 125) \\
&\text{diagonal step} \begin{pmatrix} 35040733 & 67351240 & 24050637 & b1300980 \\ 27472654 & 8ae6ca08 & ee4a6286 & e08264a8 \\ 03531247 & 1ab89238 & 54132765 & 55051040 \\ 14360705 & 73540643 & 89128902 & 70030514 \end{pmatrix} \quad (\text{weight } 186)
\end{aligned}$$

The higher weight in the original model is due to the addition carries induced by the constants $u_0, \dots, u_{15}$. A technique to avoid carries at the first round and get a low-weight output difference is to choose a message such that $m_0 = u_0, \dots, m_{15} = u_{15}$. At the subsequent rounds, however, nonzero words are introduced because of the different permutations.

Diffusion can be delayed a few steps by combining high-probability and low-weight differentials of G , using initial conditions, neutral bits, etc; for example, applying directly the differential characteristic

$$(80000000, 00000000, 80000000, 80008000) \mapsto (80000000, 0, 0, 0)$$

the diffusion is delayed one step, as illustrated below:

$$\begin{aligned}
&\text{column step} \begin{pmatrix} 80000000 & 00000000 & 00000000 & 00000000 \\ 00000000 & 00000000 & 00000000 & 00000000 \\ 00000000 & 00000000 & 00000000 & 00000000 \\ 00000000 & 00000000 & 00000000 & 00000000 \end{pmatrix} \quad (\text{weight } 1) \\
&\text{diagonal step} \begin{pmatrix} 800003e8 & 00000000 & 00000000 & 00000000 \\ 00000000 & 0b573f03 & 00000000 & 00000000 \\ 00000000 & 00000000 & ab9f819d & 00000000 \\ 00000000 & 00000000 & 00000000 & e8800083 \end{pmatrix} \quad (\text{weight } 49) \\
&\text{column step} \begin{pmatrix} 8007e4a0 & 2075b261 & 18e78828 & 9800099e \\ 5944fe53 & f178a22f & 86b0a65b & 936c73cb \\ a27f0d24 & 98d6929a & 4088a5fb & 2e39eda3 \\ a08fff64 & 2ad374b7 & 2818e788 & 1e9883e1 \end{pmatrix} \quad (\text{weight } 236) \\
&\text{diagonal step} \begin{pmatrix} 4b3cbdd2 & 0290847f & b4ff78f9 & f1e71ba3 \\ 3a023c96 & 49908e86 & f13bc1d7 & adc2020a \\ 9dca344a & 827bf1e5 & b20a8825 & fe575be3 \\ fc81fe81 & d676ffc9 & 80740480 & 52570cb2 \end{pmatrix} \quad (\text{weight } 252)
\end{aligned}$$

In comparison, for a same input difference in the linearized model we have

$$\begin{aligned}
&\text{column step} \begin{pmatrix} 80000000 & 00000000 & 00000000 & 00000000 \\ 00000000 & 00000000 & 00000000 & 00000000 \\ 00000000 & 00000000 & 00000000 & 00000000 \\ 00000000 & 00000000 & 00000000 & 00000000 \end{pmatrix} \quad (\text{weight } 1) \\
&\text{diagonal step} \begin{pmatrix} 80000018 & 00000000 & 00000000 & 00000000 \\ 00000000 & 10310101 & 00000000 & 00000000 \\ 00000000 & 00000000 & 18808080 & 00000000 \\ 00000000 & 00000000 & 00000000 & 18800080 \end{pmatrix} \quad (\text{weight } 18) \\
&\text{column step} \begin{pmatrix} 80000690 & e1101206 & 0801b818 & b8000803 \\ 1d217176 & 600fc064 & 60111212 & 22167121 \\ 90b8b886 & 16e12133 & 00888138 & 83389890 \\ 90803886 & 17e01122 & 180801b8 & 83b88010 \end{pmatrix} \quad (\text{weight } 155) \\
&\text{diagonal step} \begin{pmatrix} 44e4e456 & 133468bd & dbbda164 & 0f649833 \\ 4e20f629 & 563a9099 & a62f3969 & 7773c0be \\ feb6f508 & aabdcbf9 & 3262e291 & 87a10d6a \\ 3c2b867b & b603b05c & da695123 & f88e8007 \end{pmatrix} \quad (\text{weight } 251)
\end{aligned}$$

These examples show that, even in the linearized model, after two rounds about half of the state bits have changed when different initial states are used, on average. Similar results are obtained for a difference in the message. Using combinations of low-weight differentials and message modifications one may attack reduced versions with two or three rounds. However, differences after more than four steps seem difficult to control.

8.3.3 Invertibility

Let f^r be the function $\{0, 1\}^{512} \times \{0, 1\}^{512} \rightarrow \{0, 1\}^{512}$ that, for an initial state v and a message block m , returns the state after r rounds of the permutation of BLAKE-256. Noninteger round indices (for example, $r = 1.5$) mean the application of $\lfloor r \rfloor$ rounds and the following column step. We write $f_v^r = f^r(v, \cdot)$ when considering f^r for a fixed initial state and f_m^r when the message block is fixed. As noted above, f_m^r is a permutation for any message block m and any $r \geq 0$. In this section we use the differential properties of G to show that f_v^1 is also a permutation for any initial state v . Then we derive an efficient algorithm for the inverse of f_v^1 and an algorithm with complexity 2^{128} to compute a preimage of $f_v^{1.5}$ for BLAKE-256 (a similar method applies to BLAKE-512 in 2^{256}). This improves the round-reduced preimage attack presented in [118] (whose complexity is, respectively, 2^{192} and 2^{384} for BLAKE-256 and BLAKE-512).

8.3.3.1 A Round Is a Permutation on the Message Space

Proposition 8. *For any fixed state v , one round of BLAKE (for any index of the round) is a permutation on the message space. In particular, f_v^1 is a permutation.*

Proof. We show that, if there is no difference in the state, any difference in the message block implies a difference in the state after one round of BLAKE. Suppose that there is a difference in at least one message word. We distinguish two cases:

1. *No differences are introduced in the column step:* there is thus no difference in the state after the column step. At least one of the message words used in the diagonal step has a difference; from Corollary 1, there will be differences in at least two words of the state after the diagonal step.
2. *Differences are introduced in the column step:* from Corollary 2, output differences of the form $(0, 0, 0, 0)$, $(\Delta, 0, 0, 0)$, $(0, 0, 0, \Delta)$, or $(\Delta, 0, 0, \Delta')$ are impossible. Thus, after the first column step, there will be a difference in at least one word of the two middle rows (that is, in $v_4, \dots, v_{11}$). These words are exactly the words used as b and c in the calls to G in the diagonal step; from Proposition 4, we deduce that differences will exist in the state after the diagonal step, since $\Delta b = \Delta c = 0$ is a necessary condition to make differences vanish (see Proposition 4).

We conclude that, whenever a difference is set in the message, there is a difference in the state after one round. $\square$

The fact that a round is a permutation with respect to the message block indicates that no information of the message is lost through a round and thus can be considered a strength of the algorithm. The same property also holds for AES-128.

Note that Proposition 8 says nothing about the injectivity of f_v^r for $r \neq 1$.

8.3.3.2 Inverting One Round

Without loss of generality, we assume the constants equal to zero, that is, $u_i = 0$ for $i = 0, \dots, 7$ in the description of G . We use explicit input–output equations of G to derive our algorithms.

We first analyze the input–output equations for G . Consider the function G_s operating at round r on a column or diagonal of the state respectively. Let (a, b, c, d) be the initial state words and (a', b', c', d') the corresponding output state words. For shorter notation let $i = \sigma_r(2s)$ and $j = \sigma_r(2s + 1)$. Let $\hat{a} = a + b + m_j$ be the intermediate value of a set at line 1 of the description of G . From line 2 we get $\hat{a} = (\hat{d} \lll 16) \oplus d$, where $\hat{d}$ is the intermediate value of d set at line 2. From line 7 we get $\hat{d} = (d' \lll 8) \oplus a'$ and derive

$$a = (((d' \lll 8) \oplus a') \lll 16) \oplus d - b - m_j. \quad (8.1)$$

Below we use the following equations that can be derived in a similar way:

$$a = ((((((b' \lll 7) \oplus c') \lll 12) \oplus b) - c) \lll 16) \oplus d) - m_j - b \quad (8.2)$$

$$= a' - ((b' \lll 7) \oplus c') - m_k - b - m_j \quad (8.3)$$

$$b = (((b' \lll 7) \oplus c') \lll 12) \oplus (c' - d') \quad (8.4)$$

$$c = c' - d' - ((d' \lll 8) \oplus a') \quad (8.5)$$

$$= c' - d' - ((d \oplus (a + b + m_j)) \ggg 16) \quad (8.6)$$

$$d = (((d' \lll 8) \oplus a') \lll 16) \oplus (a' - ((b' \lll 7) \oplus c') - m_k) \quad (8.7)$$

$$a' = ((((((b' \lll 7) \oplus c') \lll 12) \oplus b) - c) \lll 16) \oplus d) + ((b' \lll 7) \oplus c') + m_k \quad (8.8)$$

$$b' = (((b \oplus (c' - d')) \ggg 12) \oplus c') \ggg 7 \quad (8.9)$$

$$d' = c' - c - ((d \oplus (a + b + m_j)) \ggg 16) \quad (8.10)$$

Observe that (8.1), (8.2), and (8.8) allow to determine m_j and m_k from (a, b, c, d) and (a', b', c', d') . Further, (8.4) and (8.5) imply Proposition 6.

We now apply these equations to invert f_v^1 and to find a preimage of $f_v^{1.5}(m)$ for arbitrary m and v . Denote by $v^i = v_0^i, \dots, v_{15}^i$ the internal state after i rounds. Again, noninteger round indices refer to intermediate states after a column step but before the corresponding diagonal step. The state v^r is the output of $f_{v,0}^r$.

We now describe how to invert f_v^1 : Given v^0 and v^1 , the message block $m = (m_0, \dots, m_{15})$ with $f_{v,0}^1(m) = v^1$ can be determined as follows:

1. determine $v_4^{0.5}, \dots, v_7^{0.5}$ using (8.4) and $v_8^{0.5}, \dots, v_{11}^{0.5}$ using (8.5);
2. determine $m_0, \dots, m_7$ using (8.2), (8.8), and (8.10);
3. determine $v_0^{0.5}, \dots, v_3^{0.5}, v_{12}^{0.5}, \dots, v_{15}^{0.5}$ using $G_0, \dots, G_3$;
4. determine $m_8, \dots, m_{15}$ using (8.2), (8.8), and (8.10).

This algorithm always succeeds, as it is deterministic. Although slightly more complex than the forward computation of f_v^1 , it can be executed efficiently.

8.3.3.3 Preimage of $f_v^{1.5}(m)$

Given some v^0 , and $v^{1.5}$ in the codomain of $f_{v^0}^{1.5}$ (thus, a preimage of $v^{1.5}$ exists), a message block m with $f_{v^0}^{1.5}(m) = v^{1.5}$ can be determined as follows:

1. guess m_8, m_{10}, m_{11} and $v_{10}^{0.5}$;
2. determine $v_4^1, \dots, v_7^1$ using (8.4), $v_8^1, \dots, v_{11}^1$ using (8.5), v_{12}^1 , and v_{13}^1 using (8.7);
3. determine $v_6^{0.5}, v_7^{0.5}$ using (8.4), m_4 (8.2), v_1^1 (8.2), $v_{14}^{0.5}$ (8.6), $v_1^{0.5}$ (8.3), $v_{11}^{0.5}$ (8.5), and $v_{12}^{0.5}$ (8.2);
4. determine $v_2^{0.5}$ (8.5), m_5 (8.8), m_6 (8.2), v_{15}^1 (8.7), $v_{15}^{0.5}$ (8.6), $v_5^{0.5}$ (8.4), v_0^1 (8.5), m_9 (8.8), and m_{14} (8.2);
5. determine $v_3^{0.5}$ (8.5), m_7 (8.8), $v_0^{0.5}$ (8.2), $v_8^{0.5}$ (8.5), m_0 (8.1), v_2^1 (8.5), v_{14}^1 (8.2), and m_{15} (8.8);
6. determine $v_4^{0.5}$ (8.9), m_1 (8.8), $v_9^{0.5}$ (8.6), v_3^1 (8.8), m_{13} (8.2), m_2 (8.2), m_3 (8.8), $v_{13}^{0.5}$ (8.7), and m_{12} (8.2);
7. if $f_{v^0}^{1.5}(m) = v^{1.5}$ output m , otherwise make a new guess.

This algorithm yields a preimage of $f_v^{1.5}(m)$ for BLAKE-256 after 2^{128} guesses in the worst case. It directly applies to find a preimage of the compression function of BLAKE reduced to 1.5 rounds and thus greatly improves the round-reduced preimage attack of [118], which has complexity 2^{192} . The method also applies to BLAKE-512, giving an algorithm of complexity 2^{256} , improving on [118]'s 2^{384} algorithm.

There are other possibilities to guess words of m and the intermediate states, but exhaustive search showed that at least four words are necessary to determine the full message block m by explicit input–output equations.

8.3.4 Impossible Differentials

An *impossible differential (ID)* is a pair of input and output differences that cannot occur. This section studies IDs for several rounds of the permutation of BLAKE. First we exploit properties of the G function to describe IDs for one and two rounds. Then we apply a miss-in-the-middle strategy to reach up to five and six rounds.

To illustrate IDs we use the following greyscale code:

	absence of difference
	undetermined (possibly zero) difference
	undetermined or partially determined nonzero difference
	totally determined nonzero difference

8.3.4.1 Impossible Differentials for One Round

The following statement describes many IDs for one round of BLAKE's permutation.

Proposition 9. *All differentials for one round (of any index) with no input difference in the initial state, any difference in the message block, and an output with difference in a single diagonal of one of the forms in Corollary 2 are impossible.*

Proof. We give a general proof for the central diagonal (v_0, v_5, v_{10}, v_{15}); the proof directly generalizes to the other diagonals of the state. We distinguish two cases:

1. *No differences are introduced in the column step:* the result directly follows from Proposition 4 and Corollary 2.
2. *Differences are introduced in the column step:* recall that, if $\Delta b \neq 0$ or $\Delta c \neq 0$, then one cannot obtain a collision for G (see Proposition 4); in particular, if there is a difference in one of the two middle rows of the state before the diagonal step, then the corresponding diagonal cannot be free of difference after.

We reason ad absurdum: if a difference was introduced in the column step in the first or in the fourth column, then there must be a difference in the corresponding b or c (for output differences with $\Delta b' = \Delta c' = 0$ are impossible after the column step, see Corollary 2). That is, one diagonal distinct from the central diagonal must have differences.

We deduce that, any state after one round with difference only in the central diagonal must be derived from a state with differences only in the second or in the third column. In particular, when applying G to the central diagonal, we have $\Delta a = \Delta d = 0$. From Proposition 2, we must thus have $\Delta a' \neq 0$, $\Delta c' \neq 0$, and $\Delta d' \neq 0$. In particular, the output differences in Corollary 2 cannot be reached.

We have shown that after one round of BLAKE, differences in the message block cannot lead to a state with only differences in the central diagonal, such that the difference is one of the differences in Corollary 2. The proof directly extends to any of the three other diagonals. $\square$

To illustrate Proposition 9, which is quite general and covers a large set of differentials, Figure 8.1 presents two examples corresponding to the two cases in the proof.

Note that our finding of IDs with zero difference in the initial and in the final state is another way to prove Proposition 8.

8.3.4.2 Extension to Two Rounds

We can directly extend the IDs identified above to two rounds, by prepending a probability-1 differential characteristic leading to a zero difference in the state after one round; for example, differences $800 \dots 00$ in m_0 and in v_0 always lead to a zero-difference state after the first round:

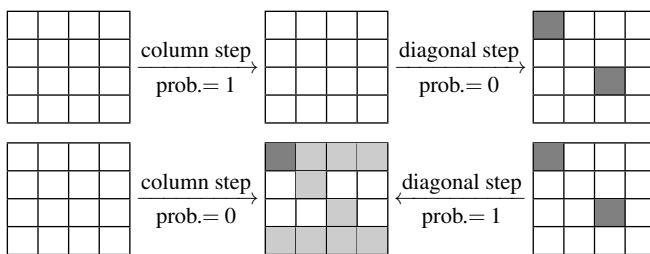
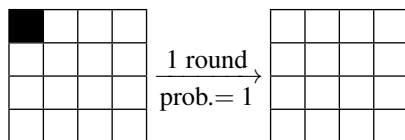


Fig. 8.1 Illustration of IDs after one round: when there is no difference introduced in the column step (top), and when there is one or more (bottom).



By Proposition 9, a state with differences only in v_0 and v_{10} cannot be reached after one round when starting from zero-difference states. Therefore, differences $800 \dots 00$ in m_0 and v_0 cannot lead to differences only in v_0 and v_{10} after two rounds. This example is illustrated in Figure 8.2.

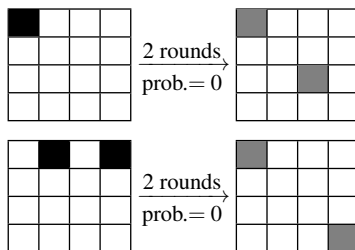


Fig. 8.2 Examples of IDs for two rounds: given difference $800 \dots 00$ in m_0 and v_0 (top), or in m_2, m_6, v_1, v_3 (bottom).

8.3.4.3 Miss-in-the-Middle Distinguisher for BLAKE-256

The technique called *miss-in-the-middle* [34] was first applied to identify IDs in block ciphers (for instance, DEAL [105] and AES [38, 87]). Let $\Pi = \Pi_0 \circ \Pi_1$ be a permutation. A miss-in-the-middle approach consists in finding a differential $(\alpha \mapsto \beta)$ of probability one for Π_1 and a differential $(\gamma \mapsto \delta)$ of probability one for Π_0^{-1} , such that $\beta \neq \delta$. The differential $(\alpha \mapsto \delta)$ thus has probability zero and so is an ID for Π . The technique can be generalized to *truncated differentials*, that is, to differentials β and δ that only concern a subset of the state. Below we apply such

a generalized miss-in-the-middle to the permutation of BLAKE. We expose separately the application to BLAKE-256 and to BLAKE-512. The strategy is similar for both:

1. start with a probability-1 differential with difference in the state and in the message so that differences vanish until the second round;
2. look for bits that are changed (or not) with probability one after a few more rounds, given this difference;
3. do the same as step 2 in the backwards direction, starting from the final difference.

Good choices of differences are those that maximize the delay before the input of the first difference, more precisely, those such that the message word with the difference appears in the second position of a diagonal step forwards, and in the first position of a column step backwards. The goal is to minimize diffusion so as to maximize the chance of probability-1 truncated differentials.

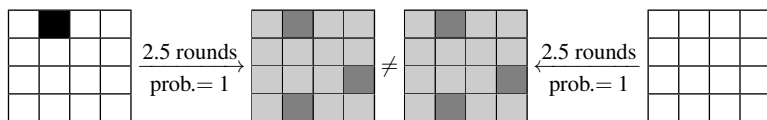


Fig. 8.3 Miss-in-the-middle for BLAKE-256, given the input difference 80000000 in m_2 and v_1 . The two differences in dark gray are incompatible, thus the impossibility. In the forward direction, 2.5 rounds are two rounds plus a column step; backwards, two inverse rounds plus an inverse diagonal step.

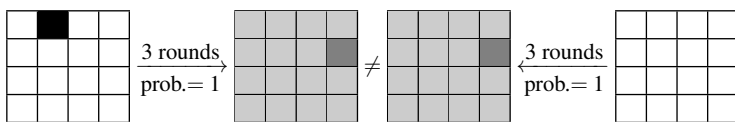


Fig. 8.4 Miss-in-the-middle for BLAKE-512, given the input difference $80 \dots 00$ in m_2 and v_1 . The two differences in dark gray are incompatible, thus the impossibility.

We consider a difference 80000000 in the initial state in v_1 , and in the message block word m_2 ; we have that

- *Forwards*, the differences in v_1 and m_2 cancel each other at the beginning of the column step and no difference is introduced until the diagonal step of the second round, in which m_2 appears as m_k in G_5 ; after the column step of the third round (that is, after 2.5 rounds), we observe that bits⁴ 35, 355, 439, and 443 are *always changed* in the state.

⁴ Here, bit 35 is the fourth most significant bit of the second state word v_1 , bit 355 is the fourth most significant bit of v_{11} , etc.

- *Backwards*, we start from a state free of difference, and m_2 introduces a difference at the end of the first inverse round, as it appears as m_j in the column step's G_2 ; after 2.5 inverse rounds, we observe that bits 35, 355, 439, and 433 are *always unchanged*.

The probability-1 differentials reported above were first discovered empirically, and could be verified analytically by tracking differences, distinguishing bits with probability-1 (non)difference, and other bits.

We deduce from the observations above that the difference 80000000 in v_1 and m_2 cannot lead to a state free of difference after five rounds. We thus identified a five-round ID for the permutation of BLAKE-256. Figure 8.3 gives a graphical description of the ID.

8.4 Properties of the Compression Function

The compression function of BLAKE consists of the initialization of the internal state, a sequence of round, and the finalization. This section reports security properties specific to that construction.

8.4.1 Finalization

At the finalization stage, the state is compressed to half its length, in a way similar to that of the cipher Rabbit [46]. The feedforward of h and s makes each word of the hash value dependent on two words of the inner state, one word of the initial value, and one word of the salt. The goal is to make the function noninvertible when the initial value and/or the salt are unknown.

Our approach of “permutation plus feedforward” is similar to that of SHA2, and can be seen as a particular case of Davies–Meyer-like constructions: denoting Enc the block cipher defined by the round sequence, BLAKE’s compression function computes

$$\text{Enc}_{m||s}(h) \oplus h \oplus (s||s),$$

which, for a null salt, gives the Davies–Meyer construction $\text{Enc}_m(h) \oplus h$. We use XORs and not additions (as in SHA2), because here additions do not increase security, and are much more expensive in circuits and 8-bit processors.

If the salt s was unknown and not fed forward, then one would be able to recover it given a one-block message, its hash value, and the IV. This would be a critical property. The counter t is not input in the finalization, because its value is always known and never chosen by the user.

8.4.2 Local Collisions

A *local collision* happens when, for two distinct messages, the internal states after a same number of rounds are identical. For BLAKE hash functions, there exist no local collisions for a same initial state (i.e., same IV, salt, and counter). This result directly follows from the fact that the round function is a permutation of the message, for fixed initial state v (and so different inputs lead to different outputs). The property generalizes to any number of rounds. The requirement of a same initial state does not limit much the result: for most applications, no salt is used, and a collision on the hash function implies a collision on the compression function with same initial state [35].

8.4.3 Fixed Points

A fixed point for BLAKE's compression function is a tuple (m, h, s, t) such that

$$\text{compress}(m, h, s, t) = h .$$

Functions of the form $\text{Enc}_m(h) \oplus h$ (like SHA2) allow the finding of fixed points for chosen messages by computing $h = \text{Enc}^{-1}(0)$, which gives $\text{Enc}_m(h) \oplus h = h$.

BLAKE's structure is a particular case of the Davies–Meyer-like constructions mentioned in Section 8.4; consider the case when no salt is used ($s = 0$), without loss of generality; for finding fixed points, we have to choose the final v such that

$$\begin{aligned} h_0 &= h_0 \oplus v_0 \oplus v_8 \\ h_1 &= h_1 \oplus v_1 \oplus v_9 \\ h_2 &= h_2 \oplus v_2 \oplus v_{10} \\ h_3 &= h_3 \oplus v_3 \oplus v_{11} \\ h_4 &= h_4 \oplus v_4 \oplus v_{12} \\ h_5 &= h_5 \oplus v_5 \oplus v_{13} \\ h_6 &= h_6 \oplus v_6 \oplus v_{14} \\ h_7 &= h_7 \oplus v_7 \oplus v_{15} \end{aligned}$$

That is, we need $v_0 = v_8, v_1 = v_9, \dots, v_7 = v_{15}$, so there are 2^{256} possible choices for v . From this v we compute the round function backward to get the initial state, and we find a fixed point whenL

- The third line of the state is $c_0, \dots, c_3$, and
- The fourth line of the state is valid, that is, $v_{12} = v_{13} \oplus c_4 \oplus c_5$ and $v_{14} = v_{15} \oplus c_6 \oplus c_7$.

Thus we find a fixed point with effort $2^{128} \times 2^{64} = 2^{192}$, instead of 2^{256} ideally. This technique also allows to find several fixed points for a same message (up to 2^{64} per message) in less time than expected for an ideal function.

BLAKE's fixed point properties do not give a distinguisher between BLAKE and a PRF, because we use here the internal mechanisms of the compression function, and not blackbox queries.

8.4.4 Fixed Point Collisions

A fixed point collision for BLAKE is a tuple (m, m', h, s, s', t, t') such that

$$\text{compress}(m, h, s, t) = \text{compress}(m', h, s', t') = h,$$

that is, a pair of fixed points for the same hash value. This notion was introduced in [9], where it is shown that fixed point collisions can be used to build multicollisions at reduced cost. For BLAKE-256, however, a fixed point collision costs about $2^{192} \times 2^{128} = 2^{320}$ trials, which is too high to exploit for an attack.

8.4.5 Pseudorandomness

One expects of a good hash function to “look like a random function.” Notions like indistinguishability, unpredictability, indifferenciability [126], and seed-incompressibility [79] define precise notions related to “randomness” for hash functions, and are used to evaluate generic constructions or dedicated designs. However they give no clue on how to construct primitives' algorithms.

Roughly speaking, the algorithm of the compression function should simulate a “complicated function,” with no apparent structure—i.e. it should have no property that a random function does not have. In terms of mathematical structure, “complicated” means, for example, that the algebraic normal form (ANF) of the function, as a vector of boolean functions, should contain each possible monomial with probability $1/2$; generalizing, this means that, when any part of the input is random, then the ANF obtained by fixing this input is also (uniform) random. Put differently, the truth table of the hash function when part of the input is random should “look like” a random bit string. In terms of input/output, “complicated” means, for example, that a small difference in the input does not imply a small difference in the output; more generally, any difference or relation between two inputs should be statistically independent of any relation of the corresponding outputs.

Pseudorandomness is particularly critical for stream ciphers, and no distinguishing attack—or any other nonrandomness property—has been identified for Salsa20 or ChaCha. These ciphers construct a complicated function by using a long chain of simple operations. Nonrandomness was observed for reduced versions with up to

three ChaCha rounds (corresponding to one and a half BLAKE rounds). BLAKE inherits ChaCha’s pseudorandomness, and in addition avoids the self-similarity of the function by having round-dependent constants. Although there is no formal reduction of BLAKE’s security to ChaCha’s, we can reasonably conjecture that BLAKE’s compression function is “complicated enough” with respect to pseudorandomness.

8.5 Security Against Generic Attacks

The security of the mode of operation of a hash function is assessed under the assumption that the core algorithm behaves “ideally.” That is, it concerns security properties of the construction that are independent of the underlying algorithms.

We first present results showing the general security of BLAKE’s mode of operation, then we discuss the applicability of state-of-the-art multicollision attacks.

8.5.1 Indifferentiability

The standard notion to establish the security of a mode of operation is that of *indifferentiability* [55, 126].

A mode of operation for a hash function is said to be indifferentiable from a random oracle if, informally, there exists no input–output relation that can be constructed more efficiently for the hash function than for an ideal hash function (assuming that the internal building blocks of the constructed hash function, e.g., compression functions or permutations, are ideal).

Formally, indifferentiability is generally proven by the construction of a “simulator” algorithm that attempts to emulate an ideal hash function upon queries of an attacker. This is the approach followed in two independent papers [4, 50] that proved BLAKE’s construction to be indifferentiable from a random oracle, assuming that its underlying block cipher is an ideal cipher (in other words, BLAKE is proven to be indifferentiable from a random oracle in the ideal cipher model, a model itself proven to be equivalent to the random oracle model [56, 83]).

What does indifferentiability mean concretely? First of all, indifferentiability is in no way a proof of security of the hash algorithm; remember that one assumes that some part of the function is ideal in the first place, so as to prove that the hash function as a whole behaves ideally. Indifferentiability thus only serves to focus cryptanalysis efforts on the components assumed perfect, and not to waste time on the construction combining those components. Also, indifferentiability proofs provide a general bound on the security of classes of hash functions, but do not guarantee that resistance to all attacks is optimal; for example, Keccak variants with capacity $c = 256$ have security guaranteed against attackers doing up to $2^{c/2} = 2^{128}$ queries, thus for a digest length of $n = 256$ nothing guarantees an optimal preimage resistance of 256 bits.

Second, there is another caveat: even if the internal components do behave ideally, indistinguishability does not capture all threat models. A counterexample was given by Ristenpart, Shacham, and Shrimpton in [154], which describes the following use case: a proof-of-storage protocol in a cloud storage system that sends back $H(M\|C)$ upon a random challenge C to prove that M is still stored. If H is BLAKE-256, and assuming (without much loss of generality) that M spans an integer number of blocks, then the server can only store *the chaining value* determined after processing M , and still respond correctly to the challenge. Clearly, this is not possible with a random oracle, and is undesirable in the context of this example [a straightforward fix would be to compute $H(C\|M)$ instead, making all M -dependent internal states also dependent on C].

8.5.2 Length Extension

Length extension is a forgery attack against MACs of the form $H_k(m)$ or $H(k\|m)$, i.e., where the key k is, respectively, used as the IV or prepended to the message. The attack can be applied when H is an iterated hash with “MD-strengthening” padding: given $h = H_k(m)$ and m , determine the padding data p , and compute $v' = H_h(m')$, for an arbitrary m' . It follows from the iterated construction that $v' = H_k(m\|p\|m')$; that is, the adversary forged a MAC of the message $m\|p\|m'$.

The length extension attack does not apply to BLAKE, because of the input of the number of bits hashed so far to the compression function, which simulates a specific output function for the last message block (cf. Section 2.4.2); for example, let m be a 1,020-bit message; after padding, the message is composed of three blocks m^0, m^1, m^2 ; the final chain value will be $h^3 = \text{compress}(h^2, m^2, s, 0)$, because the counter values are, respectively, 512, 1,020, and 0. If we extend the message with a block m^3 , with convenient padding bits, and hash $m^0\|m^1\|m^2\|m^3$, then the chain value between m^2 and m^3 will be $\text{compress}(h^2, m^2, s, 1,024)$, and thus be different from $\text{compress}(h^2, m^2, s, 0)$. The knowledge of BLAKE-256($m^0\|m^1\|m^2$) cannot be used to compute the hash of $m^0\|m^1\|m^2\|m^3$.

8.5.3 Collision Multiplication

We coin the term “collision multiplication” to define the ability, given a collision (m, m') , to derive an arbitrary number of other collisions; for example, Merkle–Damgård hash functions allow to derive collisions of the form $(m\|p\|u, m'\|p'\|u)$, where p and p' are the padding data, and u an arbitrary string; this technique can be seen as a kind of length extension attack. And for the same reasons that BLAKE resists length extension, it also resists this type of collision multiplication, when given a collision of minimal size (that is, when the collision only occurs for the hash value, not for intermediate chain values).

8.5.4 Multicollisions

A multicollision is a set of messages that map to the same hash value. We speak of a k -collision when k distinct colliding messages are known.

8.5.4.1 Joux's Technique

The technique proposed by Joux [90] (but previously described in [57,61]) finds a k -collision for Merkle–Damgård hash functions with n -bit hash values in $\lceil \log_2 k \rceil \cdot 2^{n/2}$ calls to the compression function (see Figure 8.5). The colliding messages have length of $\lceil \log_2 k \rceil$ blocks. This technique applies as well for the BLAKE hash functions, and to all hash functions based on HAIFA; for example, a 32-collision for BLAKE-256 can be found within 2^{133} compressions.

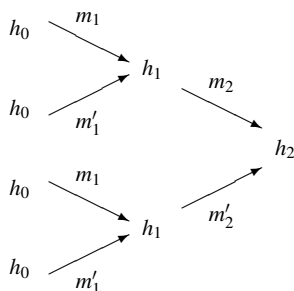


Fig. 8.5 Illustration of Joux's technique for 2-collisions, where $\text{compress}(h_0, m_1) = \text{compress}(h_0, m'_1) = h_1$, etc. This technique can apply to BLAKE.

Joux's attack is clearly not a concrete threat, which is demonstrated ad absurdum: to be applicable, it requires the knowledge of at least two collisions, but any function (resistant or not to Joux's attack) for which collisions can be found is broken anyway. Hence this attack only damages non-collision-resistant hash functions.

8.5.4.2 Kelsey–Schneier Technique

The technique presented by Kelsey and Schneier [100] works only when the compression function admits easily found fixed points. An advantage over Joux's attack is that the cost of finding a k -collision no longer depends on k . Specifically, for a Merkle–Damgård hash function with n -bit hash values, it makes $3 \cdot 2^{n/2}$ compressions and needs storage for $2^{n/2}$ message blocks (see Figure 8.6). Colliding messages have length of k blocks. This technique does not apply to BLAKE, because fixed points cannot be found efficiently, and the counter t foils fixed point repetition.

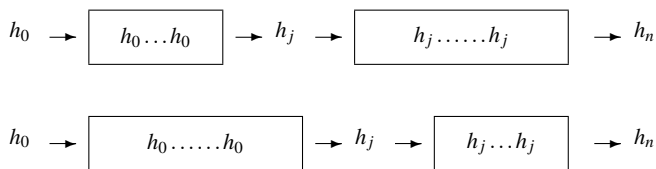


Fig. 8.6 Schematic view of the Kelsey–Schneier multicollision attack on Merkle–Damgård functions. This technique does not apply to BLAKE.

8.5.4.3 Faster Multicollisions

When an iterated hash admits fixed points and the IV is chosen by the attacker, this technique [9] finds a k -collision in time $2^{n/2}$ and negligible memory, with colliding messages of size $\lceil \log_2 k \rceil$ (see Figure 8.7). Like the Kelsey–Schneier technique, it is based on the repetition of fixed points, thus does not apply to BLAKE.

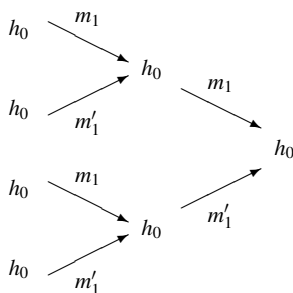


Fig. 8.7 Illustration of the faster multicollision, for 2-collisions on Merkle–Damgård hash functions. This technique does not apply to BLAKE.

8.5.5 Second Preimages

Dean [59, 5.6.3] and subsequently Kelsey and Schneier [100] showed generic attacks on n -bit iterated hashes that find second preimages in significantly fewer than 2^n compressions. HAIFA was proven to be resistant to these attacks [63], assuming a strong compression function; this result applies to BLAKE, as a HAIFA-based design. Therefore, no attack on n -bit BLAKE can find second-preimages in less than 2^n trials, unless exploiting the structure of the compression function.

8.6 Attacks on Reduced BLAKE

Below we describe some of the cryptanalytic attacks published on BLAKE. For a comprehensive list of results on BLAKE we refer the reader to <https://131002.net/blake/index.html#cr>.

At the time of writing, the most advanced attacks on BLAKE are the following:

- preimage attacks on 2.5 rounds, by Li and Xu [118], with complexity 2^{241} for BLAKE-256 and 2^{481} for BLAKE-512
- a boomerang distinguisher on 8 rounds of the keyed permutation of BLAKE-256, with complexity below 2^{200} , by Leurent [116], following previous work by Biryukov, Nikolić, and Roy [42], showed to be incorrect by Leurent [114, 115]
- a distinguisher on 6 rounds of the permutation of BLAKE-256, with complexity 2^{456} , by Dunkelman and Khovratovich [64]

These three works are described below, respectively, in Sections 8.6.1, 8.6.3, and 8.6.4.

8.6.1 Preimage Attacks

The preimage attacks by Li and Xu [118], applicable up to 2.5 rounds of BLAKE, exploit the property that, if a message block to the BLAKE compression function is modified, this leads to the attacker being able to control what one of the four output words of the G function will be after 1.5 rounds.

More precisely, if m_9 is modified, then the output word v_0 after G_4 in the first round can be controlled by the attacker. In a similar fashion, modifying m_{11} , m_{13} , and m_{15} , respectively, will lead to control of v_{12} , v_8 , and v_4 after G_5 , G_6 , and G_7 of round 1.

As v_0, v_4, v_8, v_{12} after round 1 are controlled and these four words subsequently go into G_0 within round 2, this means that the four output words from G_0 after a total of 1.5 rounds can be controlled. And so we have that, after finalization on 1.5 rounds,

$$\begin{aligned} h'_0 &\rightarrow h_0 \oplus s_0 \oplus \hat{v}_0 \oplus \hat{v}_8 \\ h'_4 &\rightarrow h_4 \oplus s_0 \oplus \hat{v}_4 \oplus \hat{v}_{12} \end{aligned}$$

Thus the output chaining values h'_0 and h'_4 can be controlled.

Recall that a preimage attack for the hash function means that, given the hash output, one aims to obtain a preimage, i.e., one or more message blocks, the above property is exploited as follows to mount preimage attacks on BLAKE: Given the initial value $h_{t-1} = h'^{-1}_0, \dots, h'^{-1}_7$ and the desired hash output $h_t = h'_0, \dots, h'_7$, the message blocks $m_9, m_{11}, m_{13}, m_{15}$ are modified to control the values of h'_0 and h'_4 after 1.5 rounds, such that a pair of such differing message blocks both map to h_t .

This technique allows to save a factor of 2^{15} in finding preimages for BLAKE-256, yielding an attack in approximately 2^{241} basic operations. When applied to BLAKE-512, a similar technique is shown to allow the finding of preimages in approximately 2^{481} .

8.6.2 Near-Collision Attack

We first describe the near-collision⁵ attack of Guo and Matusiewicz [13] on the reduced compression function of BLAKE-256. This attack only applies to a reduced version with four rounds—not the first four rounds, but rounds indexed 3 to 6—but it is remarkably simple, and has practical complexity (2^{56}).

The near-collision attack is based on the following observation: in the G function, rotations are by 16, 12, 8, and 7, where only 7 is not a multiple of 4. Therefore, if a same difference is introduced in all nibbles of a word, it may be preserved through G if it manages to avoid the 7-bit rotation. Furthermore, if two active words—that is, words with a difference in each of their nibbles—are combined by an addition, the differences may vanish (and with an XOR, they vanish with certainty). This attack thus works by “linearizing” integer additions as XORs, that is, finding an attack that works with probability 1 if all additions are replaced with XORs, and estimating the success probability as the probability that all additions behave as XORs (that is, propagate no carry).

The difference pattern that maximizes the success probability is 88888888, because it has minimal Hamming weight and ensures that the difference in the most significant nibble is satisfied with probability 1 through integer addition. Overall, the difference propagates through an integer addition like through an XOR with probability $2^{-7} = 1/128$.

Finding a position of differences that avoids the 7-bit rotation can be done with simple linear algebra methods. Then one chooses a configuration of differences that minimizes the number of active integer additions—thereby maximizing the success probability. Such a configuration has differences in m_0 and $v_0, v_3, v_7, v_8, v_{14}, v_{15}$ with starting point at round 3 and has only 8 active additions over the last three rounds. This configuration gives after feedforward final differences in h'_3, h'_4 , and h'_5 . For the first 1.5 rounds, carefully choosing chaining value and message words allows to satisfy all the constraints posed by additions “for free,” that is, with no additional complexity. This gives complexity of approximately $2^{7 \times 8} = 2^{56}$ trials.

The near-collision found is on $(256 - 24) = 232$ predetermined bits. Figure 8.8 shows how differences propagate from round 3 to 6.

⁵ A near-collision attack is a collision attack on a subset of the hash value’s bits. This subset may be a sequence of predetermined contiguous bits (say, the first 50 bits) or an arbitrary subset of randomly positioned bits.

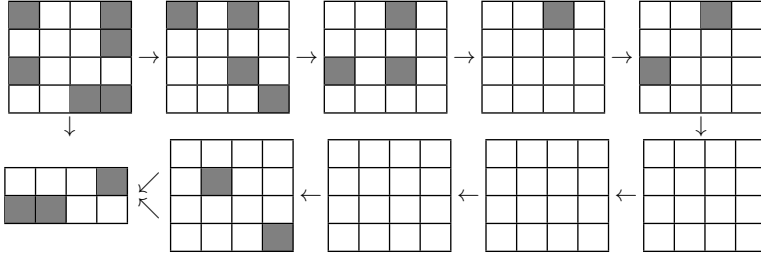


Fig. 8.8 Propagation of differences for near-collisions through rounds 3 to 6 (i.e., 8 steps). Inputs with difference are h_0 , h_3 , h_7 , s_0 , and t_0 . Gray cells denote states with differences.

8.6.3 Boomerang Distinguisher

Boomerang attacks are derived from the basic principle of differential cryptanalysis exposed in Section 8.1. The boomerang attacks on (reduced) BLAKE are so-called *distinguishers* since, contrary to the original boomerang attacks performing key recovery on block ciphers, they here only yield a tuple of blocks satisfying a specific relation—and such that the attack algorithm returns those values in much less time than a generic attack would.

Below we first introduce the principle of boomerang attacks, using the same notations and terminology as in Section 8.1.

8.6.3.1 Principle

The *boomerang attack*, introduced by Wagner in 1999 [172], works on a cipher $E = \tilde{E} \circ \hat{E}$ by exploiting a differential $\hat{\Delta} = (\hat{\Delta}_{\text{in}}, \hat{\Delta}_{\text{out}})$ for $\hat{E}$ and another differential $\tilde{\Delta} = (\tilde{\Delta}_{\text{in}}, \tilde{\Delta}_{\text{out}})$ for $\tilde{E}^{-1}$. It is based on the observation that, if an input m is such that:

1. $\hat{\Delta}$ is followed by m , that is,

$$\hat{E}_k(m) \oplus \hat{E}_k(m \oplus \hat{\Delta}_{\text{in}}) = \hat{\Delta}_{\text{out}} ;$$

2. $\tilde{\Delta}$ is followed by both $E_k(m)$ and $E_k(m \oplus \hat{\Delta}_{\text{in}})$, that is,

$$\begin{aligned} \tilde{E}_k^{-1}(E_k(m)) \oplus \tilde{E}_k^{-1}(E_k(m \oplus \hat{\Delta}_{\text{in}})) &= \tilde{\Delta}_{\text{out}} \\ \tilde{E}_k^{-1}(E_k(m \oplus \hat{\Delta}_{\text{in}})) \oplus \tilde{E}_k^{-1}(E_k(m \oplus \hat{\Delta}_{\text{in}}) \oplus \tilde{\Delta}_{\text{in}}) &= \tilde{\Delta}_{\text{out}} ; \end{aligned}$$

then we can obtain with probability $p_{\hat{\Delta}}$ the relation

$$E_k^{-1}(E_k(m) \oplus \tilde{\Delta}_{\text{in}}) \oplus E_k^{-1}(E_k(m \oplus \hat{\Delta}_{\text{in}}) \oplus \tilde{\Delta}_{\text{in}}) = \hat{\Delta}_{\text{in}} .$$

The actual attack works by querying for encryption of inputs with difference $\hat{\Delta}_{\text{in}}$, then querying for decryption of each of the values received with a difference $\hat{\Delta}_{\text{in}}$, and finally checking for a difference $\hat{\Delta}_{\text{in}}$ in the results of the last two queries.

If the forward differential characteristic is followed with probability p , and the backward differential characteristic with probability q , then the final difference is observed with probability about $(pq)^2$ (which should be significantly higher than 2^{-n} , with n the number of bits on which the difference is defined).

The *rectangle attack* [36] is a variant of the boomerang attack that works when blocks are smaller than keys. Boomerang (or rectangle) attacks were applied to build distinguishers or to mount key-recovery attacks [37, 40, 99]. The boomerang attack was first used in the context of hash functions by Joux and Peyrin [92].

8.6.3.2 Application to BLAKE

The first application of boomerang attacks to BLAKE was by Biryukov, Nikolić, and Roy [42]. The strategy used was to have differences in both the chaining value h and the message m , such that they cancel each other out as much as possible and thus minimize the number of active bits. The results claimed in [42] are a distinguisher on BLAKE-256's permutation with 8 rounds in 2^{242} , and on 7 rounds of its compression function in 2^{232} .

However, Leurent [114] showed that the results of Biryukov et al. were incorrect, due to inconsistencies in the differential characteristics used. He then demonstrated improved distinguishing attacks, partially formally verified, on BLAKE-256's internals:

- on 7 rounds of the compression function, in 2^{183}
- on 7 rounds of the keyed permutation in 2^{32}
- on 8 rounds of the keyed permutation in less than 2^{200}

Such results are of theoretical interest, however they are mostly irrelevant to the practical security of the hash function, even when used with a reduced number of rounds. Indeed, none of the results are applicable to the hash function with a reduced number of rounds, mainly because the attack model considered cannot be applied to the hash function.

8.6.4 Iterative Characteristics

As described in Section 8.1.1, an *iterative characteristic* is such that the input difference equals the output difference. More specifically, for an input pair x, x' we have $y \oplus y' = f(x) \oplus f(x') = \Delta$, with f the function attacked. An iterative differential characteristic is a useful building block in constructing a differential path through cipher and/or hash function rounds because it can be reused repeatedly, since the

difference at the output goes back to that of the input, and how many repeats can be tolerated is only limited by the feasibility of the overall probability attained.

Exploiting iterative differentials in BLAKE-256 was proposed by Dunkelman and Khovratovich [64]. They started with the G function, and focused on handling the effect on the differences by the rotation amounts 7, 8, 12, and 16.

They considered differences that are symmetric with respect to the rotation distance 8 (and therefore to any multiple thereof, like 16). This strategy is similar to that used by Guo and Matusiewicz for finding near-collisions (see Section 8.6.2); for instance, the difference 40404040 is invariant to rotation by 8, since

$$40404040 \ggg 8 = 40404040 \ggg 16 = 40404040.$$

They then searched for differentials through G such that the input entering the state due for rotation by 12 is a zero difference. To handle the rotation by 7, they chose the difference from the difference set $\{40404040, 80808080, c0c0c0c0\}$ so that rotation by 7 returns a value within the same set. Note here that $c0c0c0c0 = 40404040 \oplus 80808080$.

Having found high-probability differentials through one G, they carefully chose the best such differentials for Gs within a BLAKE round, which turned out to be the following (using 40 as a shorthand for 40404040):

- $(40, 80, 00, c0) \rightarrow (40, 80, 40, c0)$, which is satisfied upon random input values with probability 2^{-21}
- $(40, 00, 40, 0) \rightarrow (40, 00, 00, 00)$, which is satisfied upon random input values with probability 2^{-12}

These differentials are then exploited to build the following characteristic for one round (column step and diagonal step) of BLAKE-256:

$$\begin{pmatrix} 40 & 40 & 40 & 40 \\ 80 & 00 & 80 & 00 \\ 00 & 40 & 00 & 40 \\ c0 & 00 & c0 & 00 \end{pmatrix} \rightarrow \begin{pmatrix} 40 & 40 & 40 & 40 \\ 80 & 00 & 80 & 00 \\ 40 & 00 & 40 & 00 \\ c0 & 00 & c0 & 00 \end{pmatrix} \rightarrow \begin{pmatrix} 40 & 40 & 40 & 40 \\ 80 & 00 & 80 & 00 \\ 00 & 40 & 00 & 40 \\ c0 & 00 & c0 & 00 \end{pmatrix}$$

Each of the two characteristics is satisfied with probability 2^{-66} .

Based on this characteristic, [64] describes techniques to find a conforming pair for a 3-round characteristic in approximately 2^{60} operations (namely, using message modification techniques and trail backtracking, as in a number of differential attacks exploiting a complex characteristic). Then, a conforming pair is used to search a conforming pair for 6 rounds of the permutation. The effort for such a distinguisher is estimated to be of the order of 2^{456} operations.

Again, such a result helps understanding BLAKE's internals, but does not pose any security threat in any of the standard ways of using a hash function.

8.6.5 Breaking BLOKE

BLOKE is a toy version of BLAKE where the σ permutations are all set to the identity permutation—that is, no permutation of the message block words is done (see Section 3.5).

BLOKE was broken by Vidali, Nose, and Pašalic [171], who exploited the self-similarity of the round and found a fixed point such that h maps to itself, to find collisions in practical time:

They first observed that, given any internal state v , the message blocks that map v to itself can be determined efficiently (and uniquely, since one round is a permutation of the message for any fixed initial state). Then, they observed that, with such a fixed point, we have for $i = 0, \dots, 3$

$$\begin{aligned} h'_i &= h_i \oplus s_i \oplus h_i \oplus (s_i \oplus c_i) = c_i \\ h'_{i+4} &= h_{i+4} \oplus s_i \oplus h_{i+4} \oplus (t_{i/2} \oplus c_{i+4}) = s_i \oplus t_{i/2} \oplus c_{i+4} \end{aligned}$$

Therefore, in that case the new hash value h' depends only on the salt and counter, and *not on the previous chaining value*. One can thus choose two arbitrary messages of identical length, and for each of them append the message block that will yield an identical chaining value. Therefore, collisions for BLOKE can be found instantaneously.

These results support the design of BLAKE that includes round dependence within round functions.

8.6.6 Attack on a Variant with Identical Constants

We present a simple method to find collisions in $2^{n/4}$ for a toy variant of the compression function when the constants are all identical, that is, $k_i = k_j$ for all i, j .

Set $m = m_j$ for all i , and choose the chaining value, salt, and counter such that all four columns of the initial v are identical, that is, $v_i = v_{i+1} = v_{i+2} = v_{i+3}$ for $i = 0, 4, 8, 12$. Observe that G takes one input from each row, and then always uses $m \oplus u$ as input. Thus, all outputs of the four G functions in each step are identical, and so the columns remain identical through iteration of any number of rounds.

This essentially reduces the output space of the hash from 2^n to $2^{n/2}$, thus collisions can be found in $2^{n/4}$ due to the birthday paradox. However, to find a collision, we only have control over m , and this is not enough to give enough candidates ($2^{n/8}$ only) to carry out the birthday attack ($2^{n/4}$ required). We can resolve this problem by trying different (same for the collision pair) chaining values; for instance, we can set $t_0 = t_1 = 1$, and try different message values for the first $2^{n/8} + 1$ bits, then carry out the collision attack.

Note that this attack does *not* break the toy variants BLAZE and BRAKE from [15]. Indeed, these variants use no constants within G , but constants are used to initialize v . It is thus impossible to have four identical columns in the initial state.

Chapter 9

BLAKE2

BLAKE2 is a successor of BLAKE, designed in fall 2012—after Keccak was chosen as SHA3—by Jean-Philippe Aumasson, Samuel Neves, Zooko Wilcox-O’Hearn, and Christian Winnerlein. (The project partly stems from Twitter discussions, where the authors are respectively @veorq, @sevenps, @zooko, and @codesinchaos.) BLAKE2 was engineered to leverage BLAKE’s high efficiency and security, and to optimize it for modern applications, with simplicity and usability as primary considerations. In particular, a goal was that BLAKE2 be competitive in speed with MD5, since performance degradation has often been the main argument against hash function upgrades. BLAKE2 quickly gained interest from developers: a number of independent implementations and interfaces in various languages are now available, and BLAKE2 has been adopted in several projects, including the popular WinRAR archiving utility. This chapter describes BLAKE2, as well as the latest cryptanalysis results presented at the 2014 RSA Conference [74]. A large part of this chapter is adapted from the ACNS 2013 article [18] introducing BLAKE2.

9.1 Motivations

With Keccak, the SHA3 competition succeeded in selecting a hash function that complements SHA2 and is faster than SHA2 in hardware [52]. There is nevertheless a demand for fast software hashing for applications such as integrity checking and deduplication in file systems and cloud storage, host-based intrusion detection, version control systems, or secure boot schemes. These applications sometimes hash a few large messages, but more often many short ones, and hash speed directly affects the user experience.

Many systems use faster algorithms such as MD5, SHA1, or a custom function to meet their speed requirements, even though those functions may be insecure. MD5 is famously vulnerable to collision and length-extension attacks [65, 167], but it is 2.53 times as fast as SHA-256 on an Intel Ivy Bridge and 2.98 times as fast as SHA-256 on a Qualcomm Krait CPU.

Despite MD5's significant security flaws, it continues to be among the most widely used algorithms for file identification and data integrity. To choose just a handful of examples, the OpenStack cloud storage system [164], the popular version control system Perforce, and the object storage system used internally in AOL [145] (as of 2012) all rely on MD5 for data integrity. The venerable `md5sum` Unix tool remains one of the most widely used tools for data integrity checking. The Sun/Oracle ZFS file system includes the option of using SHA-256 for data integrity, but the default configuration is to instead use a noncryptographic 256-bit checksum, for performance reasons. The Tahoe-LAFS distributed storage system uses SHA-256 for data integrity, but is investigating a faster hash function [80].

Some SHA3 finalists outperform SHA2 in software; for example, on Ivy Bridge BLAKE-512 is 1.41 times as fast as SHA-512, and BLAKE-256 is 1.70 times as fast as SHA-256. BLAKE-512 reaches 5.76 cycles per byte, or approximately 579 mebibytes per second, against 411 for SHA-512, on a CPU clocked at 3.5 GHz. Some other SHA3 submissions are competitive in speed with BLAKE and Skein, but these have been less analyzed and generally inspire less confidence (e.g., due to distinguishers on the compression function).

BLAKE thus appears to be a good candidate for fast software hashing. Its security was evaluated by NIST in the SHA3 process as having a “very large security margin,” and the cryptanalysis published on BLAKE was noted as having “a great deal of depth.” But as observed by Preneel [147], its design “reflects the state of the art in October 2008”; since then, and after extensive cryptanalysis, we have a better understanding of BLAKE's security and efficiency properties.

BLAKE2 was thus proposed as an improved BLAKE with the following properties:

- **Faster than MD5** on 64-bit Intel platforms
- **32% less RAM** required than BLAKE
- Direct support, with no overhead, of:
 - **Parallelism** for many-times faster hashing on multicore or SIMD CPUs
 - **Tree hashing** for incremental update or verification of large files
 - **Prefix-MAC** for authentication that is simpler and faster than HMAC
 - **Personalization** for defining a unique hash function for each application
- **Minimal padding**, faster and simpler to implement

9.2 Differences with BLAKE

The BLAKE2 family consists of two main algorithms:

- **BLAKE2b** is optimized for *64-bit platforms*—including NEON-enabled ARM— and produces digests of any size between 1 and 64 bytes.
- **BLAKE2s** is optimized for *8- to 32-bit platforms*, and produces digests of any size between 1 and 32 bytes.

Both were designed to offer security similar to that of an ideal function producing digests of the same length. Each instance can be run on any CPU, but can be up to twice as fast when used on the CPU architecture for which it is optimized; for example, on a Tegra 2 (32-bit ARMv7-based SoC) BLAKE2s is expected to be about twice as fast as BLAKE2b, whereas on an AMD A10-5800K (64-bit, Piledriver microarchitecture), BLAKE2b is expected to be more than 1.5 times as fast as BLAKE2s.

Since BLAKE2 is similar to BLAKE, here we only describe the changes introduced with BLAKE2, and refer to Chapter 3 for the complete specification of BLAKE.

9.2.1 Fewer Rounds

BLAKE2b does *12 rounds* and BLAKE2s does *10 rounds*, against 16 and 14, respectively, for BLAKE. Based on the security analysis performed so far, and on reasonable assumptions on future progress, it is unlikely that 16 and 14 rounds are meaningfully more secure than 12 and 10 rounds (as discussed in Section 9.7). Note that the initial BLAKE submission had 14 and 10 rounds, respectively, and that the later increase [16] was motivated by the high speed of BLAKE (i.e., it could afford a few extra rounds for the sake of conservativeness), rather than by cryptanalysis results.

This change gives a direct speedup of about 25% and 29%, respectively, on long inputs. Speed on short inputs also significantly improves, though by a lower ratio, due to the overhead of initialization and finalization.

9.2.2 Rotations Optimized for Speed

The core function (G) of BLAKE-512 performs four 64-bit word rotations of, respectively, 32, 25, 16, and 11 bits. BLAKE2b replaces 25 with 24, and 11 with 63, for the following reasons:

- Using a 24-bit rotation allows SSSE3-capable CPUs to perform two rotations in parallel with a single SIMD instruction (namely, `pshufb`), whereas two shifts plus a logical OR are required for a rotation of 25 bits. This reduces the arithmetic cost of the G function, in recent Intel CPUs, from 18 single-cycle instructions to 16 instructions, a 12% decrease.
- A 63-bit rotation can be implemented as an addition (doubling) and a shift followed by a logical OR. This provides a slight speedup on platforms where addition and shift can be realized in parallel but not two shifts (i.e., some recent Intel CPUs). Additionally, since a rotation right by 63 is equal to a rotation left by 1, this may be slightly faster in some architectures where 1 is treated as a special case.

No platform suffers from these changes. Past experiments by the BLAKE designers as well as third-party cryptanalysis suggest that known differential attacks are unlikely to get significantly better (cf. Section 9.7).

9.2.3 Minimal Padding

BLAKE2 pads the last data block *if and only if necessary, with null bytes* (that is, “0” bits; recall that BLAKE2 operates on bytes as an atomic data unit, as opposed to bits for BLAKE). If the data length is a multiple of the block length, no padding byte is added. This implies that, if the message length is a multiple of the block length, no padding byte is added. The padding thus does not include the message length, as in BLAKE, MD5, or SHA2.

9.2.4 Finalization Flags

To avoid certain weaknesses, e.g., exploiting fixed points, BLAKE2 introduces *finalization flags* f_0 and f_1 , as auxiliary inputs to the compression function:

- The security functionality of the padding is transferred to a finalization flag f_0 , a word set to $\text{ff} \dots \text{ff}$ if the block processed is the last, and to $00 \dots 00$ otherwise. The flag f_0 is 64-bit for BLAKE2b, and 32-bit for BLAKE2s.
- A second finalization flag f_1 is used to signal the last node of a layer in tree-hashing modes (see Section 9.4). When processing the last block—that is, when f_0 is $\text{ff} \dots \text{ff}$ —the flag f_1 is also set to $\text{ff} \dots \text{ff}$ if the node considered is the last, and to $00 \dots 00$ otherwise.

The finalization flags are processed by the compression function as described in Section 9.2.5.

BLAKE2s thus supports hashing of data of at most $2^{64} - 1$ bytes, that is, almost 16 exbibytes (the amount of memory addressable by 64-bit processors). BLAKE2b’s upper bound of $2^{128} - 1$ bytes ought to be enough for anybody.

9.2.5 Fewer Constants

Whereas BLAKE used 8 word constants as IV plus 16 word constants for use in the compression function, BLAKE2 uses a total of 8 *word constants, instead of 24*. This saves 128 ROM bytes and 128 RAM bytes in BLAKE2b implementations, and 64 ROM bytes and 64 RAM bytes in BLAKE2s implementations.

The compression function initialization phase is modified to:

$$\begin{pmatrix} v_0 & v_1 & v_2 & v_3 \\ v_4 & v_5 & v_6 & v_7 \\ v_8 & v_9 & v_{10} & v_{11} \\ v_{12} & v_{13} & v_{14} & v_{15} \end{pmatrix} := \begin{pmatrix} h_0 & h_1 & h_2 & h_3 \\ h_4 & h_5 & h_6 & h_7 \\ \text{IV}_0 & \text{IV}_1 & \text{IV}_2 & \text{IV}_3 \\ t_0 \oplus \text{IV}_4 & t_1 \oplus \text{IV}_5 & f_0 \oplus \text{IV}_6 & f_1 \oplus \text{IV}_7 \end{pmatrix}$$

Note the introduction of the finalization flags f_0 and f_1 , in place of BLAKE's redundant counter.

The G functions of BLAKE2b (left) and BLAKE2s (right) are defined as

$a := a + b + m_{\sigma_r(2i)}$ $d := (d \oplus a) \ggg 32$ $c := c + d$ $b := (b \oplus c) \ggg 24$ $a := a + b + m_{\sigma_r(2i+1)}$ $d := (d \oplus a) \ggg 16$ $c := c + d$ $b := (b \oplus c) \ggg 63$	$a := a + b + m_{\sigma_r(2i)}$ $d := (d \oplus a) \ggg 16$ $c := c + d$ $b := (b \oplus c) \ggg 12$ $a := a + b + m_{\sigma_r(2i+1)}$ $d := (d \oplus a) \ggg 8$ $c := c + d$ $b := (b \oplus c) \ggg 7$
--	--

Note the aforementioned change of rotation counts.

Omitting the constants in G gives an algorithm similar to the BLAZE toy version (see Section 3.5). The constants in G were initially aimed to guarantee early propagation of carries, but it turned out that the benefits (if any) are not worth the performance penalty, as observed by a number of cryptanalysts. This change saves two XORs and two loads per G, that is, 16% of the total arithmetic (addition and XOR) instructions.

9.2.6 Little-Endianness

BLAKE, like SHA1 and SHA2, parses data blocks in the big-endian byte order. Like MD5, *BLAKE2 is little-endian*, because the large majority of target platforms are little-endian (AMD and Intel desktop processors, as well as most mainstream ARM systems). Switching to little-endian may provide a slight speedup, and often simplifies implementations.

Note that in BLAKE, the counter t is composed of two words t_0 and t_1 , where t_0 holds the least significant bits of the integer encoded. This (semi-)little-endian convention is preserved in BLAKE2.

9.2.7 Counter in Bytes

The counter t counts *bytes rather than bits*. This simplifies implementations and reduces the risk of error, since most applications measure data volumes in bytes rather than bits.

Note that BLAKE supports messages of arbitrary bit size for the sole purpose of conforming to NIST's requirements. However, there is no evidence of an actual need from applications to support this. Furthermore, and as observed during the first months of the competition, the support of arbitrary bit sizes was the origin of several bugs in reference implementations (including that of BLAKE).

9.2.8 Salt Processing

The modification in the salt processing simplifies the compression function, and saves a few instructions as well as a few bytes in RAM, since the salt does not have to be stored anymore. (And if the salt is supposed to be kept secret, that reduces the exposition of the salt to attackers.) Using salt-independent compression functions has only negligible practical impact on security, as discussed in Section 9.7.

9.2.9 Parameter Block

The parameter block of BLAKE2 is *XORed with the IV* prior to the processing of the first data block. It encodes parameters for secure tree hashing, as well as key length (in keyed mode) and digest length.

The parameters are described below, and the block structure is shown in Tables 9.1 and 9.2:

- General parameters:
 - **Digest byte length** (1 byte): an integer in $[1, 64]$ for BLAKE2b, in $[1, 32]$ for BLAKE2s
 - **Key byte length** (1 byte): an integer in $[0, 64]$ for BLAKE2b, in $[0, 32]$ for BLAKE2s (set to 0 if no key is used)
 - **Salt** (16 or 8 bytes): an arbitrary string of 16 bytes for BLAKE2b, and 8 bytes for BLAKE2s (set to all-NULL by default)
 - **Personalization** (16 or 8 bytes): an arbitrary string of 16 bytes for BLAKE2b, and 8 bytes for BLAKE2s (set to all-NULL by default)
- Tree hashing parameters:
 - **Fanout** (1 byte): an integer in $[0, 255]$ (set to 0 if unlimited, and to 1 only in sequential mode)

Table 9.1 BLAKE2b parameter block structure (offsets in bytes; RFU stands for “reserved for future use”).

Offset	0	1	2	3
0	Digest length	Key length	Fanout	Depth
4	Leaf length			
8	Node offset			
12				
16	Node depth	Inner length	RFU	
20	RFU			
24				
28				
32	Salt			
...				
44				
48	Personalization			
...				
60				

- **Maximal depth** (1 byte): an integer in $[1, 255]$ (set to 255 if unlimited, and to 1 only in sequential mode)
- **Leaf maximal byte length** (4 bytes): an integer in $[0, 2^{32} - 1]$, that is, up to 4 GiB (set to 0 if unlimited, or in sequential mode)
- **Node offset** (8 or 6 bytes): an integer in $[0, 2^{64} - 1]$ for BLAKE2b, and in $[0, 2^{48} - 1]$ for BLAKE2s (set to 0 for the first, leftmost, leaf, or in sequential mode)
- **Node depth** (1 byte): an integer in $[0, 255]$ (set to 0 for the leaves, or in sequential mode)
- **Inner hash byte length** (1 byte): an integer in $[0, 64]$ for BLAKE2b, and in $[0, 32]$ for BLAKE2s (set to 0 in sequential mode)

This is 50 bytes in total for BLAKE2b, and 32 bytes for BLAKE2s. Any bytes left are reserved for future and/or application-specific use, and are NULL. Values spanning more than one byte are written *little-endian*. Note that tree hashing may be keyed, in which case leaf instances hash the key followed by a number of bytes equal to (at most) the maximal leaf length.

Table 9.2 BLAKE2s parameter block structure (offsets in bytes).

Offset	0	1	2	3
0	Digest length	Key length	Fanout	Depth
4	Leaf length			
8	Node offset			
12	Node offset (cont.)		Node depth	Inner length
16	Salt			
20				
24	Personalization			
28				

We take as example an instance of BLAKE2b with:

- 64-byte digests, that is, with parameter digest length set to 40
- A 256-bit key, that is, with parameter key length set to 20
- A salt set to the all-55 string
- A personalization set to the all-ee string

BLAKE2b hashes data sequentially, thus tree parameters are set to the value specified for the sequential mode: fanout and maximal depth are set to 01, leaf maximal length is set to 00000000, node offset is set to 0000000000000000, and node depth and inner hash length are set to 00.

The parameter block for this instance of BLAKE2b is thus the following:¹

```
40200101 00000000 00000000 00000000 00000000 00000000 00000000 00000000
55555555 55555555 55555555 55555555 55555555 55555555 55555555 55555555
```

9.3 Keyed Hashing (MAC and PRF)

When keyed (that is, when the field key length is nonzero), BLAKE2 sets the first data block to the key padded with zeros, the second data block to the first message block, the third block to the second message block, etc. Note that the padded key is treated as arbitrary data, therefore:

- The counter t includes the 64 (or 128) bytes of the key block, regardless of the key length;
- When hashing the empty message with a key, BLAKE2b and BLAKE2s make only one call to the compression function.

The main application of keyed BLAKE2 is as a message authentication code (MAC). Indeed, BLAKE2 can safely be used in “prefix-MAC” mode, thanks to the indistinguishability property inherited from BLAKE [4, 50]. Prefix-MAC is always more efficient than HMAC, as it saves at least one call to the compression function. Keyed BLAKE2 can also be used to instantiate PRFs, for example, within the PBKDF2 password hashing scheme.

9.4 Tree Hashing

The parameter block supports arbitrary tree hashing modes, be it binary or ternary trees, arbitrary-depth updatable tree hashing or fixed-depth parallel hashing, etc.

¹ For readability we add a space between each 4-byte block, however the value represented is a string of bytes, not a sequence of 4-byte words (which makes a difference with respect to endianness).

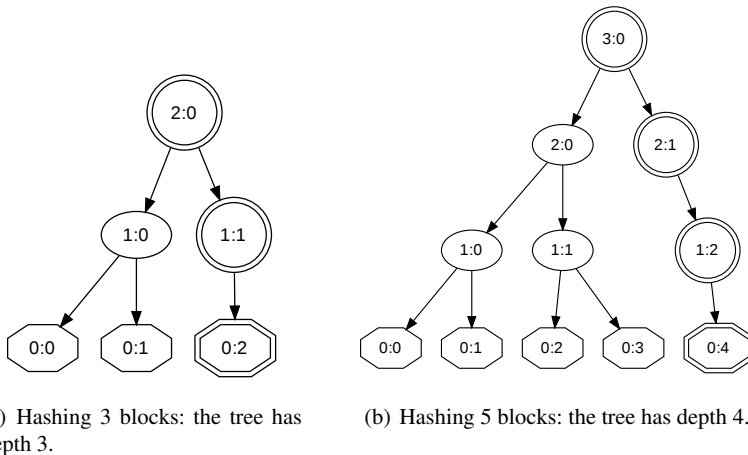


Fig. 9.1 Layouts of tree hashing with fanout 2, and maximal depth at least 4.

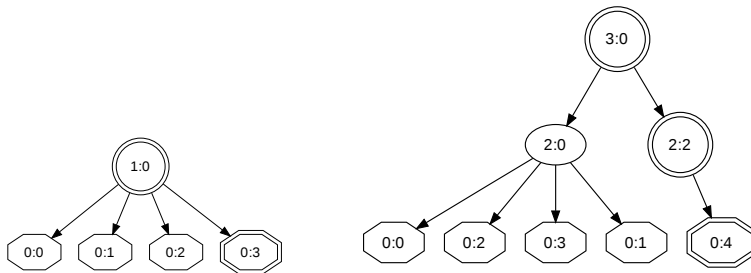
Unlike other tree hashing functions or modes, BLAKE2 does not restrict the leaf length and the fanout to be powers of 2.

9.4.1 Basic Mechanism

Informally, tree hashing processes chunks of “leaf length” bytes independently of each other, then combines the respective hashes using a tree structure wherein each node takes as input the concatenation of “fanout” hashes. The “node offset” and “node depth” parameters ensure that each invocation of the hash function (leaf or internal node) uses a different hash function. The finalization flag f_1 signals when a hash invocation is the last one at a given depth (where “last” is with respect to the node offset counter, for both leaves and intermediate nodes). The flag f_1 can only be nonzero for the last block compressed within a hash invocation, and the root node always has f_1 set to $\text{ff} \dots \text{ff}$.

Figures 9.1 and 9.2 illustrate the tree hashing mechanism, with layouts of trees given different parameters and different input lengths. In those figures, octagons represent leaves (i.e., instances of the hash function processing input data), and double-lined nodes (including leaves) are the last nodes of a layer (and thus have the flag f_1 set). Labels “ $i:j$ ” indicate a node’s depth i and offset j .

We refer to [31] for a comprehensive overview of secure tree hashing constructions.



(a) Hashing 4 blocks: the tree has depth 2.

(b) Hashing 5 blocks: the tree has depth 3.

Fig. 9.2 Layouts of tree hashing with fanout 4, and maximal depth at least 3.

9.4.2 Message Parsing

Unless specified otherwise, we recommend that data be parsed as contiguous blocks; for example, if the leaf length is 1,024 bytes, then the first 1,024-byte data block is processed by the leaf with offset 0, the subsequent 1,024-byte data block is processed by the leaf with offset 1, etc.

9.4.3 Special Cases

We highlight some special cases of tree hashing:

- **Unlimited fanout:** When the fanout is unlimited (parameter set to 0), then the root node hashes the concatenation of as many leaves as are required to process the message. That is, the depth of the tree is always 2, regardless of the maximal depth parameter. Nevertheless, changing the maximal depth parameter changes the final hash value returned. We thus recommend to set the depth parameter to 2.
- **Dealing with saturated trees:** If a tree hashing instance has fanout $f \geq 2$, maximal depth $d \geq 2$, and leaf maximal length $\ell \geq 1$ bytes, then up to $f^{d-1} \cdot \ell$ can be processed within a single tree. If more bytes have to be hashed, the fanout of the root node is extended to hash as many digests as necessary to respect the depth limit. This mechanism is illustrated on Figure 9.4. Note that, if the maximal depth is 2, then the value does not affect the layout of the tree, which is identical to that of a tree hash with unlimited fanout (see Figure 9.3).

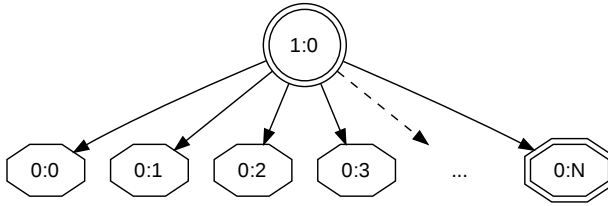


Fig. 9.3 Tree hashing with unbounded fanout (0) and arbitrary maximal depth (de facto, 2).

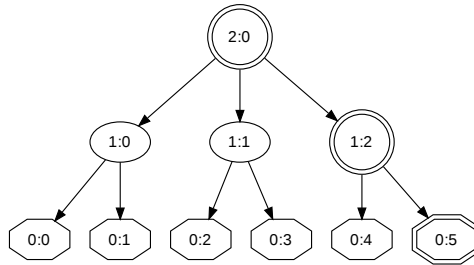


Fig. 9.4 Tree hashing with maximal depth 3, fanout 2, but a root with larger fanout due to the reach of the maximal depth.

9.4.4 Generic Tree Parameters

Tree parameters supported by the parameter block allow for a wide range of implementation tradeoffs, for example, to efficiently support updatable hashing, which is typically an advantage when hashing many (small) chunks of data.

Although optimal performance will be reached by choosing the parameters specific to one's application, we specify the following parameters for a *generic tree mode*: binary tree (i.e., fanout 2), unlimited depth, and leaves of 4 KiB (the typical size of a memory page).

9.4.5 Updatable Hashing Example

Assume that one has to provide a digest of a 1-tebabyte file system disk image that is updated every day. Instead of recomputing the digest by reading all 2^{40} bytes, one can use our generic tree mode to implement an updatable hashing scheme:

1. Apply the generic tree mode, and store the $2^{40}/4,096 = 2^{28}$ hashes from the leaves as well as the $2^{28} - 2$ intermediate hashes;

2. When a leaf is changed, update the final digest by recomputing the 28 intermediate hashes.

If BLAKE2b is used with intermediate hashes of 32 bytes, and if it hashes at a rate of 500 mebibytes per second, then step 1 takes approximately 35 minutes and generates about 16 gibibytes of intermediate data, whereas step 2 is instantaneous.

Note however that much less data may be stored: For many applications it is preferable to only store the intermediate hashes for larger pieces of data (without increasing the leaf size), which reduces the memory requirement by only storing “higher” intermediate values; for example, storing intermediate values for 4 MiB chunks instead of all 4 KiB leaves reduces the storage to only 16 MiB. Indeed, using 4 KiB leaves allows applications with different piece sizes (as long as they are powers of two of at least 4 KiB) to produce the same root hash, while allowing them to make different granularity versus storage tradeoffs.

9.5 Parallel Hashing: BLAKE2sp and BLAKE2bp

We specify two parallel hash functions, that is, with depth 2 and unlimited leaf length:

- BLAKE2bp runs *4 instances* of BLAKE2b in parallel;
- BLAKE2sp runs *8 instances* of BLAKE2s in parallel.

These functions use a different *parsing rule* than the default one proposed in Section 9.4: The first instance (node offset 0) hashes the message composed of the concatenation of all message blocks of index zero modulo 4; the second instance (node offset 1) hashes blocks of index 1 modulo 4, etc. Note that, when the leaf length is unlimited, parsing the input as contiguous blocks would require the knowledge of the input length *before* any parallel operation, which is undesirable (e.g., when hashing a stream of data of undefined length, or a file received over a network).

When hashing one single large file, and when incrementability is not required, such parallel modes with unlimited leaf length seem to be the most efficient when higher speed is desired and when sufficient CPU bandwidth and resource are available. Indeed:

- They *minimize the computation overhead* by doing only one nonleaf call to the sequential hash function;
- They *maximize the usage of the CPU (cores, ALUs, etc.)* by keeping multiple cores and instruction pipelines busy simultaneously;
- They require *realistic bandwidth and memory*.

Within a parallel hash, the same parameter block, except for the node offset, is used for all 4 or 8 instances of the sequential hash.

9.6 Performance

BLAKE2 is significantly faster than BLAKE, mainly due to its reduced number of rounds, but not only. On long messages, BLAKE2b and BLAKE2s are expected to be approximately 25% and 29% faster, ignoring any savings from the absence of constants, optimized rotations, or little-endian conversion. The parallel versions BLAKE2bp and BLAKE2sp are expected to be 4 and 8 times faster than BLAKE2b and BLAKE2s on long messages, when implemented with multiple threads on a CPU with 4 or more cores (as most desktop and server processors: AMD FX-8150, Intel Core i5-2400S, etc.). Parallel hashing also benefits from advanced CPU technologies, as previously observed [130, 5.2].

C and C# code of BLAKE2 under public domain-like license is available on <https://blake2.net>, as well as a tool `b2sum` (similar to `md5sum`).

9.6.1 Why *BLAKE2* Is Fast in Software

BLAKE2, along with its parallel variant, can take advantage of the following architectural features, or combinations thereof:

9.6.1.1 Instruction-Level Parallelism

Most modern processors are superscalar, that is, able to run several instructions per cycle through pipelining, out-of-order execution, and other related techniques. BLAKE2 has a natural instruction parallelism of 4 instructions within the G function; processors that are able to handle more instruction-level parallelism can do so in BLAKE2bp, by interleaving independent compression function calls. Examples of processors with notorious amounts of instruction parallelism are Intel's Core 2, i7, and Itanium or AMD's K10, Bulldozer, and Piledriver.

9.6.1.2 SIMD Instructions

Many modern processors contain *vector units*, which enable SIMD processing of data. Again, BLAKE2 can take advantage of vector units not only in its G function, but also in tree modes (such as the mode proposed in Section 9.5), by running several compression instances within vector registers. Microarchitectures with SIMD capabilities are found in recent Intel and AMD CPUs, NEON-extended ARM-based SoC, PowerPC and Cell CPUs.

9.6.1.3 Multiple Cores

Limits in both semiconductor manufacturing processes, as well as instruction-level parallelism have driven CPU manufacturers towards yet another kind of coarse-grained parallelism, where multiple independent CPUs are placed inside the same die, and enable the programmer to get thread-level parallelism. While sequential BLAKE2 does not take advantage of this, the parallel mode described in Section 9.5, and other tree modes, can run each intermediate hashing in its own thread. Candidate processors for this approach are recent Intel and AMD chips, the IBM Cell, and recent ARM, UltraSPARC, and Loongson models.

9.6.2 64-bit Platforms

Optimized BLAKE2 implementations were benchmarked on the eBACS [28] platform. These implementations take advantage of the AVX and XOP instructions sets, the latter being available only on AMD microarchitectures, starting with Bulldozer (released in 2011). Table 9.3 presents the timings reported for processors with two key microarchitectures: Intel’s Sandy Bridge (hydra7) and AMD’s Bulldozer (hydra6). The full set of results is available at <http://bench.cr.yp.to/results-hash.html>.

Table 9.3 Speed, in cycles per byte, of BLAKE2 in sequential mode.

Microarchitecture	BLAKE2b			BLAKE2s		
	Long	1,536	64	Long	1,536	64
Sandy Bridge	3.32	3.81	9.00	5.34	5.35	5.50
Bulldozer	5.29	5.30	11.95	8.20	8.21	7.91

- Compared with the fastest BLAKE implementations:
- On Sandy Bridge, BLAKE2b is 71.99% faster than BLAKE-512, and BLAKE2s is 40.26% faster than BLAKE-256;
 - On Bulldozer, BLAKE2b is 30.25% faster than BLAKE-512, and BLAKE2s is 43.78% faster than BLAKE-256.

Due to the lack of native rotation instructions on SIMD registers, the speedup of BLAKE2b is greater on the Intel processors, which benefit not only from the round reduction, but also from the easier-to-implement rotations.

On short messages, the speed advantage of the improved padding in BLAKE2 is quite noticeable. On Sandy Bridge, no other cryptographic hash function measured in eBACS² (including MD5 and MD4) is faster than BLAKE2s on 64-byte messages, while BLAKE2b is roughly as fast as MD4.

² <http://bench.cr.yp.to/results-hash.html#amd64-hydra7>

Like BLAKE, BLAKE2 benefits from the AVX2 instruction set, which appeared in the Haswell microarchitecture by Intel. The analysis performed in Section 5.5 for BLAKE applies to BLAKE2 as well, except for the constants, which reduce the number of instructions per compression function: techniques such as parallelized message loading or message caching can thus be applied to BLAKE2b and BLAKE2s.

As expected, the parallel versions provide a speedup of a factor close to the parallelism degree; for example, using our utility³ `b2sum` on Bulldozer, the file `ubuntu-12.04-beta1-desktop-amd64.iso` is hashed in 1.16 s with BLAKE2b, 0.33 s with BLAKE2bp (that is, 3.51 times faster), in 1.72 s with BLAKE2s, and in 0.27 s with BLAKE2sp (that is, 6.37 times faster). Similarly, on Sandy Bridge BLAKE2bp is 3.76 times faster than BLAKE2b (1.58 s versus 0.42 s) hashing the same file, while BLAKE2sp is 3.68 times faster than BLAKE2s (2.21 s versus 0.60 s). Enabling hyperthreading (with 8 virtual cores) increases the latter speedup to 5.66, hashing the file in 0.39 s. We expect these speedups to converge to 4 and 8, respectively, as implementations (and CPUs) improve.

Compared with Keccak's SHA3 final submission, BLAKE2 does quite well on 64-bit hardware. On Sandy Bridge, the 512-bit Keccak[$r = 576, c = 1,024$] hashes at 20.46 cycles per byte, while the 256-bit Keccak[$r = 1,088, c = 512$] hashes at 10.87 cycles per byte.

Keccak is, however, a very versatile design. By lowering the capacity from $4n$ to $2n$, where n is the output bit length, one achieves $n/2$ -bit security for both collisions and second preimages [30], but also higher speed. We estimate that a 512-bit Keccak[$r = 1,088, c = 512$] would hash at about 10 cycles per byte on high-end Intel and AMD CPUs, and a 256-bit Keccak[$r = 1,344, c = 256$] would hash at roughly 8 cycles per byte. This parametrization would put Keccak at a performance level superior to SHA2, but at a substantial cost in second-preimage resistance. BLAKE2 does not require such tradeoffs, and still offers much higher speed.

At the time of completing the book, the most recent benchmarks from include measurements on an Intel Xeon E3-1275 (Haswell microarchitecture) clocked at 3500 MHz. Exploiting the AVX2 instructions, BLAKE2b runs at 2.88 cycles/byte (1159 MiBps).

9.6.3 Low-End Platforms

A typical implementation of BLAKE-256 in embedded software stores in RAM at least the chaining value (32 bytes), the message (64 bytes), the constants (64 bytes), the permutation internal state (64 bytes), the counter (8 bytes), and the salt, if used (16 bytes), that is, 232 bytes, and 248 with a salt. BLAKE2s reduces these figures to 168 bytes—recall that the salt does not have to be stored anymore—that is, a gain of

³ Available from <https://blake2.net>

respectively, 28% and 32%. Similarly, BLAKE2b only requires 336 bytes of RAM, against 464 or 496 for BLAKE-512.

9.6.4 Hardware

Hardware implementations directly benefit from the 29% and 25% speedup in sequential mode, due to the round reduction, for any message length. Parallelism is straightforward to implement by replicating the logic of the sequential hash, and running independent instances in parallel circuits. BLAKE2 enjoys the same degrees of freedom as BLAKE to implement various space–time tradeoffs (horizontal and vertical folding, pipelining, etc.). In addition, parallel hashing provides *another dimension for trade-offs* in hardware architectures: depending on the system properties (e.g., how many input bits can be read per cycle), one may choose between, for example, BLAKE2sp based on eight high-latency compact cores, or BLAKE2s based on a single low-latency unrolled core.

9.7 Security

BLAKE2 builds on the high confidence built by BLAKE in the SHA3 competition. Although BLAKE2 performs fewer rounds than BLAKE, this does not necessarily imply lower security (though it does imply a lower *security margin*, which is quite an artificial notion), as explained below.

9.7.1 BLAKE Legacy

The security of BLAKE2 is closely related to that of BLAKE, since they rely on a similar core permutation.

Since 2009, at least 14 research papers have described cryptanalysis results on reduced versions of BLAKE.

As reported in Chapter 8, the most advanced attacks on the BLAKE as a hash function—as opposed to attacks on its building blocks: permutation, compression function—are *preimage attacks on 2.5 rounds* by Ji and Liangyu, with respective complexities of 2^{241} and 2^{481} for BLAKE-256 and BLAKE-512 [88].

The exact attacks as described in recent cryptanalysis papers on building blocks of BLAKE [42, 64] may not even directly apply to those of BLAKE2, due to the changes of rotation counts (typically, differential characteristics for BLAKE do not apply to BLAKE2). Nevertheless, BLAKE2 was designed with the expectation that attacks on reduced BLAKE with n rounds would adapt to BLAKE2 with at least n rounds.

9.7.2 *Implications of BLAKE2 Tweaks*

We have argued that the reduced number of rounds and the optimized rotations are unlikely to meaningfully reduce the security of BLAKE2, compared with that of BLAKE. We summarize the security implications of other tweaks:

9.7.2.1 Salt-Independent Compressions

BLAKE2 salts the hash function in the IV, rather than each compression. This preserves the uniqueness of the hash function for any distinct salt, but facilitates multicollision attacks relying on offline precomputations (see [35, 90]). However, this leaves fewer “controlled” bits in the initial state of the compression function, which complicates the finding of fixed points.

9.7.2.2 Many Valid IVs

Due to the high number of valid parameter blocks, BLAKE2 admits many valid initial chaining values; for example, if an attacker has an oracle that returns collisions for random chaining values and messages, she is more likely to succeed in attacking the hash function because she has many valid targets, rather than one. However, such a scenario assumes that (free-start) collisions can be found efficiently, that is, that the hash function is already broken. Note that the best collision-like results on BLAKE are near-collisions for the compression function with four *reordered* rounds [75, 168].

9.7.2.3 Simplified Padding

The new padding does not include the length of the message, unlike BLAKE. However, it is easy to see that the length is indirectly encoded through the counter, and that the padding preserves the unambiguous encoding of the initial padding. That is, the padding simplification does not affect the security of the hash function. Nevertheless, it may be desirable to have a formal proof.

9.7.3 *Third-Party Cryptanalysis*

At the time of writing, the only third-party cryptanalysis published on BLAKE2 is the work of Guo, Karpman, Nikolić, Wang, and Wu, presented at the 2014 RSA Conference [74]. This paper tends to confirm our initial intuition that, although BLAKE2 relaxes the cryptographic strength of some internal building blocks, this does not transfer to a reduced security of the actual hash function. Moreover, the

paper argues that “omitting the double use of the counter, as well as introducing constants IV_i , reduces the number of attacked rounds, i.e. increases the security of the compression function.”

9.7.3.1 Permutation

The main result of Guo et al. [74] on the core permutation of BLAKE2b is a distinguisher based on the observation of invariance with respect to words’ rotation: this result holds for the full 12-round version, however it has complexity of $\dots 2^{876}$ (remember that complexities as “low” as 2^{128} are considered to be an infeasible effort by today’s standards). However, in theory, observing such invariance for an ideal permutation has average-case complexity of 2^{1024} . For BLAKE2s, a similar technique can be applied to only seven rounds, with complexity of 2^{511} .

9.7.3.2 Compression Function

Guo et al. observed that, by finding a fixed point (a, b, c, d) for the G function of BLAKE2b, we have the following behavior of the round compression function:

$$\begin{pmatrix} a & a & a & a \\ b & b & b & b \\ c & c & c & c \\ d & d & d & d \end{pmatrix} \xrightarrow{\text{round}} \begin{pmatrix} a & a & a & a \\ b & b & b & b \\ c & c & c & c \\ d & d & d & d \end{pmatrix} \xrightarrow{\text{feedforward}} \begin{pmatrix} c & c & c & c \\ d & d & d & d \end{pmatrix};$$

that is, the value of the internal state is unchanged by the round function, and the feedforward with the initial chaining value (a, a, a, a, b, b, b, b) gives exactly (c, c, c, c, d, d, d, d) as the new chaining value. By finding two such fixed points with a difference of low Hamming weight in c and d , one may thus find partial collisions, on the bits with no difference. Since G does not have iterative characteristics with respect to XOR differences, Guo et al. use “rotational” differences. This leads to “partial collisions” on 304 chosen bits (of 512 in total), with complexity of approximately 2^{61} .

With a similar technique, and by modifying the IV to a carefully chosen value, collisions for the modified compression function of BLAKE2s can be found with complexity of approximately 2^{64} .

However, those methods require that the *IV be modified* to a value determined by the fixed point used. The IV specified by BLAKE2 clearly cannot be exploited, due to its asymmetry that prevents identical c and d values in the bottom rows of the state. Actually, the point of BLAKE2’s IV was precisely to avoid this type of attack by breaking symmetries in the initial value of the state.

9.7.3.3 Hash Function

The only result on the (reduced) BLAKE2 hash function in [74] is a “differential distinguisher” on a reduced version of BLAKE2b with 3.5 rounds, and with complexity of 2^{480} . Clearly, this has no implication for the security of the full 12-round BLAKE2b (and actually not even on the 3.5-round reduced version).

Chapter 10

Conclusion

If you hide your ignorance, no one will hit you and you'll never learn.

—Ray Bradbury

It should be clear that, like all the other four SHA3 finalists, BLAKE and BLAKE2 are unlikely to be broken in a meaningful way—that is, in a way that allows an attacker to compromise the security of a system where they are used in a sound way. It is not excluded that one day someone will find, using sophisticated techniques, a “distinguisher” for the full permutation of BLAKE or of BLAKE2, but that would not affect its practical security. Therefore, one can reasonably consider that BLAKE and BLAKE2 are secure for the foreseeable future, with as intrinsic limitation the 2^{112} security of BLAKE-224 against collision attacks.

BLAKE2 is a modified version of BLAKE, and BLAKE builds on HAIFA (a variant of the Merkle–Damgård mode) and ChaCha (a variant of Salsa20), such as Rijndael (AES) built on Square, and Keccak on earlier experimental designs. BLAKE and BLAKE2 are not just our work, but the outcome of years of research by the cryptographic community that helped build understanding of and confidence in its components.

We have been happy to see that BLAKE2 has been adopted in several projects, such as WinRAR and submissions to the Password Hashing Competition.¹ We hope that BLAKE2, as an improved version of BLAKE, will continue to be perceived as a reasonable alternative to SHA3, especially for applications that require fast hashing in software.

The appendices contain test vectors, reference code, as well as a list of third-party implementations. Any questions or comments regarding BLAKE or the present book can be addressed to jeanphilippe.aumasson@gmail.com.

¹ <https://password-hashing.net>

References

1. Ambainis, A.: Polynomial degree and lower bounds in quantum complexity: Collision and element distinctness with small range. *Theory of Computing* **1**(1) (2005)
2. AMD: AMD64 Architecture Programmers Manual Volume 6: 128-Bit and 256-Bit XOP, FMA4 and CVT16 Instructions. <http://developer.amd.com/documentation/guides/Pages/default.aspx#manuals> (2009)
3. Anderson, R.J., Biham, E., Knudsen, L.R.: Serpent: A candidate block cipher for the Advanced Encryption Standard. <http://www.cl.cam.ac.uk/~rja14/serpent.html>
4. Andreeva, E., Luykx, A., Mennink, B.: Provable security of BLAKE with non-ideal compression function. In: *Selected Areas in Cryptography* (2012)
5. Aoki, K., Guo, J., Matusiewicz, K., Sasaki, Y., Wang, L.: Preimages for step-reduced SHA-2. In: *ASIACRYPT* (2009)
6. At, N., Beuchat, J.L., San, Í.: Compact implementation of Threefish and Skein on FPGA. In: *NTMS* (2012)
7. Atmel: 8-bit AVR instruction set. <http://www.atmel.com/Images/doc0856.pdf>. Rev. 0856I-AVR-07/10
8. Augot, D., Finiasz, M., Gaborit, P., Manuel, S., Sendrier, N.: SHA-3 Proposal: FSB. Submission to the SHA3 Competition (Round 1) (2010)
9. Aumasson, J.P.: Faster multicollisions. In: *INDOCRYPT* (2008)
10. Aumasson, J.P., Bernstein, D.J.: Siphash: a fast short-input PRF. In: *INDOCRYPT* (2012). See also <https://131002.net/siphash/>
11. Aumasson, J.P., Dunkelman, O., Indestege, S., Preneel, B.: Cryptanalysis of Dynamic SHA(2). In: *Selected Areas in Cryptography* (2009)
12. Aumasson, J.P., Dunkelman, O., Mendel, F., Rechberger, C., Thomsen, S.S.: Cryptanalysis of Vortex. In: *AFRICACRYPT* (2009)
13. Aumasson, J.P., Guo, J., Knellwolf, S., Matusiewicz, K., Meier, W.: Differential and invertibility properties of BLAKE. In: *FSE* (2010)
14. Aumasson, J.P., Henzen, L., Meier, W., Naya-Plasencia, M.: Quark: A lightweight hash. In: *CHES* (2010)
15. Aumasson, J.P., Henzen, L., Meier, W., Phan, R.C.W.: Toy versions of BLAKE. <https://131002.net/blake/toyblake.pdf>
16. Aumasson, J.P., Henzen, L., Meier, W., Phan, R.C.W.: SHA-3 proposal BLAKE. Submission to the SHA3 Competition (Round 3) (2010). URL <https://131002.net/blake/blake.pdf>
17. Aumasson, J.P., Meier, W., Phan, R.C.W.: The hash function family LAKE. In: *FSE* (2008)
18. Aumasson, J.P., Neves, S., Wilcox-O'Hearn, Z., Winnerlein, C.: BLAKE2: simpler, smaller, fast as MD5. In: *ACNS* (2013)

19. Bai, S., Brent, R.P.: On the efficiency of Pollard's rho method for discrete logarithms. In: CATS (2008)
20. Barreto, P., Rijmen, V.: The Whirlpool hashing function. First Open NESSIE Workshop (2000)
21. Bellare, M., Canetti, R., Krawczyk, H.: Keying hash functions for message authentication. In: CRYPTO (1996)
22. Bernstein, D.J.: Cache-timing attacks on AES. <http://cr.yp.to/papers.html#cachetiming>
23. Bernstein, D.J.: ChaCha, a variant of Salsa20. <http://cr.yp.to/chacha.html>
24. Bernstein, D.J.: Snuffle 2005: the Salsa20 encryption function. <http://cr.yp.to/snuffle.html>
25. Bernstein, D.J.: The Poly1305-AES message-authentication code. In: FSE (2005). See also <http://cr.yp.to/mac.html>
26. Bernstein, D.J.: Cost analysis of hash collisions: Will quantum computers make SHARCS obsolete? In: SHARCS (2009)
27. Bernstein, D.J., Buchmann, J., Dahmen, E. (eds.): Post-Quantum Cryptography. Springer (2009)
28. Bernstein, D.J., Lange, T. (eds.): eBACS: ECRYPT Benchmarking of Cryptographic Systems (2012). URL <http://bench.cr.yp.to>. Accessed 1 November 2012
29. Bertoni, G., Daemen, J., Peeters, M., Van Assche, G.: Sponge functions. <http://sponge.noekeon.org/SpongeFunctions.pdf>
30. Bertoni, G., Daemen, J., Peeters, M., Van Assche, G.: On the indistinguishability of the sponge construction. In: EUROCRYPT (2008)
31. Bertoni, G., Daemen, J., Peeters, M., Van Assche, G.: Sufficient conditions for sound tree and sequential hashing modes. Cryptology ePrint Archive, Report 2009/210 (2009)
32. Beuchat, J.L., Okamoto, E., Yamazaki, T.: Compact implementations of BLAKE-32 and BLAKE-64 on FPGA. Cryptology ePrint Archive, Report 2010/173 (2010)
33. Biham, E.: How to make a difference: Early history of differential cryptanalysis. Invited talk at FSE 2006
34. Biham, E., Biryukov, A., Shamir, A.: Miss in the middle attacks on IDEA and Khufu. In: FSE (1999)
35. Biham, E., Dunkelman, O.: A framework for iterative hash functions - HAIFA. Cryptology ePrint Archive, Report 2007/278 (2007)
36. Biham, E., Dunkelman, O., Keller, N.: The rectangle attack - rectangling the Serpent. In: EUROCRYPT (2001)
37. Biham, E., Dunkelman, O., Keller, N.: Related-key boomerang and rectangle attacks. In: EUROCRYPT (2005)
38. Biham, E., Dunkelman, O., Keller, N.: Related-key impossible differential attacks on 8-round AES-192. In: CT-RSA (2006)
39. Biham, E., Shamir, A.: Differential cryptanalysis of DES-like cryptosystems. Journal of Cryptology 4(1) (1991)
40. Biryukov, A.: The boomerang attack on 5 and 6-round reduced AES. In: AES4 (2004)
41. Biryukov, A., Khovratovich, D.: Related-key cryptanalysis of the full AES-192 and AES-256. Cryptology ePrint Archive, Report 2009/317 (2009)
42. Biryukov, A., Nikolić, I., Roy, A.: Boomerang attacks on BLAKE-32. In: FSE (2011)
43. Black, J., Cochran, M., Shrimpton, T.: On the impossibility of highly-efficient blockcipher-based hash functions. In: EUROCRYPT (2005)
44. Black, J., Halevi, S., Krawczyk, H., Krovetz, T., Rogaway, P.: UMAC: Fast and secure message authentication. In: CRYPTO (1999). See also <http://fastcrypto.org/umac/>
45. Black, J., Rogaway, P., Shrimpton, T., Stam, M.: An analysis of the blockcipher-based hash functions from PGV. J. Cryptology 23(4) (2010)
46. Boesgaard, M., Vesterager, M., Pedersen, T., Christiansen, J., Scavenius, O.: Rabbit: A new high-performance stream cipher. In: FSE (2003)
47. Bogdanov, A., Knudsen, L.R., Leander, G., Paar, C., Poschmann, A., Robshaw, M.J.B., Seurin, Y., Vikkelsoe, C.: PRESENT: An ultra-lightweight block cipher. In: CHES (2007)

48. Brassard, G., Høyer, P., Tapp, A.: Quantum cryptanalysis of hash and claw-free functions. *SIGACT News* **28**(2) (1997)
49. Chabaud, F., Joux, A.: Differential collisions in SHA-0. In: *CRYPTO* (1998)
50. Chang, D., Nandi, M., Yung, M.: Indifferentiability of the hash algorithm BLAKE. *Cryptology ePrint Archive, Report 2011/623* (2011)
51. Chang, D., Yung, M.: Midgame attacks (and their consequences). *Rump session of CRYPTO 2012* (2012)
52. Chang, S., Perlner, R., Burr, W.E., Turan, M.S., Kelsey, J.M., Paul, S., Bassham, L.E.: Third-round report of the SHA-3 cryptographic hash algorithm competition. *NISTIR 7896*, National Institute of Standards and Technology (2012)
53. Coke, J., Baliga, H., Cooray, N., Gamsaragan, E., Smith, P., Yoon, K., Abel, J., Valles, A.: Improvements in the Intel Core 2 Penryn Processor Family Architecture and Microarchitecture. *Intel Technology Journal* **12**(3), 179–193 (2008)
54. Contini, S., Lenstra, A.K., Steinfeld, R.: VSH, an efficient and provable collision-resistant hash function. In: *EUROCRYPT* (2006)
55. Coron, J.S., Dodis, Y., Malinaud, C., Puniya, P.: Merkle-Damgård revisited: How to construct a hash function. In: *CRYPTO* (2005)
56. Coron, J.S., Patarin, J., Seurin, Y.: The random oracle model and the ideal cipher model are equivalent. In: *CRYPTO* (2008)
57. Crosby, S.A., Wallach, D.S.: Denial of service via algorithmic complexity attacks. In: *USENIX Security* (2003)
58. Daemen, J., Rijmen, V.: *The Design of Rijndael*. Springer (2002)
59. Dean, R.D.: Formal aspects of mobile code security. Ph.D. thesis, Princeton University (1999)
60. Denning, D.E.R.: *Cryptography and Data Security*. Addison-Wesley (1982)
61. Designer, S.: Designing and attacking port scan detection tools. *Phrack Magazine* **8**(53) (1998)
62. Dodis, Y., Gennaro, R., Håstad, J., Krawczyk, H., Rabin, T.: Randomness extraction and key derivation using the CBC, Cascade and HMAC modes. In: *CRYPTO* (2004)
63. Dunkelman, O.: Re-visiting HAIFA. Talk at the workshop *Hash functions in cryptography: theory and practice* (2008)
64. Dunkelman, O., Khovratovich, D.: Iterative differentials, symmetries, and message modification in BLAKE-256. In: *ECRYPT2 Hash Workshop* (2011)
65. Duong, T., Rizzo, J.: Flickr's API signature forgery vulnerability. <http://netifera.com/research/> (2009)
66. Ferguson, N., Lucks, S., Schneier, B., Whiting, D., Bellare, M., Kohno, T., Callas, J., Walker, J.: The Skein hash function family. Submission to the SHA3 Competition (Round 3), <http://www.skein-hash.info/sites/default/files/skein1.3.pdf> (2010)
67. Ferguson, N., Schneier, B., Kohno, T.: *Cryptography Engineering: Design Principles and Practical Applications*. Wiley (2010)
68. Filho, D.G., Barreto, P., Rijmen, V.: The Maelstrom-0 hash function. In: *6th Brazilian Symposium on Information and Computer Security* (2006)
69. Fischlin, M., Lehmann, A., Wagner, D.: Hash function combiners in TLS and SSL. In: *CT-RSA* (2010)
70. Floyd, R.W.: Nondeterministic algorithms. *Journal of the ACM* **14**(4) (1967)
71. Gaj, K., Homsirikamol, E., Rogawski, M., Shahid, R., Sharif, M.U.: Comprehensive evaluation of high-speed and medium-speed implementations of five SHA-3 finalists using Xilinx and Altera FPGAs. In: *Third SHA-3 Candidate Conference 2012* (2012)
72. Geer, D.E.: A witness testimony in the hearing, Wednesday 25 april 07, entitled addressing the nation's cybersecurity challenges: Reducing vulnerabilities requires strategic investment and immediate action. Submitted to the Subcommittee on Emerging Threats, Cybersecurity, and Science and Technology (2007)
73. Grover, L.K.: A fast quantum mechanical algorithm for database search. In: *STOC* (1996)
74. Guo, J., Karpman, P., Nikolić, I., Wang, L., Wu, S.: Analysis of BLAKE2. In: *CT-RSA* (2014)

75. Guo, J., Matusiewicz, K.: Round-reduced near-collisions of BLAKE-32. WEWoRC (2009)
76. Guo, X., Srivastav, M., Huang, S., Ganta, D., Henry, M.B., Nazhandali, L., Schaumont, P.: ASIC implementations of five SHA-3 finalists. In: Proceedings of 2012 Design Automation and Test in Europe Conference – DATE 2012 (2012)
77. Gürcaynak, F., Gaj, K., Muheim, B., Homsirikamol, E., Keller, C., Rogawski, M., Kaeslin, H., Kaps, J.P.: Lessons learned from designing a 65 nm ASIC for evaluating third round SHA-3 candidates. In: Third SHA-3 Candidate Conference 2012 (2012)
78. Halevi, S., Krawczyk, H.: Strengthening digital signatures via randomized hashing. In: CRYPTO (2006)
79. Halevi, S., Myers, S., Rackoff, C.: On seed-incompressible functions. In: TCC (2008)
80. Haver, E., Ruud, P.: Experimenting with SHA-3 candidates in Tahoe-LAFS. Tech. rep., Norwegian University of Science and Technology (2010)
81. Henzen, L., Aumasson, J.P., Meier, W., Phan, R.C.W.: VLSI characterization of the cryptographic hash function BLAKE. IEEE Transactions on VLSI **19**(10), 1746–1754 (2011)
82. Heyse, S., von Maurich, I., Wild, A., Reuber, C., Rave, J., Poepplmann, T., Paar, C.: Evaluation of SHA-3 candidates for 8-bit embedded processors. In: Second SHA-3 Conference (2010)
83. Holenstein, T., Künzler, R., Tessaro, S.: The equivalence of the random oracle model and the ideal cipher model, revisited. In: STOC (2011)
84. Indestege, S., Mendel, F., Preneel, B., Rechberger, C.: Collisions and other non-random properties for step-reduced SHA-256. In: Selected Areas in Cryptography (2008)
85. Indestege, S., Mendel, F., Schlaefter, M., Rechberger, C.: Practical collisions for SHAMATA. Available online (2009)
86. Intel: C++ intrinsics reference (2007). Document no. 312482-002US
87. Jakimoski, G., Desmedt, Y.: Related-key differential cryptanalysis of 192-bit key AES variants. In: Selected Areas in Cryptography (2003)
88. Ji, L., Liangyu, X.: Attacks on round-reduced BLAKE. Cryptology ePrint Archive, Report 2009/238 (2009)
89. Jonsson, J., Kaliski, B.: Public-Key Cryptography Standards (PKCS) #1: RSA Cryptography Specifications Version 2.1. RFC 3447 (Informational) (2003)
90. Joux, A.: Multicollisions in iterated hash functions. application to cascaded constructions. In: CRYPTO (2004)
91. Joux, A.: Algorithmic Cryptanalysis. Chapman and Hall/CRC (2009)
92. Joux, A., Peyrin, T.: Hash functions and the (amplified) boomerang attack. In: CRYPTO (2007)
93. Jutla, C.S., Patthak, A.C.: A matching lower bound on the minimum weight of SHA-1 expansion code. Cryptology ePrint Archive, Report 2005/266 (2005)
94. Kaliski, B.: PKCS #5: Password-Based Cryptography Specification Version 2.0. RFC 2898 (Informational) (2000)
95. Kaliski, B.: PKCS #5: Password-Based Key Derivation Function 2 (PBKDF2) Test Vectors. RFC 6070 (Informational) (2011)
96. Kaps, J.P., Yalla, P., Surapathi, K.K., Habib, B., Vadlamudi, S., Gurung, S., Pham, J.: Lightweight implementations of SHA-3 candidates on FPGAs. INDOCRYPT 2011 (2011)
97. Kaufman, C.: Internet Key Exchange (IKEv2) Protocol. RFC 4306 (Proposed Standard) (2005)
98. Kelly, S., Frankel, S.: Using HMAC-SHA-256, HMAC-SHA-384, and HMAC-SHA-512 with IPsec. RFC 4868 (Proposed Standard) (2007)
99. Kelsey, J., Kohno, T., Schneier, B.: Amplified boomerang attacks against reduced-round MARS and Serpent. In: FSE (2000)
100. Kelsey, J., Schneier, B.: Second preimages on n -bit hash functions for much less than 2^n work. In: EUROCRYPT (2005)
101. Kelsey, J., Schneier, B., Hall, C., Wagner, D.: Secure applications of low-entropy keys. In: ISW (1997)
102. Kerckhof, S., Durvaux, F., Veyrat-Charvillon, N., Regazzoni, F.: Compact FPGA implementations of the five SHA-3 finalists. ECRYPT2 Hash Workshop 2011 (2011)

103. Khovratovich, D., Rechberger, C., Savelieva, A.: Bicliques for preimages: Attacks on Skein-512 and the SHA-2 family. In: FSE (2012)
104. Klima, V., Gligoroski, D.: Generic collision attacks on narrow-pipe hash functions faster than birthday paradox, applicable to MDx, SHA-1, SHA-2, and SHA-3 narrow-pipe candidates. Cryptology ePrint Archive, Report 2010/430 (2010)
105. Knudsen, L.R.: DEAL - a 128-bit block cipher. Tech. Rep. 151, University of Bergen (1998). Submitted as an AES candidate
106. Knudsen, L.R., Meier, W.: Improved differential attacks on RC5. In: CRYPTO (1996)
107. Knudsen, L.R., Rechberger, C., Thomsen, S.S.: The Grindahl hash functions. In: FSE (2007)
108. Knuth, D.E.: The Art of Computer Programming, 2nd edn. Addison-Wesley (1981)
109. Krawczyk, H., Bellare, M., Canetti, R.: HMAC: Keyed-Hashing for Message Authentication. RFC 2104 (Informational) (1997)
110. Krovetz, T.: UMAC: Message Authentication Code using Universal Hashing. RFC 4418 (Informational) (2006)
111. Kutin, S.: Quantum lower bound for the collision problem with small range. Theory of Computing **1**(1) (2005)
112. Lai, X., Massey, J.: Hash function based on block ciphers. In: EUROCRYPT (1992)
113. Lai, X., Massey, J.L.: Markov ciphers and differential cryptanalysis. In: EUROCRYPT (1991)
114. Leurent, G.: Analysis of differential attacks in ARX constructions. In: ASIACRYPT (2012)
115. Leurent, G.: ARXtools: A toolkit for ARX analysis. In: The Third SHA-3 Candidate Conference (2012)
116. Leurent, G.: Boomerang attacks against ARX hash functions. In: CT-RSA (2012)
117. Levin, L.A.: The tale of one-way functions. CoRR **cs.CR/0012023** (2000)
118. Li, J., Xu, L.: Attacks on round-reduced BLAKE. Cryptology ePrint Archive, Report 2009/238 (2009)
119. Lipmaa, H., Moriai, S.: Efficient algorithms for computing differential properties of addition. In: FSE (2001)
120. Lipmaa, H., Wallén, J., Dumas, P.: On the additive differential probability of exclusive-or. In: FSE (2004)
121. Liskov, M., Rivest, R., Wagner, D.: Tweakable block ciphers. In: CRYPTO (2002)
122. Lucks, S.: A failure-friendly design principle for hash functions. In: ASIACRYPT (2005)
123. Manuel, S.: Classification and generation of disturbance vectors for collision attacks against SHA-1. Cryptology ePrint Archive, Report 2008/469 (2008). 20081118:202259
124. Manuel, S.: Classification and generation of disturbance vectors for collision attacks against SHA-1. Des. Codes Cryptography **59**(1-3) (2011)
125. Matyas, S., Meyer, C., Oseas, J.: Generating strong one-way functions with cryptographic algorithm. IBM Technical Disclosure Bulletin **27**(10A) (1985)
126. Maurer, U.M., Renner, R., Holenstein, C.: Indifferentiability, impossibility results on reductions, and applications to the random oracle methodology. In: TCC (2004)
127. McDonald, C., Hawkes, P., Pieprzyk, J.: Differential path for SHA-1 with complexity $\mathcal{O}(2^{52})$. Cryptology ePrint Archive, Report 2009/259 (2009). Version 20090603:102152
128. Mendel, F., Nad, T., Schläffer, M.: Improving local collisions: New attacks on reduced SHA-256. In: EUROCRYPT (2013)
129. Miyaguchi, S., Ohta, K., Iwata, M.: New 128-bit hash function. In: 4th International Joint Workshop on Computer Communications (1989)
130. Neves, S., Aumasson, J.P.: BLAKE and 256-bit advanced vector extensions. In: Third SHA-3 Conference (2012)
131. NIST: Policy on hash functions. <http://csrc.nist.gov/groups/ST/hash/policy.html> (2006)
132. NIST: The keyed-hash message authentication code (HMAC). FIPS PUB 198-1 (2008)
133. NIST: Digital Signature Standard (DSS). FIPS PUB 186-3 (2009)
134. NIST: Randomized hashing for digital signatures. SP-800-106 (2009)
135. NIST: Status report on the second round of the SHA-3 cryptographic hash algorithm competition. Available from <http://www.nist.gov/hash-competition> (2009)

136. NIST: Recommendation for password-based key derivation. SP-800-132 (2010)
137. NIST: Secure Hash Standard (SHS). FIPS PUB 180-4 (2012)
138. NIST: Third-round report of the SHA-3 cryptographic hash algorithm competition. Available from <http://www.nist.gov/hash-competition> (2012)
139. NIST: SHA-3 standard: Permutation-based hash and extendable-output functions. FIPS PUB 202 (2014)
140. van Oorschot, P.C., Wiener, M.J.: Parallel collision search with cryptanalytic applications. *Journal of Cryptology* **12**(1) (1999)
141. Osvik, D.A.: Speeding up Serpent. In: AES Candidate Conference (2000)
142. Osvik, D.A.: Fast embedded software hashing. *Cryptology ePrint Archive*, Report 2012/156 (2012)
143. Percival, C.: Stronger key derivation via sequential memory-hard functions. In: BSDCan (1998)
144. Peyrin, T.: Cryptanalysis of Grindahl. In: ASIACRYPT (2007)
145. Pollack, D.: HSS: A simple file storage system for web applications. In: LISA (2012)
146. Pollard, J.M.: Monte-Carlo methods for index computation mod p . *Mathematics of Computation* **32**(143) (1978)
147. Preneel, B.: The first 30 years of cryptographic hash functions and the NIST SHA-3 competition. In: CT-RSA (2010)
148. Preneel, B., Bosselaers, A., Govaerts, R., Vandewalle, J.: Collision-free hash functions based on block cipher algorithms. In: Carnahan Conference on Security Technology (1989)
149. Preneel, B., Govaerts, R., Vandewalle, J.: Hash functions based on block ciphers: A synthetic approach. In: CRYPTO (1993)
150. Provos, N., Mazières, D.: A future-adaptable password scheme. In: Proceedings of the FREENIX Track: 1999 USENIX Annual Technical Conference, June 6-11, 1999, Monterey, California, USA (1999)
151. Quisquater, J.J., Delescaille, J.P.: How easy is collision search? Application to DES (extended summary). In: EUROCRYPT (1989)
152. Quisquater, J.J., Girault, M.: 2n-bit hash-functions using n-bit symmetric block cipher algorithms. In: EUROCRYPT (1989)
153. Rabin, M.: Digitalized signatures. In: R. Lipton, R. DeMillo (eds.) *Foundations of Secure Computation*. Academic Press (1978)
154. Ristenpart, T., Shacham, H., Shrimpton, T.: Careful with composition: Limitations of the indistinguishability framework. In: EUROCRYPT (2011)
155. Rogaway, P., Steinberger, J.P.: Constructing cryptographic hash functions from fixed-key blockciphers. In: CRYPTO (2008)
156. Rogaway, P., Steinberger, J.P.: Security/efficiency tradeoffs for permutation-based hashing. In: EUROCRYPT (2008)
157. Saarinen, M.J.O.: Security of VSH in the real world. In: INDOCRYPT (2006)
158. Schneier, B.: Description of a new variable-length key, 64-bit block cipher (Blowfish). In: FSE (1993)
159. Schneier, B., Kelsey, J., Whiting, D., Wagner, D., Hall, C., Ferguson, N.: *The Twofish Encryption Algorithm*. Wiley (1999)
160. Schwabe, P., Yang, B.Y., Yang, S.Y.: SHA-3 on ARM11 processors. In: Third SHA-3 Conference (2012)
161. Sedgewick, R., Szymanski, T.G., Yao, A.C.C.: The complexity of finding cycles in periodic functions. *SIAM Journal on Computing* **11**(2) (1982)
162. Sharif, M.U., Shahid, R., Rogawski, M., Gaj, K.: Use of embedded FPGA resources in implementations of five round three SHA-3 candidates. *ECRYPT2 Hash Workshop 2011* (2011)
163. Shin, Y., Williams, L.: Is complexity really the enemy of software security? In: *ACM Workshop on Quality of Protection, QoP* (2008)
164. Slipetsky, R.: Security issues in OpenStack. Master's thesis, Norwegian University of Science and Technology (2011)
165. Stevens, M.: New collision attacks on SHA-1 based on optimal joint local-collision analysis. In: EUROCRYPT (2013)

- 166. Stevens, M., Lenstra, A., de Weger, B.: Predicting the winner of the 2008 US presidential elections using a Sony PlayStation 3. <http://www.win.tue.nl/hashclash/Nostradamus/> (2007)
- 167. Stevens, M., Sotirov, A., Appelbaum, J., Lenstra, A.K., Molnar, D., Osvik, D.A., de Weger, B.: Short chosen-prefix collisions for MD5 and the creation of a rogue CA certificate. In: CRYPTO (2009)
- 168. Su, B., Wu, W., Wu, S., Dong, L.: Near-collisions on the reduced-round compression functions of Skein and BLAKE. In: CANS (2010)
- 169. Teske, E.: Speeding up Pollard's rho method for computing discrete logarithms. In: ANTS (1998)
- 170. Tillich, S., Feldhofer, M., Kirschbaum, M., Plos, T., Schmidt, J.M., Szekely, A.: High-speed hardware implementations of BLAKE, Blue Midnight Wish, CubeHash, ECHO, Fugue, Grøstl, Hamsi, JH, Keccak, Luffa, Shabal, SHAvite-3, SIMD, and Skein. Cryptology ePrint Archive, Report 2009/510 (2009)
- 171. Vidali, J., Nose, P., Pašalic, E.: Collisions for variants of the BLAKE hash function. Information Processing Letters **110**(14-15) (2010)
- 172. Wagner, D.: The boomerang attack. In: FSE (1999)
- 173. Wang, X., Feng, D., Lai, X., Yu, H.: Collisions for hash functions MD4, MD5, HAVAL-128 and RIPEMD. Cryptology ePrint Archive, Report 2004/199 (2004). See also [175]
- 174. Wang, X., Yin, Y.L., Yu, H.: Finding collisions in the full SHA-1. In: CRYPTO (2005)
- 175. Wang, X., Yu, H.: How to break MD5 and other hash functions. In: EUROCRYPT (2005)
- 176. Weinmann, R.P.: AXR. <http://www.dagstuhl.de/Materials/Files/09/09031/09031.WeinmannRalfPhilipp.Slides.pdf> (2009)
- 177. Wenzel-Benner, C., Gräf, J. (eds.): XBX: eXternal Benchmarking eXtension (2012). <http://xbx.das-labor.org/trac>

Appendix A

Test Vectors

One way or another, software is always tested: either by the maintainers, by users, or by applications in production.
—Kyle Kingsbury

We provide intermediate values for hashing a one-block and a two-block message, for each of the required digest sizes. For the one-block case, we hash the 8-bit message 00000000. For the two-block case we hash the 576-bit message 000...000 with BLAKE-256 and BLAKE-224, and we hash the 1,152-bit message 000...000 with BLAKE-512 and BLAKE-384. Values are given left to right, top to bottom; for example,

```
00800000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000 00000000 00000001 00000000 00000008
```

represents

$$\begin{array}{cccccccc} m_0 & m_1 & m_2 & m_3 & m_4 & m_5 & m_6 & m_7 \\ m_8 & m_9 & m_{10} & m_{11} & m_{12} & m_{13} & m_{14} & m_{15} \end{array}$$

A.1 BLAKE-256

A.1.1 One-Block Message

IV:

```
6a09e667 bb67ae85 3c6ef372 a54ff53a 510e527f 9b05688c 1f83d9ab 5be0cd19
```

Message block after padding:

```
00800000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000 00000000 00000001 00000000 00000008
```

Salt and counter:

```
00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
```

Initial state of v :

6a09e667 bb67ae85 3c6ef372 a54ff53a 510e527f 9b05688c 1f83d9ab 5be0cd19
243f6a88 85a308d3 13198a2e 03707344 a409382a 299f31d8 082efa98 ec4e6c89

State v after 1 round:

e78b8dfe 150054e7 cabcb8992 d15e8984 0669df2a 084e66e3 a516c4b3 339ded5b
26051fb7 09d18b27 3a2e8fa8 488c6059 13e513e6 b37ed53e 16cac7b9 75af6df6

State v after 2 rounds:

9de875fd 8286272e add20174 f1b0f1b7 37a1a6d3 cf90583a b67e00d2 943a1f4f
e5294126 43bd06bf b81ecba2 6af5ceaf 4feb3a1f 0d6ca73c 5ee50b3e dc88df91

State v after 5 rounds:

5af61049 fd4a2adc 5c1dbbd8 5ba19232 9a685791 2b3dd795 a84df8d6 a1d50a83
e3c8d94a 86ccc20a b4000ca4 596ac140 9d159377 a6374ffa f00c4787 767ce962

State v after 10 rounds:

bc04b9a6 c340c7ac 4aa36daa fdb53079 0d85d1be 14500fcd e8a133e1 788f54ae
07eec484 0505399d 837ccc3f 19ad3ee7 9d3fa079 fa1c772a f0dfd074 5c25729f

State v after 14 rounds:

7a07e519 4c7e2bac 28acf9ec a5adb385 f201e161 06b69682 b290a439 232a0956
1ce6d791 bace48a4 761dd447 d40ff618 d7a1d95f 0f298ad4 8e03e31d 69d958c8

Hash value output:

0ce8d4ef 4dd7cd8d 62dfdcd9 d4edb0a7 74ae6a41 929a74da 23109e8f 11139c87

A.1.2 Two-Block Message

IV:

6a09e667 bb67ae85 3c6ef372 a54ff53a 510e527f 9b05688c 1f83d9ab 5be0cd19

A.1.2.1 First Compression

Message block after padding:

00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000

Salt and counter:

00000000 00000000 00000000 00000000 00000000 00000000 00000200 00000000

Initial state of v :

```
6a09e667 bb67ae85 3c6ef372 a54ff53a 510e527f 9b05688c 1f83d9ab 5be0cd19
243f6a88 85a308d3 13198a2e 03707344 a4093a22 299f33d0 082efa98 ec4e6c89
```

State v after 1 round:

```
cc8704b8 14af5e97 448bd7a4 7d5ed80f 88d88192 8df5c28f b11e631f 0ac6ceab
01a455ba 43baaec3 c07c7dec 4c912c63 6f8cdfec 87fd02e0 d969b7b1 b74125b6
```

State v after 2 rounds:

```
d7ed8fc3 cc0a55f2 24014945 38a9d033 8da19e93 9b91d76a 18e0448c c10a0df6
fb350b3c d894b64e f1b35175 d0dff837 54e0df8f b3131c53 64bcb7a4 819fdfea
```

State v after 5 rounds:

```
6bb8eaa1 fb2d35b9 f1c87115 8cced083 c3ccf47f ec295b60 18cf9a21 dc2ac833
1f87fba1 759ae5f0 ee2f791d 11410f9f 46c442d0 ec5be440 dc9ed226 97e6e8bc
```

State v after 10 rounds:

```
58b76f7a 24300259 ea5baee6 7abecb5c beaa0c3c 38251bb6 f0d337af ff985d99
527e3c0c 4ebfc5fa bf73d485 8b538346 03c56421 d1b9147e 63662e6c 70e9e8b2
```

State v after 14 rounds:

```
730fc16c 4ec65cf3 8cbf360f d0d11f4f 8e062a2d 07e1dc39 b87b1478 d1e60507
acb995f2 e16e3e15 088d91e1 bc2af23b b8d7be9c b50d24fe 72662a9d 70af0e4d
```

Intermediate hash value:

```
b5bfb2f9 14cfcc63 b85c549c c9b4184e 67dfc6ce 29e9904b d59ee74e faa9c653
```

A.1.2.2 Second Compression**Message block after padding:**

```
00000000 00000000 80000000 00000000 00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000 00000000 00000001 00000000 00000240
```

Salt and counter:

```
00000000 00000000 00000000 00000000 00000000 00000000 00000240 00000000
```

Initial state of v :

```
b5bfb2f9 14cfcc63 b85c549c c9b4184e 67dfc6ce 29e9904b d59ee74e faa9c653
243f6a88 85a308d3 13198a2e 03707344 a4093a62 299f3390 082efa98 ec4e6c89
```

State v after 1 round:

```
cdb79def 93a4ecb5 7565bddf 6a981300 ddc59d39 1c31c834 2733ac31 df5f9c73
b0f52f8a 6ee197f0 b9c02368 be5fd351 f28c1ca7 7c045278 350c6a3f 831429fb
```

State v after 2 rounds:

```
a860da64 9f0316a8 d4ea6ef7 306b3189 e8ff54b6 c44ef07f 47aa4dc5 b1861fe9
654bf44c 63ca0c35 499e7310 38b9fa52 161d18f7 e8f59c12 2a8f9427 9a77e537
```

State v after 5 rounds:

```
1fd187b1 5cc01f1f 498fd157 56161cc5 d27c3fe9 a6b47936 d34baa06 dc1b2684
4f4a4639 06fdd62e 3b9eb4bb 0f749e2c 257b233b f3bf6d70 88155286 574a5fc8
```

State v after 10 rounds:

```
082d579c d41f4df3 973db87a 653d77e5 1fa637c8 f4bdaa22 5dbc0eac d3e836a8
1e7cf1e0 5f1c9c3b 13cd8444 79c5abfb 4802a70c 82a926e5 4a781534 6b4bd102
```

State v after 14 rounds:

```
4da680dc 9b42342c b18edaa2 65461d92 33289ef3 88c7594d eda0117e 3a412197
2c0088f6 a2ddb7f8 dd9fc832 ee375ce3 b1b3a271 b2732537 da252f9b 1c2aca85
```

Hash value output:

```
d419bad3 2d504fb7 d44d460c 42c5593f e544fa4c 135dec31 e21bd9ab dcc22d41
```

A.2 BLAKE-224

A.2.1 One-Block Message

IV:

```
c1059ed8 367cd507 3070dd17 f70e5939 ffc00b31 68581511 64f98fa7 befa4fa4
```

Message block after padding:

```
00800000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000008
```

Salt and counter:

```
00000000 00000000 00000000 00000000 00000000 00000000 00000008 00000000
```

Initial state of v :

```
c1059ed8 367cd507 3070dd17 f70e5939 ffc00b31 68581511 64f98fa7 befa4fa4
243f6a88 85a308d3 13198a2e 03707344 a409382a 299f31d8 082efa98 ec4e6c89
```

State v after 1 round:

```
04027914 24cfdd6b 7d33f394 12cbcc67 2de38c62 6664f3d3 1d8d68fc d6cd0b0b
481423a7 2f45b4f9 21c35492 50fb35fe 1255ae24 dff2a626 9240d453 e8530b9d
```

State v after 2 rounds:

```
9fb36742 31bc5ac2 064d4095 4a2260b2 c12165d2 00d0ee58 ad1d8245 4f7b0f17
36ef0086 38dfa9e5 a67cc4b5 20963eeb f2821838 d01907d2 7d15e12d 9b9ef864
```

State v after 5 rounds:

```
aab629f7 16de3e4a 5e78a622 257ebe3c 8669ea65 99d687fd a632ea5e 511b1c46
93068ab9 67ea727c 5ec4c9a9 7212cd6a 7f90526f 6e8952f4 70e30791 16c1ebd8
```

State v after 10 rounds:

```
c9e1652f ba9e5bde 660e702e 67fc6579 be6b4c7f f5f0749a 1dfe158f 3b49131f
62a1b43d e2d6f00a 67aaa716 e006a66d 95556f38 8145a426 1ec4de7e fc75ff74
```

State v after 14 rounds:

```
ce6b0120 7f7831c3 6c4ad4f1 145018af e6fc08d7 3796581b 04d73114 acce45be
4a6a54fb 5dffce8b 2653278f 8d163884 e703278e a1ff6179 c5093076 d4125387
```

Hash value output:

```
4504cb03 14fb2a4f 7a692e69 6e487912 fe3f2468 fe312c73 a5278ec5
```

A.2.2 Two-Block Message

IV:

```
c1059ed8 367cd507 3070dd17 f70e5939 ffc00b31 68581511 64f98fa7 befa4fa4
```

A.2.2.1 First Compression

Message block after padding:

```
00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
```

Salt and counter:

```
00000000 00000000 00000000 00000000 00000000 00000000 00000200 00000000
```

Initial state of v :

```
c1059ed8 367cd507 3070dd17 f70e5939 ffc00b31 68581511 64f98fa7 befa4fa4
243f6a88 85a308d3 13198a2e 03707344 a4093a22 299f33d0 082efa98 ec4e6c89
```

State v after 1 round:

e5b52991 1fbb7ecb f7350e64 0c8d11c6 148b1e94 7c688fed c8feee1b 4046ac6e
8bc4f63c c1c7fe8c 1fa6ae53 ee4dc034 87863887 2d70805b 4fa9a232 d9860f12

State v after 2 rounds:

2f3a90e3 ebbbc331 5737a2d1 6480f282 db471183 43014abd 88924f03 5160cb72
6e8f7eeb 115d1fd6 43387c5f ffb59797 f8663d1a d5fa0ec9 0c0ed9e5 8579d4a6

State v after 5 rounds:

f729608d 8119b461 e62f4d54 7889d045 838fbd7d 1a1e5618 8728c02b e973e337
06f32665 23b502c7 fedc26fc cefdl4a6 dad6b58f 4dca0d19 31d904cb 3c7e2160

State v after 10 rounds:

d3465c90 9af58db6 77044d06 8782e7b8 f5c3f50a 78a3a751 d7923ef6 647b8d32
7b80826f 21577a7a ce253568 1b6a082b d5e512e2 e213d8e0 f39651a7 f9fdae6e

State v after 14 rounds:

8cef86c7 a53fe03f c1cf9e13 92912ab7 e666b2ce 50e0c7b4 dfcd83e6 99aaaab2
5a8c1db8 c5df5da5 5252a472 02964ce7 64f7cc82 6737018c db48674d b0d3f7d2

Intermediate hash value:

176605a7 569c689d a3ede776 67093f69 7d51757d 5f8fd329 607c6b0c 978312c4

A.2.2.2 Second Compression

Message block after padding:

00000000 00000000 80000000 00000000 00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000240

Salt and counter:

00000000 00000000 00000000 00000000 00000000 00000000 00000240 00000000

Initial state of v :

176605a7 569c689d a3ede776 67093f69 7d51757d 5f8fd329 607c6b0c 978312c4
243f6a88 85a308d3 13198a2e 03707344 a4093a62 299f3390 082efa98 ec4e6c89

State v after 1 round:

78b24f69 dd359e3b 7c75e05e 779a4316 3d2bfbee ea479686 de701096 e01398e5
8907b84d 855fb196 d682ed6c 5487d95e caee46bb 33a39bbd 9c28f332 5ff502f1

State v after 2 rounds:

bc5a4c4c ad7d995a 00bba35d 0bea4495 d6c0f1cf 891eca54 8eb95e77 d1614112
73e586ab 40cae9c9 19c689dd 624bc7b7 7729314c 0fc7b802 e269ed89 b4c40dd1

State v after 5 rounds:

9664b1e6 c7329a7a 37db4880 779d1981 b05ecafd 49f78a02 16983441 80c80ab1
601c3551 0db868ec 7ad02138 691fc82e 118c8093 be617947 42ddda59 8862b2f2

State v after 10 rounds:

ad49264a f50b2055 29c2ec7b f8398abb fb6bba47 c9fc2626 1cd31e08 e3e75a78
144a402c ecda2a07 1ccaeed0 b73ac43b 2bb70fbb 71a9e691 4f9c2e99 8b78fc0e

State v after 14 rounds:

a1e9fee4 99180b3c 8f8629e3 c825f8de 48e8af2e 712c0633 87373eea 4e0ce59f
4325fb9e d33c2442 3868bc3a d4708103 bd34589b ee0ac28b dbb008e2 fae58bb1

Hash value output:

f5aa00dd 1cb847e3 140372af 7b5c46b4 888d82c8 c0a91791 3cfb5d04

A.3 BLAKE-512

A.3.1 One-Block Message

Message block after padding:

0080000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000001 0000000000000000 0000000000000008

IV:

6a09e667f3bcc908 bb67ae8584caa73b 3c6ef372fe94f82b a54ff53a5f1d36f1
510e527fade682d1 9b05688c2b3e6c1f 1f83d9abfb41bd6b 5be0cd19137e2179

Salt and counter:

0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000008 0000000000000000

Initial state of v :

6a09e667f3bcc908 bb67ae8584caa73b 3c6ef372fe94f82b a54ff53a5f1d36f1
510e527fade682d1 9b05688c2b3e6c1f 1f83d9abfb41bd6b 5be0cd19137e2179
243f6a8885a308d3 13198a2e03707344 a4093822299f31d0 082efa98ec4e6c89
452821e638d0137f be5466cf34e90c64 c0ac29b7c97c50dd 3f84d5b5b5470917

State v after 1 round:

98957863d61905b3 2064357139454e43 391fb64bd757fb63 a77c0e00bbe362b5
86d4b6c41f60c7e1 823f30053beb147c 68e6fc038d3b0b70 d93165f3477733df
ded9d48a51dde68f 3b73bb8b500c22b1 03f92332a668036b e2f0b698ea636bb9
a40103908a3fd2ae 016613ad1a47c604 bfbcb229c63e28b76 02a5ddf1aff95a3a

State v after 2 rounds:

```
84dac4b310f8b76b 01ce15a3aa8d8b2e f12c708c9d10a8b0 778c288779642198
13d4f878f30c3f5e 5b049744b1932015 0fcfc0dee2c0f4a0 80b67926a85e5ad8
8d0e3fb6c987be2b a1e68630be9171c7 06d755881837e80f b8729cfe5d112fa0
9226c2a7d8ad1f76 8265c86d8c126bc1 c0bfc6fee0cff19b e48fa8828eec436a
```

State v after 5 rounds:

```
efd689a66bdc0a95 2253dde0cb058ffc 886b8a405ae244fa ca317dfe42522691
fb5123461df359e7 17efb7c5fd09f586 8e07fe0bd4918c29 e3ae0acdf25d6303
6d4719e51f4a0833 27218b65bd7d4bc0 9227b3ea1497ad64 72b2c922552b72f9
855c5d1c44dd57a4 fc1340ae55773e39 03b57f827be2f1cd b43f42f4aa368791
```

State v after 14 rounds:

```
1c803aadb0c3622b 055eb72e5a0615b3 4624e5b1391e8a33 7b2a7aa93e27710a
f7ea864e4d591df7 34e2ff788dbd71a7 01d13a3673488668 390d346d5cb82ecf
00d6ac4e1b3d8de0 58cd6e304b8ad357 33e864217d9c1147 c9c686a43790d49f
8c76318c3b9e3c07 20952009e26ae7a1 e63865aec6b7e10c 2faffdcb74ade2de
```

State v after 16 rounds:

```
a4c49432d99d5e8d e90f2891abd6b4a6 49c0415e4a303c04 0411becca4309ea7
d84c660093c4cabd 1da7328a685c8535 af04db28c411cfe1 148facbcaf9cd9fe
595b67d2dcf8e77f e805a26c2b41f54c 8f13bb9aae41cd1d a413194ad2feb3b2
76d336c6c8bc63d1 3e99bb3b08feef23 aed8a237b480f33c 7b6aea4550ab4634
```

Hash value output:

```
97961587f6d970fa ba6d2478045de6d1 fabd09b61ae50932 054d52bc29d31be4
ff9102b9f69e2bbd b83be13d4b9c0609 1e5fa0b48bd081b6 34058be0ec49beb3
```

A.3.2 Two-Block Message

IV:

```
6a09e667f3bcc908 bb67ae8584caa73b 3c6ef372fe94f82b a54ff53a5f1d36f1
510e527fade682d1 9b05688c2b3e6c1f 1f83d9abfb41bd6b 5be0cd19137e2179
```

A.3.2.1 First Compression

Message block after padding:

```
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
```

Salt and counter:

```
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000400 0000000000000000
```

Initial state of v :

```
6a09e667f3bcc908 bb67ae8584caa73b 3c6ef372fe94f82b a54ff53a5f1d36f1
510e527fade682d1 9b05688c2b3e6c1f 1f83d9abfb41bd6b 5be0cd19137e2179
243f6a8885a308d3 13198a2e03707344 a4093822299f31d0 082efa98ec4e6c89
452821e638d01777 be5466cf34e9086c c0ac29b7c97c50dd 3f84d5b5b5470917
```

State v after 1 round:

```
1be45837f23baee5 2111f54a79ad333d f51f6f4bdbdacc64 bfd3af47522ba647
3cbd1a03babee0b1 4c1679e18847bed0 65375dda217af370 fc804555ea9c61c0
13dca8e50fcbbea2 a028a1030a7f2907 a8486683a019458c 6f50bbc1baad52d1
26ff0c474e8a8e46 3661dba5d8adce89 fb6e1530f3fa0cd2 29f3d982476d1c5b
```

State v after 2 rounds:

```
078a7f4ab38b51a3 3cc938d334f088ae c9688433013eb5f4 963a2028d731f262
a2e4f2f9127a623e 7df540dffec115f7 539403ccff3e7eda 4039a268638b91e7
6de0d9bf908ef408 d9747550eadaflb2 5cbeb17148553d5c cc40fd3e15dd6c42
528f6d54b521156e ce320314e7255341 c374721ddc0feeb2 f64047d64aed39a9
```

State v after 5 rounds:

```
7ce663efb2f3997d ca831a13aeladea2 1b489b08d9c77613 8449elf48bf74a4a
d7f36f5dad19b6f0 1b79a03b9dadcc93 0c5a6120750e5b4a 4d74c0055fea4d29
91ecb03ddfb95f46 d12929425d257265 4436f30ba8fda059 8f5ea5d22a3cfc07
1591886653094950 a98739e101b44d3a 78556c535f2905f2 e5bc8eddac0176df
```

State v after 14 rounds:

```
bae5b20438ebd1ae fb9eb556d67be6cd 1dd32aa12cb2c411 42374bfece90fa65
807e55b199234ecc 7fc73b526fadc9d8 760b6b884ba1b098 b77d0e14ccb094dd
fb079b4d09cda172 ee56fd3b622f28ac a4c9c6924b60c4b9 244e57a15b596644
7c86caace54a8e3e 71782ef1771e5aba 5fce8f0139cba368 d3f1a57a2bd841f4
```

State v after 16 rounds:

```
8ace4588105ef7e8 1cc36907319943be 40e0ac4199c96848 d758207628a2fcb1
0da86b4b6f335c80 40cda4c168a9570b 1a58bbb86dfe6baf c95c785976a6b38f
9c9dc23d05ee6893 933b75529e2be1fe 11b14581561a7ccc 288df0a868b9453d
e96ab70c1614870c 6437ba76484c940f 835fc973c1218ec7 63a773992264bd92
```

Intermediate hash value:

```
7c5a61d2e60c5673 349fb2d02b78057b 6d3f1ab23147ecaf 5a9a25e41f068f7d
b5cc8e38d4c1595d bfff763b0bdbaf1b 8684ab60579e5803 f11bc6d947bc2f64
```

A.3.2.2 Second Compression

Message block after padding:

```
0000000000000000 0000000000000000 8000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000001 0000000000000000 0000000000000480
```

Salt and counter:

```
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000480 0000000000000000
```

Initial state of v :

```
7c5a61d2e60c5673 349fb2d02b78057b 6d3f1ab23147ecaf 5a9a25e41f068f7d
b5cc8e38d4c1595d bfff763b0bdbaf1b 8684ab60579e5803 f11bc6d947bc2f64
243f6a8885a308d3 13198a2e03707344 a4093822299f31d0 082efa98ec4e6c89
452821e638d017f7 be5466cf34e908ec c0ac29b7c97c50dd 3f84d5b5b5470917
```

State v after 1 round:

```
7dc6e2217b190bd3 2d69c6d6aeda0572 c445cfa1ee378343 8761913893dac34f
d7ab98024a5de598 dd3c50178ba6cfe0 26ac7f783c286112 af357137bf5b27fa
537a754e12075d1e 08ae7d22952e350f 892b8373958f8500 edc023ef5fc2b9c3
3cee042f8e124fa5 ebccea756d5ddbdc 44eef37d26631b07 cbb87f4cc2dd2d13
```

State v after 2 rounds:

```
cc056856c518d859 7344abcd0d8a6950 ca67e04fb09d817b 1d8c4e9daaea72d1
e6b340711eca08bf 73c3ff68cf47f1f1 d2207fe16aba76e7 fa938a0bc99e8b07
1d18cc99351e737e 8fe782ca928829ff 02bb3600e4fdf376 b8c00d91ea6c13ea
3f91b8f1e4a84e64 cc0f5b8510b363b5 44b84d4f9533710e 65e10f27e5e5bffa
```

State v after 5 rounds:

```
93c53a007170b925 1a2fdd068c9d5f6e 00ac49ae15ab9892 037c2596c191739d
4ab00ac40c224583 335d1755fe36617f c5563c085f95a304 5186037e4bc146b7
413bdf4a9610b8ae 8b00f63774a69126 423466af367f81ae b07234da1883cd37
83dc32ec57dc0c0b e51c59511cffa5e1 38b2f87608ec0ed5 b77e9446582f3042
```

State v after 14 rounds:

```
23897e7c9eab8a3f 34125e009632ab3b 07ffb519e17e078d 7f488875753a238e
91e58ecf92563d9f c246847e756f98b3 2dd4f6bf4750bb17 07ce0e79086f7852
79103890fb73058d 53aac95c31b3b84e 64ee88c4fb103b29 c68ed0a58b94204f
ca2842ea101cf14b 251e178d430a7e37 c3e3c40fe82f826b f90d61b845d1c180
```

State v after 16 rounds:

```
c2961e406275c096 1b37a68dbee2abd6 4f8f5b9710a90b23 315bda6d8a014764
0837cd44dd4e7025 f773fbc58d201d97 e2ae133356abb427 6d44168b6b9d94b9
8ffb68448c905990 a2630aed65596132 e3e0f3f02115d479 7793504008324236
ae8ffbfdbf8235500c af7a62874c4addae aa34dcce6f3441b1 159dc3567175e603
```

Hash value output:

```
313717d608e9cf75 8dcbleb0f0c3cf9f c150b2d500fb33f5 1c52afc99d358a2f
1374b8a38bba7974 e7f6ef79cab16f22 ce1e649d6e01ad95 89c213045d545dde
```

A.4 BLAKE-384

A.4.1 One-Block Message

Message block after padding:

```
0080000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000008
```

IV:

```
cbbb9d5dc1059ed8 629a292a367cd507 9159015a3070dd17 152fec8f70e5939
67332667ffc00b31 8eb44a8768581511 db0c2e0d64f98fa7 47b5481dbefa4fa4
```

Salt and counter:

```
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000008 0000000000000000
```

Initial state of v :

```
cbbb9d5dc1059ed8 629a292a367cd507 9159015a3070dd17 152fec8f70e5939
67332667ffc00b31 8eb44a8768581511 db0c2e0d64f98fa7 47b5481dbefa4fa4
243f6a8885a308d3 13198a2e03707344 a4093822299f31d0 082efa98ec4e6c89
452821e638d0137f be5466cf34e90c64 c0ac29b7c97c50dd 3f84d5b5b5470917
```

State v after 1 round:

```
5b063a05f1a479bb 82ca717b7a4f6f94 4f58dfbdab593ffb f826c578573bec7e
c0836949c0fa750a 99fd9aa2e726bf09 32f52e2cbfc45a64 80686c4ae126cda9
5eb10a738bf891ee 3df23e84618c549f f2c230e414f34299 9191632bee7ee45e
c83cf461edc79b6d 8ff3fb919a781656 9be2fd02dfe1b98a 5b64934e1fe8370d
```

State v after 2 rounds:

```
5b2b57c1586feea6 7413d0fe48c32be2 535ca6f699c38d80 bbee0c0cbd530269
9e3cd39f1c1868da a4d8c74d2a7aa0f5 7524f4211494ef12 a94a548795a319ec
b9f9689afc6aeda6 ebc0e49c45a1e9aa 260d24a2d818cb43 ba3914617a2d98ec
f7ba66dc1aeb284c 9c362fbce59789d9 74b3b2650c513d2c d53eb118a489c053
```

State v after 5 rounds:

```
4292009f26c4caa5 17df7cf80e7a6542 24ca7fe6607b8393 c91ddca2afecd146
7ecaf3b6bc20cfd7 00d47510478c61b9 f1a2f95870eaf7b0 52ad845da7d26918
a0e941f5b18548fa bfc96fc91f31717 4b9f4584075d75c4 bf9c0ee7e53657ff
cb09e853ba91c13d fd46e7fe45aa85e3 ce6e1c891ffaaef9 2c9e50427598264a
```

State v after 14 rounds:

```
1dd69f386c168b30 eb4b1ad311c7c265 42044aa20151c2a0 1bd8cbe637dfb25d
94abf0918d4b9749 6a59118b73ab159b 56ee21c11395b066 00bb340a4c94c03b
2ec5d56650765851 b84bf78188e22a8d 5149df33128faac1 8e52cd242adb8ea8
88ea30691a1873aa dabf685d0556d4af 51168ca096930c62 e42652ffb6d559cf
```

State v after 16 rounds:

```
36512bf3e39351f8 9477606c71836a24 0efcb83c910deed8 23cc167714d245a0
71d6f1d7f5ada777 19b7c2f855b20b15 14ceb36724144e05 d8ae8c3ebba6cf13
edc2a9c9c3a3262a 1e05cb635dcaea33 38bc8f1c767f147e 01d7c4b422fe1dc5
3fdcc9354fd88b6b 84a44af8a049c603 85cf0f5d20038e18 2fb4fd1f72850c85
```

Hash value output:

```
10281f67e135e90a e8e882251a355510 a719367ad70227b1 37343e1bc122015c
29391e8545b5272d 13a7c2879da3d807
```

A.4.2 Two-Block Message

IV:

```
cbbb9d5dc1059ed8 629a292a367cd507 9159015a3070dd17 152fec8f70e5939
67332667ffc00b31 8eb44a8768581511 db0c2e0d64f98fa7 47b5481dbefa4fa4
```

A.4.2.1 First Compression

Message block after padding:

```
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
```

Salt and counter:

```
0000000000000000 0000000000000000 0000000000000000 0000000000000000
00000000000000400 0000000000000000
```

Initial state of v :

```
cbbb9d5dc1059ed8 629a292a367cd507 9159015a3070dd17 152fec8f70e5939
67332667ffc00b31 8eb44a8768581511 db0c2e0d64f98fa7 47b5481dbefa4fa4
243f6a8885a308d3 13198a2e03707344 a4093822299f31d0 082efa98ec4e6c89
452821e638d01777 be5466cf34e9086c c0ac29b7c97c50dd 3f84d5b5b5470917
```

State v after 1 round:

```
3bbf567d6d8e7c9a 826ab1796f4b2f2a d3589ab1a73a76fb 7ffb66ffaaa078b4
1f7bfe2284b78162 elf997f6b243cd2a 70b6ba23b832f52d b5418f66ec6d2031
ada82f0dd0769947 c23086272083f261 f6a871c70393f9fa 8d515b125606ead
c802f0cf294f6269 c6f36399df7e1e35 8f20eddf0ba7d74a de4472f1d1506e6f
```

State v after 2 rounds:

```
ea85a242a7f6cfce 89a54c23487ca8bf 5c8893d38ef63bf3 46b087aa28d56be5
5d085c4433f1929c 8134381eee29381f 36505ec762dab50c d71519e8814d4e39
f4a2235795910f0f 58ad370d224cb9b0 47d1e79a61966b91 0563f8e3ba681dbd
48d6e244313c9d0c d079de27cba8f3c8 dd134c5a6384efac 7e27a4ac04cf472d
```

State v after 5 rounds:

```
802c1f2e2198ae80 ee5b58bb836a1d70 8157b2da7fb7781d 9295e0c42dc728fc
d88df0e4bfc0adab 7871bb15b4555cab f89864b706e11f5f f01f54f3cb2b4e5f
014c1c71f0918e4d ea826f742daa21d0 33c03f7dfb0166dc 11442f58cfc88765
0d2fb5dcd1ade0ae 7c972bbfef957fb5 7d874f206dd2e3fb 8cfe8958c6233803
```

State v after 14 rounds:

```
48d2abeec2d71cc5 453acf7bb753bbf1 8ad951b5121e15f2 6d70d249d39a715a
af9fde1ee3cad40d c661f45a89950adc 843a9ee5d8169bd5 c74bc1121b511e1a
12d0217d0e74e5b1 cc7bd5e254c52b17 8636bf1d9b6e636b e5fdf466195146e0
16dac45878471174 cdae5b050c98e92a 121004668dbab665 aef35f816cea29f2
```

State v after 16 rounds:

```
3712b6e9cb7b63f2 37af7025586b6460 257ed91309eb62a0 c8e2f10f4c47949f
2a4a05037b5cddfa b5e117ff1e5a553e e1695e955cc18fe4 3100b996720399c7
b547462aecf8b55e db5bd016009287b3 a1e6cda8e4d58aab f25a251ec5a5da6e
cc6204cfc9023e98 9939a01e93e2ebdc 6d666072608b942f 5d6505e5b9649428
```

Intermediate hash value:

```
49ee6d9ee6864874 8e6e89196e8536d4 15c115e1dd4e351c 2f9738c97eec17c8
811b27ab4d9ee853 a26cfd66e5e0abf3 570310ea58b3946c 2bd0f46e759d424b
```

A.4.2.2 Second Compression

Message block after padding:

```
0000000000000000 0000000000000000 8000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000480
```

Salt and counter:

```
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000480 0000000000000000
```

Initial state of v :

49ee6d9ee6864874 8e6e89196e8536d4 15c115e1dd4e351c 2f9738c97eec17c8
811b27ab4d9ee853 a26cfd66e5e0abf3 570310ea58b3946c 2bd0f46e759d424b
243f6a8885a308d3 13198a2e03707344 a4093822299f31d0 082efa98ec4e6c89
452821e638d017f7 be5466cf34e908ec c0ac29b7c97c50dd 3f84d5b5b5470917

State v after 1 round:

006be95a66625251 79f3d0100619fe3f c0ac9991bbcbf7cd 8b84444c9ad96764
4f171ad0f3a3dea9 b1c7f7e6c97afff5 2e13ab4e1ebabb9f 49eb4a1d9e1f91f6
517d276924fefc3b ca0ee442f7580c9b 621cd230958bff1b 964c1f3a7f395ac4
86a45a4c3d9a424c 0b2d58ec8066608c 491952b97a0292cd 0fd9f18eb607b1f2

State v after 2 rounds:

9bba5065d0ddf6bd 18e52994739a91e0 72cd02f348c9ba19 a258f47a2f3e0a96
374e2ddcc60df1ef 0c442933ac2eb70e c4aefcdcabaeafb0 44965da93d4cc1a6
f2ede0ac437259f6 560175cb6a65f093 9755239e63b2d96a 51691777590cb37a
0d44f5e2447e7879 535f8292919e08e6 e47b361174c3d2f3 692fc37673f90e04

State v after 5 rounds:

9775064d5300cb4d c8dc04c98f8eeb4f f262d279cee88953 1d6822f8de090ddd
a86eb858c7914981 4257b029f13117a2 80bb47e2dc61fbbd 89f13f71786cdec3
0ccfacd927c99da8 22e7bee29f3fd1d5 ae62dc2965f57ee4 703573f8124518a0
683890980c63d04b f95d5141b985aedd 45a265f29715cfc7 fd9664f57fad2407

State v after 14 rounds:

4542b3975a2c224d 9046de63f984b8e6 75cd7a39321aede6 56c1820db8185b88
c63697063579ddfc 7c24c051f35bbbc4 da28ef56d97b2ae0 99bbf8b121ec6ad4
fe1e0776a0df6bb7 726de26c49f7939a 4c13939d3ca296d7 eb2d11499200ef0b
6a7c50324336de37 8b06973e8e5a5560 90097fd9bc7c9e8c f9f031f90127d78f

State v after 16 rounds:

a075e77b2d789059 694a9dfcecc350da bddd2a4edb40816a 2350b07555e4584b
317f8a79881aa9a8 e56eb3614a02d706 358c9dbb7621380e 66a32913135d8ed9
e203cf38896bbe0 4c533f44179417e1 56313dbef76725a1 6a7dfc286ccd8266
d91ca6ff6fe28549 63a0a229f2eb6bb9 48df2388ccde1001 fb66bfb8e1939963

Hash value output:

0b9845dd429566cd ab772ba195d271ef fe2d0211f16991d7 66ba749447c5cde5
69780b2daa66c4b2 24a2ec2e5d09174c

Appendix B

Reference C Code

Programming is fun, so can be cryptography; however they should not be combined.

—Charles Kreitzberg & Ben Shneiderman

This chapter provides reference C code for the four instances of BLAKE. All the code in this chapter is released under the CC0 1.0 Universal (CC0 1.0) Public Domain Dedication license. You can copy, modify, distribute and perform the work, even for commercial purposes, all without asking permission. Details and legal text are available on <http://creativecommons.org/publicdomain/zero/1.0/>.

B.1 blake.h

```
#include <string.h>
#include <stdio.h>
#include <stdint.h>

#define U8TO32_BIG(p) \
    (((uint32_t)((p)[0]) << 24) | ((uint32_t)((p)[1]) << 16) | \
     ((uint32_t)((p)[2]) << 8) | ((uint32_t)((p)[3])      ))

#define U32TO8_BIG(p, v) \
    (p)[0] = (uint8_t)((v) >> 24); (p)[1] = (uint8_t)((v) >> 16); \
    (p)[2] = (uint8_t)((v) >> 8); (p)[3] = (uint8_t)((v)      );

#define U8TO64_BIG(p) \
    (((uint64_t)U8TO32_BIG(p) << 32) | (uint64_t)U8TO32_BIG((p) + 4))

#define U64TO8_BIG(p, v) \
    U32TO8_BIG((p),      (uint32_t)((v) >> 32)); \
    U32TO8_BIG((p) + 4, (uint32_t)((v)      ));

typedef struct
{
    uint32_t h[8], s[4], t[2];
    int buflen, nullt;
    uint8_t buf[64];
}
```

```

} state256;

typedef state256 state224;

typedef struct
{
    uint64_t h[8], s[4], t[2];
    int buflen, nullt;
    uint8_t buf[128];
} state512;

typedef state512 state384;

const uint8_t sigma[][16] =
{
    { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15 },
    { 14, 10, 4, 8, 9, 15, 13, 6, 1, 12, 0, 2, 11, 7, 5, 3 },
    { 11, 8, 12, 0, 5, 2, 15, 13, 10, 14, 3, 6, 7, 1, 9, 4 },
    { 7, 9, 3, 1, 13, 12, 11, 14, 2, 6, 5, 10, 4, 0, 15, 8 },
    { 9, 0, 5, 7, 2, 4, 10, 15, 14, 1, 11, 12, 6, 8, 3, 13 },
    { 2, 12, 6, 10, 0, 11, 8, 3, 4, 13, 7, 5, 15, 14, 1, 9 },
    { 12, 5, 1, 15, 14, 13, 4, 10, 0, 7, 6, 3, 9, 2, 8, 11 },
    { 13, 11, 7, 14, 12, 1, 3, 9, 5, 0, 15, 4, 8, 6, 2, 10 },
    { 6, 15, 14, 9, 11, 3, 0, 8, 12, 2, 13, 7, 1, 4, 10, 5 },
    { 10, 2, 8, 4, 7, 6, 1, 5, 15, 11, 9, 14, 3, 12, 13, 0 },
    { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15 },
    { 14, 10, 4, 8, 9, 15, 13, 6, 1, 12, 0, 2, 11, 7, 5, 3 },
    { 11, 8, 12, 0, 5, 2, 15, 13, 10, 14, 3, 6, 7, 1, 9, 4 },
    { 7, 9, 3, 1, 13, 12, 11, 14, 2, 6, 5, 10, 4, 0, 15, 8 },
    { 9, 0, 5, 7, 2, 4, 10, 15, 14, 1, 11, 12, 6, 8, 3, 13 },
    { 2, 12, 6, 10, 0, 11, 8, 3, 4, 13, 7, 5, 15, 14, 1, 9 }
};

const uint32_t u256[16] =
{
    0x243f6a88, 0x85a308d3, 0x13198a2e, 0x03707344,
    0xa4093822, 0x299f31d0, 0x082efa98, 0xec4e6c89,
    0x452821e6, 0x38d01377, 0xbe5466cf, 0x34e90c6c,
    0xc0ac29b7, 0xc97c50dd, 0x3f84d5b5, 0xb5470917
};

const uint64_t u512[16] =
{
    0x243f6a8885a308d3ULL, 0x13198a2e03707344ULL,
    0xa4093822299f31d0ULL, 0x082efa98ec4e6c89ULL,
    0x452821e638d01377ULL, 0xbe5466cf34e90c6cULL,
    0xc0ac29b7c97c50ddULL, 0x3f84d5b5b5470917ULL,
    0x9216d5d98979fb1bULL, 0xd1310ba698dfb5acULL,
    0x2fffd72dbd01adfb7ULL, 0xb8e1afed6a267e96ULL,
    0xba7c9045f12c7f99ULL, 0x24a19947b3916cf7ULL,
    0x0801f2e2858efc16ULL, 0x636920d871574e69ULL
};

```

[illegible]

B.2 blake224.c

```
#include "blake.h"

void blake224_compress( state224 *S, const uint8_t *block )
{
    uint32_t v[16], m[16], i;
#define ROT(x,n) (((x)<<(32-n))|( (x)>>(n)))
#define G(a,b,c,d,e) \
    v[a] += (m[sigma[i][e]] ^ u256[sigma[i][e+1]]) + v[b]; \
    v[d] = ROT( v[d] ^ v[a],16); \
    v[c] += v[d]; \
    v[b] = ROT( v[b] ^ v[c],12); \
    v[a] += (m[sigma[i][e+1]] ^ u256[sigma[i][e]])+v[b]; \
    v[d] = ROT( v[d] ^ v[a], 8); \
    v[c] += v[d]; \
    v[b] = ROT( v[b] ^ v[c], 7);

    for( i = 0; i < 16; ++i ) m[i] = U8TO32_BIG( block + i * 4 );

    for( i = 0; i < 8; ++i ) v[i] = S->h[i];

    v[ 8] = S->s[0] ^ u256[0];
    v[ 9] = S->s[1] ^ u256[1];
    v[10] = S->s[2] ^ u256[2];
    v[11] = S->s[3] ^ u256[3];
    v[12] = u256[4];
    v[13] = u256[5];
    v[14] = u256[6];
    v[15] = u256[7];

    if ( !S->nullt )
    {
        v[12] ^= S->t[0];
        v[13] ^= S->t[0];
        v[14] ^= S->t[1];
        v[15] ^= S->t[1];
    }

    for( i = 0; i < 14; ++i )
```

```

{
    G( 0,  4,  8, 12,  0 );
    G( 1,  5,  9, 13,  2 );
    G( 2,  6, 10, 14,  4 );
    G( 3,  7, 11, 15,  6 );
    G( 0,  5, 10, 15,  8 );
    G( 1,  6, 11, 12, 10 );
    G( 2,  7,  8, 13, 12 );
    G( 3,  4,  9, 14, 14 );
}

for( i = 0; i < 16; ++i )  S->h[i % 8] ^= v[i];

for( i = 0; i < 8 ; ++i )  S->h[i] ^= S->s[i % 4];
}

void blake224_init( state224 *S )
{
    S->h[0] = 0xc1059ed8;
    S->h[1] = 0x367cd507;
    S->h[2] = 0x3070dd17;
    S->h[3] = 0xf70e5939;
    S->h[4] = 0xffc00b31;
    S->h[5] = 0x68581511;
    S->h[6] = 0x64f98fa7;
    S->h[7] = 0xbefa4fa4;
    S->t[0] = S->t[1] = S->buflen = S->>nullt = 0;
    S->s[0] = S->s[1] = S->s[2] = S->s[3] = 0;
}

void blake224_update( state224 *S, uint8_t *in, uint64_t inlen )
{
    int left = S->buflen;
    int fill = 64 - left;

    if( left && ( inlen >= fill ) )
    {
        memcpy( ( void * ) ( S->buf + left ), ( void * ) in, fill );
        S->t[0] += 512;

        if ( S->t[0] == 0 ) S->t[1]++;

        blake224_compress( S, S->buf );
        in += fill;
        inlen -= fill;
        left = 0;
    }

    while( inlen >= 64 )
    {
        S->t[0] += 512;

```

```

    if ( S->t[0] == 0 ) S->t[1]++;

    blake224_compress( S, in );
    in += 64;
    inlen -= 64;
}

if( inlen > 0 )
{
    memcpy( ( void * ) ( S->buf + left ), \
            ( void * ) in, ( size_t ) inlen );
    S->buflen = left + ( int )inlen;
}
else S->buflen = 0;
}

void blake224_final( state224 *S, uint8_t *out )
{
    uint8_t msglen[8], zz = 0x00, oz = 0x80;
    uint32_t lo = S->t[0] + ( S->buflen << 3 ), hi = S->t[1];

    if ( lo < ( S->buflen << 3 ) ) hi++;

    U32TO8_BIG( msglen + 0, hi );
    U32TO8_BIG( msglen + 4, lo );

    if ( S->buflen == 55 )
    {
        S->t[0] -= 8;
        blake224_update( S, &oz, 1 );
    }
    else
    {
        if ( S->buflen < 55 )
        {
            if ( !S->buflen ) S->>nullt = 1;

            S->t[0] -= 440 - ( S->buflen << 3 );
            blake224_update( S, padding, 55 - S->buflen );
        }
        else
        {
            S->t[0] -= 512 - ( S->buflen << 3 );
            blake224_update( S, padding, 64 - S->buflen );
            S->t[0] -= 440;
            blake224_update( S, padding + 1, 55 );
            S->>nullt = 1;
        }

        blake224_update( S, &zz, 1 );
        S->t[0] -= 8;
    }
}

```

```

    S->t[0] -= 64;
    blake224_update( S, msglen, 8 );
    U32TO8_BIG( out + 0, S->h[0] );
    U32TO8_BIG( out + 4, S->h[1] );
    U32TO8_BIG( out + 8, S->h[2] );
    U32TO8_BIG( out + 12, S->h[3] );
    U32TO8_BIG( out + 16, S->h[4] );
    U32TO8_BIG( out + 20, S->h[5] );
    U32TO8_BIG( out + 24, S->h[6] );
    U32TO8_BIG( out + 28, S->h[7] );
}

void blake224_hash( uint8_t *out, uint8_t *in, uint64_t inlen )
{
    state224 S;
    blake224_init( &S );
    blake224_update( &S, in, inlen );
    blake224_final( &S, out );
}

```

B.3 blake256.c

```

#include "blake.h"

void blake256_compress( state256 *S, const uint8_t *block )
{
    uint32_t v[16], m[16], i;
    #define ROT(x,n) (((x)<<(32-n))|( (x)>>(n)))
    #define G(a,b,c,d,e) \
        v[a] += (m[sigma[i][e]] ^ u256[sigma[i][e+1]]) + v[b]; \
        v[d] = ROT( v[d] ^ v[a],16); \
        v[c] += v[d]; \
        v[b] = ROT( v[b] ^ v[c],12); \
        v[a] += (m[sigma[i][e+1]] ^ u256[sigma[i][e]])+v[b]; \
        v[d] = ROT( v[d] ^ v[a], 8); \
        v[c] += v[d]; \
        v[b] = ROT( v[b] ^ v[c], 7);

    for( i = 0; i < 16; ++i ) m[i] = U8TO32_BIG( block + i * 4 );

    for( i = 0; i < 8; ++i ) v[i] = S->h[i];

    v[ 8] = S->s[0] ^ u256[0];
    v[ 9] = S->s[1] ^ u256[1];
    v[10] = S->s[2] ^ u256[2];
    v[11] = S->s[3] ^ u256[3];
    v[12] = u256[4];
    v[13] = u256[5];
    v[14] = u256[6];
    v[15] = u256[7];
}

```

```

if ( !S->nullt )
{
    v[12] ^= S->t[0];
    v[13] ^= S->t[0];
    v[14] ^= S->t[1];
    v[15] ^= S->t[1];
}

for( i = 0; i < 14; ++i )
{
    G( 0,  4,  8, 12,  0 );
    G( 1,  5,  9, 13,  2 );
    G( 2,  6, 10, 14,  4 );
    G( 3,  7, 11, 15,  6 );
    G( 0,  5, 10, 15,  8 );
    G( 1,  6, 11, 12, 10 );
    G( 2,  7,  8, 13, 12 );
    G( 3,  4,  9, 14, 14 );
}

for( i = 0; i < 16; ++i )  S->h[i % 8] ^= v[i];

for( i = 0; i < 8 ; ++i )  S->h[i] ^= S->s[i % 4];
}

void blake256_init( state256 *S )
{
    S->h[0] = 0x6a09e667;
    S->h[1] = 0xbb67ae85;
    S->h[2] = 0x3c6ef372;
    S->h[3] = 0xa54ff53a;
    S->h[4] = 0x510e527f;
    S->h[5] = 0x9b05688c;
    S->h[6] = 0x1f83d9ab;
    S->h[7] = 0x5be0cd19;
    S->t[0] = S->t[1] = S->buflen = S->nullt = 0;
    S->s[0] = S->s[1] = S->s[2] = S->s[3] = 0;
}

void blake256_update( state256 *S, uint8_t *in, uint64_t inlen )
{
    int left = S->buflen;
    int fill = 64 - left;

    if( left && ( inlen >= fill ) )
    {
        memcpy( ( void * ) ( S->buf + left ), ( void * ) in, fill);
        S->t[0] += 512;

        if ( S->t[0] == 0 ) S->t[1]++;

        blake256_compress( S, S->buf );
    }
}

```

```

    in += fill;
    inlen -= fill;
    left = 0;
}

while( inlen >= 64 )
{
    S->t[0] += 512;

    if ( S->t[0] == 0 ) S->t[1]++;

    blake256_compress( S, in );
    in += 64;
    inlen -= 64;
}

if( inlen > 0 )
{
    memcpy( ( void * ) ( S->buf + left ), \
            ( void * ) in, ( size_t ) inlen );
    S->buflen = left + ( int )inlen;
}
else S->buflen = 0;
}

void blake256_final( state256 *S, uint8_t *out )
{
    uint8_t msglen[8], zo = 0x01, oo = 0x81;
    uint32_t lo = S->t[0] + ( S->buflen << 3 ), hi = S->t[1];

    if ( lo < ( S->buflen << 3 ) ) hi++;

    U32TO8_BIG( msglen + 0, hi );
    U32TO8_BIG( msglen + 4, lo );

    if ( S->buflen == 55 )
    {
        S->t[0] -= 8;
        blake256_update( S, &oo, 1 );
    }
    else
    {
        if ( S->buflen < 55 )
        {
            if ( !S->buflen ) S->>nullt = 1;

            S->t[0] -= 440 - ( S->buflen << 3 );
            blake256_update( S, padding, 55 - S->buflen );
        }
        else
        {
            S->t[0] -= 512 - ( S->buflen << 3 );
            blake256_update( S, padding, 64 - S->buflen );
        }
    }
}

```

```

        S->t[0] -= 440;
        blake256_update( S, padding + 1, 55 );
        S->>nullt = 1;
    }

    blake256_update( S, &z0, 1 );
    S->t[0] -= 8;
}

S->t[0] -= 64;
blake256_update( S, msglen, 8 );
U32TO8_BIG( out + 0, S->h[0] );
U32TO8_BIG( out + 4, S->h[1] );
U32TO8_BIG( out + 8, S->h[2] );
U32TO8_BIG( out + 12, S->h[3] );
U32TO8_BIG( out + 16, S->h[4] );
U32TO8_BIG( out + 20, S->h[5] );
U32TO8_BIG( out + 24, S->h[6] );
U32TO8_BIG( out + 28, S->h[7] );
}

void blake256_hash( uint8_t *out, uint8_t *in, uint64_t inlen )
{
    state256 S;
    blake256_init( &S );
    blake256_update( &S, in, inlen );
    blake256_final( &S, out );
}

```

B.4 blake384.c

```

#include "blake.h"

void blake384_compress( state384 *S, const uint8_t *block )
{
    uint64_t v[16], m[16], i;
    #define ROT(x,n) (((x)<<(64-n))|((x)>>(n)))
    #define G(a,b,c,d,e) \
        v[a] += (m[sigma[i][e]] ^ u512[sigma[i][e+1]]) + v[b]; \
        v[d] = ROT( v[d] ^ v[a],32); \
        v[c] += v[d]; \
        v[b] = ROT( v[b] ^ v[c],25); \
        v[a] += (m[sigma[i][e+1]] ^ u512[sigma[i][e]])+v[b]; \
        v[d] = ROT( v[d] ^ v[a],16); \
        v[c] += v[d]; \
        v[b] = ROT( v[b] ^ v[c],11);

    for( i = 0; i < 16; ++i ) m[i] = U8TO64_BIG( block + i * 8 );

    for( i = 0; i < 8; ++i ) v[i] = S->h[i];

    v[ 8] = S->s[0] ^ u512[0];

```

```

v[ 9] = S->s[1] ^ u512[1];
v[10] = S->s[2] ^ u512[2];
v[11] = S->s[3] ^ u512[3];
v[12] = u512[4];
v[13] = u512[5];
v[14] = u512[6];
v[15] = u512[7];

if ( !S->>nullt )
{
    v[12] ^= S->t[0];
    v[13] ^= S->t[0];
    v[14] ^= S->t[1];
    v[15] ^= S->t[1];
}

for( i = 0; i < 16; ++i )
{
    G( 0, 4, 8, 12, 0 );
    G( 1, 5, 9, 13, 2 );
    G( 2, 6, 10, 14, 4 );
    G( 3, 7, 11, 15, 6 );
    G( 0, 5, 10, 15, 8 );
    G( 1, 6, 11, 12, 10 );
    G( 2, 7, 8, 13, 12 );
    G( 3, 4, 9, 14, 14 );
}

for( i = 0; i < 16; ++i ) S->h[i % 8] ^= v[i];

for( i = 0; i < 8 ; ++i ) S->h[i] ^= S->s[i % 4];
}

void blake384_init( state384 *S )
{
    S->h[0] = 0xcbbb9d5dc1059ed8ULL;
    S->h[1] = 0x629a292a367cd507ULL;
    S->h[2] = 0x9159015a3070dd17ULL;
    S->h[3] = 0x152fec8df70e5939ULL;
    S->h[4] = 0x67332667ffc00b31ULL;
    S->h[5] = 0x8eb44a8768581511ULL;
    S->h[6] = 0xdb0c2e0d64f98fa7ULL;
    S->h[7] = 0x47b5481dbefa4fa4ULL;
    S->t[0] = S->t[1] = S->buflen = S->>nullt = 0;
    S->s[0] = S->s[1] = S->s[2] = S->s[3] = 0;
}

void blake384_update( state384 *S, uint8_t *in, uint64_t inlen )
{
    int left = S->buflen;
    int fill = 128 - left;

```

```

if( left && ( inlen >= fill ) )
{
    memcpy( ( void * ) ( S->buf + left ), ( void * ) in, fill );
    S->t[0] += 1024;

    if ( S->t[0] == 0 ) S->t[1]++;

    blake384_compress( S, S->buf );
    in += fill;
    inlen -= fill;
    left = 0;
}

while( inlen >= 128 )
{
    S->t[0] += 1024;

    if ( S->t[0] == 0 ) S->t[1]++;

    blake384_compress( S, in );
    in += 128;
    inlen -= 128;
}

if( inlen > 0 )
{
    memcpy( ( void * ) ( S->buf + left ), \
            ( void * ) in, ( size_t ) inlen );
    S->buflen = left + ( int )inlen;
}
else S->buflen = 0;
}

void blake384_final( state384 *S, uint8_t *out )
{
    uint8_t msglen[16], zz = 0x00, oz = 0x80;
    uint64_t lo = S->t[0] + ( S->buflen << 3 ), hi = S->t[1];

    if ( lo < ( S->buflen << 3 ) ) hi++;

    U64TO8_BIG( msglen + 0, hi );
    U64TO8_BIG( msglen + 8, lo );

    if ( S->buflen == 111 )
    {
        S->t[0] -= 8;
        blake384_update( S, &oz, 1 );
    }
    else
    {
        if ( S->buflen < 111 )
        {
            if ( !S->buflen ) S->>nullt = 1;

```

```

    S->t[0] -= 888 - ( S->buflen << 3 );
    blake384_update( S, padding, 111 - S->buflen );
}
else
{
    S->t[0] -= 1024 - ( S->buflen << 3 );
    blake384_update( S, padding, 128 - S->buflen );
    S->t[0] -= 888;
    blake384_update( S, padding + 1, 111 );
    S->>nullt = 1;
}

    blake384_update( S, &zz, 1 );
    S->t[0] -= 8;
}

S->t[0] -= 128;
blake384_update( S, msglen, 16 );
U64TO8_BIG( out + 0, S->h[0] );
U64TO8_BIG( out + 8, S->h[1] );
U64TO8_BIG( out + 16, S->h[2] );
U64TO8_BIG( out + 24, S->h[3] );
U64TO8_BIG( out + 32, S->h[4] );
U64TO8_BIG( out + 40, S->h[5] );
}

void blake384_hash( uint8_t *out, uint8_t *in, uint64_t inlen )
{
    state384 S;
    blake384_init( &S );
    blake384_update( &S, in, inlen );
    blake384_final( &S, out );
}

```

B.5 blake512.c

```

#include "blake.h"

void blake512_compress( state512 *S, const uint8_t *block )
{
    uint64_t v[16], m[16], i;
#define ROT(x,n) (((x)<<(64-n))|( (x)>>(n)))
#define G(a,b,c,d,e) \
    v[a] += (m[sigma[i][e]] ^ u512[sigma[i][e+1]]) + v[b];\
    v[d] = ROT( v[d] ^ v[a],32); \
    v[c] += v[d]; \
    v[b] = ROT( v[b] ^ v[c],25); \
    v[a] += (m[sigma[i][e+1]] ^ u512[sigma[i][e]])+v[b]; \
    v[d] = ROT( v[d] ^ v[a],16); \
    v[c] += v[d]; \
    v[b] = ROT( v[b] ^ v[c],11);

```

```

for( i = 0; i < 16; ++i ) m[i] = U8TO64_BIG( block + i * 8 );

for( i = 0; i < 8; ++i ) v[i] = S->h[i];

v[ 8] = S->s[0] ^ u512[0];
v[ 9] = S->s[1] ^ u512[1];
v[10] = S->s[2] ^ u512[2];
v[11] = S->s[3] ^ u512[3];
v[12] = u512[4];
v[13] = u512[5];
v[14] = u512[6];
v[15] = u512[7];

if ( !S->>nullt )
{
    v[12] ^= S->t[0];
    v[13] ^= S->t[0];
    v[14] ^= S->t[1];
    v[15] ^= S->t[1];
}

for( i = 0; i < 16; ++i )
{
    G( 0, 4, 8, 12, 0 );
    G( 1, 5, 9, 13, 2 );
    G( 2, 6, 10, 14, 4 );
    G( 3, 7, 11, 15, 6 );
    G( 0, 5, 10, 15, 8 );
    G( 1, 6, 11, 12, 10 );
    G( 2, 7, 8, 13, 12 );
    G( 3, 4, 9, 14, 14 );
}

for( i = 0; i < 16; ++i ) S->h[i % 8] ^= v[i];

for( i = 0; i < 8 ; ++i ) S->h[i] ^= S->s[i % 4];
}

void blake512_init( state512 *S )
{
    S->h[0] = 0x6a09e667f3bcc908ULL;
    S->h[1] = 0xbb67ae8584caa73bULL;
    S->h[2] = 0x3c6ef372fe94f82bULL;
    S->h[3] = 0xa54ff53a5f1d36f1ULL;
    S->h[4] = 0x510e527fade682d1ULL;
    S->h[5] = 0x9b05688c2b3e6c1fULL;
    S->h[6] = 0x1f83d9abfb41bd6bULL;
    S->h[7] = 0x5be0cd19137e2179ULL;
    S->t[0] = S->t[1] = S->buflen = S->>nullt = 0;
    S->s[0] = S->s[1] = S->s[2] = S->s[3] = 0;
}

```

```

void blake512_update( state512 *S, uint8_t *in, uint64_t inlen )
{
    int left = S->buflen;
    int fill = 128 - left;

    if( left && ( inlen >= fill ) )
    {
        memcpy( ( void * )( S->buf + left ), ( void * ) in, fill );
        S->t[0] += 1024;

        if ( S->t[0] == 0 ) S->t[1]++;

        blake512_compress( S, S->buf );
        in += fill;
        inlen -= fill;
        left = 0;
    }

    while( inlen >= 128 )
    {
        S->t[0] += 1024;

        if ( S->t[0] == 0 ) S->t[1]++;

        blake512_compress( S, in );
        in += 128;
        inlen -= 128;
    }

    if( inlen > 0 )
    {
        memcpy( ( void * ) ( S->buf + left ), \
                ( void * ) in, ( size_t ) inlen );
        S->buflen = left + ( int )inlen;
    }
    else S->buflen = 0;
}

void blake512_final( state512 *S, uint8_t *out )
{
    uint8_t msglen[16], zo = 0x01, oo = 0x81;
    uint64_t lo = S->t[0] + ( S->buflen << 3 ), hi = S->t[1];

    if ( lo < ( S->buflen << 3 ) ) hi++;

    U64TO8_BIG( msglen + 0, hi );
    U64TO8_BIG( msglen + 8, lo );

    if ( S->buflen == 111 )
    {
        S->t[0] -= 8;
        blake512_update( S, &oo, 1 );
    }
}

```

```

    }
    else
    {
        if ( S->buflen < 111 )
        {
            if ( !S->buflen ) S->nullt = 1;

            S->t[0] -= 888 - ( S->buflen << 3 );
            blake512_update( S, padding, 111 - S->buflen );
        }
        else
        {
            S->t[0] -= 1024 - ( S->buflen << 3 );
            blake512_update( S, padding, 128 - S->buflen );
            S->t[0] -= 888;
            blake512_update( S, padding + 1, 111 );
            S->nullt = 1;
        }

        blake512_update( S, &z0, 1 );
        S->t[0] -= 8;
    }

    S->t[0] -= 128;
    blake512_update( S, msglen, 16 );
    U64TO8_BIG( out + 0, S->h[0] );
    U64TO8_BIG( out + 8, S->h[1] );
    U64TO8_BIG( out + 16, S->h[2] );
    U64TO8_BIG( out + 24, S->h[3] );
    U64TO8_BIG( out + 32, S->h[4] );
    U64TO8_BIG( out + 40, S->h[5] );
    U64TO8_BIG( out + 48, S->h[6] );
    U64TO8_BIG( out + 56, S->h[7] );
}

void blake512_hash( uint8_t *out, uint8_t *in, uint64_t inlen )
{
    state512 S;
    blake512_init( &S );
    blake512_update( &S, in, inlen );
    blake512_final( &S, out );
}

```

Appendix C

Third-Party Software

It ain't over till it's over.
—Yogi Berra

A number of third-party software projects enable the integration of BLAKE or BLAKE2 in programs written in various languages, through either native implementations or wrappers around C code.

Below we list the public projects known to us at the time of writing. These are generally open-source projects available under a permissive license. (For example, the reference C code of BLAKE2 was published under the public domain-like CC0 license). The list below does not constitute an endorsement or a recommendation.

C.1 BLAKE

The fastest implementations of BLAKE by Neves, Leurent, Pornin, and others are available in the SUPERCOP benchmarking suite (these also include some assembly implementations, for AVX instruction sets or ARM architectures): <http://bench.cr.yp.to/supercop.html>.

- AVR: <https://bitbucket.org/vmingo/blake256-avr-asm/> (von Maurich)
- C (HMAC-BLAKE): <https://github.com/davidlazar/BLAKE> (Lazar)
- C#: <http://www.dominik-reichl.de/projects/blakesharp/> (Reichl)
- Dart: <https://github.com/dchest/dart-blake> (Chestnykh)
- Go: <https://github.com/dchest/blake256> (Chestnykh)
- Haskell: <https://github.com/killerswan/Haskell-BLAKE> (Cantu)
- Java: <http://code.google.com/p/blake-512-java-implementation/> (Greim)
- JavaScript: <http://github.com/drostie/sha3-js> (Drost)
- JavaScript: <http://www.scottlogic.co.uk/2012/02/blake-512-in-javascript/> (Rhodes)
- Perl: <http://search.cpan.org/~gray/Digest-BLAKE-0.04/lib/Digest/BLAKE.pm> (Gray)
- PHP: <http://www.sinfocol.org/2011/01/blake-hash-extension-for-php> (Correa)

- Python: <http://www.seanet.com/~bugbee/crypto/blake/> (Bugbee)
- AVR: <http://www.das-labor.org/wiki/AVR-Crypto-Lib/en> (Otte)

C.2 BLAKE2

Reference and optimized implementations by Neves are available on <https://blake2.net>.

- C: <https://github.com/floodyberry/blake2b-opt> (Floodyberry)
- C: <https://github.com/cmr/libblake2> (Richardson)
- C (for PPC AltiVec): <https://github.com/blake2-ppc/blake2-ppc-altivec> (Sverdrup)
- Dart: <https://github.com/dchest/blake2-dart> (Chestnykh)
- Go: <https://github.com/dchest/b2sum> (Chestnykh)
- Go: <https://github.com/dchest/blake2b> (Chestnykh)
- Go: <https://github.com/dchest/blake2s> (Chestnykh)
- Java: <https://github.com/alphazero/Blake2b> (Houshyar)
- Node.js: <https://github.com/sekitaka/node-blake2> (sekitaka)
- Perl: <http://search.cpan.org/~gunya/Digest-BLAKE2-0.01/> (Suenaga)
- Python: https://github.com/buggywhip/blake2_py (Bugbee)
- Python: <https://github.com/dchest/pyblake2> (Chestnykh)
- Python: <https://github.com/darjeeling/python-blake2> (Bae)
- JavaScript: <https://github.com/dchest/blake2s-js> (Chestnykh)
- PHP: <https://github.com/strawbrary/php-blake2> (Akimoto)

Index

- AES (Rijndael), 4, 112, 115, 118, 119, 124, 128
- AVX2, 70, 179
- BLAKE-224, 43
- BLAKE-256, 37
- BLAKE-384, 43
- BLAKE-512, 41
- BLAKE2, 165
- BLAZE, 44, 163, 169
- BLOKE, 44, 163
- Boomerang attacks, 160
- BRAKE, 44, 163
- ChaCha (cipher), 122, 124, 125
- Checksum, 10
- Collision resistance, 18, 20, 110, 152
- Collisions multiplication, 155
- Commitment, 15
- Compression functions, 24, 28, 38, 42, 182
- Constants, 37, 168
- Constants (rationale), 128
- Data identification, 14
- Differential characteristic, 132
- Differential characteristics (iterative), 161
- Differential cryptanalysis, 131
- Differentials (impossible), 147
- Diffusion, 135, 142
- Distinguishers, 19
- Endianness, 6, 24, 40, 58, 169, 171
- Fixed points, 26, 137, 152, 163, 182
- FLAKE, 44
- Forgery, 10
- Grøstl (SHA3 finalist), 34
- HAIFA (iteration mode), 6, 27, 122
- Hash functions, 1, 17
- Hash functions (keyed), 18
- HMAC, 50
- Implementation (ARM), 62
- Implementation (ASIC), 98, 180
- Implementation (AVR), 60
- Implementation (C), 55, 177
- Implementation (C, vectorized), 64
- Implementation (FPGA), 100
- Implementation (Go), 58
- Implementation (Haskell), 59
- Implementation (Python), 59
- Indifferentiability, 20, 154
- Indistinguishability, 12
- Iteration modes, 24, 122
- JH (SHA3 finalist), 34
- Keccak (SHA3 finalist), 34
- Key derivation, 13
- Key update, 14
- LAKE (hash function), 122
- Length extension, 26, 110, 155
- MD5, 15, 165
- Meet-in-the-middle, 22
- Merkle–Damgård (iteration mode), 24, 122
- Message authentication codes (MACs), 10, 50, 172
- Miss-in-the-middle, 149
- Modification detection, 9
- Multicollisions, 25, 156
- Near-collision resistance, 159
- NEON, 83

- Padding, [24](#), [40](#)
- Parameter block (BLAKE2), [170](#)
- Password hashing, [13](#)
- PBKDF2, [13](#), [53](#)
- Permutations, [38](#)
- Permutations (rationale), [126](#)
- Preimage resistance, [15](#), [17](#), [110](#), [147](#), [158](#), [180](#)
- Proof-of-work, [14](#)
- Pseudorandom functions (PRFs), [12](#), [172](#)
- Pseudorandomness, [19](#), [153](#)

- Quantum computers, [23](#)

- Requirements (BLAKE), [121](#)
- Requirements (SHA3, general), [107](#)
- Requirements (SHA3, technical), [109](#)
- Robustness, [119](#)
- Rounds, [38](#), [39](#), [42](#), [167](#)
- Rounds (rationale), [128](#)

- S-boxes, [124](#)
- Salsa20 (cipher), [123](#)
- Salt, [38](#), [49](#), [170](#), [181](#)

- Second-preimage resistance, [18](#), [26](#), [110](#), [157](#)
- SHA1, [31](#)
- SHA2, [32](#), [110](#), [166](#)
- SHA3 competition, [2](#), [31](#), [34](#), [107](#), [165](#)
- Signatures, [11](#)
- Simplicity, [115](#)
- Skein (SHA3 finalist), [35](#), [122](#)
- Sponge functions (iteration mode), [27](#)
- SSE2, [64](#)
- SSE4.1, [70](#)
- SSSE3, [70](#)

- Timestamping, [15](#)
- Tree hashing, [79](#), [166](#), [172](#)

- Unpredictability, [19](#)

- Versatility, [120](#)

- Wide-pipe (iteration mode), [27](#), [122](#)

- XOP, [79](#)